The Enterprise Engineering Series

Monique Snoeck

Enterprise Information Systems Engineering

The MERODE Approach

Springer

The Enterprise Engineering Series

Series Editors

Jan Dietz
Erik Proper
José Tribolet

Editorial Board

Terry Halpin
Jan Hoogervorst
Martin Op 't Land
Ronald G. Ross
Robert Winter

More information about this series at
http://www.springer.com/series/8371

Monique Snoeck

Enterprise Information Systems Engineering

The MERODE Approach





Monique Snoeck
KU Leuven
Leuven
Belgium

ISSN 1867-8920 ISSN 1867-8939 (electronic)
ISBN 978-3-319-10144-6 ISBN 978-3-319-10145-3 (eBook)
DOI 10.1007/978-3-319-10145-3
Springer Cham Heidelberg New York Dordrecht London

Library of Congress Control Number: 2014950110

© Springer International Publishing Switzerland 2014
This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed. Exempted from this legal reservation are brief excerpts in connection with reviews or scholarly analysis or material supplied specifically for the purpose of being entered and executed on a computer system, for exclusive use by the purchaser of the work. Duplication of this publication or parts thereof is permitted only under the provisions of the Copyright Law of the Publisher's location, in its current version, and permission for use must always be obtained from Springer. Permissions for use may be obtained through RightsLink at the Copyright Clearance Center. Violations are liable to prosecution under the respective Copyright Law.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
While the advice and information in this book are believed to be true and accurate at the date of publication, neither the authors nor the editors nor the publisher can accept any legal responsibility for any errors or omissions that may be made. The publisher makes no warranty, express or implied, with respect to the material contained herein.

Printed on acid-free paper

Springer is part of Springer Science+Business Media (www.springer.com)

To all women in computing.

Preface

Many strategic IT projects fail, meaning that organisations are unable to reap the benefits of their IT investments to the fullest extent. One of the key reasons for this failure is the lack of coherence and congruence among the various components of an enterprise. This lack of congruence inevitably translates into a lack of integration among the many IT projects. Because of the penetration of IT in many aspects of an organisation, the need for a unified and integrated perspective on the organisation and its support with enterprise information system has become more important than ever before. One of the main goals of enterprise engineering is to foster the development of a congruent and coherent perspective of the organisation, in which all parts of the organisation are arranged in a harmonious and skilful way. In particular, the development of an encompassing 'Enterprise Architecture' is to offer a global perspective on an organisation's design, facilitating in this way better project and programme management by ensuring the mutual fit of individual projects. This book takes a design perspective to enterprise engineering and focuses on creating comprehensive and cohesive descriptions (models) of certain aspects of the enterprise. Such models allow gaining insight in the core structure of the enterprise, and they can be used to create the required essential structures of its supporting information systems. This book focuses on *domain modelling* as a way to foster the development of a common language to talk about essential business concepts and to develop a shared understanding of the rules that govern these business concepts. The MERODE approach offers a practical method for domain modelling and using these domain models for enterprise information system engineering.

The name 'MERODE' originally stood for '**m**odel-driven **e**ntity **r**elationship **o**bject-oriented **de**velopment', referring in this way to the roots of the method. The most outstanding feature of MERODE is that it uses the principle of existence dependency. This contributes to the completeness and flexibility of models while at the same time enabling intelligent consistency checks. Second, the method makes a clear distinction between specification and implementation, implementation being

achieved by transforming the specifications, not by elaborating them. Finally, MERODE is different from most other approaches in that it guarantees an extremely large independency between the enterprise layer and the application layer. This feature further improves flexibility and extensibility of the resulting enterprise information systems.

Over time, MERODE has integrated 25 years of research on requirements engineering, conceptual modelling, Entity Relationship Modelling, Jackson Systems Development, object orientation, process algebra, UML, BPMN, Enterprise Architecture frameworks, Services-Oriented Architecture, event-driven architecture and model-driven engineering performed within the Management Information Systems group of the Faculty of Business and Economics of the KU Leuven. The MERODE approach combines on the one hand best-of-breed industrial practices but—due to its academic development—also has a formal foundation that is based on the process algebra CSP, theory of Finite State Automata and Petri Net theory. This formal underpinning enables an intelligent approach to model quality uncommon in industrial standards, while at the same time, practitioners don't need to care about formal definitions or algebraic formulas.

This book is primarily intended for practitioners, students and researchers:

- Enterprise architects will find in this book practical advice on how to obtain a clear separation between demand and supply and a practical approach for the demand side of software development or delivery. MERODE is compliant with Enterprise Architecture frameworks such as TOGAF and ZACHMAN, but more importantly, it complements these architecture frameworks by offering very practical guidelines on how to obtain a correctly layered architecture and how to build a domain model and what quality criteria a good domain model should satisfy. They will also find a good example of how model-driven engineering can be put at work using a template-based approach.
- Both business analysts and students will find in this book practical advice on how to create complete models that combine structural and behavioural views of a system-to-be and that are ready to be transformed into a code using a generative approach and how to evaluate the quality of those models.
- Researchers in the area of conceptual modelling or enterprise modelling will find in this book a good integrated overview of the main research results related to the MERODE project.

This book has benefit from the work of many researchers that collaborated on the MERODE project (in reverse order of appearance): Gayane Sedrakyan, Tom Huysegoms, Pieter Hens, Manu De Backer, Geert Monsieur, Raf Haesen, Wilfried Lemahieu, Geert Poels, Anne-Marie Depuydt, Guido Dedene and, last but not least, Maurice Verhelst. The method has also benefit from the contributions of those who did not only use the method in their work but also took the time to challenge its practical relevance, engage in discussions with the researchers and guide master thesis students investigating particular topics of the method (in reverse order of appearance): Marc De Decker, Jan Wirix and Bert Dingemans. Finally, I'm also

indebted to the many, many students who have attended my classes and challenged the approach with interesting questions. These questions were a source of constant improvement.

Leuven, Belgium Monique Snoeck

Guide to the Reader

This book consists of four parts.

Part I introduces the basic philosophy and principles behind the approach proposed in this book and that will be refined in the subsequent parts of the book.

Chapter 1 introduces the concepts of enterprise engineering, Enterprise Architecture and enterprise modelling and the role of conceptual modelling in enterprise information systems engineering. It does this by first focusing on the discipline of Enterprise Architecture in order to understand the role of enterprise modelling. Next we focus on what modelling is about and answer the questions of what to model, from which perspective and how to do it. Finally, we go beyond modelling only and explain in the last section how model-driven engineering caters for a radically new approach for developing enterprise information systems.

Although the focus of this book lies on enterprise modelling and the demand side of software development, bridging the gap between demand and supply is also of prime importance. Therefore, the MERODE approach builds on two main principles. The first of these principles is the layered organisation of a system. This is a well-known high-level architectural pattern for information system architecture, but MERODE also follows this principle during requirements engineering. The second main principle that ensures a smooth transition from demand to supply is the quality assurance of the artefacts delivered by the enterprise architects to the software builder. These principles and how they benefit to the demand side as well as to the supply side are explained in more details in Chap. 2.

Chapter 3 concludes the introductory part with a general overview of the approach by means of a small library example. It should give the reader some insight in all the techniques and principles that are present in MERODE, without going too much in detail.

Part II gives an in-depth presentation of each of the domain modelling techniques. Chapters 4, 5 and 6 give a more detailed explanation of, respectively, data modelling, interaction modelling and life cycle modelling. Chapter 7 explains attributes and constraints which provide more detail to models. Chapter 8 presents how to use inheritance in conceptual modelling.

Each chapter is concluded with a section presenting the formal underpinning and a section that presents the MERODE meta-model. These sections are mainly intended for researchers and tool builders and can be safely skipped by the reader who wishes to focus on the practical application of the approach.

Part III addresses the remaining layers. Chapter 9 gives a brief account of techniques to use in information system service modelling and Chap. 10 presents the business process layer and explains how to integrate business process modelling with domain modelling and the information system services.

Finally, part IV focuses on model transformation. Chapter 11 explains the main principles of model-driven engineering and illustrates this by means of the transformation rules and the transformation technology used to transform MERODE models in working prototypes. Finally, Chap. 12 concludes by explaining how the approach can be adapted to application and component integration.

The MERODE approach is also supported by a tool, called JMermaid (for Java MERODE modelling aid), and a code generator environment. This software is free and can be downloaded from the MERODE website: http://merode.econ.kuleuven.be.

There is still a lot of material that didn't make it to this book. For *students* there is a large collection of cases with solutions. These are made available to the students through the e-learning platform. Of interest to both *students* and *practitioners* are the domain modelling patterns that have been developed as a result of practice and research. Some of these are available as publications; most will be made available via the website. *Researchers* will find quite some extra material in the many publications on MERODE-related topics. Most of these can be retrieved from the author's personal KU Leuven page.

Contents

Part I Introduction

1 Enterprise Modelling 3
1.1 Enterprise Engineering 3
1.2 Enterprise Architecture 5
1.3 How to Develop Enterprise Models 12
1.3.1 What Exactly Is Modelling? 12
1.3.2 Which Aspects to Include into Models? 15
1.3.3 From Which Perspective Should Models Be Made? . . . 17
1.3.4 Where to Start, How to Go On and When to Stop? . . . 20
1.4 Modelling and Then What? 20
1.4.1 Using Models for Software Development 21
1.4.2 Following a Model-Driven Engineering Approach 23
1.5 Summary 29

2 From Demand to Supply: Layers and Model Quality 31
2.1 A Layered System Architecture Both for Requirements and System Architecture 32
2.1.1 Enterprise, Information Services and Business Process Layer 32
2.1.2 Layers Versus Requirements Gathering and Engineering 35
2.2 Formal Verification of Models 40
2.2.1 Model Quality 40
2.2.2 Modelling Language 44
2.2.3 Tool Support for Quality Control 46
2.2.4 Quality Checking in MERODE and JMERMAID 48
2.3 Summary 49

3 Overview of MERODE 51
3.1 The Modelling Phases 51
3.2 Domain Modelling 52

3.2.1 Business Object Types . . . 53
3.2.2 Business Event Types . . . 54
3.2.3 A Small Library Example . . . 59
3.3 Information System Service Modelling . . . 70
3.4 Business Process Modelling . . . 72
3.5 Implementation . . . 74
3.6 A Note to the Reader . . . 75

Part II Domain Modelling Techniques

4 The Existence-Dependency Graph . . . 79
4.1 Why Existence Dependency? . . . 79
4.1.1 Frozen Versus Modifiable Association Ends in UML . . . 80
4.1.2 Association, AssociationClass or Class? . . . 81
4.1.3 Improved Consistency Checking . . . 82
4.1.4 Improved Possibilities for Transformation to Code . . . 83
4.2 The Existence-Dependency Graph . . . 83
4.2.1 Defining Existence Dependency . . . 83
4.2.2 The Existence-Dependency Graph . . . 84
4.2.3 Multiplicity of Existence Dependency . . . 85
4.2.4 Graphical Representation . . . 86
4.2.5 Life Cycle Implications of Existence Dependency . . . 88
4.3 How to Create an Existence-Dependency Graph . . . 94
4.3.1 UML Class . . . 95
4.3.2 Binary Association . . . 95
4.3.3 Unary Association . . . 97
4.3.4 N-ary Associations . . . 98
4.3.5 Aggregation . . . 98
4.3.6 Describing Versus Shaping the Real World . . . 101
4.4 Some Final Considerations . . . 103
4.5 Formal Definition of the Existence-Dependency Graph . . . 103
4.6 Meta-Model . . . 104

5 Object Interaction . . . 107
5.1 What Is a Business Event? . . . 107
5.2 The Object-Event Table . . . 112
5.2.1 Definition . . . 112
5.2.2 Graphical Representation . . . 113
5.3 Existence-Dependency Graph Versus Object-Event Table . . . 114
5.3.1 Propagation Rule . . . 114
5.3.2 Type of Involvement Rule . . . 117
5.3.3 Contract Rule . . . 120
5.4 Multiple Propagation . . . 122

5.5 Formal Definition of the Object-Event Table 123
5.6 Meta-Model 125

6 Object and System Behaviour 127
6.1 Object Behaviour Modelling 127
6.1.1 Finite-State Machines 127
6.1.2 Basic Correctness Checks 130
6.1.3 Stratification of Finite-State Machines 131
6.1.4 Exploiting Parallelism 132
6.2 Global System Behaviour 134
6.3 Consistency Checking 136
6.3.1 Consistency Checking with the OET 137
6.3.2 Consistency Checking with the EDG 138
6.3.3 Checking Global Behaviour 141
6.4 Formal Definition of Interaction by Joint Participation to Business Events 144
6.5 Meta-model 147

7 Attributes and Constraints 149
7.1 Defining Attributes 149
7.1.1 What Are Attributes? 149
7.1.2 The Difference Between Object Types and Attributes 150
7.1.3 Data Types 153
7.2 Defining Constraints 154
7.2.1 Uniqueness Constraints 155
7.2.2 Attribute Constraints 156
7.2.3 Method Constraints 156
7.2.4 Referential Integrity and Sequence Constraints 157
7.3 Defining Methods 161
7.4 Aliases 161
7.5 Meta-model 168

8 Inheritance 171
8.1 Inheritance in the Class Diagram 171
8.1.1 Definition of Generalisation/Specialisation 171
8.1.2 Inheritance 176
8.2 Behavioural Aspects of Inheritance 177
8.2.1 Specialisation of Event Types 177
8.2.2 Propagating Along Inherited Dependencies 181
8.2.3 More Examples 185
8.3 On the Use of Generalisation/Specialisation Hierarchies and Roles 191
8.3.1 Attribute-Defined Subclass 192
8.3.2 Existence-Defined Subclass 193

8.3.3 State-Defined Subclass 196
8.3.4 Guidelines for Using Roles and Generalisation/ Specialisation 197
8.4 Formal Definition of Inheritance 200
8.5 Meta-model 200

Part III The Information System Layer and the Business Process Layer

9 The Information System Service Layer 205
9.1 Information Objects 206
9.2 Information System Services 207
9.2.1 What Is an Information System Service? 207
9.2.2 Output Services 208
9.2.3 Input Services 209
9.2.4 Complex Services 211
9.3 Refining the Architecture of the Information System Layer 215
9.3.1 The Event Handling Layer 215
9.3.2 Consistent Event Types and Transactions 215
9.3.3 Cross-Cutting Concerns 218
9.4 Case Study: The JMermaid Architecture 219
9.4.1 Information Object Types 219
9.4.2 Event Handling 220
9.5 Meta-model 222

10 Bridging Business Process Modelling and Domain Modelling 223
10.1 Business Process Modelling 223
10.2 Linking Business Processes to the Enterprise Layer and the IS Service Layer 226
10.3 Distributing Constraints Across Layers 230
10.3.1 General Principles 230
10.3.2 An Order Handling Example 233
10.3.3 Cross-Layer Consistency 238
10.4 Meta-model 240

Part IV Model Transformation

11 Model Transformation 245
11.1 Model-Driven Development 245
11.2 Transformation Rule Examples 247
11.2.1 General Implementation Architecture 247
11.2.2 The Enterprise Layer 248
11.2.3 The Event Handling Layer 255
11.3 Transformation Technology Example 256
11.3.1 The mxp File 256
11.3.2 Template-Based Approach to Transformation 258

12 Application and Component Integration 261
12.1 Business Event-Based Coordination 261
12.2 The Component-Event Table 264
12.3 Architectures for Event Handlers 265
12.4 Contract Management Capabilities 267
12.5 Benefits of Event-Based Coordination 270

References 271

Index 277

List of Examples

Example 3.1	The Library: Coordinating Across Business Objects	55
Example 3.2	The Library: Small Library Case Text	59
Example 3.2	The Library: Analysis of the Small Library Case Text	59
Example 3.2	The Library: Analysis of the Small Library Case Text (continued)	61
Example 3.3	The Library: Book Acquisition Process	72
Example 4.1	The Library: Life Spans of Copies and Loans	84
Example 4.2	The Library: Existence Dependencies	85
Example 4.3	The Library: Existence-Dependency Graph	87
Example 4.4	Customer and Orders	92
Example 4.5	Invoicing Bus Trips	93
Example 4.6	Aggregation: Kitchen Furniture Set	99
Example 4.7	Salesperson and Order Management	101
Example 4.8	Pharmacy	102
Example 5.1	Ordering: Handling a Modify Quantity Event	113
Example 5.2	The Library: Simplified and Incomplete Object-Event Table	114
Example 5.3	The Library: Constraints Spanning Several Object Types	115
Example 5.4	The Library: Fully Specified OET	116
Example 5.5	Project Administration: EDG and OET	116
Example 5.6	The Library: Type of Involvement Rule	118
Example 5.7	Project Administration: Wrong OET for Closing Projects	119
Example 5.8	The Library: Contract Rule for Reservation Event Types	120
Example 5.9	The Library: Formal Definition of the OET	124
Example 6.1	The Library: Behaviour Definition	128
Example 6.2	The Library: Correctness Check of a Finite-State Machine for ACCOUNT	130
Example 6.3	The Library: Global System Behaviour	136
Example 6.4	The Library: Verification of the Restriction Rule	140
Example 6.5	House Owners: 'More Deterministic than' Not Resulting from Existence Dependency	141

Example 6.6 The Library: Calculating Global Behaviour 142
Example 6.7 The Library: Applying the Projection to Scenarios 144
Example 6.8 The Library: Applying the Projection to Regular Expressions 144
Example 7.1 Stock Management 152
Example 7.2 Product Pricing 153
Example 7.3 Constraint: No More than 5 Loans 154
Example 7.4 Constraint: Return Date After Loan Date 156
Example 7.5 Constraint: Closing Projects Requires Closed Registrations ... 156
Example 7.6 Constraint: No Refunding After the Deadline 157
Example 7.7 Constraint: No Overdrawing of Accounts Beyond Credit Limit 157
Example 7.8 Sales and Billing: Multiple Propagation Constraint 161
Example 7.9 Team Relationships: Aliases and Constraints 163
Example 7.10 Part-Type Prerequisites: Aliases and Constraints 165
Example 7.11 Pharmacy: Multiple Propagation 166
Example 8.1 Animals: Generalisation/Specialisation Hierarchy 173
Example 8.2 Loans at the Bank 177
Example 8.3 Material Products and Services 185
Example 8.4 The Library: Specialised Items 185
Example 8.5 The Pharmacy: Free and Restricted Products 189
Example 8.6 Books: Attribute-Defined Subclass, Based on Colour 192
Example 8.7 Cars: Attribute-Defined Subclass, Based on Price 193
Example 8.8 Managers of Departments: Existence-Defined Subclass 194
Example 8.9 Conference Organisation 194
Example 8.10 Ships: State-Defined Subclass 196
Example 8.11 Real Estate Management 199
Example 9.1 A Car Rental Company 211
Example 9.2 Assigning Employees to Departments 216
Example 9.3 Student Grading: Authorisation as Cross-Cutting Concern ... 218
Example 10.1 Borrowing a Book: Relationship Between Different Kinds of Events 227
Example 10.2 Book Acquisition: Linking the Business Process Model to the Domain Model 228
Example 10.3 Returning Parcels 231
Example 11.1 Code Examples for Class, Attribute and Associations for the Library Example 250
Example 11.2 Code Example for Transforming Finite-State Machines for the Library Example 252
Example 11.3 Code Snippets for Preconditions 252
Example 11.4 Cardinality and Referential Integrity Verification for COPY 254

Part I
Introduction

Chapter 1
Enterprise Modelling

1.1 Enterprise Engineering

As enterprises are complex systems involving both human and technological aspects and are highly influenced by the environment in which they operate, architecting an enterprise and the information systems that support its functioning is not an easy task. In the usual way of working, enterprise information systems development is guided by strategy development, programme management and Enterprise Architecture [72].

Strategy development is a discipline that focuses on the formulation of an organisation's strategy taking into account factors such as suppliers, buyers, competitors, environment, etc. Business-IT alignment is a major concern in such strategy development and its subsequent operationalisation. Henderson and Venkatraman's framework [33] is an example of a framework specifically focusing on the mechanisms of aligning IT strategy and business strategy and on aligning strategy to operations. Once business and IT strategy have been formulated, their operationalisation will typically give rise to a number of IT-projects, each of which aims to realise a distinct part of the to-be situation. In order to ensure that each project contributes to the global goal, coherence and consistency between projects need to be managed. This is the area of programme management. A programme manager will be responsible to define a road map that sets out how each project sets a step to realise the strategy and delivers business benefits.

Nevertheless, as project and programme management focus on the realisation of business benefits within time, resource, and budget constraints, problems may still arise due to a lack of global perspective. As a consequence of adjustments of individual projects, their mutual sets of results may show gaps, overlaps, and inconsistencies. Imagine a team of interior architects each designing and optimising one room (the kitchen, the bathroom, the study, the living room, etc.). Without the possibility of referring to a global plan, chances would be rather small that these

© Springer International Publishing Switzerland 2014

M. Snoeck, *Enterprise Information Systems Engineering*, The Enterprise Engineering Series, DOI 10.1007/978-3-319-10145-3_1

individual projects could be assembled to a decent house without interface problems. As stated in Postulate 1 of the Enterprise Engineering Manifesto[1]: 'In order to perform optimally and to implement changes successfully, enterprises must operate as a unified and integrated whole. Unity and integration can only be achieved through deliberate enterprise development (comprising design, engineering, and implementation) and governance'. Therefore, one of the main goals of enterprise engineering is to offer a global perspective facilitating better project and programme management by ensuring the mutual fit of individual projects. In their paper on the discipline of enterprise engineering [20], Dietz, Hoogervorst et al. report that there is a plethora of evidence that a major reason for failing to implement a strategy is the lack of coherence and consistency among the various components of an enterprise. One of the major goals of enterprise engineering is therefore to achieve organisational *concinnity*, i.e. the harmonious and skilful arrangement and fitting together of the enterprise's components.

Besides the need for coherence and consistency, also manageability is a major concern. Indeed, the fast evolving world puts high demands on our enterprises and organisations in terms of constant need for flexibility, adaptation and innovation and the desire to proactively exploit new developments in order to create new business opportunities. The more enterprises and organisations rely on IT support for their functioning, the more the same flexibility, adaptation and innovation are required from enterprise information systems. So, while in information systems engineering very often an analogy with bricks and mortar construction is made, one needs to be aware that organisations and their supporting information systems are required to evolve much faster than is usually required from bricks and mortar constructions, making manageability an important issue.

As enterprises or organisations are created intentionally to serve a certain purpose, it means that they are actively 'designed'. As will become clear through subsequent discussions in this chapter, the activity of design will comprise both the understanding of the enterprise (looking backward) and its 'shaping' or creation such as to achieve the desired outcome (looking forward). Furthermore, the empowerment, skills and learning capabilities of employees are essential for the organisation's productivity, product and service quality, learning and innovation [20]. Enterprise engineering therefore also takes a human-centred view on the organisations implying that design decisions should always be evaluated for their impact on the human aspects of the organisation.

Given the complexity of organisations and the goal of achieving consistency and coherence, enterprise engineering should be based on well-developed foundational theories. While the field of enterprise engineering is still under development, a number of theories or classes of theories start to emerge as foundations of this field. In [20] the different classes of theories and their relationships are discussed. The initial development of the approach in this book was started long before the discipline of enterprise engineering emerged. Yet as the development of the

[1] http://www.ciaonetwork.org/publications/EEManifesto.pdf

MERODE approach was driven by the desire of providing a sound and coherent approach to enterprise modelling through the use of algebra, its grounding in mathematics allows to position the approach in the context of enterprise engineering theories.

The theoretical foundation of enterprise engineering relies on *philosophical theories* (such as mathematics, logic, epistemology and phenomenology). While such theories are very broad and general, they form a solid ground on which more specific theories can be built. Mathematics and logic are, for example, instrumental for enabling reasoning about the truthfulness and completeness of statements. The MERODE approach relies on sound mathematical data and modelling theories (set theory) and on process algebra. These mathematical theories are extensively relied on to manage and ensure the quality of enterprise models. A second class of theories are the *ontological* theories, which are theories about the nature of things that are rooted in philosophical theories. While enterprise modelling may rely on the existence of or contribute to a certain extent to developing an enterprise ontology, the MERODE approach was not yet rooted in ontological theory. A third class of theories are the *ideological* theories, which are theories that address the aspects of goals of people and of enterprises in particular. Also these aspects have not yet been integrated in the MERODE approach. These three families of foundational theories provide the fundament for the development of a fourth family of *technological theories* that can be used to design and make systems. In particular, we need theories to bind development, design, engineering and implementation into a consistent *methodology*. In addition, care needs to be taken to minimise the impact of changes and avoid a combinatorial explosion of ripple effects, requiring 'normalising' system design. As will become clear in the next parts of this book, care has been taken to integrate the data and behavioural perspective into a sound approach for modelling. The resulting models serve as a 'design' for (aspects of) the enterprise but also to 'understand' and 'engineer' an organisation. As the models are directly used to develop enterprise information systems from, a natural binding arises between design, software development and implementation yielding a consistent methodology. Furthermore, Chap. 2 will explain how—to a certain extent—also the isolation of the impact of enterprise information system changes into separate layers is pursued, contributing in this way to the 'normalisation' of the enterprise information system. As a result, the approach in this book can itself be categorised into the family of technological theories.

1.2 Enterprise Architecture

While a true enterprise engineering approach is still in its infancy, throughout the past decades, several attempts have been made to devise practical guidelines for managing the development of enterprise information systems starting from a global plan and a global vision of the enterprise. This has mainly been done from design and governance perspectives, resulting in 'Enterprise Architecture

(EA) Frameworks'. There exist a large number of EA frameworks (also referred to as a jungle of frameworks[2]) and some have been around for many years. As an example, the initial Zachman framework was first published in 1987 in the IBM systems journal [101]. Although there has been increased research on Enterprise Architecture since the start of the century, most EA frameworks find their origin in the practitioners' community and lack a solid scientific foundation, which hampers both theoretical and practical progress. This has triggered the initiative to develop the discipline of enterprise engineering as a true scientific discipline (cfr. the Enterprise Engineering Manifesto and [20]).

Despite their lack of formal and theoretical grounding, existing EA frameworks still have some merit for their aim of offering a practical guide to the enterprise architects to develop and maintain the architecture of an organisation by offering tools to identify and describe components and their relationships, taking into account objectives, stakeholders and their viewpoints, and by offering guidelines to help managers to establish a smooth path from an as-is to the envisaged to-be situation and to gain insight in the factors that hamper strategy realisation. This architectural design perspective is an essential instrument to achieve alignment, integration and flexibility [30].

The purpose of this section is to frame the MERODE approach in the current context of Enterprise Architecture. EA frameworks are mainly characterised by their particular orientation. Some of these frameworks focus on providing a global plan for the enterprise (the design perspective), while others cater for the need for evolution and ensure that adaptability and flexibility are built into systems (the governance and complexity management perspectives). An example of the second type of framework is The Open Group Architecture Framework (TOGAF) [93], while the Zachman framework [90, 101] on the other hand is a catalogue (an ontology) defining a structured set of essential descriptions required to architect a system. ArchiMate finally is a framework that focuses on the core concepts and their relationships as used by modelling languages used for creating architectural descriptions [94]. To gain a better understanding in what Enterprise Architecture stands for, we very briefly outline the cornerstones of these three frameworks.

The TOGAF framework consists of different parts, of which the three main parts are the Architecture Development Method (ADM) which defines a step-by-step approach to developing an Enterprise Architecture (Fig. 1.1); the Architecture Capability Framework which discusses the organisation, processes, skills, roles and responsibilities required to establish and operate an architecture function within an enterprise; and the Architecture Content Framework which provides an overview of typical architecture deliverables and addresses their information structure and the reuse of architectural artefacts.

The Zachman framework on the other hand (Fig. 1.2) rather focuses on architectural descriptions and organises these along the question words what, how, who,

[2] http://en.wikipedia.org/wiki/Jaap_Schekkerman#How_to_Survive_in_the_Jungle_of_Enterprise_Architecture_Frameworks

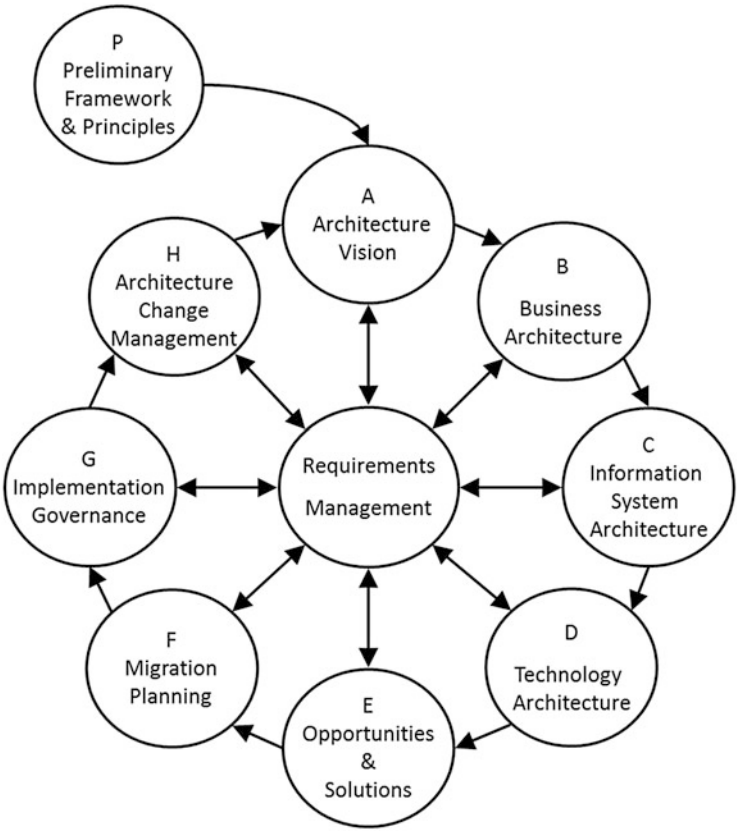


Fig. 1.1 The TOGAF Architecture Development Method. Adapted from [93, p. 48]

when, where and why and along different abstraction levels, more or less corresponding to perspectives held by different stakeholders and the different construction phases a project goes through: the owner and the architect dealing with scoping, domain understanding and information system requirement and the constructor dealing with the choice of technological platforms, design and implementation.

While the Zachman framework identifies and organises the descriptions that would constitute a complete architectural description, ArchiMate focuses on the languages and the modelling concepts to use to specify architectural deliverables. The ArchiMate Enterprise Architecture modelling language has been developed to provide a uniform representation for such architecture descriptions. It offers an integrated architectural approach that describes and visualises the different architecture domains and their underlying concepts, relations and dependencies. At the highest abstraction level, ArchiMate's architectural framework distinguishes three core concepts (passive structure, behaviour and active structure) and three layers (business, application, technology); see Fig. 1.3. The layers correspond to the

	What *Inventory*	How *Process Flows*	Where *Distribution Network*	Who *Responsibility Assignment*	When *Timing Cycles*	Why *Motivation Intentions*
1. Scope Contexts *(Scope Identification Lists)*						
2. Business Concepts *(Business Definition Models)*						
3. System Logic *(System Representation Models)*						
4. Technology *(Technology Specification Models)*						
5. Components *(Configuration Models)*						
6. Operations Instances *(Implementation)*						

Fig. 1.2 The Zachman Framework for information system architecture. Adapted from http://www.zachman.com/about-the-zachman-framework

	Passive Structure	Behaviour	Active Structure
1. Business			
2. Application			
3. Technology			

Fig. 1.3 ArchiMate's architectural framework. Adapted from ([72], p. 69)

identically named layers in TOGAF. An active structure element is defined as an entity that is capable of performing behaviour; a behaviour element is defined as a unit of activity performed by one or more active structure elements and passive structure elements are the objects on which behaviour is performed. All these elements are refined and related. ArchiMate 2.0 also provides for a motivational extension corresponding to the 'why' column of the Zachman framework.

For an encompassing introduction to the field of EA and a more complete overview and discussion of EA frameworks, the reader is referred to [72]. Following the definitions given in this book on p. 34, three perspectives on the role of Enterprise Architecture can be distinguished:

> '– A regulations oriented perspective—which manifests itself as a prescriptive notion governing the design of an enterprise. When taking this perspective, one will focus on principles, leading to rules, guidelines and standards, focusing on the enterprise's design freedom in the direction of its success.
>
> – A design-oriented perspective—which emphasizes the comprehensive and cohesive specification of an enterprise in all its facets, as a high level design. This perspective focuses on essential design decision, as well as its core structures. When taking this perspective, one typically produces models that describe the design of actual systemic artefacts and their interrelations.
>
> – A pattern-oriented perspective—which focuses on the use of design patterns. This perspective forms a bridge between the regulative and the design perspectives. To meet the regulations set out in the regulative perspective, during design activities, suitable patterns can be applied'.

This book will mainly emphasise the design perspective and focuses on creating comprehensive and cohesive descriptions of certain aspects of the enterprise in order to gain insight in the core structure of the enterprise and the required essential structures of its supporting information systems. From this perspective, two of the key concepts of Enterprise Architecture are models and views (next to concerns of stakeholders, principles and frameworks) [72]. To cater for the design perspective, an Enterprise Architecture is developed through a series of models that each describe certain aspects (views) of the enterprise in either the 'as-is' state or the targeted 'to-be' state.

The need for separate models and views arises from the immense complexity of organisations and the operationalisation of an organisation's strategy, as a result of which creating a single encompassing model is not feasible nor desirable. Similar to construction plans and as outlined in the next section, views are used to reduce a model's complexity in terms of specific concerns or a set of related concerns. In this way, models are purposeful descriptions that can be used to study the impact of decision from a specific perspective.

A generally accepted high-level set of views separates the description of the Business Architecture, the Information System Architecture and the Technology Architecture. As an example, according to The Open Group Architecture Framework (TOGAF), enterprise system architecture can be divided into the following major domains:

- Business Architecture
- Information System Architecture, consisting of:
  - Application Architecture
  - Data and Information Architecture
- Technology Architecture

The *Business Architecture* is concerned with constructing a business model to describe how the business operates. It focuses on business roles, business actors, business processes and any supporting structures that comprise the organisational business model. The *Application Architecture* deals with the application layer of an enterprise that includes software components, software applications or information systems. The *Data Architecture* deals with the data structure used by a business and its applications. Data Architecture also describes how data is processed, stored and used within the system. The primary focus of *Information Architecture* is information and knowledge required for business operations and decisions. The *Technology Architecture* aims to model the technology layer of the enterprise, such as system software, devices, networks and infrastructure services.

Given the complexity of enterprises, developing an enterprise's architecture in all facets requires a substantial investment of time and effort. Although often an analogy is made with the plans of a house, for most complex organisations an analogy with city planning and development would be more appropriate. Organisations very often follow a maturity growth path in developing their Enterprise Architecture, evolving from no architecture process at all to a managed and continuously optimised architecture process. The role of models in this architecture process will not only depend on the organisation's architectural maturity level but also on the culture and the specific architecture approach that fits best with this organisation. Some organisations will favour a planned development of architectural descriptions prior to starting a set of projects, while other organisations will favour an on-the-go approach developing the Enterprise Architecture piece by piece as projects are executed. Similarly, the emphasis put on the different types of architecture (business, information system or technology) will differ, depending on its maturity and culture and the goals it wishes to pursue with Enterprise Architecture development.

This book takes a design perspective and focuses on creating comprehensive and cohesive descriptions of certain aspects of the enterprise in order to gain insight in the core structure of the enterprise and in order to use these descriptions to create the required essential structures of its supporting information systems. It is therefore also important to have clear view on roles and the resulting division of responsibilities in creating architectural descriptions.[3] In building construction projects, there are three roles: the owner of the project who orders, the architect who makes the plans and the constructor company that builds according to the plans. In construction projects, the architect operates on the side of the owner of the project to create the plans that define the demand side of the project. The construction company acts as the supplier. The demand side and the supply side of a project are clearly separated. Notice, for instance, that in Belgium for building construction projects, it is forbidden by law that architect and constructor belong to the same company. These same roles exist for software projects. First, there is the owner's view: this defines the Business Architecture. It is an essential view in achieving

[3] This is not to be confused with roles and actors in *business* processes.

business-ICT alignment as it defines the environment and its playing rules for which information systems are developed. In the second place we find the information system architecture which is the specification of the *required* information and application support for the business functions. Both architectural views together define the 'demand' side of a project. On the supply side, there is the implementation: the technical solution that is to be realised by the 'constructor company' and that should provide the functionality as required in the specification. The solution is described by means of a Technology Architecture that pictures the baseline hardware and software platform types and by means of the implemented Application Architecture and Data and Information Architecture that describe the *realised* services and information offered to support business functions.

In software projects however, it is not always clear on which side architects operate. Very often the 'business' will ask a 'supplier' to build an enterprise information system for the organisation. In such cases, often the supplier will also act as architect, resulting in a confusion between demand and supply. Such confusion is hampering the development of generic, flexible and adaptable solutions. Moreover, it entails an inherent problem of conflict of interests on the side of the architect-supplier and hence puts the demand side in a weaker negotiation position.

From an Enterprise Architecture perspective, this book places the main emphasis on the demand side perspective. Consequently, the book focuses on the Business Architecture. When developing a Business Architecture, different architectural descriptions can be created: domain models, goal models, process models, etc. To avoid confusion, we state here the definitions of a number of terms as we will use them in the remainder of this book:

1. Enterprise Architecture: encompassing approach to the architecture of complex and large enterprise systems. The main goal of Enterprise Architecture is to achieve a better operational realisation of the organisation's strategic goals. The term *enterprise* includes the whole complex, socio-technical system that comprises people, information, technology and business (e.g. operations).
2. Business Architecture: one step ((B) in TOGAF's Architecture Development Method and the highest layer in ArchiMate's framework) dealing with the description of how a business operates in terms of business actors, goals, processes, business objects, etc.
3. Enterprise model: second row of the Zachman framework; it is the artefact that describes the Business Architecture.
4. Domain model: part of the enterprise model, namely, the part that describes business objects and their relationships. In case of an object-oriented domain model, this includes the description of the behaviour and interaction of business objects as well.
5. Business process model: part of the enterprise model describing activities and the executing actors in an organisation.

1.3 How to Develop Enterprise Models

The previous section explained the role of Enterprise Architecture and what position enterprise models take in EA, but we still are left with the question of how exactly to develop such models. For the sake of completeness, we first address the question of what modelling is about.

1.3.1 What Exactly Is Modelling?

In the most general sense, a model is anything used in any way to represent something else. Models can be physical in the sense of an object that represents another in more or less detail. Often, physical models are built to scale to represent larger object: a toy car that is a model of a 'real' car will faithfully represent the outer looks of that particular car. Conceptual or formal models are symbolic representations of a reality. For example, Fig. 1.4 is a (conceptual) process model that represents the sequence of steps to make bread with a bread-making machine. It uses rounded boxes, circles and arrows to, respectively, represent activities, events and sequences. Both physical and conceptual models are often used to help us understand and study the reality they represent: a new dress can be first realised in bale cotton to study and optimise its design and Markov chains may be used to study the behaviour of queues. At the same time, an essential characteristic of models is that they make abstraction of certain dimensions of the reality. The toy car will faithfully represent the bodywork of a car, but not its engine. The bale cotton dress represents the shape of the final dress, but not its fabric, colour and finishing. The process model makes abstraction of the concrete person making the bread and of the place where the bread making takes places. The Markov Chain model makes abstraction of the concrete items in the queue.

In many scientific disciplines formal models are used intensively to study the behaviour of objects, e.g. by investigating their properties by means of mathematical reasoning. Markov chains can, for example, be used to calculate throughput, turnaround times and waiting times. Likewise, process model such as in Fig. 1.4 is used to optimise business processes. In information systems development, conceptual models play a prevalent role as plan from which the final system is to be made. Sometimes also physical models, called prototypes, are built to study the future information system. We can most easily compare this to the use of plans and cardboard models by architects in bricks and mortar construction. As shown in

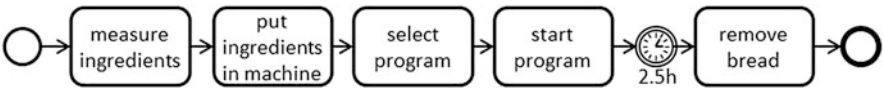


Fig. 1.4 Model representing the process of baking bread with a bread-making machine

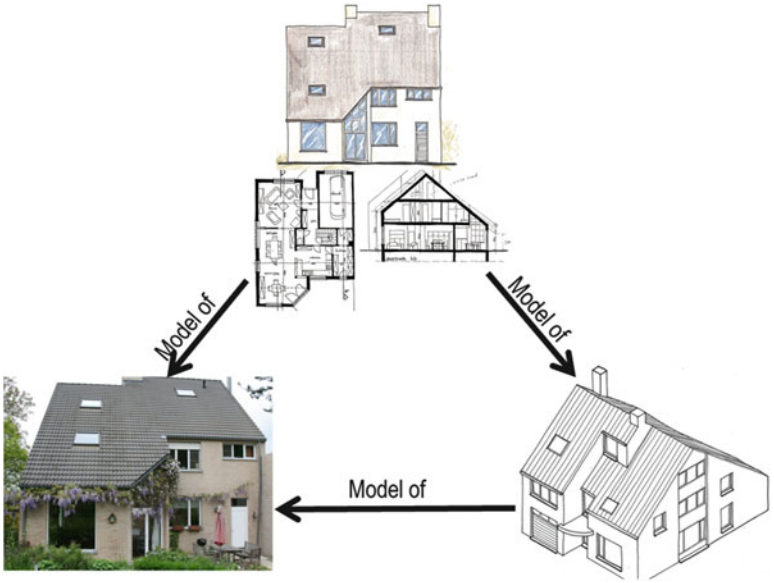


Fig. 1.5 Conceptual models are used to create both physical models and the real object. *Plans published with kind permission of © M. Van de Wouwer 2000. All rights reserved. Photo published with kind permission of © M. Snoeck 2014. All rights reserved*

Fig. 1.5, conceptual plans can be used both to create a physical prototype and to construct the final system.

Although models and plans make abstractions of certain aspects of the real-world object, they can themselves become very complex when more aspects of a (complex) system need to be described into the plans.

In order to manage complexity, it is therefore very common to make use of different plans for an object or a system, each of them describing certain aspects depending on the characteristics that need to be studied or understood based on that particular plan. At my physiotherapist, several pictures (i.e. plans or models) of the human body hang on the wall: an overall picture of the skeleton, an overall picture of the muscles, detailed pictures of neck and shoulders, etc. Likewise, in (bricks and mortar) construction different plans of a house are made. These plans will on the one hand depict different aspects of the house but on the other hand also target different audiences.

As an example, Fig. 1.6a, b shows different plans of the house used to discuss the future house with the owner or project sponsor while Fig. 1.6c shows an excerpt from the plans for the same house but used to convey the building instructions to the masons.

This illustrates how models are purposeful descriptions that may differ depending on the purpose they are used for:



Fig. 1.6 Different plans for a family house. *Published with kind permission of © M. Van de Wouwer 2000. All rights reserved*

- Plans of a house are used for price estimation. Rough plans are sufficient for rough estimations; detailed plans will be required for more precise estimations.
- Figure 1.6a, b are plans used to communicate with the (future) owner of the family house construction project. In order to discuss the essence of the house, the owner's view will focus on aspects that concern a house as 'a place to live'. How many people are there in the family? What do they use the rooms in the house for: to relax, to work, to cook, to play, to receive guests, etc.? How would you typically circulate in the house? Which rooms do you want close to each other and where do you need doors? Figure 1.6c is an excerpt from a plan used to communicate with the construction company. It has therefore more technical details and uses a symbolic language adapted to that purpose.
- High-level plans are used to guide further development. The floor plan of Fig. 1.6a will be used as global plan to fit in smaller design projects that target specific rooms of the house. The detailed plan of the kitchen will start from the dimension of the kitchen as set out in the global floor plan and will have to take the positions of the doors to the corridor and the dining room into account.

Conceptual models are often also represented in a graphical, symbolic way. This necessitates the use of a *modelling language*, a collection of symbols that represent concepts. Such languages also embody a set of rules, a grammar, that define how symbols are to be combined into meaningful models. Such modelling languages are adapted to the context they serve. As an example, the symbols used in the owner's view are much simpler and more intuitive than those used in Fig. 1.6c because in the owner's view abstraction can be made of technical details that need to be present and represented in the technical plan.

Information systems development makes extensive use of models to guide the development of software. As a result, a plenitude of modelling languages is available for documenting all kinds of views and aspects of software. The 'Unified Modelling Language' (UML) is an example of a general-purpose language that is extensively used in this field. Other languages target specific aspects, for example, 'Entity Relationship Modelling', which specifically targets data modelling, or the 'Business Process Modelling Notation' (BPMN) which specifically targets the description of business processes.

1.3.2 Which Aspects to Include into Models?

Despite the large number of Enterprise Architecture frameworks, little concrete advice is to be found on how exactly to create enterprise models. Nevertheless, although different EA frameworks tend to propose different sets of viewpoints, there seems to be a consensus that the following viewpoints should be included in a Business Architecture description [5]:

- The 'why' describing the goals and the motivations of why things are done a certain way
- The 'what' describing the main concepts or business objects of the organisation
- The 'how' describing the business processes that are performed at the operational level
- The 'who' describing the actors and resources used to perform tasks in the business process models

In this book, we concentrate on the 'what' and the 'how'. For the 'why' aspect, we refer to goal modelling techniques such as, e.g. advocated by I* [100] or KAOS [96]. Goal trees allow to decompose high-level goals down to more operational goals, until the level is reached where actual business processes can be described. The 'who' aspect is partly dealt with by means of business process models: these models focus on the 'how' dimension but also allow to describe actors and resources used to perform tasks ('who'). Techniques that specifically focus on the 'who', e.g. organisation charts, are out of scope of this book.

The 'what' aspect will be described by means of a domain model and the 'how' aspect will be captured both by the domain model and the business process model.

Domain models focus on defining business objects and their relationships. Very often, it is assumed that this perspective equals a purely data perspective on the enterprise (the 'what' aspect). However, in the approach proposed in this book, we want to take advantage of benefits offered by an object-oriented approach. According to the object-oriented approach, each object should embody its behaviour next to its data in a single module, thereby offering a single place of setting rules for business object behaviour. As an example, a bank account should not only hold its balance but also the rules that govern the withdrawal and deposit of money. In this way, such rules can be defined in a single place, rather than being scattered across all possible information system services that allow to register a withdrawal or deposit of money. Splitting the data aspects from business objects from their behaviour by putting behavioural aspects solely in the business process model will make it impossible to take advantage of the object-oriented approach. Therefore, this book adheres to an object-oriented approach, and so, next to capturing data aspect (=the 'what' aspect), the domain model will additionally capture the behaviour of individual objects by defining an object life for each business object type. In this way, the domain model hence groups the 'how' aspects per business object type. Business processes on the other hand also capture the 'how' of an enterprise (in combination with the 'who') but do this from the perspective of work

organisation. Of course, both models are strongly related: through its tasks, one business process may affect multiple business objects. Reversely, one business object may be affected by multiple business processes.

If we consider the layers of the Zachman framework, then the domain model and the business process model together match the business concepts layer (=layer 2), whereas information system services and automated support for business process enactment are positioned in the system logic layer (=layer 3). In this way, the Zachman framework clearly separates the owner's view from the information system perspective. In the Zachman framework, the conceptual layer is called the 'business concepts layer' or 'enterprise model' [72]. So, in terms of Zachman's terminology, building an enterprise model is the resulting artefact of developing a 'Business Architecture' as defined in the TOGAF framework and in ArchiMate. Such enterprise model consists (among others) of a domain model (covering the what column and aspects of the how column) and of a business process model (how column). Figure 1.7 shows the relationships between the different terms.

Compared to ArchiMate's core concept, the business objects will correspond to passive structures on which behaviour is performed. The domain model describes the business objects and their mutual relationships (see Chap. 4). Business events (see Chap. 5) correspond to behavioural concepts. Process models will combine structural concepts such as actors and roles with behaviour concepts such as processes. The behaviour of business objects (see Chap. 6) and the business process model (see Chap. 10) correspond to the behaviour and active structures identified by ArchiMate.

To a large extent, the domain model developed according to the MERODE approach conforms to Postulate 4 of the Enterprise Engineering Manifesto[4]:

> *In order to manage the complexity of a system (and to reduce and manage its entropy), one must start the constructional design of the system with its ontological model. This is a fully implementation independent model of the construction and the operation of the system.*

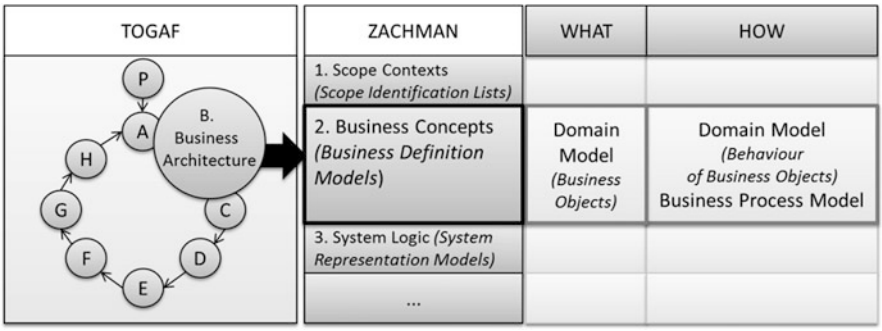


Fig. 1.7 Terminology

[4] http://www.ciaonetwork.org/publications/EEManifesto.pdf

Moreover, an ontological model has a modular structure and its elements are (ontologically) atomic. For enterprises the meta-model of such models is called enterprise ontology. For information systems the meta-model is called information system ontology.

Note. At any moment in the life time of a system, there is only one ontological model, capturing its actual construction, though abstracted from its implementation. The ontological model of a system is comprehensive and concise, and extremely stable.

An important remark needs to be made about the definition of the word 'enterprise'. TOGAF defines 'enterprise' as 'The highest level (typically) of description of an organization and typically covers all missions and functions. An enterprise will often span multiple organizations' (see Sect. 3.34 in [93]). On the other hand, for large and complex enterprises, it may be more practical to limit the scope of a single architecture to one or several business units or a specific (intra- or even inter-organisational) functional domain. These independently developed architectures need then to be integrated to form a federated architecture. As the notion of 'enterprise' as handled in a single architecture development project may range from multiple organisations to a single business unit, often the term 'universe of discourse' is used to refer to the part of the enterprise that is in scope of the project. In the remainder of this book, the scope of the 'enterprise' will be considered as the scope of a single architecture project, which may or may not cover all business units of an enterprise as defined above. In case the architecture is part of a federated architecture, it is however assumed that integration of the domain level is a crucial issue that needs to be dealt with as well to ensure the interoperability of the individual domain models. A domain model should indeed preferably be developed in an enterprise-wide manner (rather than on a per project base) and strive for an enterprise-wide agreement on the definitions of business objects, their mutual relationships and the rules that govern their behaviour. It is intended as a stable, concise and fully implementation-independent ontological definition of business objects.

1.3.3 From Which Perspective Should Models Be Made?

An important guiding principle in developing domain models is that given their role in understanding and studying the functioning of an enterprise, an important issue is to keep the focus on the enterprise itself, i.e. the owner's view, and not let oneself be guided by pure 'data modelling' or information system considerations. In construction projects, the owners view concentrates on the essential characteristics of the family house and makes abstraction of the materials used to construct the house (which does however not mean that this aspect is not discussed at some point in time with the owner). It also makes abstraction of the finishing: painting the walls or covering them with wallpaper, the choice of furniture, etc. are not represented in this plan, and these aspects will usually not be at the core of the discussion.

When designing a house, we could of course let ourselves be guided by our favourite materials and interior designs of desired furniture. However, design

fashion changes over time. So to achieve a timeless way of building, a good architect will rather focus on the essence of a place to live, to work and to relax.

Focusing on the essence does not compromise adaptability and flexibility: repainting or refurnishing the house can be done without affecting the structure of the house. On the other hand, the wrong shaping of rooms and the wrong placing of doors, walls and windows are costly in terms of later modifications. Adding or moving a door, a wall or a window is a much more radical change that most likely will affect multiple rooms and require the repainting of the concerned rooms.

Likewise, when developing the domain model, one will take care to concentrate on the essence of the business. For example, the essence of banking resides in accounts holding an amount of money and the fact that money can be deposited on and withdrawn from the accounts. The fact that the way in which we can withdraw money has evolved as technology has evolved is to be compared to the fact that houses can be redecorated according to the latest trends.

As domain models represent (aspects of) the way a business or organisation operates, this conceptual model will also serve as the plan to create enterprise information systems from. Indeed, an important and specific aspect of enterprise information systems is that they can themselves be considered as models of our organisations as they attempt to mimic our organisation. As an example, a library system is expected to mirror the real-world library:

- The library system catalogue should exactly represent the books that are available in the library.
- The shelf numbers that I find in the library system should tell me on which physical shelf the book is to be found.
- If a book is borrowed or returned to the library, the availability information for this book should be updated accordingly.

Lots of effort is taken to ensure the faithfulness of the representation in the real world. Books are, for example, equipped with electronic tags, and entrance and exit doors are equipped with detectors so that no one can enter or leave the library with a book without this being registered into the system. Our enterprise information systems are also used to study the real-world enterprise: evolution of numbers of visits and visitors are monitored to take decisions on opening hours, expansion of the collection, or conversion of the library to a study centre with meeting rooms and quiet study places.

Moreover, as technology evolves, the real world is increasingly virtualised: whereas a bank account still was a tangible booklet in the second half of the twentieth century, by now our bank accounts only exist in the bank's computer systems. As a result of this, the way we can consult our bank statements has changed over the years. However, the concept of bank account and the fact that we can deposit and withdraw money has not changed.

So, although we advise to focus on the enterprise itself and not let oneself be distracted by information system aspects, one cannot escape the fact that the enterprise 'is' the enterprise information system and vice versa, at least to a certain extent. So, when making enterprise models, one is actually making the model of the

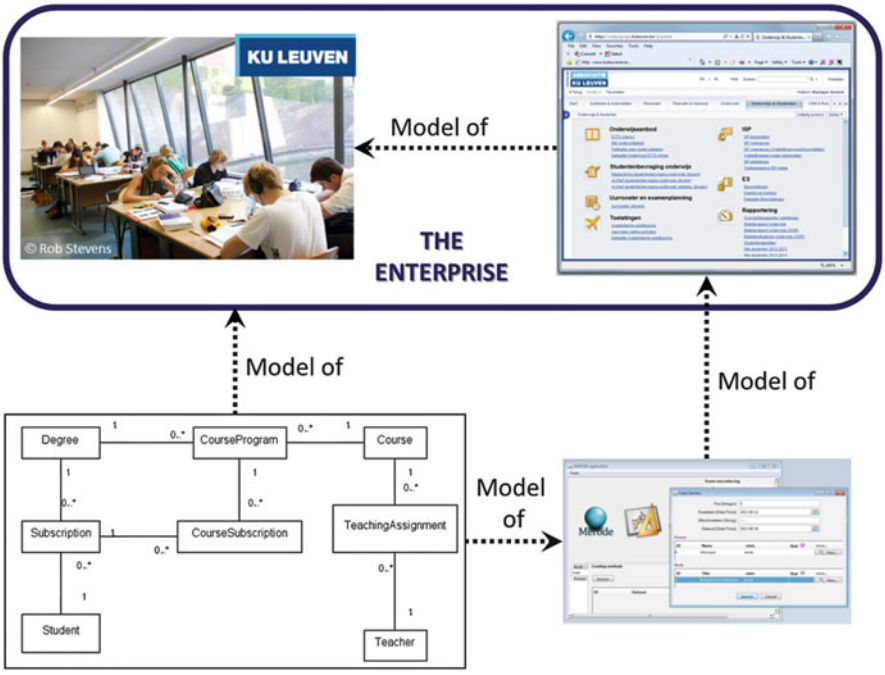


Fig. 1.8 'Model of' relationship between enterprise, enterprise information system, domain model and its prototype. *Photo published with kind permission of © R. Stevens 2014. All rights reserved*

enterprise information system, and reversely: when one is making plans for an enterprise information system, one is actually making a model of the enterprise.

As a result of this, domain models become more than building plans: the conceptual model of an enterprise is more than ever the way to understand the functioning of an organisation and of the information systems used to manage that organisation. The more enterprises are virtualised and embodied in their enterprise information systems, the more the domain model is essential in building the enterprise information systems right. Figure 1.8 shows how an enterprise, its enterprise information systems, the domain model and prototype systems are 'models of' each other. In the example of the KU Leuven, one can state that the KU Leuven partly is embodied in its enterprise application 'KULoket': a student is only subscribed to a course if this is registered in KULoket. No matter how often a student attends classes in the real world, the subscription for a course does not exist unless it is registered in KULoket. The same holds for exams: if a student did not registered for an exam in KULoket, the 'real world' filled in examination copy is considered as non-existing when the information system registration of the exam does not exist. In this respect, KULoket 'models' part of KU Leuven but also partly 'is' KU Leuven as it even supersedes the 'real world' for certain aspects. The domain model describes ('models') the business objects of KU Leuven but also

serves as a plan to create the enterprise information system from. If a prototype is generated from that domain model, then the prototype application is also a model of the enterprise information system (and therefore indirectly from part of KU Leuven as well).

1.3.4 Where to Start, How to Go On and When to Stop?

Given that we know what particular aspect we wish to model and what perspective to take, we still face the question: where to start, how large or detailed should it be and when is the model 'good' or 'good enough'? For that, we need a *modelling method*. A modelling method will give advice on which elements should be captured in a model, and which not. Moreover, a (good) method embodies a particular philosophy with respect to what the key elements are to ensure high quality of the models and will therefore (usually) be quite prescriptive. A true method should give advice on how to evaluate the quality of a model and not just say 'check quality'. For example, it should give directions on how to create a domain model (e.g. answer the question 'what are business objects and how do I find them?') and give advice on when a model is good or bad.

One could expect modelling languages to provide some guidance in creating models. But just as dictionaries and grammar books don't tell us how to write a novel or an academic textbook, modelling languages do nothing more than explaining the meaning of symbols and providing rules to create syntactically correct models. Likewise, the 'universal modelling language' (UML) offers numerous diagramming techniques with descriptions of the rules to compose a diagram or model, but UML gives no guidance at all on how to create and validate models. This gap on 'how to create and validate models' is exactly what will be addressed in the remaining chapters of this book.

1.4 Modelling and Then What?

From a design perspective, the domain model and the business process model are not only used to understand or shape the functioning of an organisation, but they will also be used as plans to build systems from. As such they take on the role of a software requirements statement. As organisations become more highly dependent on their supporting enterprise information systems, we further elaborate the use of models from a software development perspective.

1.4.1 Using Models for Software Development

Generally speaking, the software engineering process includes a series of stages. The classical model of a software development process comprises the following phases:

- Analysis
- Design
- Implementation
- Testing
- Maintenance

The analysis phase may start with a feasibility study during which general options are investigated. This will be followed by a detailed and careful analysis of the business domain and the users' requirements. While elaborating the requirements, qualities such as clarity, consistency (in terms of internal contradictions), completeness and feasibility of the requirements should be carefully managed. The design phase will deliver the architecture document(s) of the internal design of a software system. In addition to system architecture standards, other important factors such as technology, time and cost will be also taken into consideration while designing a software system. During the implementation phase the software itself will actually be programmed by a software developer team. Once programmed, the software system also needs to pass through a careful testing and validation process to ensure that defects are revealed and addressed as soon as possible. Testing may reveal problems in all previous phases of the development process: bugs in the code, problems in the design but also missing or wrongly understood requirements. The result of the testing phase may therefore lead to reconsidering and redoing some (or a lot) of the previous work. Finally, when all tests yield a satisfying result, the software is released for use. The software engineering process does however not end with the release of a software system. Maintaining and enhancing the software is of no less importance than the prior phases of software engineering process. It may be necessary to correct a defect, to adapt the software to a new infrastructure or to add new functionalities in the software. Coping with newly discovered problems or new requirements can take far more time than the initial development of the software; very often it will even exceed by far the actual cost of the software creation. Figure 1.9 depicts the relationships between the different phases in a software development cycle.

Models will typically be used in the analysis and design stages. Analysis models will have a higher level of abstraction, whereas design models will have more (technical) details. As such, domain models and business process models play the role of analysis model. The level to which modelling is effectively used may however be very different from project to project. In general, the field of software engineering addresses all types of software that can be developed: real-time software, embedded software, gaming software, enterprise information systems, etc. These different types of software have very different characteristics, and as a result

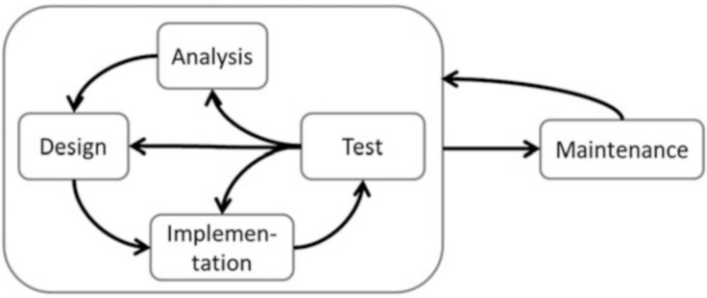


Fig. 1.9 Phases in a software development life cycle

the way one needs to go through the different phases of software development can differ depending on the specific needs of the type of software to be developed and the context in which and for which it is developed. For example, in one case a plan-driven approach of development may be required, leading to a strict sequential path through the stages of analysis, design, etc. In such case a development project will start with an analysis phase covering the entire scope of the project. This is referred to as the waterfall model. In other cases an iterative approach could be preferred, developing a system through a series of time boxes during each of which all phases are gone through for *a part* of the system, allowing in this way the incremental development of a system. Yet in other cases an even more agile model of development would be preferable with very quick iterations over analysis, (re)development and testing of parts of the system. As a result, different *software development life cycle models* (SDLM) have been proposed, for example, the Rational Unified Process (RUP) [37], the Agile Unified Process [1, 3], Scrum [91], etc. Such models are used as instruments to shape the planning and management of a software development project. They are, for example, documented as libraries of process descriptions, and a project manager will need to 'compose' the project's life cycle depending on the characteristics of the project, the available budget, people, etc.

As a result of this, many people consider software development life cycle models (such as Agile, RUP, etc.) as 'a method' (see, e.g. http://en.wikipedia.org/wiki/Software_development_methodology). However, most of the time, little advice is present on the 'how to' at the more detailed level. In this book, we call such software development life cycle models a *software development* method and we consider a *modelling* method as a framework that gives advice on 'how to' both on a global level and (and most importantly) at the level of specific tasks in the software development process. A software development life cycle model will, for example, embody a requirements phase and advice to 'manage requirements' and to 'model visually'. A modelling method will then give advice on which elements in a requirement should be captured in a model and which not and how to check quality.

Due to these different practices in terms of software development processes, there is no uniform practice in terms of the way models are used for software development. Requirements and specification documents may range from purely textual to highly formal, mathematical documents, with visual models somewhere

in between the range of textual and formal. What we do know, however, is that ambiguous and incomplete requirements are a source of errors in many projects. As explained by Pohl [76], requirements should be developed along three dimensions: the specification dimension (they should be detailed and complete), the representation dimension (they should be represented in an adequate format, preferably a semiformal one) and the agreement dimension (all stakeholders should agree on what the specifications are).

In practice though, requirements are still often partly or even largely documented textually, complemented with a number of visual models. The inherent ambiguity of natural language makes that each stakeholder sees his/her own truth in the requirements documents. It is then the role of the programmers to 'translate' these requirements and models into code and to resolve remaining ambiguities and incompleteness by gradually elaborating the specifications with more details until a working and satisfying piece of software is obtained. Very often, this results in programmers re-doing the analysis work. It is exactly this difficulty or even inability of developing 'good' requirements in a reasonable amount of time that can be conveyed to and understood by the software construction team that has led to highly iterative software development methods, some of which even completely skip detailed analysis and documentation of requirements.

Nevertheless, models can play a role in improving the software development life cycle, on the condition that they are of sufficient quality and can be created in a reasonable amount of time.

And here is another contribution of the method proposed in this book: MERODE is a modelling method that enables to create high-quality domain models. The method is grounded in process algebra, which enables mathematical reasoning on models. Thanks to this, models can be checked for internal consistency and mutual completeness. The automated reasoning also caters for autocomplete functionality, which allows creating correct models faster. The formal grounding of the method is kept hidden for practitioners. To ensure the utility of the method for practitioners, the modelling method uses class diagrams and finite-state machines from the UML and process diagrams from the BPMN as main modelling techniques. The method can be combined with a variety of software development life cycle models.

1.4.2 Following a Model-Driven Engineering Approach

Already in the early 1980s, M. Jackson pioneered in his 'systems development method' the idea that the activity of requirements analysis should in principle be completely independent of the target implementation environment [38]. It is during the design process that the specifications should be *transformed* while taking into account the specific technological requirements. As a result, the transition to design, rather than being a (presumed) easy and seamless elaboration of specifications into code, should rather be a 'transformation' that is specific for each particular implementation environment. The same specification can be transformed to several

distinct implementation environments. This insulates system specifications from technological development. When technology evolves, system specifications need no change. Part of them or the whole of them needs only to be transformed to the new technological environment. Next to a clear distinction between specification and implementation, this contributes significantly to the long life of the system.

This idea of a transformational approach is exactly the idea behind *model-driven engineering (MDE)*. MDE is a model-centric and transformational approach to software engineering. In particular, rather than focusing on algorithms and the elaboration of code, it focuses on the creation of models as representations of the software to be built. These models are then used to be transformed into other models or into code. The transformations exploit architectural knowledge available in the form of recurring design patterns [24, 28, 29] (conforming to the pattern-oriented perspective as the third role of Enterprise Architecture, cfr. p. 8). Key aspects of this transformational approach are that it speeds up the creation of software, that it enhances software quality and that it facilitates the portability and interoperability of software.

The first artefact that an MDE approach will require to create is a model that is a good representation of the domain knowledge and that can be used as a basis to create applications for this domain. Once defined, such a conceptual domain model can be used to refine the information system requirements with business management and/or end-users. The completed model then can be processed further to develop an application (software artefact). The *software artefact* can be created either by a development team or—in a true MDE approach—by an automated engine that will accept the model as an input and translate it into other (intermediate) models and then finally into computer code. The process of model-to-code mapping may thus pass through a number of transformations from one model to another before a software artefact can be generated. In software development this process is referred to as *model-to-model* and *model-to-code mappings*, and the engines producing models or code from a model are called, respectively, *transformation engines* or *code generators*.

Compared with the classical software development life cycle, MDE induces two important changes in the way software is developed. First, in classical development, requirements gathering and analysis are largely informal steps that are *manually* transformed to design and then to code. In particular, the use of the same paradigm of object orientation from analysis to implementation has enforced the belief that object-oriented development is a 'seamless' elaboration process. In a classical development approach, it is therefore not considered as a severe problem when requirements and analysis artefacts are incomplete or ambiguous: they will be interpreted and refined by humans during design and coding anyway. In model-driven engineering, however, models are transformed to other models by means of model-to-model mappings and to code by means of model-to-code mappings. An important issue is that in principle there is no human interpretation of requirements anymore. MDE assumes completeness and sufficiency of detail to enable the *automated* transformations of models. The result of the requirements analysis phase should therefore be formal and precise enough to be manipulated by a transformation engine.

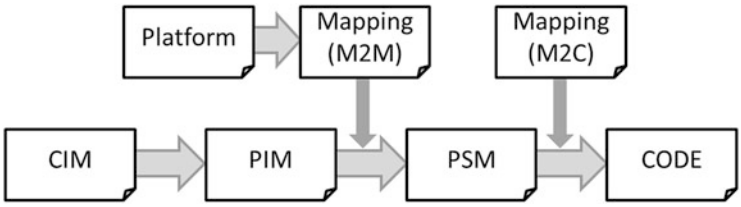


Fig. 1.10 The OMG's Model-Driven Architecture

A second important difference pertains to the way systems are corrected or enhanced. In a classical approach, when testing code, bugs will be corrected directly in the code or in the design. But even when during testing missing functionalities are discovered, this seldom leads to the adaptation of requirements and analysis documentation: also these corrections are usually done directly in the code and the design of the software. In an MDE approach, however, deficiencies discovered during testing should be corrected in the models (or in the transformations), which can then be used to regenerate the code.

The *Model-Driven Architecture (MDA)* is an architectural framework for MDE developed by the OMG. It offers a combination of concepts, tools and techniques to put MDE at work in practice. In MDA the development of a system will start with the creation of a 'computation independent model' and a 'platform independent model'. These are then transformed to 'platform specific models' and to code (see Fig. 1.10).

According to the MDA Guide V1.0.1 (2003),[5] the definitions of a CIM and PIM are as follows:

> *(CIM) A computation independent model is a view of a system from the computation independent viewpoint. A CIM does not show details of the structure of systems. A CIM is sometimes called a domain model and a vocabulary that is familiar to the practitioners of the domain in question is used in its specification. It is assumed that the primary user of the CIM, the domain practitioner, is not knowledgeable about the models or artifacts used to realize the functionality for which the requirements are articulated in the CIM. The CIM plays an important role in bridging the gap between those that are experts about the domain and its requirements on the one hand, and those that are experts of the design and construction of the artifacts that together satisfy the domain requirements, on the other.*
>
> *(PIM) A platform independent model is a view of a system from the platform independent viewpoint. A PIM exhibits a specified degree of platform independence so as to be suitable for use with a number of different platforms of similar type.*

In the MDA the UML is used as the cornerstone language to create models.

As the development of MERODE has been highly influenced by the ideas of M. Jackson, the method has incorporated the principles of MDE right from the beginning of its development in the nineties and was in this respect way ahead of its time. The method has been shaped by this objective of automated transformation to code. In particular, it has strengthened the desire for unambiguous diagram

[5] Downloaded from http://www.omg.org/mda/specs.htm

semantics and automated consistency checking to ensure the fitness of model for automated transformation. If we position MERODE within OMG's MDA standards, we can state it as follows:

- MERODE focuses on the development of a CIM as it targets the creation of a domain model as prime artefact.
- MERODE delivers a complete PIM for (a part of) a software system: the models delivered by MERODE are platform independent and sufficiently complete to generate other models and/or code from it. MERODE in fact provides for a practical approach to model-driven engineering with the major focus on modelling method and models as primary artefacts of the engineering process.
- The necessity for complete and unambiguous diagram semantics in order to ensure the ability to transform models automatically to code motivates the use of a limited set of constructs from the UML. MERODE therefore uses only part of the UML notation for the diagramming techniques (class diagram and finite-state machine) and further refines the concepts of the class diagram with the concept of existence dependency (see further and in Chap. 4).
- The necessity of platform independence (in particular the independence of object-oriented implementation) has led to a proprietary approach for the modelling of object interaction. MERODE complements the UML class diagram and finite-state charts with a kind of CRUD-matrix,[6] a technique borrowed from Information Engineering [53].
- Because of the latter, MERODE is supported by a proprietary tool, but with possible exchanges with UML compliant tools by means of XML. The tool allows to generate working prototype applications from models, enabling in this way the fast prototyping of requirements.
- MERODE proposes a set of model quality checking rules to define precise, complete and transformable models and to assess their quality.

In this book we will exemplify a number of transformations that show how a MERODE model can be transformed to code. As explained before, the process of model-to-code mapping may pass through a number of transformations from one model to another before a software artefact can be generated (*model-to-model* transformations). In this book we will however skip addressing possible intermediate model-to-model transformations in the code generation process, since MDE is not the core subject of the book. We'll restrict the MDE aspects to demonstrating that MERODE allows creating models that are sufficiently complete and have sufficient detail to be fit as transformation input. The MERODE tool also provides an architect with the code generation tool to automatically generate a ready-to-use software prototype (i.e. a 'cardboard application') from a model he/she designed. This, in turn, will allow him/her to test and refine the system model based on detected defects in the conceptual business model given its fully working prototype,

[6] CRUD-matrix is a table containing functions of an application and with each function entry indicating the type of interaction with a database for that function: create, read, update or delete.

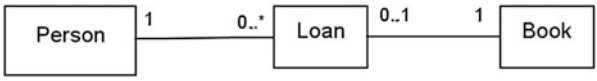


Fig. 1.11 Data model for a very simple library

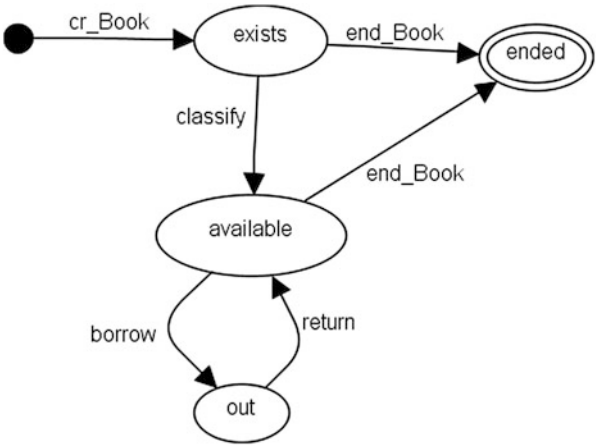


Fig. 1.12 Behaviour model for a book

actually enabling checking the external validity of models through simulation of models. Moreover, the models are embedded in the prototype application (simulated model) and explicitly referred to during application execution. This allows tracing the behaviour of the application back to the model which contributes to a better understanding of the implications of modelling choices.

Let's look at a simple library example. Suppose the domain model contains three business objects: person, book and loan. The objects are related such that each book can be on loan at most once (0..1) at one point in time, while each person can have zero to many (0..*) ongoing loans at any point in time (Fig. 1.11). A loan always refers to a specific (1) person and a specific (1) book. The behaviour of a book is defined in such a way that a book needs to be classified before it can be borrowed (Fig. 1.12).

Based on these models, a prototype application is generated that allows to register people, books and loans. Figure 1.13 shows the user interface of this application when trying to register borrowing a book. Hitting the borrow button in the borrow-window will trigger an error (see Fig. 1.14 on the left). Indeed, the model defines that a loan always refers to a specific (1) person and a specific (1) book. So, if one then tries to register a loan without indicating which person borrows the book, the application will refuse to register the 'borrow'. The user is offered the possibility to view the model part that causes the refusal (see Fig. 1.14 on the right); in this way the behaviour of the application can easily be traced back to the model.

Likewise if one attempts to borrow the book before it has been classified, this action will be refused and the user will receive feedback on the part of the model that causes the refusal (see Fig. 1.15).

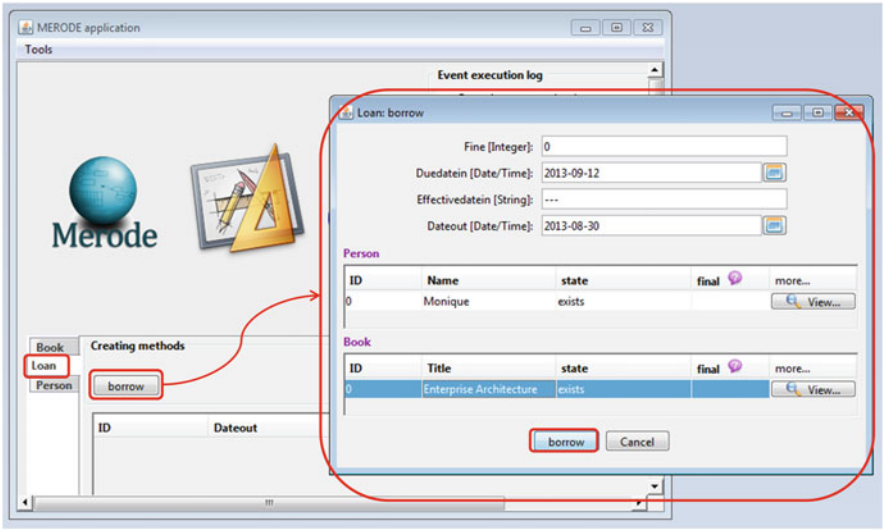


Fig. 1.13 Borrowing a book in the prototype application of the library

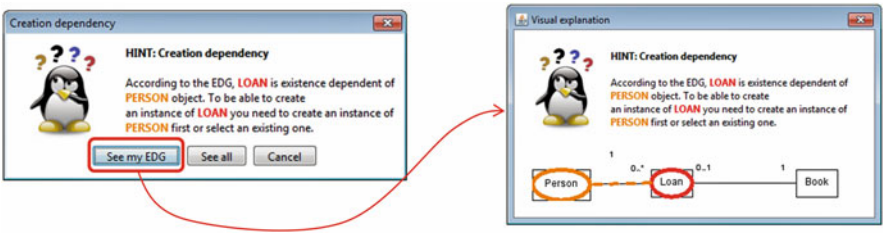


Fig. 1.14 Model-driven feedback on the data model

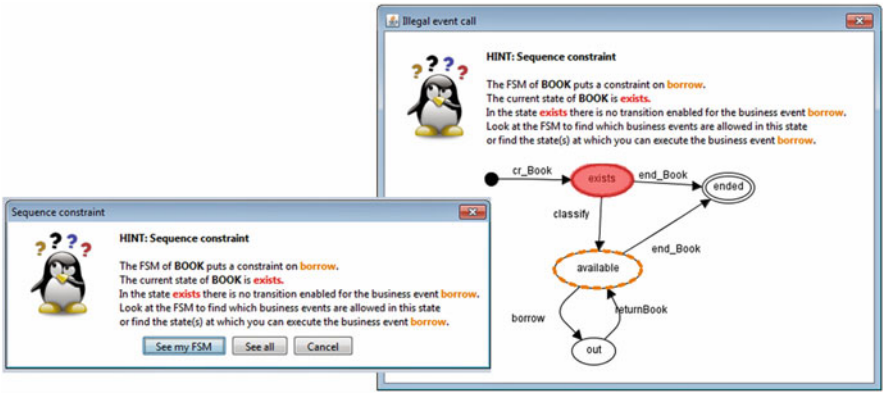


Fig. 1.15 Model-driven feedback on business object behaviour

1.5 Summary

To summarise, MERODE's position in terms of enterprise engineering and Enterprise Architecture can be stated as follows.

MERODE is a methodology that unifies modelling, design and implementation into a consistent and practical approach (methodology) for enterprise (domain) modelling and the development of maintainable and flexible enterprise information systems. The mathematical foundation is not limited to formalisation but also provides effective support for design quality (see Chap. 2).

MERODE aims at a clear separation between demand and supply and offers a practical approach for the demand side of software development or delivery. In this respect, using TOGAF's terminology, it primarily focuses on the Business Architecture and it defines a high-level Information System Architecture (see Chap. 2) in order to identify the required information system support for an organisation's business processes. In terms of modelling artefacts, MERODE addresses (in part) the Scope Identification Layer, the Business Concept Definition and the System Representation Model of the ZACHMAN framework. In particular, it proposes a method for domain modelling and for the identification of required information system services and it explains how to cross-link domain model, information system services and business processes ensuring the overall consistency of the architecture. As such, MERODE addresses both 'how to' and 'what' aspects of Enterprise Architecture in that it offers a set of guidelines on how to build models but also clearly defines the deliverables of each step in the process. In this respect, MERODE is compliant with Enterprise Architecture frameworks such as TOGAF and ZACHMAN, but more importantly, it complements these architecture frameworks by offering very practical guidelines on how to obtain a correctly layered architecture and on how to build a domain model and what quality criteria a good domain model should satisfy.

MERODE is a modelling method that enables to create high-quality domain models. The method is grounded in process algebra, which enables mathematical reasoning on models. Thanks to this, models can be checked for internal consistency and mutual completeness. The automated reasoning also caters for autocomplete functionality, which allows creating correct models faster. The formal grounding of the method is kept hidden for practitioners. To ensure the utility of the method for practitioners, the modelling method uses class diagrams and finite-state machines from the UML and process diagrams from the BPMN as main modelling techniques. The method can be combined with a variety of software development life cycle models.

MERODE fosters a model-driven engineering approach to software development. It targets the creation of a domain model that is platform independent and sufficiently complete to enable its transformation to platform-specific models and to code. In order to achieve its goals of automated transformation of models, MERODE limits the use of UML to a number of well-defined constructs with

clear semantics and complements this with the notion of existence dependency and a proprietary approach to object interaction modelling.

MERODE is supported by a tool that allows to create models and check the models for consistency and readiness for transformation. A companion code generator allows to generate a fully working prototype. By embedding the models into the application, the behaviour of the prototype can easily be traced back to the models.

The MERODE approach does not claim to be a 'complete' enterprise engineering approach. Rather, it covers a subset of enterprise engineering aspects but does so in a theoretically sound way. In this respect it distinguishes itself from many other modelling approaches. One of the goals of the approach is to equip modellers with a set of sound basic principles and a deep understanding of enterprise modelling techniques. These capabilities can then be used in many different ways to tackle the complexities of today's enterprise engineering projects.

Chapter 2
From Demand to Supply: Layers and Model Quality

Although the development of an appropriate Business Architecture is essential in realising organisation's strategy, reflection on the desired information system service functionality and information system architecture is needed as well. Moreover, from an enterprise information systems development perspective, we need to help software engineers in creating generic, flexible and (therefore) future-proof systems. Focusing in the first place on the demand side of a software project and ensuring the quality of enterprise models is one means to achieve this. At the same time, bridging the gap between demand and supply is also of prime importance. Therefore, the MERODE approach builds on two main principles which, next to being of benefit to the demand side, benefit to the supply side as well.

The first of these principles is the layered organisation of a system. This is a well-known high-level architectural pattern for software architecture, but MERODE also follows this principle during requirements engineering. As a result, the method guarantees an extremely large degree of independence between the domain model and the information system services. This feature further improves flexibility and extensibility of the resulting system. Moreover, the method's advice is to focus first on the enterprise layer whereas many business analysts would gather requirements in an information system service driven way, starting with the identification of desired information system support and user interfaces.

A second main principle that ensures a smooth transition from demand to supply is the quality assurance of the artefacts delivered by the enterprise architects to the software builder. This focus on quality is the natural consequence of the transformation-based approach. As models need to be precise and complete enough to be manipulated by a transformation engine, this transformation-based approach poses stringent requirements on the quality of the specifications.

In the next section, we will first explain the essentials of the layered information system architecture and further motivate how this benefits the enterprise architect and the constructor of software. The second section will then outline essential aspects of quality assurance and explain how this contributes to modelling practice and to the construction of software.

© Springer International Publishing Switzerland 2014

M. Snoeck, *Enterprise Information Systems Engineering*, The Enterprise Engineering Series, DOI 10.1007/978-3-319-10145-3_2

2.1 A Layered System Architecture Both for Requirements and System Architecture

2.1.1 Enterprise, Information Services and Business Process Layer

When gathering requirements, users express their desires about many aspects (functional and non-functional) of the system-to-be. Moreover, as enterprises are functioning in a highly dynamic environment, requirements and expectations towards information system support change rapidly too. The need for flexible and adaptable systems is therefore very real. One way to contribute to the adaptability of a software system is to organise it as a set of layers.

From a construction perspective, the guiding principle of the organisation of the layers is that each layer is able to use the functionality of inner (and more stable) layers but is agnostic of the functionality of outer layers. As a result of this, a modification of an inner layer will have a ripple effect through all outer layers. However, an outer layer can be modified or stripped off and replaced by a new version, without affecting inner layers. Consequently, the kernel of the system should provide stable functionalities that do not change very often. Around this kernel, layers are arranged such that the outer layers incorporate functionality with a lower level of stability than the inner layers they are built on.

This idea of layering software code along the dimension of stability obviously also applies to enterprise information systems but moreover can easily be applied to requirements as well. If we consider different types of requirements, we see that essential business concepts tend to be quite stable. Business rules and information needs will change more frequently and user interfaces and business processes show the highest change rate. Logically then, software components that embody the stable part of requirements should form the kernel layer of a system, whereas software components that embody highly variable aspects requirements should be put in the outer layer.

Domain modelling focuses on capturing the requirements pertaining to the essential business concepts that remain valid even if there is no information system at all: ideally a domain model is fully information system agnostic. The domain object classes form a stable core on top of which information system functionality is modelled as a set of independent information system services. So the domain model provides the model for the stable kernel of the enterprise information systems: the enterprise layer. Around this stable layer built according to the domain model, we put an outer layer of information system services (ISS for short). In order to mirror the state of the real world in the enterprise information system, input services allow capturing information about events in the real world and register this information as new, modified or terminated business objects. Output services enable user to 'look into the mirror' and extract information about the enterprise through output services such as reports, dash boards, etc. An additional user interface layer glues the individual services together and provides the user with facilities to trigger, interrupt

and resume the execution of services. If in addition we take care of keeping the information system services perfectly independent from each other by requiring that all interaction goes through the enterprise layer, we obtain a highly modular and flexible system: information system services can be plugged in and out without affecting the enterprise layer.

On top of the information system service layer, we find the business process layer. Business processes define the work organisation as sequences of tasks to be performed by actors. In order to perform their tasks, actors may use the information system services. If the business processes are supported by software such as a business process management system, this can be implemented as a third layer on top of the information system services layer (see Fig. 2.1).

If we position this set of layers against the models of the Business Architecture, we notice that the Business Architecture is actually spread across the outmost and the innermost layer. Indeed, the Business Architecture is defined by on the one hand activity models (business process models) and on the other hand by a model of the core business concepts (the domain model). Business process models belong to the outer layer. The domain model describes the core business objects about which the business needs information, the relations between these business objects and their individual behaviour. Because the business objects identified in this domain model are key concepts in the Enterprise Architecture, the bottom layer or enterprise layer is sometimes also called the 'Business Object Layer'.

Finally, this layered information system architecture should not be confused with a three-tier architecture for distributed systems, a concept that belongs to the Technology Architecture. In that kind of three-tier implementation architecture, a separation is made between presentation, application logic and data. In terms of the aforementioned layers, the information system service layer contains the application logic as well as the application data and the enterprise layer contains the business object data as well as the business logic (see Fig. 2.2).

The separation between an enterprise layer, an information system service layer and a business process layer leads to a natural partition of systems in three weakly

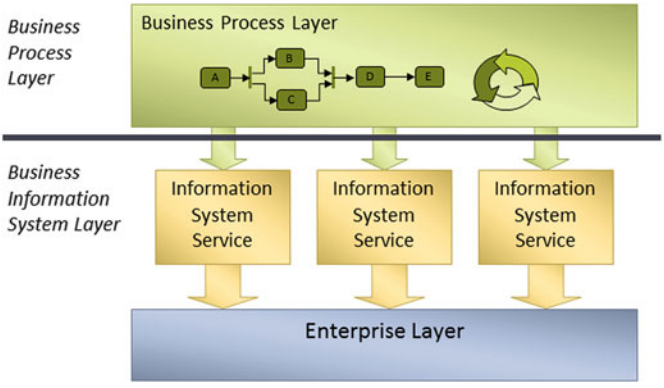


Fig. 2.1 Architectural layers

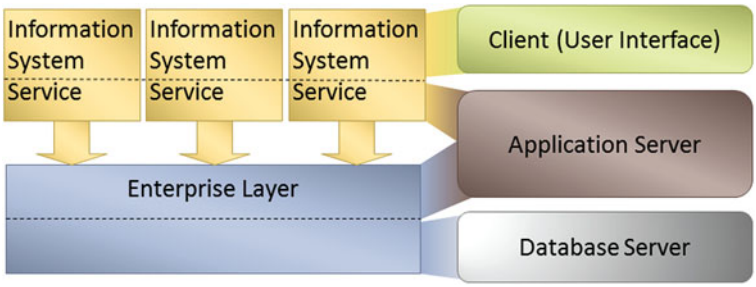


Fig. 2.2 Layers versus Application and Technology Architecture

coupled subsystems, each corresponding to a single aspect of the problem and evolving independently of all other aspects. The three independent aspects are:

- The business logic (the enterprise layer)
- The work organisation (the input services and the business process layer)
- The information needs (the output services)

As a result, on the construction side of a project (i.e. the supply side), this layered approach ensures the adaptability of the software and hence lowers maintenance cost. In response to the changing environment and the need for innovation, organisations need to be able to swiftly adapt their business processes. As these are put in the outmost layer, they can be adapted without ripple effects on the lower layers. Innovation may also drive changes in information system services: new technologies and the need for more or other information may require the implementation of new or adapted input and output information services. Logically, such new services may induce changes in the business processes as well. Again, when information system services are implemented as independent services, ripple effects will remain limited. Only business processes using these services will be affected. Finally, changes to essential business concepts will affect the kernel of enterprise information systems, which is the natural consequence of the fact that they embody the 'essence' of an organisation. Changes to the enterprise layer may affect information system services as well as business processes.

Next to significant productivity gains during maintenance, the layered architecture also offers better insight in the cost of changes. Changes in the enterprise layer may affect all outer layers. Such changes are therefore more expensive and should therefore be scarce. Changes to the business process layer and the information system layer do not affect the lower layers and should be easier to manage. As information system services can more easily be plugged in and out, changes to these services are easier to perform and therefore less expensive. In addition, because information services are always based on a specific part of the domain model, the set of services that is affected by a modification of business objects is easy to determine. This allows to better estimate the cost of modifying the domain model.

A final advantage that benefits both the enterprise architect and the software builder is that the layered approach using a domain model as kernel also allows

improved management of servicing users operating in a similar context, but with different preferences. An example is the development of a software package for different customers that operate in the same business domain. The layered approach facilitates the development of a product line in the following way. The domain model can be offered as a unifying kernel, common to the whole product line. In this way, the domain model is an instrument to harmonise requirements across users and to reduce variability. It creates a common language that defines a common view on essential business objects. If one succeeds to let users agree on a common domain model, variability can then be realised by offering each user a set of information system services tailored to his/her preferences. Higher levels of variability can be achieved by adding proprietary extensions to the domain model and corresponding information system services.

2.1.2 Layers Versus Requirements Gathering and Engineering

The order of the layers is in principle independent of the steps in the requirements gathering and engineering phase. In practice, however, the requirements gathering step occurs most frequently in a top-down manner: many software engineering methods will propose to elicit requirements by interviewing users about their business processes and the use cases for the information system. During requirements elicitation, when users formulate system requirements, they usually tend to mix domain knowledge specifications with information system service requirements. Figure 2.3 shows a subset of specifications for a medical software package for general practitioners. It is clear that only statements 2 and 5 are defining the

1. The software should be web-based.
2. Each consultation is for a single patient. If a visit concerns issues of several patients (e.g. a parent coming with two children), treating the issues of the different patients is considered as having one consultation per patient.
3. The general practitioner should be able to consult reports on medical examinations at the hospital through a secure connection, but only for patients that gave a prior consent for this.
4. The general practitioner should be able to send reports on a medical examinations to a colleague through a secure connection.
5. A medical report is always for a single patient and a single examination.
6. It must be possible to access the file of a patient by means of a maximum of five mouse clicks.
7. Shortcut keys must be provided for quick access to system functionalities (e.g. ctrl + P for printing).

Fig. 2.3 Example of user requirements

domain, while statements 3, 4, 6 and 7 are a description of the required information system services. Finally, statement 1 reflects a technology requirement. One of the most important tasks for a system analyst is then to separate the different types of specifications in order to identify business requirements and information system service requirements.

During requirements engineering, we will classify the different requirements according to the different layers. This means we will separate requirements that describe objects in the problem domain, information system service requirements and business process requirements and put all these requirements in the appropriate layer.

Requirements engineering methods do not always have such clear distinction between domain modelling and information services modelling. Often, the specification of a business object contains not only the core business attributes and routines but also input and output services. We advocate that such a grouping should not be allowed at the modelling stage. If specific reasons apply, the technical architect may still choose to group things that belong to different layers in a single software component when transforming the models to implementation.

An important characteristic of MERODE is that, next to the organisation of requirements into layers, MERODE advises to engineer requirements in a bottom-up way and to focus on creating the domain model first. The benefits of doing so are that this will:

- Foster problem orientation rather than solution orientation and in this way avoid an early implementation bias
- Contribute to generic thinking and hence more future-proof systems
- Stimulate the development of a common language across business users, hence contributing to a better business-IT alignment

In essence, the method proposed in this book is a requirements engineering method. So it concerns the initial phases in software development, where users are questioned on their expectations about the new information system. The focus of MERODE is on requirements *engineering*[1] rather than requirements *gathering*. Requirements gathering can be considered as collecting the pieces of a jigsaw puzzle, whereas requirements engineering pertains to sorting pieces, throwing away irrelevant pieces, detecting and searching for missing pieces and making sure that everything fits together. We already explained that user requirements differ in stability depending on the system aspect they pertain to and therefore belong to different layers. That's one aspect of 'sorting' the pieces of the jigsaw puzzle.

[1] Some authors define requirements engineering as consisting of requirements gathering and requirements analysis. In this book we define requirements engineering as a phase that follows the requirements gathering step and that not only consists of analysing the requirements but also of building a model where every piece of requirements fits in well. Because of this construction aspect, we prefer the term requirements engineering over the term requirements analysis.

In addition to this 'sorting' of requirements, this method also advocates a *focus on domain modelling first* in order to foster problem orientation and to ensure the genericity of the system to be. If one simply lets users tell their requirements, one will notice that most users tend to formulate solutions rather than problems and only report on their current use of a system. As a result, simple requirements gathering is indeed not enough: it should be followed by a thorough analysis and engineering phase to make sure that the final model captures the right problem in the correct way.

One of my former students told me this wonderful story about users formulating solutions rather than their true problems:

> *As a young analyst, one of his first assignments was to investigate the request of a user, who requested that the pink fill of the header of a form would be made yellow. Upon asking for the reasons for this change of colour the user explained that the form was used for some contractual agreement (e.g. a quote). It was therefore faxed to the customer who had to sign it and fax it back. By going twice through the fax machine, the pink fill turned black, making the customer's signature illegible. So the user had figured out that if pink was changed into yellow, this would solve his problem.*

This little story beautifully illustrates that one should always search for the real problem behind the problem formulated by the user. In the above case, the true problem was not about colours but about obtaining a customer's signature, which was eventually solved by providing the means for electronic signatures, making the use of the fax obsolete and contributing to business process optimisation. Users also tend to describe their requirements based on their current use of the system: the user is used to work with a faxing machine and therefore formulated his request in terms of a solution with a faxing machine. This story illustrates how problem-oriented thinking contributes to the development of generic and future-proof systems.

This problem-oriented thinking receives little attention in current practice. Requirements gathering is supported (and encouraged) by many techniques such as semi-structured interviews, storyboards, scenario walkthroughs and use case analysis. Most of these techniques focus on the way information systems are currently used by the users and/or desired future uses of an information system. The results can, for example, be documented using the technique of use case diagrams from the UML.

Documenting the desired 'uses' of a system is without any doubt an essential aspect of requirements gathering. But in order to ensure the creation of generic and future-proof systems, a major point of attention is to pay attention to the fact that a system should not only support today's use cases but should also be capable of supporting tomorrow's use cases.

Suppose, for example, that when interviewing users, the first one comes up with the request to calculate $1 + 1$, the second one needs the calculation of $15 + 15$ and the third of $7 + 7$. What the users probably need is some facility to add numbers, either twice the same or possibly two different numbers. Maybe future uses even motivate the development of a simple calculator, a scientific calculator or even a full computer. Although each of these solutions offers increasing genericity, they all come with a different development cost and a price in terms of ease of use.

Unfortunately, many users are limited in their ability of generic thinking about information systems, mainly because they are unfamiliar with information system design or because they are unfamiliar with the potential services that can be realised with information technology. As a result, they may fail to envisage interesting future use cases. Therefore, discovering possibilities for genericity is an important task of a good business analyst. The business analyst can achieve this by focusing on the problem, rather than on the solution. Obviously, solution-oriented thinking cannot be completely avoided. But one should always keep in mind that different solutions address different problems: the simple calculator, the scientific calculator and the computer all address different problems with different levels of complexity. The business analyst is therefore responsible to discuss and evaluate alternative options with the users: genericity comes with a price, and it remains up to the user to decide how much genericity he/she is willing to pay for, depending on the problems he/she expects to face in the future.

Analysing potential areas for generic and future-proof solutions and assessing the impact of different solutions on the work organisation of an enterprise require thorough requirements analysis and engineering on top of requirements gathering.

Even when focusing on current *and future* use cases of an information system, the use case perspective on requirements gathering still suffers the problem mentioned before that use cases are often used to collect requirements about all aspects of the system, in this way blurring together domain model aspects, business rule aspects, information system aspects, user interface aspects and workflow aspects (see Fig. 2.4). So, whereas use cases are a very practical technique for requirements gathering, use case modelling needs to be followed by an engineering phase during which requirements are categorised according to the identified layers (the 'sorting' part) and further analysed and engineered to define good models.

The question remains then how such 'engineering' of the requirements should be done. As pioneered by the Jackson Systems Development (JSD) methodology [38], modelling the real world or the 'domain' allows to construct a solid core for a software system. The complexity of most systems is caused by the complexity of

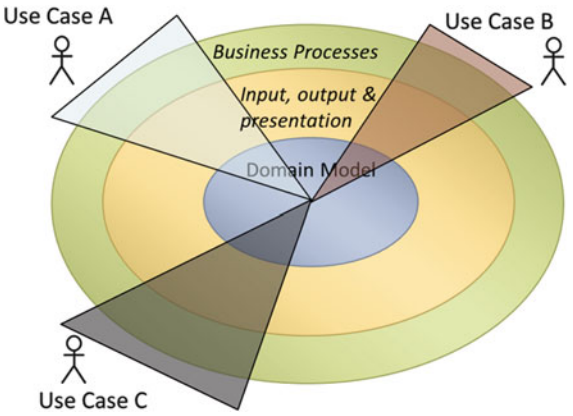


Fig. 2.4 Use cases blur different layers together

the reality they have to deal with. In addition, statements about the required information system functionality always have some underlying assumptions about the real world. It thus makes sense to build such a real-world model prior to the development of an information system. On the other hand, it makes no sense to build a real-world model without having some information system functionality in mind. Similarly the required information system services are mostly determined by the business processes, and reversely, business processes are designed with some available information system service in mind. Hence, both models are an integral part of an analysis model with equal importance.

In this book, following the idea of Jackson, the focus will be on domain modelling first, meaning that the requirements *engineering* process will start with the creation of the domain model. This is not such an evident choice: for users it is much easier to start telling about their business processes. However, if the entire requirements engineering process is executed as a top-down process (starting with business processes, then identifying the required information services to support the tasks and then deriving the required business objects from there), the danger exists that the domain model, which is after all the kernel of the system, will be entirely focused and tailored to the currently envisaged business processes. This way of working is quite natural and easy but severely limits generic thinking about information systems. A requirements *gathering* process can start with business process modelling. However, the requirements *engineering* process should start with domain modelling, then continue with possible information system services that can be build and finally end with defining the business processes to-be. As domain modelling focuses on essential business concepts and business rules, creating an enterprise-wide conceptual model forces the business to develop a common view on how the business operates. In this way domain modelling helps to avoid the fragmentation due to a use case by use case view and stimulates focusing on the true problem and devising generic solutions. Also, the focus on domain modelling can help to overcome the limitations of a project-based view and hence facilitate the alignment of IT with business needs.

In practice, the entire requirements gathering and engineering process will be as follows:

1. Requirements gathering starts with identifying current and envisaged business processes. During this first step, a first set of information system services and business objects are filtered out of the requirements as the result of the 'sorting' of the pieces of the jigsaw puzzle.
2. The requirements engineering phase starts with constructing a first version of the domain model.
3. The completeness and validity of the domain model is verified by checking whether this model provides all that is needed to support the envisaged information system services and business processes identified in step 1. Potentially new information system services and new business processes are identified.
4. The domain model is revisited such as to account for incompleteness and validity problems identified in step 3.
5. Steps 3 and 4 are repeated until a stable domain model is reached.

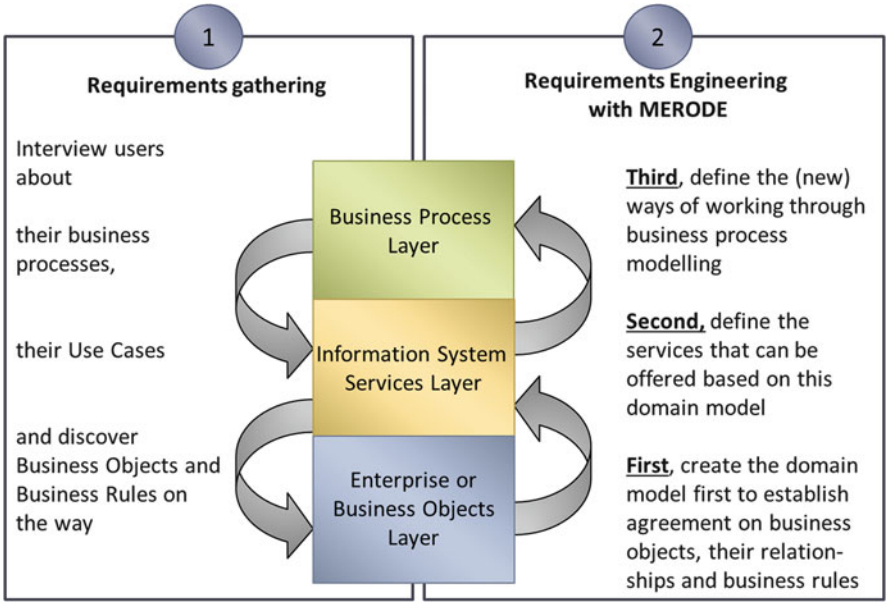


Fig. 2.5 Requirements gathering versus requirements engineering with MERODE

6. An inventory is made of the required information system services (by-product of step 3, further completed in this step).
7. The business processes are finalised (by-product of step 3, further completed in this step).

Figure 2.5 gives an overview of how requirements gathering is a top-down activity through the layers, whereas according to MERODE requirements engineering follows a bottom-up approach through the layers (modulo the necessary iterations to achieve a satisfying level of validity and completeness).

2.2 Formal Verification of Models

2.2.1 Model Quality

Hidden, unspoken requirements, ambiguously formulated requirements or a lack of agreement on requirements may all affect software development projects in a negative way. The quality of requirements and hence also of an enterprise model is of prime importance for the successful development of enterprise information systems. A wrong, incomplete or ambiguous domain model on which there is no agreement reflects the fact that the business is not clearly understood in the same way by all stakeholders.

To achieve a certain level of quality in the production of software, software engineering takes a lot of inspiration from the manufacturing process. In manufacturing, one way of guaranteeing the quality of final products is to standardise the production process. For example, in an assembly line, the production steps are completely standardised. As a result, all end products are identical. Standardising the production method is a way to ensure a certain level of quality for all delivered products. Research on software process improvement (SPI) and models such as ISO 9000 and the capability maturity model (CMM) [73] all focus on the software development *process*. By rigorously defining tasks and activities in this development process, such frameworks hope to ensure the quality of the final product. Information systems development is however quite different from manufacturing: each development project aims at the construction of a system that is perfectly tailored to the needs of the users. Therefore, it is impossible to completely standardise the software development process, although many software development frameworks would like us to believe otherwise.

This impossibility to achieve quality by focusing on the process only should however not prevent us from demanding a certain level of quality for the final result. The way to do this is to establish a number of quality criteria that must be met by the final result. The same is true when I hire someone to clean my house: I do not tell this person to sweep twice here and three times there; I only want my house to be clean at the end. The exact intermediate process is of lesser importance. For the same reason, this book does not focus primarily on development processes. In contrast to many other approaches, this book focuses on a number of strict criteria that will be defined to ensure the quality of the outcome of the process: the final outcome of a process is subject to a list of very stringent norms that will qualify it for further development. In particular, as the focus lies on domain modelling, the method in this book will give some guidance on the process of modelling but will put most emphasis on the quality of the produced models. Concentrating on the quality of the modelling process is like ensuring the cleanness of water pipes. This is beyond doubt an important element in quality assurance. But as judiciously remarked by J. Voas [97], clean pipes can produce dirty water. And thus, we believe that an essential component in the development of a modelling method is the elaboration of tests that can assure the 'quality of the water', hence, model quality. This way of working is equivalent to Mintzberg's coordination mechanism by standardisation of output, as opposed to coordination by standardisation of process [59].

Furthermore, in a model-driven approach, the outcome of domain modelling is a platform-independent model that will be used as input for transformations to other models or code. As it should be possible to *automatically* transform such a model to models or code by means of model-to-model and model-to-code transformations, the model should be absolutely unambiguous and precise. Indeed, in case of *automated* transformations, there is no human intervention, and so remaining errors and ambiguities cannot be corrected during the design and implementation step. Transformation engines obey the law 'garbage in, garbage out'.

In the past, different frameworks for model quality have been proposed. Modelling quality frameworks define levels of quality that a certain model should accomplish. There are many frameworks that attempt to bring order and enlightenment to the quality of conceptual modelling representations and to the quality of the conceptual modelling process. Some frameworks stand out from the mass: the first is a framework introduced by Lindland, Sindre and Sølbverg [47] known as SEQUAL in its latest versions [43, 44]. It builds on Morris' semiotic theory and defines several quality aspects based on the relationships between a model, a body of knowledge, a domain, a modelling language and the activities of learning, taking action and modelling. The second is a framework developed by Wand and Weber based on Bunge's ontological theory (the Bunge–Wand–Weber representational model: BWW) [98]. Both frameworks have solid theoretical foundations, and they look at quality in conceptual modelling from two different perspectives. The SEQUAL framework focuses on the product of conceptual modelling whereas the BWW framework focuses on the process of conceptual modelling. Focusing on the integration of the SEQUAL and BWW frameworks in 2012, a third Conceptual Modelling Quality Framework (CMQF) [62] appeared. This framework is useful for evaluating the end result of the conceptual modelling process, the conceptual representation and the quality of the modelling process itself in terms of the knowledge aspects that come into play during the modelling process. A detailed explanation of this framework being out of scope for this book, we limit the definition of model quality to the basic cornerstones (for a complete explanation, see [62]).

Models can be considered as sets of statements that can be checked by comparing them to other statements or verifying them against a number of rules. The following basic types of quality can be identified.

2.2.1.1 Syntactic Quality

This refers to checking whether all statements expressed by a model are well formed, according to the conventions of the modelling language the model has been expressed in. For example, class diagrams are composed of classes (rectangles) and associations (connectors), and each association must always have at least two association ends. An association with only one end does not satisfy the syntax of a class diagram. If you compare this with verifying a text, syntax checking means that the text is verified for spelling and grammar. A good modelling language offers a clear definition of the syntax of its modelling techniques so as to enable this type of verification.

2.2.1.2 Intentional Quality

Next to obeying the syntax imposed by a modelling language, the model should also obey the intended meaning of the symbols. Symbols shouldn't be (mis)used to

represent something else than they are defined to represent. An architect shouldn't use the symbol for a window where he/she intends to have a door simply because he/she doesn't know how to draw a door. Likewise, in a domain model, a class/rectangle is intended to represent a set of business objects from a same type (e.g. the set of customers). Hence, using the class symbol to represent a collection of events (e.g. mouse clicks) is an example of bad intentional quality of a model.

2.2.1.3 Semantic Quality

This refers to level to which the set of statements in the domain model is a 'good' representation of the 'real world'. Semantics is sometimes also referred to as the 'meaning' of a model. Two types of semantic quality can be defined. **Completeness** means that all relevant statements about the domain have been included in the model. **Validity** means that all statements contained in the model are a correct representation of the domain at hand. Compared to the verification of a text, validity means that everything that is said in a text is true, and completeness means that no omissions are observed. Validity can further be subdivided into *external validity* and *internal validity*. External validity means that statements are verified against the truth as defined by the 'external' user. Internal validity verifies whether different statements that compose a model do not contradict each other. It is also referred to as 'consistency checking'. Stated differently, external validity asks the question 'Is this the right model?', whereas internal validity focuses on the question 'Is the model right?'.

Checking the semantic quality of models can be very hard. First of all, each individual diagram should be valid and complete. This in itself is already a challenge and requires a precise and unambiguous definition of the modelling techniques (preferably in a formal way). The intrinsic difficulties arise from the fact that different diagrams are used to define different aspects of the problem and that all these diagrams need to be consistent with each other. The first type of verification is called intra-diagram verification and the second type, inter-diagram verification. As an example, consider again the plans in Fig. 2.6. On the left hand is a floor plan with a separation wall between the sitting room and the dining room (indicated by the arrow). On the right-hand side is a transversal cut plan, used to define the height of roof, ceilings and windows and where the separating wall has been drawn at the other side of the dining room. This is an example of bad internal quality: it is a problem of inter-diagram inconsistency. Notice that consistent plans do not guarantee the external validity: it yet remains to be seen where the wall should be built in the real-world house according to the owner's requirements.

The same type of problem can appear in domain modelling. Consider an approach in which object behaviour is specified by means of one diagram and object interaction is specified in some other diagram. Whatever technique is used, in any way, the behaviour of the final system is a composition of all the individual object behaviour definitions and the definition of how objects interact with each other. For large systems it is difficult, if not impossible, to have a complete mental

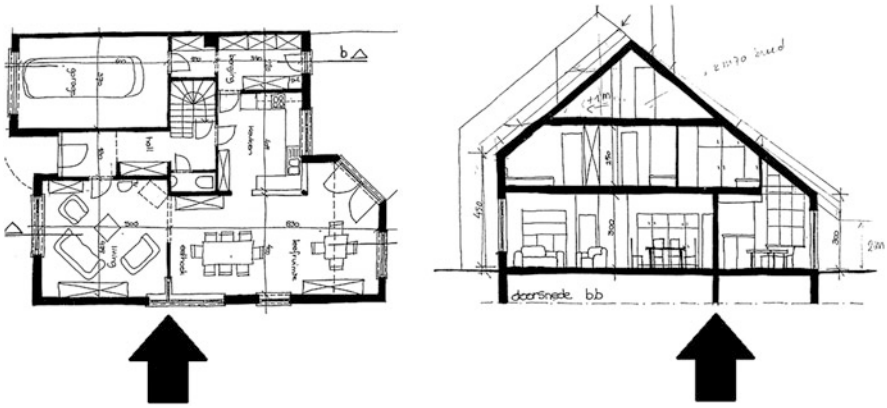


Fig. 2.6 Inconsistent plans for a house. Published with kind permission of © M. Van de Wouwer 2000. All rights reserved

model of what the behaviour of the overall system will be. As a result it is likely that the overall system will show some undesired behaviour such as starvation or deadlocks. Although some of these errors can be discovered by careful testing of *the implemented system*, it is much better if the conceptual modelling language provides the necessary formal underpinning enabling the use of verification tools and algorithms to check and *prove* the correctness of specifications *during model construction* [17, 82].

2.2.1.4 Pragmatic Quality

This quality dimension refers to the level to which the statements expressed by the model are correctly understood by the actors that will read the model. This quality dimension is often referred to as understandability of a model. Ensuring pragmatic quality can be positively affected by choosing the right modelling language and the right representation for the right public. Also, using adapted representations for the same conceptual model, depending on the targeted public, may positively influence model understanding.

As an example, converting the two-dimensional plans of a house to a computer-animated 3D model can help the owners to create a mental model of what their future house will look like.

2.2.2 Modelling Language

The choice of a modelling language will affect model quality in different ways. First of all, syntactic and intentional quality will be affected by the level to which

the symbols, their meaning and their composition rules are well defined and algorithms can be used to verify the syntax. Semantic quality will be affected by the level to which the languages provide algorithms to verify consistency and completeness. Finally, the level to which a language is widely known and understood will affect pragmatic quality.

Several authors have advocated the use of formal languages [15] to provide for better quality of models and the necessary support for model verification. An important observation is that formality on its own is not sufficient to achieve high quality.

Consider, for example, the statement:

John is a person.
Any student is a person.
Any employee is a person.

We can translate this statement to mathematics as follows:

$$\text{John} \in \text{PERSON}$$
$$\text{STUDENT} \subseteq \text{PERSON}$$
$$\text{EMPLOYEE} \subseteq \text{PERSON}$$

Despite this formality, we still don't know whether in our universe all people are either students or employees and we also don't know whether one can be a student and an employee at the same time. Formally, one still needs to check whether the following statements are true in our universe of discourse:

$$\text{PERSON} = \text{STUDENT} \cup \text{EMPLOYEE}$$
$$\text{STUDENT} \cap \text{EMPLOYEE} = \varnothing$$

Although formality does not offer a guarantee for completeness of specifications, it does offer the major benefit that formal languages enable reasoning (in a mathematical sense) about properties of models [14]. This reasoning enables one to investigate models for omissions, ambiguities and contradictions. Eventually, one may even *prove* that certain anomalous system behaviour, such as deadlock, cannot occur [17, 82]. The formal underpinning of modelling techniques enables quality control at a level that is impossible to reach with informal techniques.

When talking about formal techniques, one nearly automatically thinks of complex specification languages with many mathematical symbols and that require a lot of training before they can be used. Fortunately, formal techniques are not necessarily mathematical specification languages but can be graphical techniques as well, provided that the syntax and semantics of these techniques are precisely described. The useful thing about formal languages is the reasoning they enable. This reasoning can fortunately be hidden behind more user-friendly graphical symbolic representations.

As language, ideally we would opt for a formal language, given its advantages for formal verification. However, such a language has the severe drawback that it would make models very hard to read, understand and validate by the users, jeopardising pragmatic quality. This book therefore opts for a domain-specific language, largely based on the widely known UML. As the UML is a general-purpose modelling language, it offers quite a lot of techniques and features, suited for all possible areas of software development: it can be used for models in the domain of enterprise information systems as well as for modelling real-time systems or embedded systems. It also supports the description of artefacts in all stages of the development process: from high-level conceptual models such as domain models down to the specification of design models close to implementation. Given that the language is widely known in the software development community, the choice for UML ensures (in terms of understandability) a much better fit with our users than a formal language. On the other hand, the multitude of constructs holds the danger of low intentional quality. Also, we still need the formality to be able to offer the users the power of formal reasoning and code generation. To achieve this, we limit our use of UML to the most frequently used techniques and complement the existing UML semantics with a further formalisation by means of process algebra. The formal reasoning power is kept hidden behind the known graphical representation of UML, such as to shield the business analyst from the complexities of the MERODE-process algebra. The basis of the MERODE domain modelling approach is the UML techniques for class diagramming and life cycle modelling (state machines), augmented with the object-event table (borrowed from James Martin's Information Engineering method [53]) to model object interaction. The MERODE approach is a domain-specific use of UML in the sense that:

1. It doesn't use all the UML techniques, but only a relevant subset for the context of domain modelling.
2. It doesn't use all the features of the chosen UML techniques, but is limited to the features relevant for domain modelling.
3. It complements the UML techniques with the CRUD-matrix of the Information Engineering approach.
4. It complements the known semantics of UML with a more thorough formalisation by means of a CSP-like process algebra [35, 82, 17].

2.2.3 Tool Support for Quality Control

From the point of view of quality of the modelling process, the formal definition of a modelling technique will allow for correctness checking. As models serve as plans for software development, error-free models substantially reduce development costs. But evenly important, the precise definition of the syntax of a modelling technique is a prerequisite for the development of a supporting computer-aided software engineering (CASE) tool: the precise definition of semantics and the availability of a formal procedure for checking consistency between views allow

to add intelligence to such a CASE-tool. Without these features, CASE-tools cannot offer much more support than simple diagram editing. CASE-tools in general and modelling tools in particular are for models what a word processor is for a text. They offer the possibility to capture requirements in the form of visual models, called diagrams. Diagram editors range from simple drawing tools to intelligent requirements engineering tools, capable of thorough quality checks.

For the verification of consistency (internal validity), three basic approaches can be distinguished [87]. A first approach is consistency by analysis, meaning that an algorithm is used to detect all inconsistencies between two deliverables (each representing a particular view of the same system), and a report is generated thereafter for the developers. In this kind of approach, the requirements engineer can freely construct the different views of the system. At the end of the specification process or at regular intervals, the algorithm is run against the models to spot errors and/or incompleteness in the various views. The verification can be done manually, but obviously building the algorithm into a CASE-tool will substantially facilitate the consistency checking procedure.

The second approach can be denoted as consistency by monitoring, meaning that a tool has a monitoring facility that checks every new specification against the already existing specifications in the CASE-tool's repository. Whenever an attempt is made to enter a specification that is inconsistent with some previously entered specification, the new specification is rejected. The advantage of this approach is that the model is constantly consistent. Whereas the first approach puts the burden of correcting inconsistencies on the requirements engineer, the second approach avoids the input of inconsistencies. At the end of the specification process, the model must still be verified for completeness. The possible disadvantage of this approach is that a too stringent verification procedure will turn the input of specifications into a frustrating activity. The two approaches can be compared to two spelling and grammar checking strategies in word processing: the first checks spelling and grammar by running the spelling and grammar checker periodically, whereas the second approach is the equivalent of the option 'check spelling and grammar as you type'.

A third approach is consistency by construction, meaning that a tool generates one deliverable from another and guarantees completeness (to a certain extent). Whenever specifications are defined in one view, those elements in other views that can automatically be inferred are included in those views. Also in this approach, the requirements engineer can only define consistent models. The major advantage is however that the specifications are more or less constructed in an automated way: everything that can automatically be inferred is generated by the CASE-tool. This saves a lot of input effort while at the same time contributing to the completeness of models. In addition, whereas the monitoring approach leads to a CASE-tool that generates error messages at every attempt to enter inconsistent specifications, the self-constructing approach avoids the input of inconsistent specifications by completing the entered specifications with their consistent consequences. The result is a much more user-friendly environment. This type of feature can be compared to an autocomplete functionality.

Text processing	Requirements engineering
Text	Model
Language	Modelling language
Word processing tool	CASE-tool
Checking spelling and grammar	Verifying syntactic quality, i.e. the correct use of modelling techniques
Cross referencing	Verifying consistent naming of model items
Checking the content of the text	Verifying semantic quality (validity and completeness)
Checking the understandability of the text	Verifying pragmatic quality
Check spelling and grammar afterwards	Consistency by analysis
Check spelling and grammar as you type	Consistency by monitoring
Autocomplete	Consistency and completeness by construction

Fig. 2.7 Comparing requirements engineering to text processing

Figure 2.7 compares the different types and approaches of quality control based on the analogy with a word processing tool.

Although CASE-tools can offer substantial help in verifying the quality of models, the human verification remains very important. Moreover, external validity can, for example, only be checked having models checked by domain experts. Although the method presented in this book tries to achieve maximal support by means of algorithmic checking, we nevertheless need to ensure that we can rely on users that are knowledgeable in the business domain, so that they can validate models. Interestingly, the interaction with domain experts will be stimulated by advanced CASE-tool support. When a tool detects an inconsistency, inconsistency resolution will require interrogating the domain experts about what is correct and what not. Referring to the example given in Fig. 2.6, the owner will have to decide which of both plans has the correct representation of the position of the separation wall. Also the autocompletions generated by the tool will have to be verified with domain experts. In this way, the formal verification is a powerful mechanism that substantially contributes to the completeness and external validity of models.

2.2.4 Quality Checking in MERODE and JMERMAID

One of the crucial elements in the development of MERODE is that we want the method to provide effective support for model quality. First, this has been achieved by formalising the domain model by means of process algebra. This book translates all the formal definitions in natural language and illustrates them with many examples. As a result, the user of the methodology does not necessarily need to understand these formal definitions for being able to use MERODE. Notwithstanding, the formal definitions have an added value in that they define concepts more precisely than ever could be done in natural language.

Secondly, to ensure the ease of modelling, a lot of effort has been spent on making all concepts orthogonal. There is a one-to-one mapping between graphical

representation and underlying formal concept and concepts can be composed without restriction: the composition of valid specifications is in its turn a valid specification.

Finally, quite some effort has been put in offering adequate tool support for quality control. This has resulted in the JMermaid tool, a proprietary CASE-tool that specifically supports the creation of MERODE domain models.[2] The tool implements a mix of the three quality assurance strategies. Whenever possible, the tool implements the 'consistency by construction' strategy and autocompletes specifications. This feature can be turned off as well. Next, the input of new specifications is always monitored against the quality rules outlined in this book for each of the modelling techniques. Finally, a final check can be run against a model, to check whether the model satisfies all quality criteria and, if desired, to check whether the model is fit for the generation of Java code.

2.3 Summary

In terms of information system architecture, MERODE advises a layered architecture, organising enterprise information systems (from kernel to outer layer) into an enterprise layer, an information system service layer and a business process layer. This layered architecture should also be applied to functional requirements.

Next to the organisation of requirements into layers, MERODE advises to engineer requirements in a bottom-up way and to focus on creating the domain model first. The benefits of doing so are that this will:

- Foster problem orientation rather than solution orientation and in this way avoid an early implementation bias.
- Contribute to generic thinking and hence more future-proof systems.
- Stimulate the development of a common language across business users, hence contributing to a better business-IT alignment.

On the supply side, the layered information system architecture ensures the adaptability of the software and hence lowers maintenance cost.

The quality checking approach of MERODE and its companion tool JMermaid offer a number of benefits on the demand side of a project:

- The quality checking rules contribute to the ease of modelling as they offer practical guidelines on how to create and improve models.
- The autocomplete functionality of the JMermaid tool but also the quality checking rules in general contribute to the completeness and the external validity of models as each quality check will generate questions stimulating and feeding the discussion between architects and business users.

[2] JMermaid stands for Java MERODE modelling aid.

- The internal validity of models is ensured, making them fit for implementation and for a model-driven approach to software engineering.
- Thanks to the MDE approach, models can immediately be transformed to a working prototype, which can be used by the architect to validate the models with the business users. In other words, the automatic generation of code makes a MERODE model truly executable.

On the supply side, the quality assurance ensures that the architect delivers complete and consistent models that enable a model-driven approach to software engineering. Transformations can already be developed at the start of the project, in parallel with the requirements engineering phase which facilitates the iterative development of software. Moreover, transformations are reusable across projects and therefore allow to substantially speed up the development of software.

Chapter 3
Overview of MERODE

This chapter concludes the introductory part with a general overview of the approach by means of a small library example. It should give the reader scome insight in all the techniques and principles that are present in MERODE, without going too much in detail. Part II of this book will then present each technique separately in a more elaborated and formal way. The chapter starts with a global overview of the modelling process and then reviews the models from inner layer to outer layer: first domain modelling, then information system service identification and finally business process modelling.[1]

3.1 The Modelling Phases

The main phases of the MERODE modelling process are the enterprise modelling phase and the transformation to implementation phase. Enterprise modelling is then further divided according to the structure of user requirements. We therefore make a distinction between domain modelling, information system service identification and business process modelling. Within these three phases, the following steps are undertaken:

1. Domain modelling phase
   1.1 Modelling business object types and business event types
      (a) Identification of business object types

[1] A very earlier version of the topics handled in Chapters 3–6 has been published as the journal paper 'Existence Dependency: The Key to Semantic Integrity Between Structural and Behavioural Aspects of Object Types' by Snoeck M. and Dedene G., which appeared in *IEEE Transactions on Software Engineering*, Vol. 24, No. 4, pp. 233-251, April 1998 © 1998 IEEE. The material from this paper has been integrated in this book with the kind permission of IEEE.

© Springer International Publishing Switzerland 2014

M. Snoeck, *Enterprise Information Systems Engineering*, The Enterprise Engineering Series, DOI 10.1007/978-3-319-10145-3_3

(b) Construction of the existence-dependency graph
(c) Identification of business event types
(d) Construction of the object-event table

1.2 Sequence constraint specification
1.3 Specification of attributes and object-event methods

(a) Definition of domain object type attributes
(b) Definition of object-event methods
(c) Definition of constraints

2. Information system service identification phase

2.1 Identification of information object types
2.2 Identification of all required services, with for each service:

(a) Identification of the required business object type and information object type attributes
(b) For input services: identification of the generated business events
(c) Specification by means of use case diagrams

3. Business process modelling phase

3.1 Identification of relevant business processes
3.2 Business process modelling
3.3 Relating tasks to business events
3.4 Relating tasks to input and output services
3.5 Relating data to domain object types and their attributes

As will be explained in Sect. 3.6, the modelling process of MERODE is strongly iterative and allows for a lot of freedom and parallelism. The order of presentation does not imply that the requirements engineering process should follow the same sequence. Rather, the whole process is mainly guided by a set of rigorous verification rules that assess the quality of the models.

3.2 Domain Modelling

The basic building blocks of a MERODE domain model are business object types and business event types, which are related to each other and defined in more detail through three views or sub-models: a data model or class diagram, called an existence-dependency graph (EDG), an interaction model called object-event table (OET) and life cycle definitions for each of the business objects, called finite-state machines (FSM). We will illustrate each of these techniques by means of a small library example.

3.2.1 Business Object Types

The first step in domain modelling is the identification of business object types that occur in the described reality and which, at the same time, are relevant for the system to be constructed. In other words, one needs to identify the business objects that need to be mirrored in the enterprise information system, because information about these objects is required to manage business operations, planning and strategy. To be a valid candidate business object type, an element must satisfy the following characteristics: a business object has an identity, it has a state described by means of attributes and it shows some behaviour. In addition, business objects have to correspond to real-world concepts, which leads to additional criteria. As a result, business object types should satisfy the following set of requirements:

- Since information about business objects is important to manage business operations, an object **can be described by a number of properties**. For example, a customer can be described by means of a name, a birth date, an address, etc. A date is not an object but a describing element: for example, as 'birth date of', it describes a customer. Notice that the distinction between a property and an object is dependent on the business domain (see Chap. 7). The fact that the user is willing to see a concept as an object can be determining. For example, the country where a person lives will normally be considered as a property of a person. But in a certain organisational context, it might be necessary to view country as an object, because relevant properties can be attached to this concept (e.g. number of inhabitants, surface, political regime, etc.).
- An object **has an identity**, not necessarily explicit and visible to the user (in other words, the concept of identity is *not* the same as the concept of a (primary) key in the relational model).
- An object **corresponds to a real-world concept: it has a real-world counterpart** which may or may not be tangible. For example, a 'customer' is a tangible object in the real world. A 'loan' in a library is not a tangible object, but it is a real-world object as well. When a book is borrowed, no tangible object is created, but nevertheless the loan exists. Notice that this requirement of a real-world counterpart is typical for domain modelling. Objects in an information system implementation model need not necessarily have a real-world counterpart.
- An object **exists for a certain period of time**. For example, a loan has a certain duration: it starts when the book is borrowed and it ends some time later when the book is returned. A check-in at the hotel desk has no duration; that's to say, it can be considered as occurring at one point in time. It is the transition between a reservation and a stay and is in fact a business event type.
- Because of the previous requirement, an object is **always involved in at least two business events**: its creation and its ending. CUSTOMER is an object type because at a certain point in time, a customer starts to exist for an enterprise and may after a while cease to exist; 'date' and 'age' are not, because they do not

start and/or cease to exist. In addition, a business object may be subject to state[2] changes, due to the occurrence of a business event. A customer has a state and can have a change of state. But a similar reasoning does *not* apply to date nor to age.

An analysis model only describes object **types**, not individual objects. Object types are designated by a noun and, in this book, they are written in small capitals.

Apart from identifying object types with the purpose of including them in models, each object type should have a proper and agreed-upon designation of its meaning. In this way, business object types form (a part of) a business ontology and define a common language in the form of an agreed-upon terminology to designate important business concepts.

3.2.2 Business Event Types

The domain modelling method proposed in this book follows the principles of object orientation. So, as already mentioned before, the domain model will also capture the behaviour and interaction of the business objects. According to the object-oriented paradigm, the modelling of interaction relies on the following principles:

- The identification of operations for a class
- Message passing, that is, the fact that an object can invoke the operations of an object it has a link to.

These principles are the basic way the behaviour of a piece of code is shaped in an object-oriented programming language. Most software engineering methods use these interaction concepts in the same way in the design and analysis phases of software development. However, as requirements engineering should not be biased by implementation considerations, a domain model should only be based on those principles that can be supported by all possible implementation technologies. In practice, all object-oriented programming languages offer support for the principles of the object-oriented paradigm, albeit in different flavours. Older and more traditional implementation technologies such as relational databases and third-generation languages have no problems in dealing with, for example, the notions of class, encapsulation and information hiding. They have however never been designed to offer support for message passing and inheritance. Fortunately, a number of workarounds and best practices do exist. As will be explained in Chap. 8, inheritance can be used in domain modelling, provided a number of

[2] The state of an object should not be confused with its 'life cycle state'. The state refers to the more general concept of visible state, that is, the whole of values of the object's attributes, while the 'life cycle state' explicitly refers to states as they appear in state transition diagrams such as finite-state machines and Harel Statecharts.

considerations on the different interpretations of the concept at domain modelling versus programming level are taken into account.

The use of concept of message passing for modelling interaction in a domain model poses however a different kind of problem. The main inconvenience of the concept of message passing is that it implies a fairly, even too detailed level of interaction modelling and that an interaction schema most of the time involves objects outside the domain model.

Example 3.1 The Library: Coordinating Across Business Objects

Imagine, for example, a library where members can borrow copies of books. In this type of domain, MEMBER and COPY are examples of business object types. When a member borrows a book, the book becomes unavailable, so there clearly is some kind of interaction between a member object and a book object. If message passing is used to model that interaction, the fact that a member borrows a copy can be modelled either by means of a message from the member to the copy or by a message from the copy to the member (see Fig. 3.1).

The problem becomes even more apparent if we consider the addition of an object type LOAN. As a LOAN object needs to be created as well, three objects need to be coordinated. In that case at least 12 different message passing scenarios can be identified (see Fig. 3.2).

At the level of domain modelling, it would be irrelevant to debate the direction and sequence of the messages between business objects: it is unlikely (and would be a bad practice) that in the implemented system one of the *business* objects would send a message to the other. What is much more likely is that messages to business



Fig. 3.1 Interaction scenarios between COPY and MEMBER

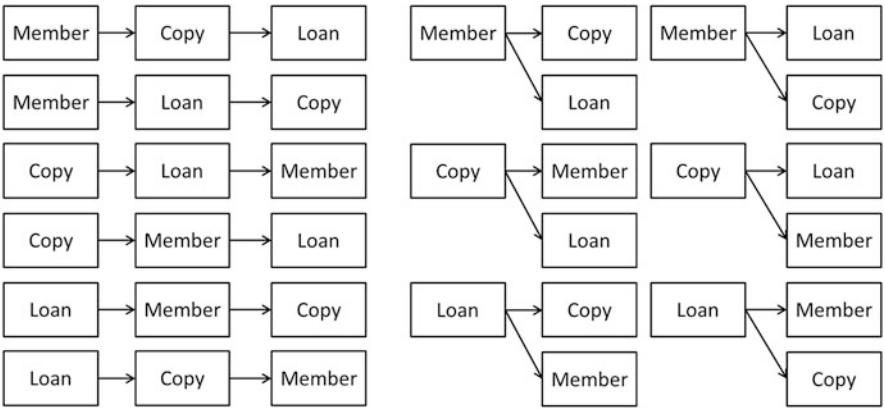


Fig. 3.2 Potential interaction scenarios between COPY, MEMBER and LOAN

objects will be invoked by objects in the information system service layer (e.g. a transaction object) or even the user interface layer. These layers are however absent in the domain model, and therefore, we cannot model the interaction properly, at least not when we wish to stick to a pure domain modelling approach.

At the domain modelling level, object interaction can be specified in a natural way by identifying business events and indicating which objects are involved in these events. For the library example, this means that we define a business event called '*borrow*' and that the three domain objects MEMBER, COPY and LOAN are *involved* in this event. In order to specify the effect of the business event *borrow*, all three involved objects are equipped with an operation (called a 'method' in object-oriented terminology) that defines the reaction of this object on the *borrow* event and it is agreed that these methods will all three be executed upon occurrence of a *borrow* business event (see Fig. 3.3). For simplicity, we assume that the method that defines the reaction of an object type to a particular type of event carries the same name as that event.[3]

This interaction concept is called 'multiparty interaction' and it means that objects synchronise upon joint participation to events. It is a known principle from process algebra. CSP [35] and ACP [4] are examples of algebras that allow to model process that synchronises on common events, while message passing is more similar to CCS [58]. One of the major advantages of this principle of communications is that the complexity remains the same, no matter how many objects participate in an event. As has been demonstrated with the library example, interaction schemes based on message passing grow exponentially in complexity when additional objects need to be added to the interaction scheme.

Moreover, using multiparty interaction based on joint participation to business events offers numerous additional advantages:

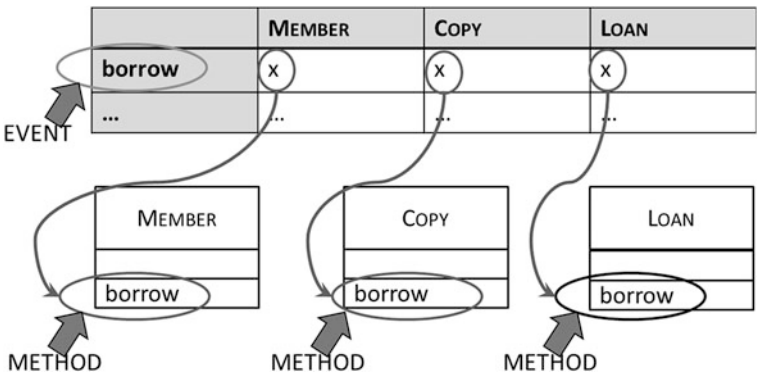


Fig. 3.3 Modelling interaction between business objects

[3] To distinguish between event names and method names, in JMermaid, event names are by default prefixed with EV_ and method names with ME_.

- In the context of domain modelling, it leads to the explicit identification of business events.
- It can be represented by a very simple modelling technique: a C(R)UD table which indicates which business events **c**reate, **u**pdate or **d**elete objects of a certain type.[4] As the table matches objects against events, it will be called the object-event table (or 'OET' for short).
- The notion of business events nicely integrates with the notion of task.
- The notion of event combines well with state machines, where events are the triggers for transitions from one state to another.
- Behaviour and object interaction are easy to formalise.
- It allows for an event-driven implementation, which is easy to realise with non-object-oriented implementation tools (see Part IV).
- It does not preclude an object-oriented implementation with message passing as interaction mechanism. Multiparty interaction by joint participation to business events allows for the transformation of models to any implementation environment in a nearly straightforward manner. Seamless transition to object-oriented implementation technology and transition to non-object-oriented technology are both possible.

These arguments are further explained in Chap. 5 and the remainder of this book.

In the context of domain modelling, we therefore choose to model the behavioural aspect in terms of business event participation. To identify something as a business event type, it should satisfy the following requirements:

- A business event of a certain type **occurs or is recognised at one point in time**; its possible duration can be abstracted. A good example is the event 'pay'. When we include 'pay' in the list of relevant business event types, it refers to the moment at which the activity of paying terminates. For example, consider a payment in cash. Such an activity will last a certain time: the customer has to take his/her wallet, count money, etc. but the actual duration of all this is of no interest for the model. In a domain modelling perspective, the 'pay' event occurs at the moment that the activity of paying has been (successfully) performed. On the other hand, the activity of 'owning' a property is not an event: you cannot make abstraction of the duration of the ownership. In fact at least two business event types are involved in an ownership: 'buy' and 'sell' that indicate the start and the end of the ownership. Being in a state (like owning one million Euro) is never an event. It is the change of state that is an event (like going from owning one million Euro to owning two million Euro). Similarly, heating a pan is not an event, but a process. It is the start and ending of the heating process that are events. If during heating the temperature of the pan crosses a threshold, this crossing can be modelled as an event as well, if relevant in the universe of discourse.

[4] The R in CRUD stands for **r**ead. As will be explained in Chap. 5, we will not model the reading of objects at domain modelling level.

- A business event matches something that **happens in the real world** within the universe of discourse; it is not an information system event. For example 'release mouse button' or 'push mouse button' is not acceptable in a domain model. Events that happen within the universe of discourse remain relevant for the domain model, even if there is no information system.
- A business event is **atomic**; it cannot be split into sub-events. Consider, for example, an association where members can register and are allowed to pay their membership fee later. Some members however announce themselves as a member by directly paying their membership fee. In that case, 'become a member' is a business event type and so is 'pay membership fee'. But 'become a member by paying membership fee' is not: it is a combination of the previously mentioned business event types.

  The fact that a business event is not decomposable does not require that the object base remains in a consistent state after the occurrence of a business event: a business event is not a unit of consistency. As shall be seen in a later chapter, it is possible to specify constraints that imply the grouping of events in order to keep the set of existing objects in a consistent state. For example, in a course administration case, it might be required that a student is always enrolled for at least one course. This means that when a student is created, an enrolment must be created at the same time. Similarly, when the last enrolment comes to an end, either the life of the student must end as well or a new enrolment must be created. These groups of events (e.g. a *create_student* event combined with an *enrol* event) are called ***consistent*** groups of events. They are composed of atomic events. In contrast with the latter, they always keep the set of existing objects in a consistent state: before and after the occurrence of a ***consistent*** group of events, all specified rules are satisfied. Consistent groups of events will be handled in more detail in Sect. 9.3.2. For the purpose of domain modelling and for the specification of sequence restrictions, we use ***atomic***, non-decomposable events.

Event types should preferably be designated by the infinitive of a verb rather than by a conjugated tense: *pay* but not *pays*. In this book they will be written in *italics*.

Not all elements satisfying these criteria will be retained as relevant business object types and business event types. One of the purposes of scope modelling is to determine the relevant area for enterprise modelling. Elements outside this scope are not retained. For example, a *create_customer* event type models the moment at which a customer becomes relevant for the business, which is not necessarily his/her birth. Another consideration is that only elements that are necessary to deliver the functionality that we have in mind should be retained. The domain model completely determines which functionality **can** be delivered. In this sense it is important to include enough object types and event types. But retaining too many elements will increase development cost and is therefore not desirable.

3.2.3 A Small Library Example

Example 3.2 The Library: Small Library Case Text

In the small library, members can borrow books, possibly several books at a time. The privileges of a member determine for how many days a book can be borrowed. Members can ask for a renewal to postpone the day of return. If a person has a book that is more than one week overdue, the library system automatically generates a reminding email to this member. If after a certain time the book is not returned yet, the book is registered as lost and possibly the member loses some privileges. The library provides its members with an on-line catalogue where all individual copies are classified.

Variations on the library example will be used throughout the subsequent chapters to illustrate concepts and techniques of domain modelling.

In the project definition phase, it has been defined that the library wants an information system for the administration of members, loans and books. The accounting and ordering administration are at this point in time out of scope.

3.2.3.1 Identifying Business Object Types and Business Event Types

First of all, we have to search for event types and object types that occur within the given scope. In the given example this means we look for object types and event types for the administration of members, loans and books, but not for accounting and ordering administration. Looking for the relevant business event types and business object types can be done in several ways. One method is to extract from the description of the situation a list of all verbs and of all nouns: verbs are candidate business event types and nouns are candidate business object types. Thinking of the life cycle of an object type also allows discovering relevant event types. Conversely, thinking of all objects, things and persons that are involved in an event allows discovering object types. Finally, reasoning on an example by studying a practical situation (i.e. reasoning at the level of object occurrences) can also help to find object and event types. For the above description, applying the first method results in Fig. 3.4.

Example 3.2 The Library: Analysis of the Small Library Case Text

Let us first examine the list of object types. As we consider the enterprise information system as the mirror of the real world, or even the enterprise information system to *be* the virtualised enterprise, LIBRARY and LIBRARY SYSTEM are duplicates: they refer to the same object type. Also there is only one occurrence of this object type (there is only one library in our universe of discourse) and it always exists. We can remove LIBRARY and LIBRARY SYSTEM because there is no relevant creation and ending event for this object type within the given scope: we will not create and end libraries. In the scope defined above, also all possible events that we might specify are related to this object type, for the simple reason that the library *is* the universe of discourse. In general, such an object type will be rejected because it

Candidate business object types	Candidate business event types
library	borrow
member	determine days
book	ask for renewal
privilege	postpone day of return
day	generate email
renewal	return
return	register as lost
person	lose privileges
week	classify
library system	
email	
catalogue	
copy	

Fig. 3.4 Candidate business object types and business event types for the library

is deemed to be superfluous. The candidate object types MEMBER and BOOK are retained because they satisfy the definition of business object type. The object type CATALOGUE is rejected because the catalogue is nothing else than the list of available books. It refers therefore to a *service*, namely, the possibility to consult the list of available books. We will therefore capture this in the next layer as a required service. But, because of this catalogue, we must make a difference between book titles and physical copies of books. We therefore remove the object type BOOK and replace it by the object type TITLE, which is the item in the catalogue of which possibly many copies can be on shelf. And we add the object type COPY to refer to a physical copy of a book. The object type PRIVILEGE is removed because we consider the privileges to be part of the description of a member. Therefore, in this small example, it is modelled as a property of a member rather than as an object type. The object type DAY does not fit the given definition of business object types: it does not exist for a certain period of time only. It has no life cycle and it is not involved in events such as *create_day* and *end_of_day*. The same arguments hold for WEEK. The object types RENEWAL and RETURN are in fact the event types *renew* and *return* and belong as such to the list of business event types. The object type PERSON is removed because we are not interested in any person before he/she has become a member of the library or after he/she has ceased to be a member. The concept of 'person' therefore coincides with the concept of 'member' in our universe. Finally, the candidate object type EMAIL refers to the service of generating an email, rather than to a business object type.

Let us now go through the list of candidate business event types. We have already added the event type *renew*. *Ask for renewal* and *postpone day of return* are synonyms for *renew*. Based on the same arguments used to remove EMAIL from the list of object types, the event type *generate email* should be removed as well (it's an information system service). *Register as lost* expresses the fact that we

Candidate business object types	Candidate business event types
MEMBER	borrow
TITLE	return
COPY	renew
	classify
	change_member_info
	lose
	register_member
	stop_membership
	acquire
	dispose
	create_title
	end_title

Fig. 3.5 Final list of business object types and business event types for the library

would like to be able to register the occurrence of the business event type *lose* and is thus renamed as such. *Determine days* and *lose privileges* refer to the update of the privilege attribute of the object type MEMBER, which gives rise to the event type *change_member_info*. The completeness of the list can be checked by considering the definition of object types. Objects have a duration and are involved in at least two events: one for their creation and one for their ending. In the light of this statement, we should add events for the creation and ending of members, titles and copies, respectively: *register_member* and *stop_membership* (for MEMBER), *create_title* and *end_title* (for TITLE) and *acquire* and *dispose* (for COPY). Indeed, whenever possible, one should try to find a more meaningful name than the defaults *create_object_type* and *end_object_type*. Finally, the event type *classify* denotes the fact that a copy is assigned a location and added to the catalogue. From then on it is available for borrowing. The result is the list of object types and event types of Fig. 3.5.

3.2.3.2 Data Modelling: Constructing an Existence-Dependency Graph

The static or structural aspects of object types can be documented by a UML class diagram. Determining the required associations and the multiplicities of the associations requires a thorough rereading of the text. Nouns that appear together in one sentence are an indication of the existence of an association. The use of singular and plural is often an indication of the desired multiplicities. One will also be frequently confronted with implicit requirements: an association is made explicit in one direction (or even not) and remains implicit in the other direction.

Example 3.2 The Library: Analysis of the Small Library Case Text (continued)

The phrase '*members can borrow books*' indicates the existence of an association between members and books which starts to exist upon the occurrence of a *borrow* event. The verb 'can' implies that it is optional for members to borrow books. The plural *book<u>s</u>* indicates that a member can borrow several books. Not said

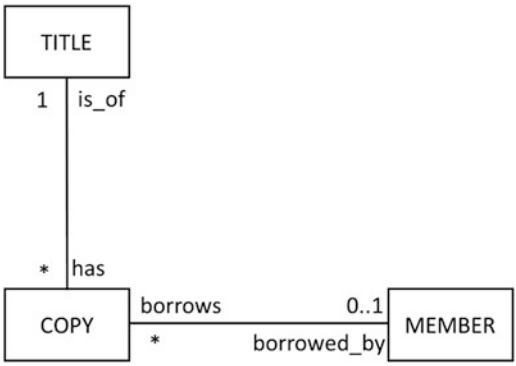


Fig. 3.6 UML class diagram for the library

explicitly is that several books can be borrowed *simultaneously*. If we assume that this is the case, we can conclude that a member can be related to zero, one or more books at one point in time by means of the association 'borrows'. The reverse direction of the association is not stated explicitly in the text. We infer from the text that 'a book can be borrowed by at most one member at one point in time'. Implicit requirements inferred from the text should be validated with the help of domain experts.

The UML class diagram for the library is shown in Fig. 3.6. The cardinalities of the relationships are indicated by means of intervals according to the UML notation (see [67]). The diagram should therefore be read as follows. The association borrows between the two object types MEMBER and COPY specifies that a copy can be borrowed by at most one member at a time (indicated by the interval [0..1] next to MEMBER) and that a member can, at the same time, have zero, one or more copies in loan (which is depicted by the * next to COPY). The relationship type is_of represents the fact that a physical copy is always a copy of one particular title (indicated by the 1 next to TITLE) and that a title can have zero, one or any number of copies (depicted by the * next to COPY).

As will be explained in Chap. 4, in MERODE it is required that in a final domain model, all associations should express existence dependency. An object (called '*dependent object*') is existence dependent on another object (called '*master object*') if the life of the first object is always fully contained in the life of the latter. The is_of relationship indeed expresses existence dependency: once a copy is associated with a title, it can never be associated anymore with another title. However, the borrows relationship does not express existence dependency: the life of a member is not fully contained in the life of the borrowed copy, nor is the life of a copy fully contained in the life of the borrowing member.

In order to obtain a class diagram in which all associations express existence dependency, the association between COPY and MEMBER is instantiated to a new object type that we call LOAN. This LOAN object type captures everything that relates to the time that a member and a copy are associated with each other. In this sense it can be seen as a **contract** object type between MEMBER and COPY. This new LOAN object type is existence dependent on both MEMBER and COPY: a loan always refers

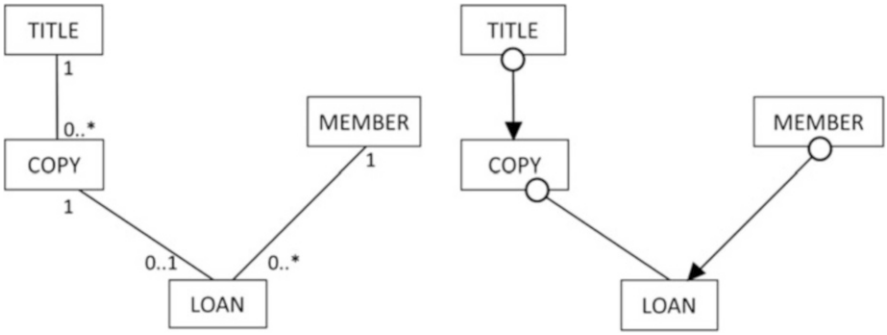


Fig. 3.7 Existence-dependency graph for the library in UML notation (*left*) and MERODE notation (*right*)

to one and the same member and to one and the same copy during its whole life (it can't 'change' from member nor copy during its lifetime) and information from that member or that copy should not be removed as long as loans are registered for that member or for that copy. Figure 3.7 shows on the left the resulting UML class diagram. In MERODE we call this refined UML class diagram an **existence-dependency graph**. As will be explained in Chap. 4, one can use a MERODE-specific notation to draw the existence-dependency graph to make the distinction with a UML class diagram. This notation is shown in the right part of Fig. 3.7.[5]

Notice that the specifications could have been stated such as to contain the noun 'loan' in the first place. For example, the second sentence could have been as follows: '*The privileges of a member determine the duration of the loan'*. In that case, by applying the strategy of considering nouns as candidate object types, the LOAN object type would have been detected right from the start. We can conclude that the existence-dependency principle allows us to discover object types which are relevant but which do not appear in the original formulation of the specifications. This is one of the benefits gained from the existence-dependency requirement, which is the cornerstone of MERODE. Also, requiring all associations to express existence dependency allows organising all the object types of the domain model in a top-down manner, with independent object types at the top and dependent object types at the bottom of the graph.

Figure 3.7 specifies that there can exist many simultaneous copies for the same title, that a member can have several simultaneous loans but that there is at most one loan at one point in time for a single copy. The major benefits of working with existence dependency will be explained in Chap. 4:

1. It provides the class diagram with more precise semantics.
2. It allows enhancing the modelling process by removing ambiguities.

[5] The notation is further explained in Chap. 4.

3. It allows to discover new relevant business objects.
4. It enables improved modelling and consistency checking of the behavioural aspects.
5. As a result of the previous elements, it enables improved possibilities for the automatic transformation of specifications to code.

These benefits are the main reason that MERODE requires to transform a class diagram into an existence-dependency graph.

3.2.3.3 Interaction Modelling: Constructing an Object-Event Table

The dynamic aspects of object types are described by means of two diagrams. In the first place, an object-event table (OET) identifies the relevant business event types and specifies which event types create, modify or end occurrences of a particular object type. It is a tabular representation of associations between business object types and business event types. The OET for the library is depicted in Fig. 3.8. Relevant event types for this administration are those of Fig. 3.5:

borrow = a copy is borrowed.
renew = a loan is renewed.
return = a copy is returned to the library.
change_member_info = information about a member is modified (e.g. his/her privileges).
lose = a copy is registered as lost.
register_member = a person becomes a new member.
stop_membership = a person ceases to be a member.
acquire = a new copy is acquired.
classify = a copy is entered in the catalogue.
dispose = an old copy is disposed of.
create_title = a new title is created in the catalogue.
end_title = a title is removed from the catalogue.

In the OET, we first indicate the ***owner***[6] ***participations***. Participations themselves can be indicated by a 'C', 'M' or 'E' depending on whether an event type creates (C), modifies (M) or ends (E) occurrences of an object type. An object class A is an ***owner class*** for an event type if this object class A is involved in this event type and there is no class which is existence dependent on A and which is also involved in this event type. In assigning owner classes to event types, we put a 'C', 'M' or 'E' at the intersection of the row of the event type and the column of the object type. This leads to the table shown in Fig. 3.8.

[6] See Chap. 5 for a detailed definition of 'owner participation' and 'owner class'.

	TITLE	COPY	LOAN	MEMBER
borrow			O/C	
return			O/E	
renew			O/M	
change_member_info				O/M
lose			O/E	
register_member				O/C
stop_membership				O/E
acquire		O/C		
classify		O/M		
dispose		O/E		
create_title	O/C			
end_title	O/E			

Fig. 3.8 Owner participations for the library

After that, each event which is marked for the owner class is also marked for classes on which the owner class is existence dependent. This is called 'propagation of event types'.[7] The motivation for this propagation is that nothing can happen to a dependent object without it happening at the same time to the master object. For example, if something happens to a loan, e.g. it is renewed, automatically the copy and the member are also involved in this renewal (at least indirectly). For the given OET, it means that event participation is propagated from LOAN to COPY and MEMBER and from COPY to TITLE. This leads to the table of Fig. 3.9. Each entry in this table represents a method in the class corresponding to the column of the entry and is named as the event type corresponding to the column of the entry.[8] Propagating event types along the existence-dependency relation has the advantage that all possible places for information gathering and constraint control are identified, which does not necessarily mean that all marked object-event participations will have a meaningful content. Nevertheless, this automatic identification of possible slots for information gathering and business rules is a second advantage gained by using the existence-dependency principle. For example, in the case at hand, the *borrow* method of the class MEMBER is the right place to count the number of copies a member has in loan and to check a business rule such as 'a member can have at most five copies in loan at the same time'. The *borrow* method of the class COPY is the right place to count the number of times a copy has been borrowed. Finally, the *borrow* method of the class TITLE is the place to count the number of times a title has been borrowed. At implementation time, empty methods can of course be removed to increase efficiency.

[7] See Chap. 5 for a more detailed explanation.

[8] This naming convention applies to the development of new software. For existing software, the cell can be used to link an event type to an existing method of a software component (see Sect. 12.2).

	TITLE	COPY	LOAN	MEMBER
borrow	A/M	A/M	O/C	A/M
return	A/M	A/M	O/E	A/M
renew	A/M	A/M	O/M	A/M
change_member_info				O/M
lose	A/M	A/E	O/E	A/M
register member				O/C
stop membership				O/E
acquire	A/M	O/C		
classify	A/M	O/M		
dispose	A/M	O/E		
create_title	O/C			
end_title	O/E			

Fig. 3.9 Object-event table for the library

3.2.3.4 Life Cycle Modelling: Constructing Finite-State Machines

Events are not allowed to occur in a random order during the life cycle of an object. They are subject to sequence constraints. For example, a copy cannot be returned if it was not borrowed earlier. Actually, the object-event table already specifies some of these sequence constraints. Indeed, for each column, it holds that creating events should always precede modifying and ending events and that ending events cannot be followed by any kind of event. We can thus see in the OET that a LOAN starts with a *borrow* of a copy and is then subject to 0, 1 or possibly many *renew* events and that it is ended by a *return* or a *lose* event.

The OET however only specifies the 'trivial' sequence constraints. That is, an object has to be created before it can be modified and an ending event ends the life cycle of an object such that nothing more should happen to this object afterwards. In some cases, more stringent sequence constraints have to be imposed on events of some object types. For such object types a life cycle diagram is drawn according to the principles of finite-state machines (FSM). This type of diagram specifies life cycle states and transitions from one state to another. Figure 3.10 shows how the basic control structures of sequence, choice and iteration can be expressed with an FSM. In UML, state charts allow for more construct than in basic FSMs. In our approach, we will stick to the basic FSM constructs. Such basic FSM are a well-known kind of automaton that generates a 'regular language'. The equivalent regular language definition is given in the third column of Fig. 3.10. As will become clear in Chap. 6, the theory of automata and their languages will prove useful in checking the behavioural model for consistency.

The methods in the FSMs of TITLE, MEMBER, COPY and LOAN (Fig. 3.11) correspond exactly to those cells for which a C, M or E is drawn in the OET. In a finite-state machine, states are represented as circles and transitions as arrows labelled with the

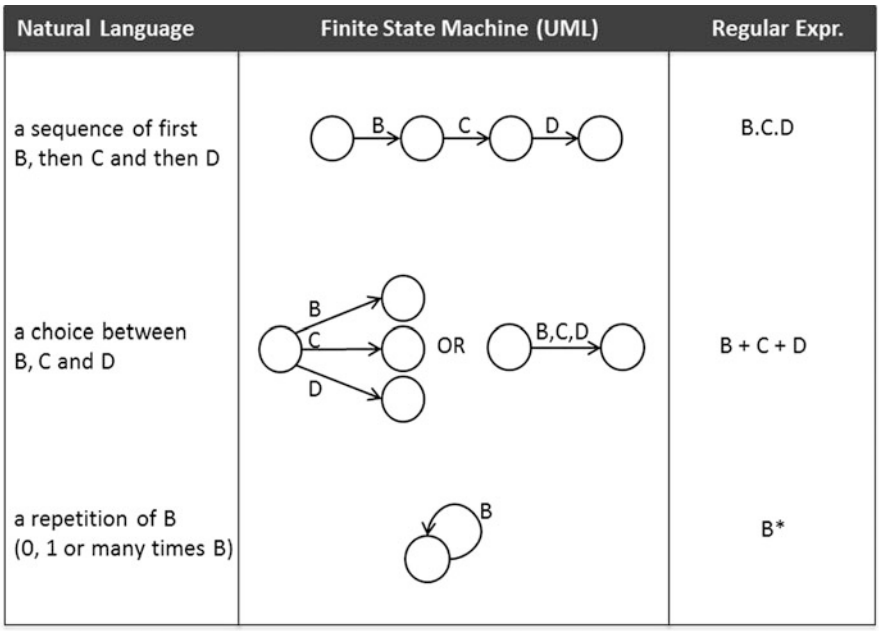


Fig. 3.10 Natural language, FSM and regular expression for sequence, selection and iteration

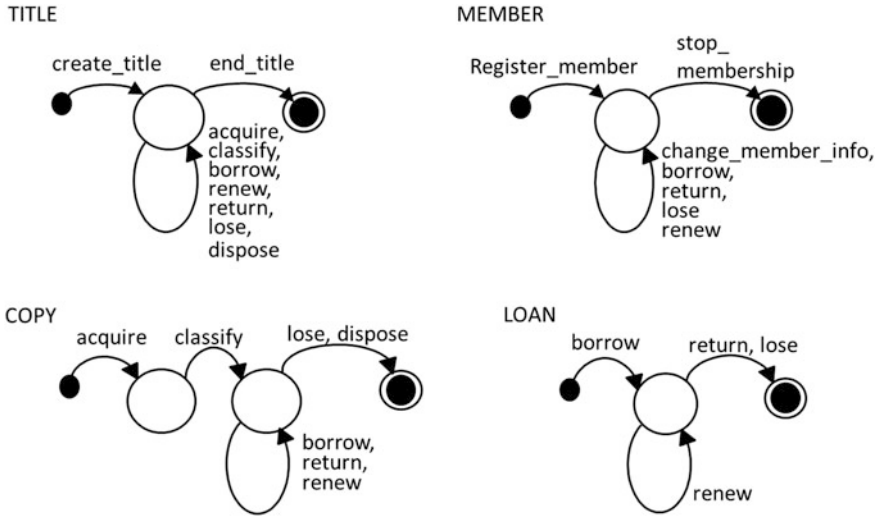


Fig. 3.11 Finite-state machines for TITLE, MEMBER, COPY and LOAN

names of the methods. The initial state is indicated by a black fill and the final state as a circle with a double line and a black fill of the inner circle. A more detailed description of these techniques will be given in Chaps. 5 (OET) and 6 (FSM).

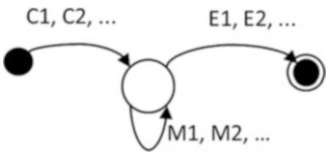


Fig. 3.12 Default life cycle represented as a finite-state machine

Because the object-event table also contains the indication of the type of involvement (create, modify or end), drawing finite-state machines is in fact only necessary for the specification of additional constraints that are not yet specified by the object-event table. Indeed, the type of involvement defines a (trivial) default life cycle which is the same for each object type: an object can start its life in one or more ways (a selection of the C-entries), it can then undergo an arbitrary number of modifications and these events are allowed to happen in any order (iteration of a selection of the M-entries) and finally the life of an object is ended in one way or another (a selection of E-entries). This kind of default life cycle is represented as a finite-state machine in Fig. 3.12. The sequence constraints for TITLE, LOAN and MEMBER represented in Fig. 3.11 are examples of such default life cycles. Drawing the default life cycles is obviously completely superfluous. Only the life cycle of COPY shows nontrivial behaviour (because the M-event *classify* should occur only once in the entire life cycle) and therefore needs to be explicitly specified by means of an FSM.

Notice that the restriction that *borrow* and *return* events need to alternate is not modelled in the diagram of COPY because this is already captured by the combination of the diagram of LOAN and the EDG that specifies that there can be at most one loan for the same copy at one point in time. Since all constraints apply *simultaneously*, there is no need to duplicate such sequence constraints in multiple FSMs (see Sect. 6.3.3). This simplifies the modelling process substantially.

Let us also remark that isolating sequence constraints from the rest of the system lead to a large degree of flexibility. If the constraints change, we only need to redraw a uniquely identifiable diagram.

3.2.3.5 Specification of Attributes and Object-Event Methods

Once the static and dynamic aspects of object types are defined, the domain model is further extended by adding attributes to object types and event types and by specifying what happens to an object type when an event of a particular type occurs. The definition of attributes is required to concretely identify which information about object types is of interest and relevant for the enterprise. Another way to put it is that without attributes, objects remain invisible. Likewise, the definition of a method makes the effect of an event concrete. Moreover, a minimum set of attributes are required to make it possible to prototype a domain model. The result is one class definition for each object type, which contains the set of attributes for that object type, and one method for each event type in which this object type is

```
class MEMBER {
Attributes
        registration-number  // unique identification of person;
        name                 // name of this person;
        address              // address of this person;
        category             // depending on the category, persons
                             // have different privileges;

Methods
...
register member (input-registration-number; input-name; input-address; input-category) {
        // Creates and initialises a new member
                registration-number := input-registration-number
                name := input-name
                address := input-address
                category := input-category
        } // register member
...
} // class member
```

Fig. 3.13 Class definition for MEMBER

involved. We exemplify this by giving the class definition for MEMBER in pseudocode and the event method for *register member* in Fig. 3.13.

Once attributes have been defined, the domain model can also be extended with additional business rules. These rules can take the form of constraints on attribute domains, uniqueness constraints or preconditions for event types (other than the current life cycle state). An example could be a rule that requires a member not to have more than five books simultaneously in loan or a rule that requires the name of a member not to be empty. More details on how to specify such constraints will be given in Chap. 7.

3.2.3.6 Consistency Checking Between the Three Domain Modelling Diagrams

The existence-dependency graph is one of the features that makes MERODE unique compared to other conceptual modelling methods. It is the cornerstone because inter-diagram consistency checking between the three schemes is achieved by ensuring consistency between the existence-dependency graph and each of the other diagrams. By using existence dependency as the key to consistency checking, it is possible to achieve a much better quality of domain model than when solely checking for syntactic accordance between diagrams. Chapter 4 defines the existence-dependency graph (EDG). Chapters 5 and 6 define the object-event table (OET) and the behaviour schema (the sequence constraints), respectively.

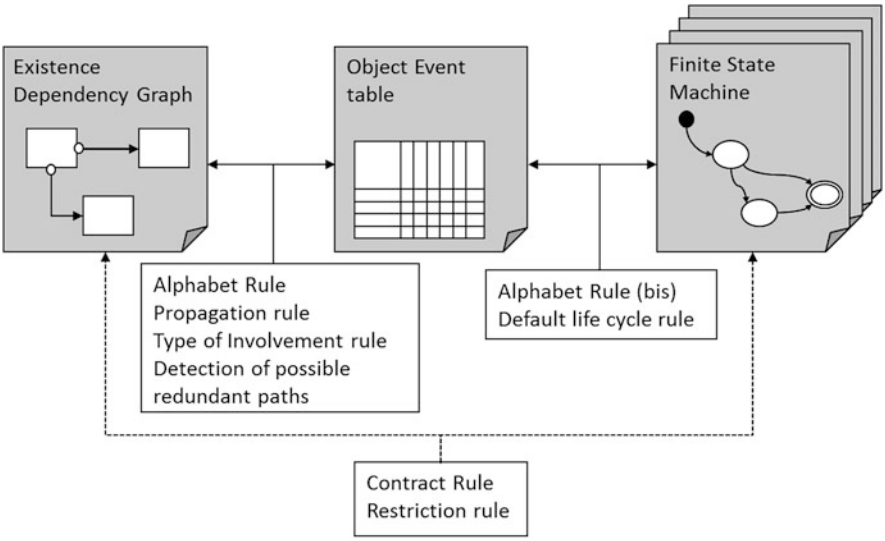


Fig. 3.14 Consistency checking between the three subschemes

For each of these diagrams, it is explained how consistency with the EDG is ensured. In addition, behaviour must also be checked for consistency with the OET. Figure 3.14 gives an overview of all the verification rules that will be explained in the next chapters.

Although this book is mainly about domain modelling, it might be clarifying to take a look at the other steps in the requirements engineering phase (information system service modelling (Sect. 3.3) and business process modelling (Sect. 3.4) and at the implementation phase (Sect. 3.5).

3.3 Information System Service Modelling

The information system service model defines how information system services provide end-users of the information system with input and output facilities. The information system services are designed as a layer on top of the domain model. They are based on two types of facilities:

- *Queries:* information system services can query the value of object attributes and provide the user with output in the form of web-based or to-be-printed reports. We call this type of services *output services.*
- *Business event generation:* information system services can provide input facilities to end-users by recording information from the environment and generating events that are transmitted to the domain model. We call this type of services *input services.*

Information system services can be offered through different channels such as web interfaces, client or server applications and web services. Input services will often be of an interactive nature and combined with output services. Assume, for example, an input service to register a change of address of an existing customer. Such a service can be implemented with a pick-list that allows users searching for the customer whose address is to be modified. Although the registration of an address change is an input service, the pick-list is in fact an output service. Interactive services are therefore often a combination of output and input facilities. Another way to discriminate services is to look at the initiator: services can be initiated by some external agent, typically the user interface, a timer or a business process engine.

Examples of services for the library that are:

1. *Interactively triggered by the user via the user interface:*

   S1 = register a new member (input service).
   S2 = classify a new copy (input service).
   S3 = register a loan (input service).

2. *Triggered by a timer:*

   S4 = for each member who has a loan that is more than a week overdue, generate an email to remind him/her to return the copy (output service).
   S5 = print a list of all copies that should have been returned yesterday (output service).
   S6 = withdraw privileges of a member when this member has a copy that is more than two months overdue (input service).

The information system service model with the six services defined above is shown in Fig. 3.15. Services are always allowed to query the state of objects in order to retrieve information. Here this is depicted by means of an 'access' dependency from the service to the enterprise layer. Generated business events are depicted as 'send' dependencies. Initiators of services are the actors, represented as a stickman for a human actor and as a clock icon for the time-triggered use cases.

Let us, for example, take a closer look at service S3. This (self-) service is initiated upon request by the member, which is indicated by the connection to the stickman 'member'. To validate the loan, the service needs to query the state of the copy that is borrowed and the state of the member. This query is represented as the 'access' dependency to the enterprise layer. The access dependencies could also be drawn to copy and member, but this would clutter the diagram too much. Finally, the service will send the *borrow* event to the business model, which is represented by the 'send' dependency labelled with 'borrow'. Figure 3.15 also demonstrates that services are modular components independent of each other, which can easily be plugged in and out of the domain model. Services never introduce essential changes to the domain model. Information system service modelling is discussed more extensively in Chap. 9.

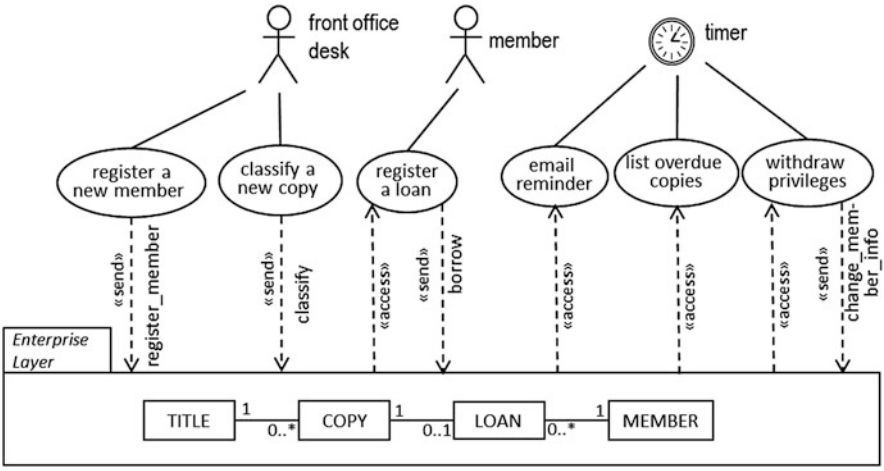


Fig. 3.15 Information system service model for the library

3.4 Business Process Modelling

The topmost layer in our architecture is the business process modelling layer. Business processes encompass much more than the information system as they consider the whole work organisation perspective, which also takes other aspects into account such as manual activities and organisational aspects.

Example 3.3 The Library: Book Acquisition Process

A researcher obtains information about a book via some channel and decides that the book should be acquired by the library. Let us assume that the process starts with this decision. In order to start the acquisition process, the researcher needs to send an email with the details of the book to the acquisition officer in the library. The acquisition officer will verify whether the book is already available in the library. If not, he/she will verify if there are sufficient funds left for the acquisition. If so, he/she orders the book from the editor. This can be done by email, fax, phone or the editor's website. The title of the ordered book is already registered in the library system. When the book has been delivered, the copy is assigned a classification number and put in the glass display at the library entrance. It will stay there for a month and then be moved to the regular shelves. The acquisition officer sends out an email to the researcher to notify him/her of the availability of the book.

This process is shown in Fig. 3.16. Some of the tasks will require the use of an information system service. Output services will be required to look up information, whereas input services will be used to register information in the system. Figure 3.17 gives an overview of the relations between business process tasks, information system services and business events for the given example. From this table it becomes apparent that the business process can be put as a layer on top of

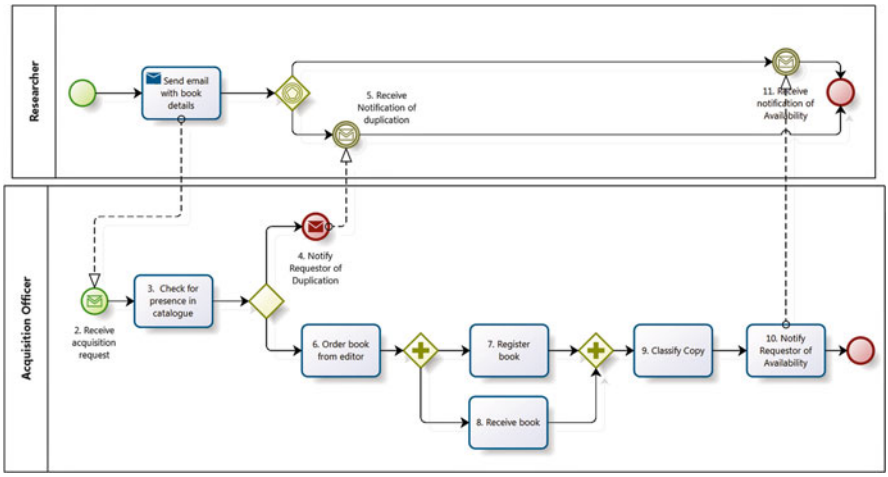


Fig. 3.16 Acquisition business process in the library

Task	Information system service	Triggered business event
1. Send email with book details	*Outside the modelled information system*	
2. Receive acquisition request	*Outside the modelled information system*	
3. Check for presence in catalogue	**Output service** that allows to consult the list of registered titles and copies	
4. Notify requestor of duplication	*Outside the modelled information system*	
5. Receive notification of duplication	*Outside the modelled information system*	
6. Order book	*Outside the modelled information system*	
7. Register book	**Input service** which allows to register the title	cr_title cr_copy
8. Receive book	*Outside the modelled information system*	
9. Classify copy	**Input service** which allows to assign a placement number to the copy	classify
10. Notify requestor of availability	**Output service** that enables to look up recently acquired copies and their details in order to compose the email (manual)	
11. Receive notification of availability	*Outside the modelled information system*	

Fig. 3.17 Relating tasks, information system services and business events

the information system services. Each task is executed either outside the information system or by using the provided information system services.

As business processes define sequences of tasks, which in their turn may invoke input services that trigger business events, the business processes implicitly define

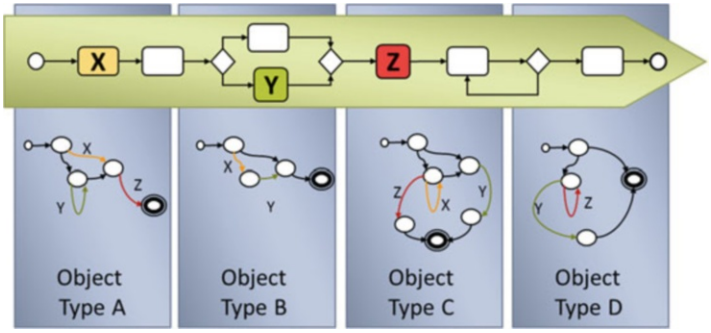


Fig. 3.18 Business process versus object life cycles

sequences on business events. If we compare this form of sequencing to the sequencing defined in the object types' life cycles, then we could say that a business process groups business events according to work organisation (shown as a horizontal dimension in Fig. 3.18) whereas finite-state machines group behaviour on a per object basis (shown as a vertical dimension in Fig. 3.18). Although both dimensions are orthogonal to each other, they interact because a task that triggers an input service may induce state changes in several objects, namely, the objects participating in the triggered events. In Fig. 3.18 this is depicted by means of the colours: the first task labelled Y triggers a business event that induces the transitions labelled Y in each of the four object types.

3.5 Implementation

In a classic approach to software development, the requirements and analysis stages are a largely informal and iterative process. At a certain point in time, specifications are handed over to the implementation team, which proceeds with the technical design, coding and testing. In model-driven development, models are the prime artefact based on which software is developed. Requirements analysis should result in formal 'platform independent models' (PIMs), which are then transformed into 'platform specific models' (PSMs) by means of model-to-model (M2M) transformations and then into code by means of model-to-code (M2C) transformations.

Since MDE is not the core subject of this book, intermediate model-to-model mappings are skipped; Chapter 11 is limited to the demonstration of how models can be directly transformed into code.

The object-event table plays a crucial rule in achieving coordinated updates in the enterprise layer. Interestingly, this principle of coordination can also be used to apply the approach to application and component integration. This is explained in Chap. 12.

3.6 A Note to the Reader

The library example was kept quite small because describing a larger example would have needed the introduction of too many concepts at the same time. Chapters 4, 5 and 6 describe the three modelling techniques in more detail.

Besides a detailed description of the modelling techniques, these chapters also contain the formal (i.e. mathematical) definition of the underlying concepts and a meta-model definition of the diagramming techniques. The formal foundation by means of process algebra enables rigorous quality control. Fortunately, understanding the mathematical background is not a prerequisite for using the techniques correctly. As a result, readers whose main interest is in applying the method to practical problems can safely skip the formal definitions. Likewise, the meta-model definition fosters the clear and unambiguous understanding of the proposed diagramming techniques and is useful for people interested in tool support of model transformations. At the same time, the meta-model provides a nontrivial example of how the method can be applied to define itself. Yet also these parts can safely be skipped by those who focus on the practical use of the method.

Finally, the examples in this book are quite small and are always presented in a linear way. In practice though, a much more iterative approach needs to be taken to modelling, as has been explained in Sect. 2.1.2. Also, as explained at the end of Sect. 1.3.2, the scope of the 'enterprise' may be first limited to, e.g. only some functional domains. This will also lead to an iterative approach in which the scope of the universe of discourse is gradually extended with more functional domains. Each iteration will allow for the development of additional information system services and the support of more business processes.

Part II
Domain Modelling Techniques

Chapter 4
The Existence-Dependency Graph

4.1 Why Existence Dependency?

One of the main tasks in domain modelling is the identification of relevant concepts in the universe of discourse. Typical kinds of concepts are classes, attributes, methods and relationships between classes. In this chapter, we primarily deal with classes and relationships between classes. Business object types are classes that represent a set of similar objects in the universe of discourse of an enterprise. For example, in the universe of discourse of a library, we can identify members and books as business object types. As business objects do not exist independently, but coexist and interact with other business objects, relationship types called 'associations' are used to capture the semantics according to which business objects are related.

The concepts of class and association used in this book are identical to the concepts of class and association in UML or the concept of entity and relationship of the Entity Relationship Model. The UML term 'class' can be used to denote any kind of class in any layer of a system. This conforms to the definition of 'class' in UML: 'A class describes a set of objects that share the same specifications of features, constraints, and semantics'. On the other hand, the term 'business object type' refers specifically to classes in the enterprise layer.

Some of the relationships or associations organise business object types (or classes) into lattice structures. The 'A Kind Of' (or Is-A or generalisation/specialisation) and the 'A Part Of' (or Part-Of or aggregation) relationship are two primary ways of organising business objects. In particular, the generalisation/specialisation lattice is considered crucial to object-oriented conceptual modelling due to its ability to enable reuse of specifications and code. This chapter presents another such relationship, called *existence dependency*, that also organises classes into a lattice structure. How this relates to the 'A Part Of' relation is explained in Sect. 4.3.5 and Chap. 8 addresses the 'A Kind Of' relationship.

© Springer International Publishing Switzerland 2014

M. Snoeck, *Enterprise Information Systems Engineering*, The Enterprise Engineering Series, DOI 10.1007/978-3-319-10145-3_4

The use of existence dependency as primary type of association (next to inheritance) is motivated by four main arguments:

1. It provides the class diagram with more precise semantics.
2. It allows enhancing the modelling process by removing ambiguities.
3. It enables improved modelling and consistency checking of the behavioural aspects.
4. As a result of the previous elements, it enables improved possibilities for the automatic transformation of specifications to code.

Before defining the concept of existence dependency in more details in the next paragraph, we first elaborate on these arguments.

4.1.1 Frozen Versus Modifiable Association Ends in UML

When using UML class diagramming to model the static aspects of the universe of discourse, we are faced with an incompleteness in the commonly used notation that requires clarification to achieve a correct understanding of the domain and to achieve diagram consistency checking. The incompleteness also prevents swift and easy translation to code. Consider the class diagram in Fig. 4.1. It identifies three object types[1] (CUSTOMER, ORDER and SALESPERSON) and two associations: an ORDER *is placed by* a CUSTOMER and *is managed by* a SALESPERSON. Although both associations exhibit exactly the same graphical notation in the UML diagram, there is a substantial difference between the two associations. Whereas the customer of an order remains the same for the whole duration of the order, the salesperson managing the order may change over time. In other words, the association end *is placed by* labelled with a '1' next to customer is not modifiable or frozen, whereas the association end *is managed by* labelled with a '1' (next to salesperson) is modifiable. Figure 4.1 is not incorrect but is incomplete as it does not allow to discern between frozen and modifiable association ends labelled with multiplicity '1'. In order to obtain a model that is truly precise and complete and can be correctly and automatically transformed to code, we need to clarify the difference between modifiable and non-modifiable association ends. Making the model more complete eases for the transition to code but is first and foremost beneficial for the requirements process itself: the modifiable association end may need additional requirements gathering to determine, e.g. under what conditions a salesperson can be reassigned and what the consequences are for the previously assigned salesperson, the new salesperson, the order or even the customer. For example, when the yearly bonus of a salesperson depends on the realised orders, the reassignment of salespeople to orders may have an (undesired) effect on their earnings.

[1] In the remainder of this book, we will follow the convention that object types are written in SMALL CAPS and that association names will be underlined.



Fig. 4.1 Example class diagram

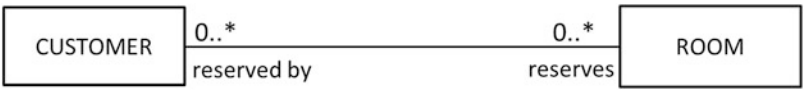


Fig. 4.2 Reservation as an association

4.1.2 Association, AssociationClass or Class?

In certain cases, associations play such an important role that they are or could be treated as business objects in their own right. For example, in a hotel, a reservation can be described as an association between a customer and a room (see Fig. 4.2).

However, a reservation carries semantics of its own. For example, the date of reservation does not belong to the room nor to the customer, but is an attribute of the reservation itself. A reservation also has behavioural features: it can be created, confirmed and terminated by a cancellation (in which case a cancellation fee might be due) or by a 'check-in' by the customer.

Moreover, the same customer may make several reservations for the same room. In that case, the association contains several tuples linking the same customer to the same room. UML states that it is possible to have several links associating the same set of instances but that in such a case, links carry an additional identifier apart from their end values. This is again an indication that associations can be considered as object types, since they can have their own identity.

Associations that carry attributes and operations can be modelled in UML by making use of the concept of 'AssociationClass'. UML defines this concept as follows:

> *A model element that has both association and class properties. An AssociationClass can be seen as an association that also has class properties, or as a class that also has association properties. It not only connects a set of classifiers but also defines a set of features that belong to the relationship itself and not to any of the classifiers.*

Figure 4.3 shows how to model RESERVATION as an AssociationClass.

It finally also is possible to model a reservation simply as an object type in its own right as shown in Fig. 4.4, whereby it should be noted that the association ends linking a reservation to its reserving customer and the reserved room are not modifiable.

In the process of requirements gathering, it is practical to start developing a data model without bothering too much about the choice whether to model something as an association, as an AssociationClass or as a class. Usually one will start initially to model something as an association, and as more requirements are gathered and the

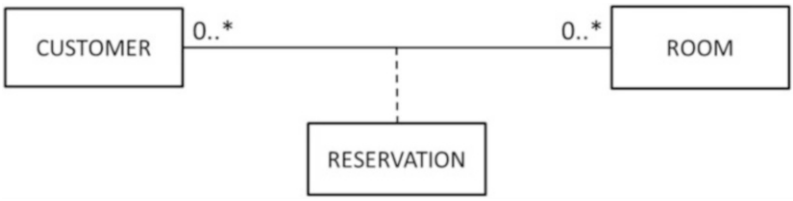


Fig. 4.3 Reservation as an AssociationClass

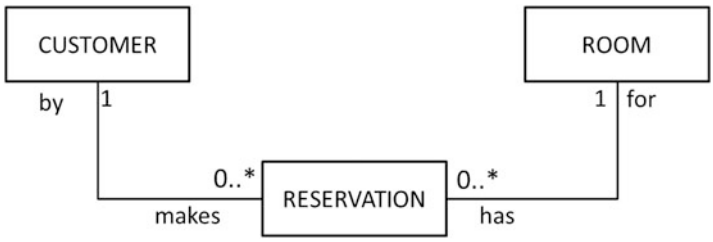


Fig. 4.4 Reservation as a class

requirement engineering step makes apparent that certain associations need to be enriched, associations can be transformed into business object types of their own right. For the final model, one should however end with a model where no doubts are left whether or not to transform certain associations into a class or not. A simple way to do this is to organise classes according to existence dependency. When an association end with multiplicity '1' is frozen (meaning that the link cannot be modified), it carries the semantic of existence dependency: an order is existence dependent on the customer, but not on the salesperson. Similarly, when an association is turned into a class, the new business object type will be existence dependent on the classes associated with the association ends: a reservation is existence dependent of the reserving customer and of the reserved room.

4.1.3 Improved Consistency Checking

A very important motivation for classifying object types according to existence dependency is the ability to allow for quality control at a very high level. In object-oriented conceptual modelling, static and dynamic aspects of the universe of discourse are modelled with equal emphasis. Usually different techniques are used to capture different aspects. But as explained in Sect. 2.2.1, some of these techniques have overlapping semantics. This means that the same aspects may be modelled in more than one scheme. It is clear that some kind of consistency checking between schemes is required to ensure the quality of the conceptual model. This consistency checking can vary from a simple syntactic correspondence

to a full semantic match between diagrams. The concept of existence dependency is based on the notion of 'life' of an object. The life of an object is the time span between the point in time of its creation and the point of time of its ending. As a result, when objects are related through existence dependency, this induces a natural sequencing of creating and ending of objects. In the hotel example, the customer and the room will have to be created before a reservation can be created by that customer for that room, and reversely, due to referential integrity rules, the reservation will have to be terminated before life of the customer or the room can be ended. An existence-dependency diagram is—in spite of its appearance—therefore not a pure data model but also defines a default behavioural model. This facilitates consistency checking with other diagrams intended to capture the behavioural aspects of the domain.

4.1.4 Improved Possibilities for Transformation to Code

As the use of the existence-dependency relationship provides a data model with more precise semantics and enables consistency checking with other diagrams, the use of existence dependency substantially eases the transformation of models to other models or to code. Indeed, ambiguities such as association ends being frozen or modifiable need to be resolved before a model can be transformed to code. In a classical development process, a human programmer will resolve such issues, but in a model-driven approach, there is no human intervention. Depending on the transformation rules, for a model like in Fig. 4.1, both association ends would either be implemented as modifiable or as frozen, as there is no way to make a distinction based on the UML class diagram. Additionally, consistency checking is vital to ensuring transformation to a working and bug-free application.

4.2 The Existence-Dependency Graph

4.2.1 Defining Existence Dependency

Existence dependency (ED for short) is defined at two levels: at the level of object types or classes and at the level of object occurrences. The existence-dependency relation is a partial ordering on objects and object types which is defined as follows:

Definition Let P and Q be object types. P is existence dependent on Q (notation: $P \leftarrow Q$) if and only if the life of each occurrence p of type P is embedded in the life of one single and always the same occurrence q of type Q. p is called the *dependent object* (P is the dependent object type) and is existence dependent on q, called the *master object* (Q is the master object type).

A more informal way of defining existence dependency is as follows:

> If each object of a class P always is associated with minimum one, maximum one and always the same occurrence of class Q, then P is existence dependent on Q.

In terms of life cycles, existence dependency means that the life of the existence-dependent object cannot start before the life of its master. Similarly, the life of an existence-dependent object ends at the latest at the same time that the life of its master ends.

Example 4.1 The Library: Life Spans of Copies and Loans

The life span of a loan of a copy (of a title) is always embedded in the life span of the copy that is on loan. Indeed, we cannot have a loan for a copy if the copy doesn't exist. And the life cycle of the copy cannot end as long as the life cycle of the loan is not ended. In addition, a loan always refers to one and the same copy for the whole time of its existence. Hence the object type LOAN is existence dependent on the object type COPY.

4.2.2 The Existence-Dependency Graph

The existence-dependency relationship defines a partial ordering on object types. The following definition establishes what we understand by a syntactically correct existence-dependency graph.

Definition The existence-dependency graph (EDG) is a relation that connects object types of the domain model and on which we impose the restriction that the graph has to be acyclic:

1. An object type is never existence dependent on itself.
2. The master of an object type can never be a dependent (directly or indirectly) of that same object type.

The two restrictions are motivated as follows:

1a. The life span of an object is always embedded in itself. As a result, one could say that an object is existence dependent on itself. However, in the context of domain modelling, it is the relation between *different* business objects that is of interest. In this sense, saying that an object is existence dependent on itself does not provide us with additional information.

1b. Assume that an object *type* P would be existence dependent on itself, whereby each occurrence of class P depends on the existence of another occurrence of the same class P. It would then be impossible to create occurrences of class P. Indeed, since the life of the existence-dependent object cannot start before the life of its master, creating the existence-dependent object requires the existence of a master object. But this master is in turn existence dependent on another object of the same class, which should already exist before the

master is created. As a result, allowing an object type to be existence dependent on itself creates a problem of circular prerequisites. Hence we define that an object type cannot be existence dependent on itself.
2. Similarly, allowing cycles in the existence-dependency graph leads to circular prerequisites as well. Hence we require the existence-dependency graph to be acyclic.

Example 4.2 The Library: Existence Dependencies

In a library environment, the entity type TITLE refers to the common features of several copies of the same book, that is to say, the combination of authors, title, text, publisher, number of pages and so on. The library can possibly keep zero, one or many physical copies of a single title. Copies are always the physical materialisation of a title. As in addition each copy is the materialisation of exactly one title for its whole life, the object type COPY is existence dependent on the object type TITLE. MEMBERS can borrow COPIES. But members are not existence dependent on copies, nor are copies existence dependent on members. There is however an association between MEMBER and COPY: some copies are borrowed by a member. This association is modelled by the object type LOAN, which is existence dependent on the object types that are involved in the association, namely, COPY and MEMBER. Indeed, a loan always refers to exactly one and always the same member and to exactly one and always the same copy. For this small library example, the set of object types and the existence-dependency relationships are as follows:

There are four business object types:

- LOAN, TITLE, COPY, and MEMBER.
- LOAN is existence dependent of COPY and MEMBER.
- COPY is existence dependent of TITLE.

4.2.3 Multiplicity of Existence Dependency

The existence-dependency graph also defines the multiplicity of the existence-dependency relationship. This multiplicity defines how many occurrences of the dependent object type can be dependent on one master object at one point in time. It can be represented according to the UML notation for multiplicities[2]: *<lower bound>.. <upper bound>*, whereby a '*' signifies an unlimited upper bound and 1 is used as abbreviation for 1..1. When used as adornment to text elements (such as attributes or in the formal definitions below), multiplicities are placed between square brackets. When used as adornment for symbols (such as in class diagrams), the square brackets are omitted.

Definition P [0..1] ← Q if P ← Q and an occurrence q of Q can have at most one existence-dependent occurrence of P at one point in time.

[2] See [67], Sect. 7.3.32 Multiplicity Element (from Kernel).

P [1..1] ← Q if P ← Q and an occurrence q of Q always has exactly one existence-dependent occurrence of P *at one point in time*. The occurrence of P is allowed to change over time.

P [0..*] ← Q if P ← Q and an occurrence q of Q can have more than one existence-dependent occurrence of P *at one point in time*.

P [1..*] ← Q if P ← Q and an occurrence q of Q has at least one but can have more than one existence-dependent occurrence of P *at one point in time*.

When the minimum multiplicity is 1, we say that the association is mandatory for the master. When the minimum multiplicity is 0, we say that the association is optional for the master.

Notice that the clause 'at one point in time' is essential in the definition of the multiplicities. Over time, most objects of a certain type can have many existence-dependent objects of another type. For example, in a library, a copy of a book can be on loan at most once at one point in time but can be borrowed many times as time goes on. So the multiplicity of the association borrowed by from COPY to LOAN is [0..1] at one point in time but [0..*] over time. In the existence-dependency graph, we will always interpret multiplicities at one point in time. Transformation to a database schema may require transforming the multiplicity of [0..1] to [0..*], e.g. to enable keeping track of an archive of past loans.

In case an association expresses existence dependency, the multiplicity of the link from the dependent to the master always is [1..1], and moreover, the link is frozen. In other words, the frozen end indicates the master of the association. This is the consequence of the fact that the dependent object always refers to exactly one *and always the same* master. Hence, one could write P[0..*] ← [1]Q in full instead of the shorthand notation P[0..*] ← Q.

4.2.4 Graphical Representation

For the graphical representation, we have two options. To be aligned with the standard practice of UML notation, we can use that notation with an additional convention to resolve the frozen-modifiable association end issue (see further). The alternative is to use the MERODE proprietary notation, to make a clear distinction between a general UML class diagram, used during requirements gathering and early requirements engineering phases, and the existence-dependency graph, as final result of the requirements engineering phase.

Figure 4.5 shows the UML and MERODE notation side-to-side. In UML we represent an existence-dependency association as a UML association with a multiplicity of one on the side of the master and the appropriate multiplicity on the side of the dependent. As there is no graphical way to indicate that the association end on the side of the master is frozen, we need to set an additional convention on the way the UML notation is used in order to correctly reflect the semantics of existence dependency. To be able to make a distinction between a frozen [1] on the side of the

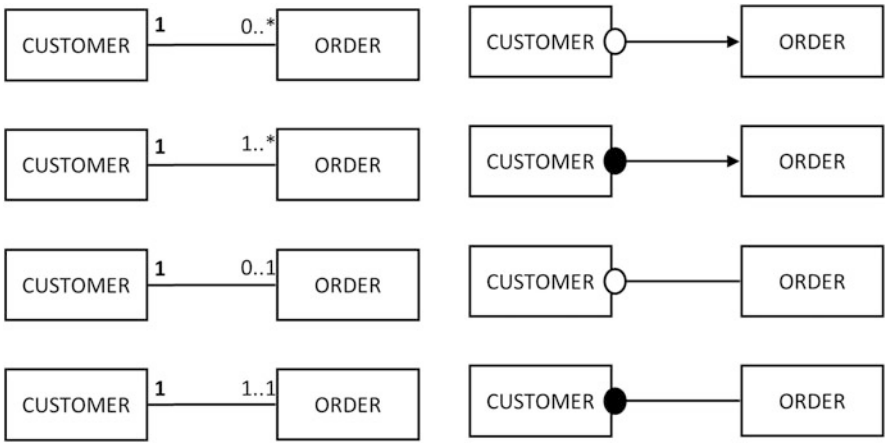


Fig. 4.5 UML and MERODE notation for existence-dependency associations

master and a modifiable [1..1] on the side of the dependent, we use the convention that on the side of the master we always use the abbreviated notation [1], whereas on the side of the dependent, we use the full notation [1..1]. So, A[1..1] ← [1]B means that B is the master and A is dependent. Each object of type A depends on exactly one and always the same object of type B, and each object of type B has exactly 1 object of type A, but this object of type A can be replaced by another object of type A over time.

The MERODE notation is an adaptation of the (old) Bachman notation. On the side of the dependent, it uses a straight line to denote a maximum multiplicity of one and an arrow for a maximum multiplicity of many. The minimum multiplicity is indicated by a dot *on the side of the master*: a black dot when it is mandatory to have a dependent and a white dot when it is optional to have a dependent.

Some people prefer the UML notation because it is better known and it explicitly shows the multiplicity. Other people prefer the MERODE notation, because it is easier to see which object is the master. In this way it eases thinking about the behavioural aspects defined by the model. In this book, when layout permits, we will use both notations: most diagrams will be shown in both notations next to each other.

Example 4.3 The Library: Existence-Dependency Graph

In the library example, at one point in time, a copy can be involved in at most one loan, a member can have several loans going on and a title can have several physical copies.

Figure 4.6 gives a representation of a model for this specification both in UML and in MERODE notation.

An existence dependency can also be represented using the Entity Relationship notation. According to the definitions of P. Chen [10], a weak entity in the Entity Relationship Model as defined has no own identity but derives its identity from another entity it is related to through a weak relationship. The weak entity therefore

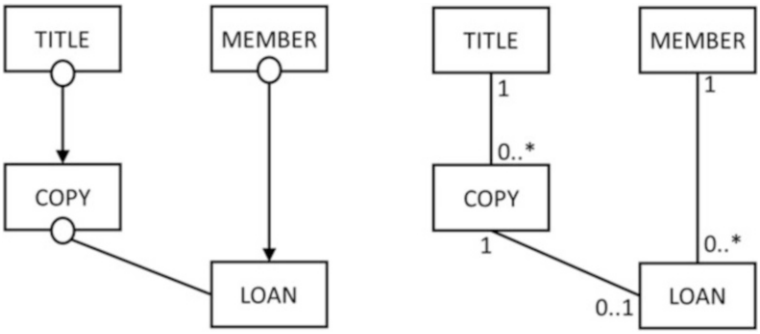


Fig. 4.6 EDG with cardinalities for the library, on the left-hand side according to the UML notation and on the right-hand side according to the MERODE notation

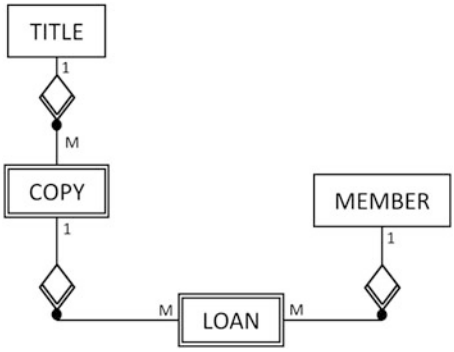


Fig. 4.7 Library model using the ER notation

depends for its existence on the master entity it derives its identity from. The main difference between the concept of weak entities and existence dependency is that in an existence-dependency graph, the dependent object type can have its own identity. Figure 4.7 shows the library model using the ER notation.

4.2.5 Life Cycle Implications of Existence Dependency

Each object type is always involved in at least two event types: one for its creation and one for its ending. As explained before, the concept of existence dependency is based on the notion of 'life' of an object. It therefore induces implications on the order in which master and dependent objects can be created and ended. As a result, an existence-dependency diagram is—in spite of its appearance—therefore not a pure data model but also defines a default behavioural model.

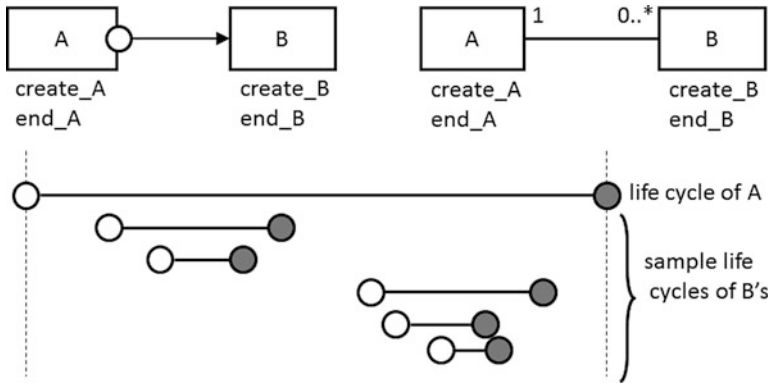


Fig. 4.8 Behavioural implications of a [0..*] dependency

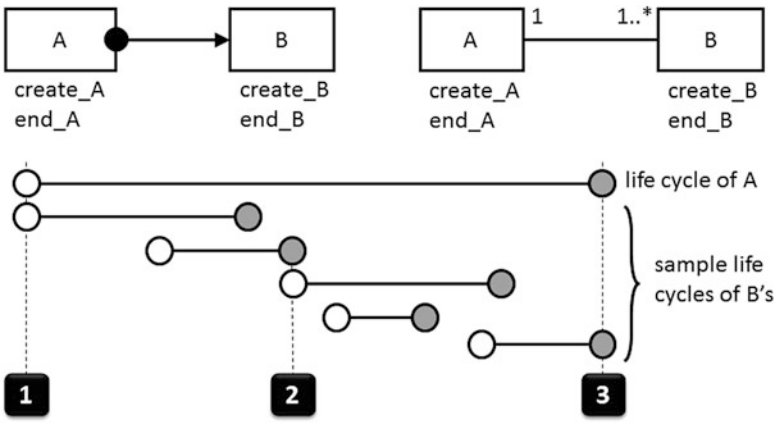


Fig. 4.9 Behavioural implications of a [1..*] dependency

In Chaps. 5 and 6, we will use this to achieve consistency checking with, e.g. life cycle definitions by means of finite-state charts. Figures 4.8, 4.9, 4.10 and 4.11 show the behavioural implications of the four different types of dependencies.

– A [0..*] dependency has no other implications than that the creation of the dependents always happens after the creation of the master and that the ending of the dependents has to happen before the ending of the master. This is similar to the referential integrity rules defined when using foreign keys in a relational model.
– A [1..*] dependency imposes the additional constraints that there should always be a dependent alive. So, when creating the master object, at least one dependent should be created at the same time (see point 1 in Fig. 4.9). When ending the last living dependent, either a new dependent should be created to replace the last

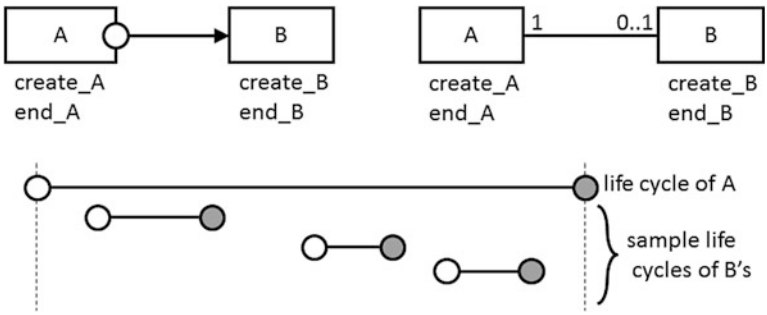


Fig. 4.10 Behavioural implications of a [0..1] dependency

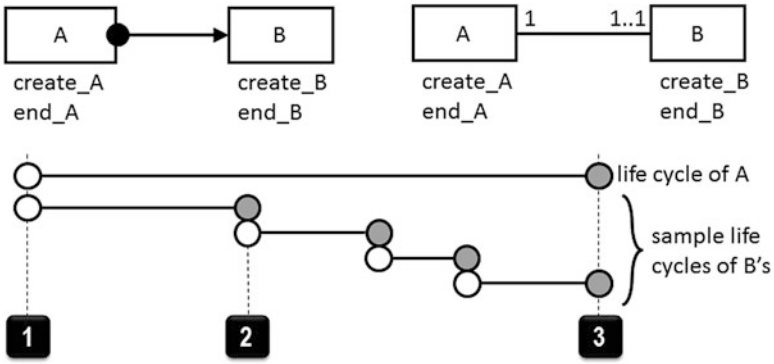


Fig. 4.11 Behavioural implications of a [1..1] dependency

dependent (see point 2 in Fig. 4.9) or the master should be terminated as well (see point 3 in Fig. 4.9).

- A [0..1] dependency imposes the constraints that there should always be at most one dependent alive (cfr. Fig. 4.10).
- A [1..1] dependency implies that there should always be exactly one dependent. So, when creating the master object, a dependent should be created at the same time (see point 1 in Fig. 4.11). When ending the dependent, either a new dependent should be created to replace this dependent (e.g. point 2 in Fig. 4.11) or the master should be terminated as well (see point 3 in Fig. 4.11).

Figure 4.12 gives a graphical representation of the relationship between the life cycle of a copy and those of its related loans and between the life cycle of a member and those of its related loans.

Let us remark that, although the multiplicity captures some behavioural aspects of the domain, it is not sufficient to specify behaviour completely. In addition, multiplicities may need to be updated in the existence-dependency graph when behaviour is considered in more detail during life cycle modelling. Imagine, for

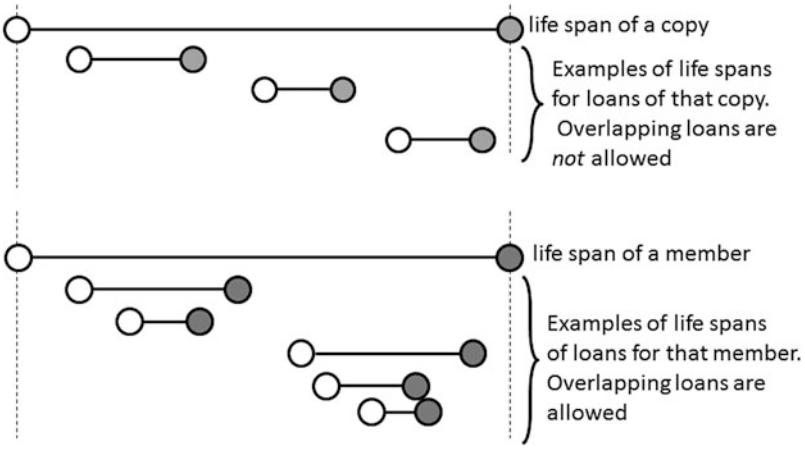


Fig. 4.12 Examples of life spans for a copy, a member and loans

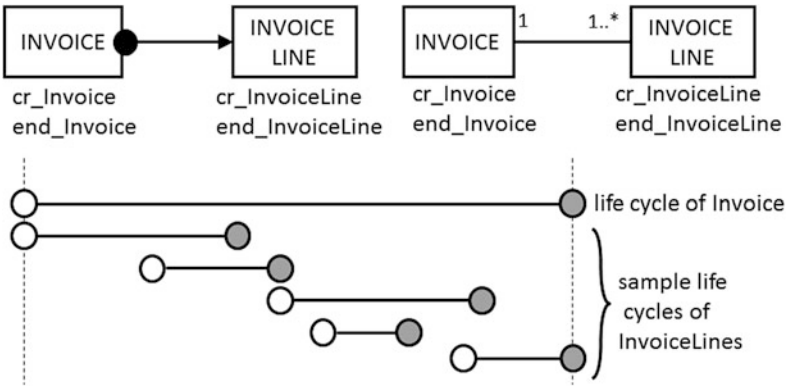


Fig. 4.13 Invoices and invoice lines

example, that the library model is extended with the notion of fines: members are fined when a copy is overdue. The fine is calculated when the copy is returned to the library and the payment of the fine is registered by the event *pay* in the life cycle of a loan. If there can be a space of time between returning the copy and paying the fine, a copy might be involved in more than one loan at a time. It might be that one loan is not ended yet because there is still a fine to pay while the next loan for that copy has already started. In this case the multiplicity of the existence-dependency relationship between COPY and LOAN is 'many'.

A final remark should be given on modelling mandatory associations. Suppose, for example, that during requirements gathering the statement is made that each INVOICE should contain at least one INVOICE LINE. This would give rise to a model as in Fig. 4.13. This EDG implies that as soon as an INVOICE is created, an INVOICE LINE

should be created as well. In such a case, one needs to consider the impact of such a model on work organisation. When modelled with an optional-many association, more flexibility is allowed in the invoice registration process: it is possible to register an invoice and postpone the registration of the invoice line to a later moment in time. Such considerations need to be done case by case. For invoicing, the flexibility of registering the first invoice line later on may not be really necessary, but for ordering it might be interesting to allow registering the header of the order (with invoicing address and delivery address), without demanding to register a first-order line at the same time.

One should also be conscious of the point in time at which something is mandatory. A multiplicity of minimum one holds at any point in time in the life of the participating objects. So, it holds immediately and for the whole duration of their life. If the obligation only holds at certain points in time, but not for the whole duration of the life of the objects, mandatory associations are not suitable and should be replaced by optional associations with minimum multiplicity zero.

Example 4.4 Customer and Orders

Suppose that in a shop, customers are registered when they place an order for the first time. No customers are registered out of the blue. In that case, *at the start of its life*, a customer mandatorily has an order. It would however be wrong to model the association between CUSTOMER and ORDER with a multiplicity of minimum one, as this would imply that a customer needs at least one (living) order *at any point in time* (see Fig. 4.14).

In this case, the correct model has an optional association between customer and order, as shown in Fig. 4.15. At the start of its life (denoted with 1 in Fig. 4.15), the creation of the customer coincides with the creation of the first order. In the middle of its life, it may happen that a customer temporarily has no outstanding orders (period 2 in his/her life). Typically, customers will be ended when they have had no outstanding orders for a longer period of time (denoted 3 in Fig. 4.15).

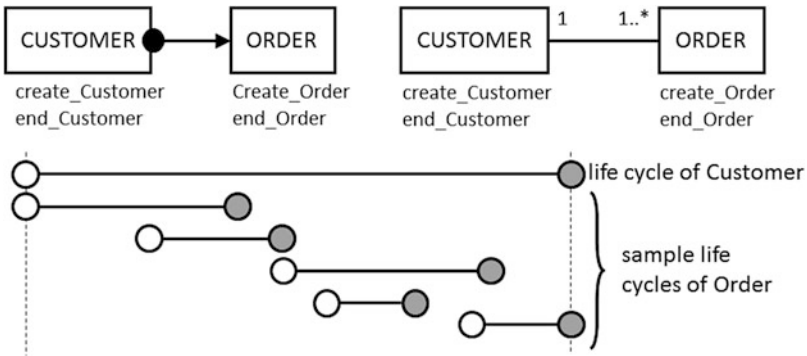


Fig. 4.14 Mandatory association between CUSTOMER and ORDER

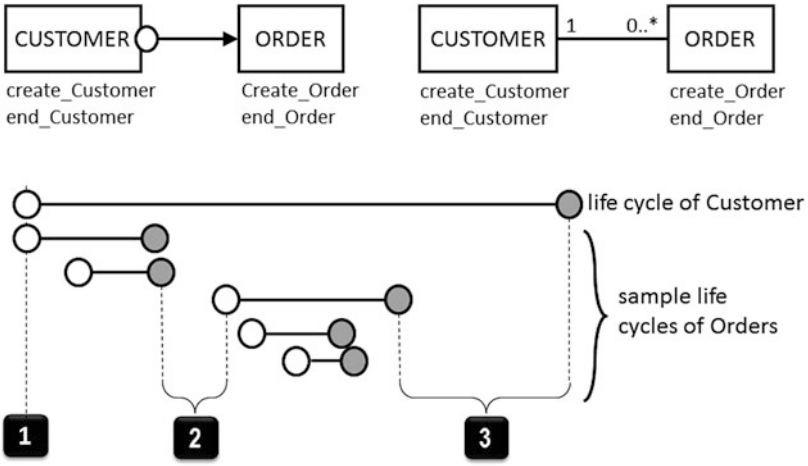


Fig. 4.15 Customer once in a while without (living) orders

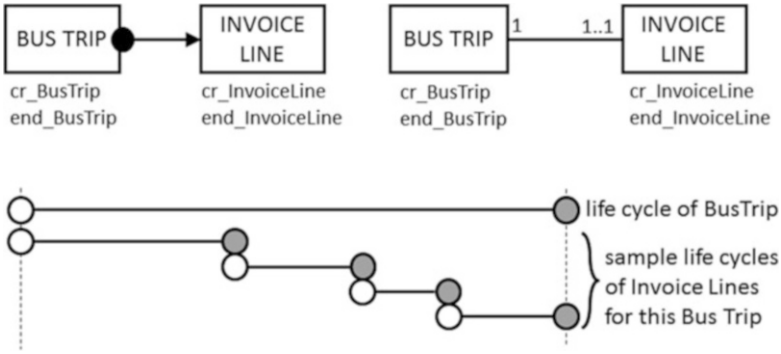


Fig. 4.16 Bus trip and mandatory invoice line

Although an existence-dependency graph implies certain behaviour, it is not intended to, nor can be used to, fully model behaviour. The fine details of behaviour and additional behavioural constraints need to be modelled by means of life cycle diagrams and general constraints.

Example 4.5 Invoicing Bus Trips

Assume that a bus company offers its customers a bus pass allowing post payment of bus use. Customers entering the bus have to validate their pass in the ticketing machine. At the end of each month, the customer receives an invoice on which all the bus trips of that month are invoiced: one trip per invoice line. It is obviously required that each bus trip is invoiced *at some point in time* (meaning that it appears on an invoice line of the month during which the trip was made). It would however be wrong to model this as a mandatory relationship as in Fig. 4.16. This

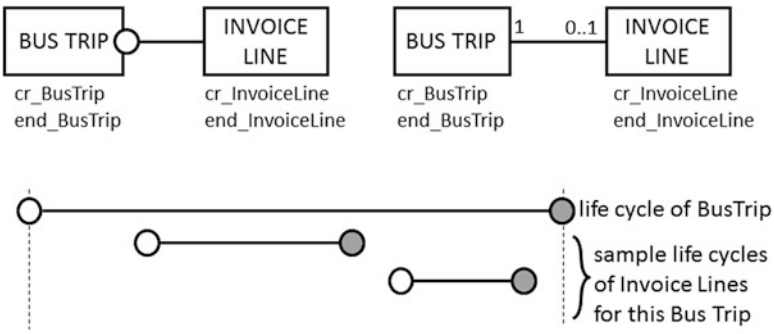


Fig. 4.17 Bus trip and optional invoice line

EDG implies that as soon as a bus trip is registered, an invoice line should be created as well. The actual intended behaviour is that a bus trip has to be invoiced at some point in time: it is mandatory to invoice the bus trip, but it is allowed to do it later on. A better model is therefore the model of Fig. 4.17. This model still allows to invoice the same bus trip several times. But as we will see in a later chapter, we can refine and improve the behavioural model by defining the life cycle of bus trip so as to ensure that invoicing happens and so as to prevent invoicing a second time.

4.3 How to Create an Existence-Dependency Graph

Most conceptual modelling approaches use some sort of class diagram to model static aspects. These class diagrams can be written down using the UML notation, Entity Relationship notation, ORM or any other data modelling language. In the MERODE approach it is the existence-dependency graph that fulfils this purpose: all object types have to be related according to existence dependency. The easiest way to create an existence-dependency graph is to create a conventional class diagram first (written in UML or any similar language) and then analyse the multiplicity and frozen or modifiable nature of each association end. Depending on the outcome, each association is turned into a class or not. In this way, each conventional class diagram can be refined into an existence-dependency graph. At first sight it does not seem so obvious that organising object types according to existence dependency is always possible. By means of a few examples, we will demonstrate that this is in fact pretty straightforward. In the remainder of this section, each example starts from a conventional class diagram written down in UML. The derived EDG is written down according to the MERODE notation. The general procedure to transform a conventional class diagram into an existence-dependency graph is as follows:

- Every UML class is a class in the EDG.
- When a UML association expresses existence dependency, it is an existence-dependency association in the EDG.
- When a UML association does not express existence dependency, it is transformed to a class in the EDG that represents the period in time the participating objects are related.

We review sequentially the frequently used UML class diagramming concepts:

- One to many association that does not express existence dependency
- Many-to-many association
- Unary association
- N-ary association
- Aggregation

4.3.1 UML Class

Each UML class is an object type in the EDG. Reversely, a class in the EDG is not necessarily a class in the UML class diagram. When transforming the UML class diagram to an EDG, additional classes may be added to the EDG, namely, for each association that is transformed into a class (see further).

4.3.2 Binary Association

Imagine a conceptual model with two related object types. Either the relationship between the object types expresses existence dependency or it does not. In the first case, one object type is existence dependent on the other. For example, ORDER and ORDERLINE are related to each other by means of an existence-dependent relationship with multiplicity [0..*]: each order can have zero to many existence-dependent order lines. If the association does not express existence dependency, then that association should be instantiated to a new object type that exists during the time that the relationship exists. For example, assume a domain model where EMPLOYEE is related to DEPARTMENT. The association between EMPLOYEE and DEPARTMENT is a non-existence-dependent one-to-many association: each department has zero to many employees and each employee is assigned to at most one department. However, the existence of an employee does not depend on the existence of a department, and conversely, the existence of a department does not depend on the existence of an employee. Even if the association end on the side of department is mandatory for employees, that is, an employee must always be related to a department, there is no existence dependency because we can assume that an employee can be moved to another department. Since the relationship that models the allocation of employees to departments does not express existence dependency,

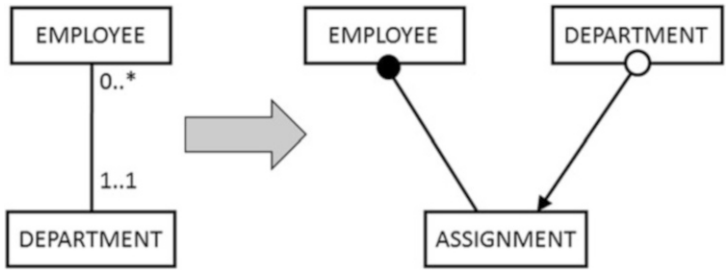


Fig. 4.18 Transformation of UML class diagram to an EDG for a one-to-many association

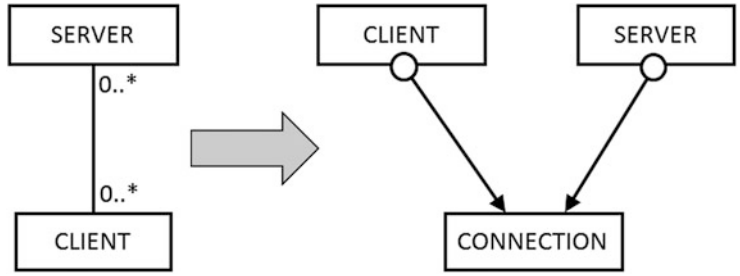


Fig. 4.19 Transformation of UML class diagram to an EDG for a many-to-many association

the existence-dependency graph will contain a third object type ASSIGNMENT that models this relationship. It is a kind of contract object type that models the intersection of the life cycles of the employee and the department. The new object type ASSIGNMENT is existence dependent on both EMPLOYEE and DEPARTMENT, with multiplicity [1..1] for EMPLOYEE and [0..*] for DEPARTMENT. That is, an assignment always refers to exactly one and the same employee, and since an employee can be allocated to at most one department at the same time, each employee is referred to by at most one assignment object at the same time. An assignment always refers to exactly one and always the same department, and a department can be referred to by many assignments at one point in time, namely, one per employee assigned to that department. Figure 4.18 shows the UML class diagram and the resulting existence-dependency graph using the MERODE notation.

Notice that with this conversion of an association to an existence-dependent object type, we assume that relationships are unchangeable: when a link is changed, a new association instance is created. For example, if an employee is assigned to a new department, this means that the old assignment object ends and a new assignment object is created.

The same reasoning applies to one-to-one and many-to-many associations. For example, if in a network management model client computers can be connected to many server computers and server computers serve many client computers, the connection must be modelled as a separate object type. This is illustrated in Fig. 4.19.

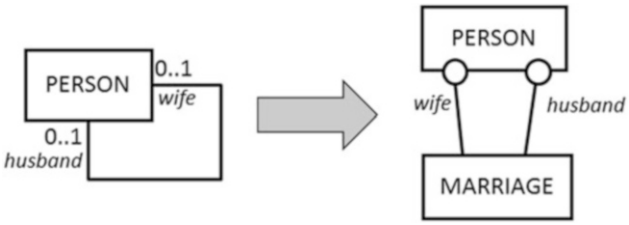


Fig. 4.20 Transformation of UML class diagram to an EDG for a unary association (marriage)

4.3.3 Unary Association

Unary associations never express existence dependency, because an object type cannot be existence dependent on itself. Therefore, unary associations are always converted to an existence-dependent object type. For example, if marriage is modelled as a unary association from person to person, the existence-dependency graph will contain two object types, namely, PERSON and MARRIAGE. As the existence of a marriage depends both on the existence of a husband and on the existence of a wife, there are two existence-dependency associations between MARRIAGE and PERSON (see Fig. 4.20). Generally speaking, each time we transform a unary association into an existence-dependent object type, we will have two existence-dependency relations between the same master and dependent. To be able to discern between these two associations, they should be given two different names, e.g. according to the role the master plays in the relation. In this example, a person can play [0..1] times the role of wife and [0..1] times the role of husband.[3]

Another example is a model for an organisational hierarchy of departments: the hierarchy relationship should be modelled as a separate object type (Fig. 4.21). The left existence-dependency association has a cardinality of [0..1] because a department can play the role of subdepartment at most once: it is subdepartment of at most one department. The association on the right has cardinality [0..*]: a department does not play the role of head department when it is at the bottom of the hierarchy. Higher up in the hierarchy, a department plays the role of head department as many times as it has subdepartments.

[3] By making use of constraints (see Chap. 7), we can additionally impose that these two roles are further constrained based on the gender of the person.

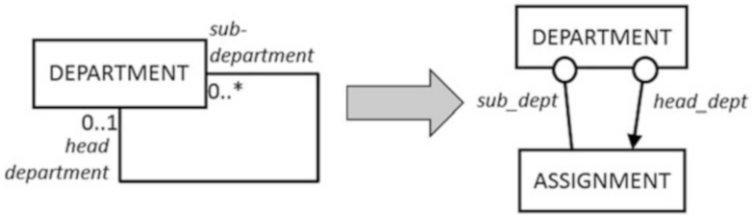


Fig. 4.21 Transformation of UML class diagram to an EDG for a unary association (subdepartment of)

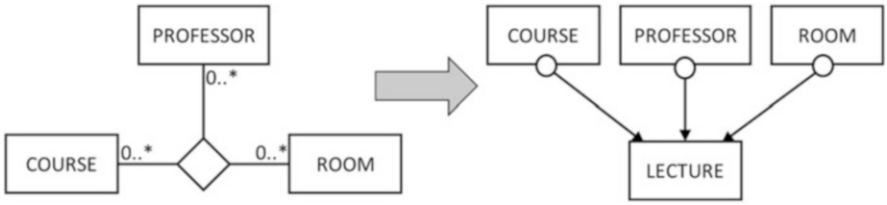


Fig. 4.22 Transformation of UML class diagram to an EDG for an N-ary association (lectures at the university)

4.3.4 N-ary Associations

Associations with an arity[4] larger than 2 will also be converted to new classes. For example, assume a university model where professors teach courses in a particular classroom. This can be modelled as an association with arity 3 in a UML class diagram. In the existence-dependency graph, the association is modelled as a class LECTURE existence dependent of PROFESSOR, ROOM and COURSE (see Fig. 4.22).

4.3.5 Aggregation

Although the Part-Of relationship is essentially an association between object types just as any other association, many designers of modelling languages have estimated that it deserves special attention and notation [11, 23, 40, 67, 77, 79, 99]. This is probably due to the fact that the notion of 'Part-Of' embodies some aspects of existence dependency and propagation of events [77]. However, existence dependency and the Part-Of relation are no equivalent concepts: some Part-Of relations

[4] Cf. http://en.wikipedia.org/wiki/Arity: 'In logic, mathematics, and computer science, the arity of a function or operation is the number of arguments or operands that the function takes. The arity of a relation is the number of domains in the corresponding Cartesian product. The term springs from such words as unary, binary, ternary, etc.'

are existence dependent and some are not. For example, wheels are part of a car, but if a car is disassembled, the wheels still exist as objects. Order lines, on the contrary, usually cannot exist independent from the order of which they are a part. If the parts are not existence dependent on the aggregate, propagation of events is not straightforward. Disassembling a car ends the life of a car-object but does not end the life of the constituent parts. On the other hand, deleting an order implies the deletion of the order lines.

UML has two types of aggregation: shared and composite. The UML semantics guide defines that for a composite aggregation, the composite object has responsibility for the existence and storage of the composed objects (parts). Although this is an implementation-oriented view, one can assume that the intended semantics are those of existence dependency. In the semantics guide of UML, the precise semantics of shared aggregation are however left open as variation point: they are assumed to vary by application area and modeller.

One of the reasons the definition of aggregation is left open in UML is that, as pointed out in [15], there is no clear consensus on the exact semantics of the Part-Of relation. The use of aggregation is not clear-cut and the semantics of the Part-Of relationship are not precisely defined. It is not always clear whether parts are existence dependent on the aggregation or not, when and which operations should be propagated and whether the Part-Of relation is transitive or not [6, 11, 12, 40, 77, 79]. Sometimes the definitions in different methods contradict each other.

The existence-dependency relation can be used as a valuable alternative for the Part-Of relation. A Part-Of relation with existence-dependent parts can simply be replaced by existence dependency: in case of existence-dependent components, the existence-dependency relation is identical to the Part-Of relation. In case of non-existence-dependent parts, we can model the involvement of the part in the aggregate as a separate object type. This new object type is a kind of *contract* between the part and the aggregate for the duration of the part-of relationship. As a result, we do not really need a special notation for the Part-Of relation. Figure 4.23 illustrates the first case with an example. The Part-Of relation is drawn according to the notation of UML with a black diamond to denote composite aggregation.

Example 4.6 Aggregation: Kitchen Furniture Set

Figure 4.24 gives an example of non-existence-dependent parts. Let us assume that a KITCHEN FURNITURE SET is configuration assembled from a set of pieces. The UML diagram states that a KITCHEN FURNITURE SET consists of zero to many KITCHEN CABINETS; exactly one KITCHEN SINK, COOKING PLATE and REFRIGERATOR; zero to one DISH WASHER; and zero to one OVEN. Each component can exist without being part of a KITCHEN FURNITURE SET, indicated by the optional cardinality next to KITCHEN FURNITURE SET. In the equivalent EDG, the multiplicities express the fact that each component can be in use by at most one KITCHEN FURNITURE SET at one point in time and that a KITCHEN FURNITURE SET consists of optional-many KITCHEN CABINETS; mandatorily one KITCHEN SINK, COOKING PLATE and REFRIGERATOR; zero to one DISH WASHER; and zero to one OVEN. Reuse of components is possible. This is expressed in the EDG by having the '..._USE' objects that model the period in time that a

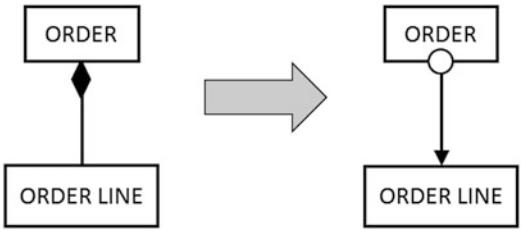


Fig. 4.23 From composite aggregation to existence-dependent parts: UML class diagram and EDG

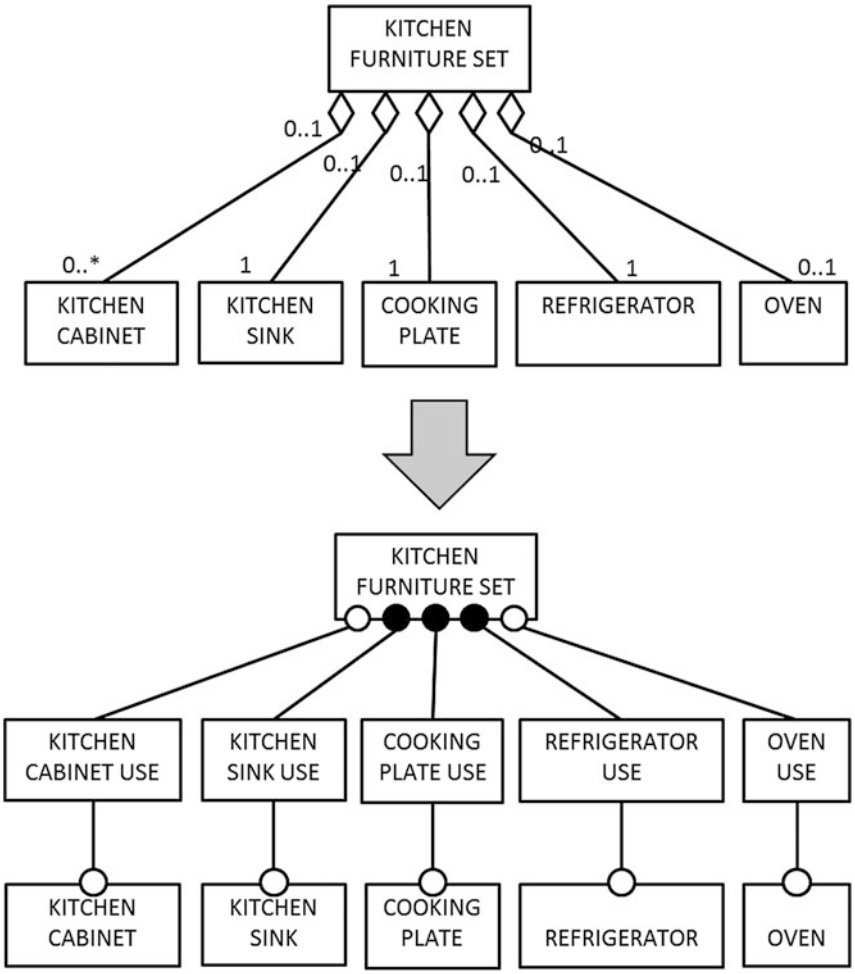


Fig. 4.24 From aggregation to non-existence-dependent parts: UML class diagram and EDG

component has been used as part of a KITCHEN FURNITURE SET. A . . ._USE object will start to exist when a component is added to a KITCHEN FURNITURE SET and will cease to exist when a component is removed from a KITCHEN FURNITURE SET.

4.3.6 Describing Versus Shaping the Real World

A final consideration is that domain models serve both the purpose to understand and to design an enterprise. When using domain modelling for the purpose of 'understanding' an organisation, a domain model is often referred to as a 'real-world' model. In such case, we can say that the domain model looks 'backward' to describe the as-is situation. On the other hand, domain modelling is not only about capturing reality but also about shaping reality of an enterprise when it is used for design purposes. In such case, we can say that the domain model looks 'forward' to design the to-be situation.

Example 4.7 Salesperson and Order Management

Let us consider the example of Fig. 4.1. An order is existence dependent on a customer, but it is not existence dependent on a salesperson as the salesperson can leave the company or receive another task within the company. So far we are capturing reality. But when we define the intermediate-dependent class ORDER MANAGEMENT ASSIGNMENT (see Fig. 4.25), we can reflect on how we would like the sales department to operate. We could, for example, decide to allow at most one order manager per order. On the other hand, we could also decide to allow for teamwork. In case however the salary of a salesperson is augmented with a bonus depending on the total amount of realised orders, an additional reflection is required on how to assign shares to different order managers managing the same order, both in the case of single and team assignments. In case of single assignments, the impact of the handover to another salesperson on the bonus assignment needs to be determined, and in the case

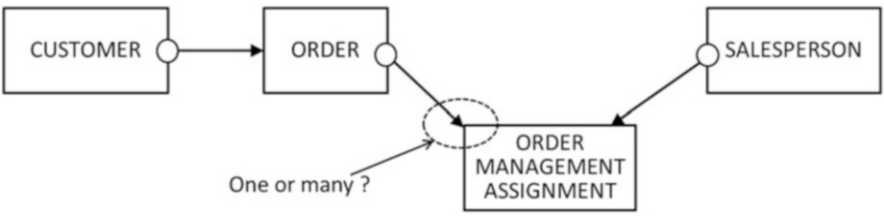


Fig. 4.25 Order management

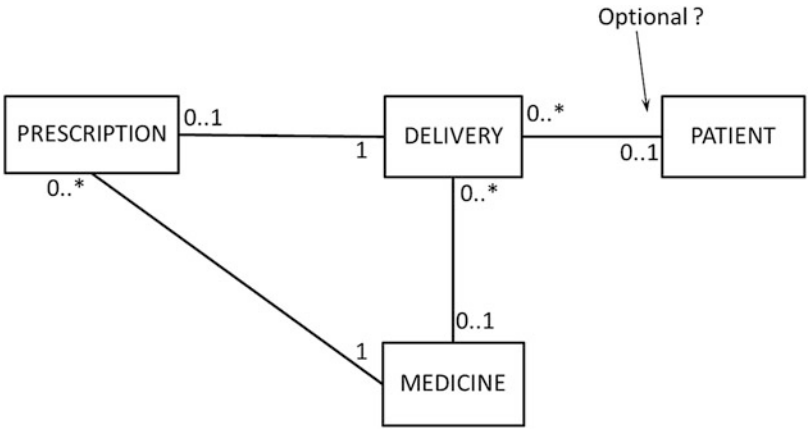


Fig. 4.26 A partial domain model for a pharmacy

of team assignment, how to handle the sharing of the bonus among the team members needs to be decided upon.

Example 4.8 Pharmacy

Assume the universe of a pharmacy. Medicines are delivered to patients, sometimes free and sometimes on prescription. When discussing this example with people unfamiliar with modelling, they will assert that each delivery is mandatorily related to a person. Indeed, in the real world, no medicine will leave the pharmacy, unless there is a person requesting that medicine. However, a domain model is meant to become the core of a management information system. It is therefore biased by an information needs perspective. In this case, the question to ask is whether we really want to know for each delivery to whom it was delivered (Fig. 4.26). If yes, this means we have to register every single person that wants to buy something in our pharmacy, even if something like a moistening cream, a pacifier or some throat tablets. In such a case, a delivery can be modelled as existence dependent of patient. If we would like to allow for 'anonymous' sales, then delivery is *not* existence dependent of patient.

Hence, although in the real world each delivery is to a person, in the domain model, we shape our reality by registering only for some of the deliveries to whom they were. It is up to the business people to decide which deliveries can be anonymous or not and whether or not it would be interesting to make use of fidelity cards to reduce the number of anonymous sales.

4.4 Some Final Considerations

This chapter has presented a classification principle for object types, called existence dependency. The main advantages are that we clarify the differences between frozen and modifiable association ends, that we avoid the use of the concept of 'association as a class' and hence that we will benefit from improved possibilities for consistency checking and for code generation.

By de facto modelling non-existence-dependent relationship types as new classes, the number of classes in a model will increase, but the logical complexity of individual classes will decrease. Overall, this can be considered as a positive effect because volume is easier to deal with than logical complexity. Of course, at implementation time, classes can be merged. In the example of the employees and departments (Fig. 4.18), the object type ASSIGNMENT resulting from the relationship type that models the assignment of employees to a department could well be merged in the object type EMPLOYEE as a foreign key to DEPARTMENT (see Sect. 11.2.2.1). As such efficiency improvements can have certain negative impacts on maintainability, such decisions are to be taken at implementation time when compromises between efficiency and maintainability are considered.

Proposing a conceptual modelling approach where object types can only be classified according to generalisation/specialisation and existence dependency is in contrast with the trend to provide analysts with techniques with vast semantic richness and as much expressiveness as possible. However, we should be aware that semantically rich techniques may be much more difficult to use. In addition, they are difficult to define in a perfectly unambiguous way. As a result, schemes built according to semantically rich techniques often need an accompanying interpretation to be understandable by users, and what is worse, the interpretation of a single model may vary from person to person. With existence dependency, users have a simple technique at hand, the application of which is so clear-cut and unambiguous that discussions about possible interpretations become superfluous. Moreover, there is no loss of expressivity.

During the creative process of developing a domain model, it might be convenient to use a less constraining approach that allows for more types of associations between objects. However, the final model should be expressed in terms of object types, existence dependency and generalisation/specialisation only. Only in this way it is possible to ensure the syntactical and semantic integrity of domain models.

4.5 Formal Definition of the Existence-Dependency Graph

Informal definitions have the disadvantage that often different interpretations are possible. For this reason, at the end of each chapter, we will include a formal definition of the modelling techniques as initially published in [82–84]. Fortunately,

understanding these formal definitions is not a prerequisite for applying the method correctly. Hence, readers who are more interested in the practical application than in the formal foundations of the method can safely skip these sections. The full formal definition can be found in [82, 84].

Let $\mathcal{M}$ be the set of object types in the conceptual schema. The existence-dependency graph (EDG) is a relation $\leftarrow$ which is a bag[5] over $\mathcal{M} \times \mathcal{M}$ such that $\leftarrow$ satisfies the following restrictions:

1. An object type is never existence dependent on itself:

$$\forall P \in \mathcal{M}: \quad (P, P) \notin \leftarrow$$

2. Existence dependency is acyclic. This means that:

$$\forall n \in \mathbb{N},\ n \geq 2, \forall P_1, P_2, \ldots, P_n \in \mathcal{M}:$$
$$(P_1, P_2), (P_2, P_3), \ldots, (P_{n-1}, P_n) \in \ \leftarrow\!\!\Rightarrow (P_n, P_1) \notin \leftarrow 1$$

$\twoheadleftarrow$ is the non-reflexive transitive closure of $\leftarrow$.

$\twoheadleftarrow \subseteq \mathcal{M} \times \mathcal{M}$ such that:

1. $\forall$ P, Q $\in$ $\mathcal{M}$: (P,Q) $\in \leftarrow \Rightarrow$ (P,Q) $\in \twoheadleftarrow$.
2. $\forall$ P, Q, R $\in$ $\mathcal{M}$: (P,Q) $\in \leftarrow$ and (Q,R) $\in \twoheadleftarrow \Rightarrow$ (P,R) $\in \twoheadleftarrow$

For the library model of Fig. 4.6, this amounts to:

$\mathcal{M}$ = {LOAN, TITLE, MEMBER, COPY}
LOAN $\leftarrow$ COPY, LOAN $\leftarrow$ MEMBER, COPY $\leftarrow$ TITLE

4.6 Meta-Model

A meta-model is a conceptual model of a modelling technique. Meta-models are used to provide a (more) formal definition of a modelling technique. At the same time, meta-models provide the basis for developing tool support. At the end of each chapter, we will also include the meta-model definition of the modelling approach defined so far.

Figure 4.27 shows the meta-model for the existence-dependency graph both in MERODE and in UML notation. The EDG consists of object types which are

[5] Bags can contain the same element more than once (as opposed to sets).

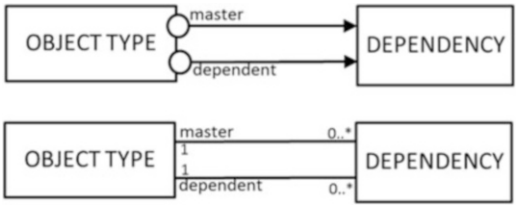


Fig. 4.27 Meta-model for the existence-dependency graph both in MERODE and in UML notation

related through existence-dependency relations. Each dependency depends for its existence on two object types: a master object type and a dependent object type. Each object type can play zero to many times the role of master and zero to many times the role of dependent.

Chapter 5
Object Interaction

The dynamic aspects of a system are defined by the combination of the dynamic behaviour of the individual components of the system and the way components interact. In MERODE, these aspects are respectively modelled by means of sequence constraints and the object-event table. This chapter deals with modelling object interaction by means of the object-event table. Sequence constraints are the subject of Chap. 6.

5.1 What Is a Business Event?

In UML, behavioural aspects associated to objects are modelled as dynamic features of the class the objects belong to. The dynamic features are also called 'methods'.[1] Objects will interact with each other by having one object's methods call another object's features. This principle is called 'message passing'. This rather fine-grained way of modelling a system's behaviour is not so practical for domain modelling for two main reasons.

First of all, when considered from an information systems point of view, a domain model only defines part of the object types of the overall system. Apart from the enterprise layer, a complete information system will consist of at least an additional information system services layer, containing classes that embody application logic and user interface logic. Possibly, there can even be a business process layer on top of this. The overall system behaviour will consist of message exchanges across all these layers. Typically, an interactive service will start in a user interface object, which will call upon an application logic object, which will in turn call upon a business object. For example, in a banking application, a user interface object showing an overview of a customer's accounts will call upon an

[1] Static features are commonly called 'attributes', cfr. Chap. 7.

© Springer International Publishing Switzerland 2014

M. Snoeck, *Enterprise Information Systems Engineering*, The Enterprise Engineering Series, DOI 10.1007/978-3-319-10145-3_5

application object that will retrieve all of the customer's accounts and for each account retrieve the balance by invoking the features of the business class ACCOUNT. In practice, a complete interaction scenario will rarely involve only business objects; it will also involve objects in the outer layers. It is therefore difficult and not advisable to design interaction scenarios based on business object types only. So, in a domain model, we can only define incomplete interaction scenarios by means of message passing.

A second reason is that the design of interaction scenarios is a design activity rather than a requirements engineering activity. Interaction scenarios express the way that activities involving multiple objects are coordinated. Coordination of activities across multiple participants addresses the 'how to' question and can be designed in many different ways. At requirements engineering level, we are primarily concerned with the question of 'what' happens and which business objects are involved. In other words, modelling behavioural aspects right away by means of interaction scenarios is at a too detailed level. Before developing interaction scenarios, we need to model behavioural aspects of the enterprise at a higher level of abstraction.

In enterprise modelling, modelling behavioural aspects is—among others—performed by defining business processes. Business processes describe the way an enterprise functions, mainly from a perspective of work organisation. The finest level of granularity in business processes is called activity or task. A task represents a unit of work handled by a single actor at a single moment of time. The tasks that compose business processes also involve business objects: the tasks in a business process will manipulate objects such as customers, products, orders, bank accounts and so on. The work organisation and hence the business processes tend to change frequently and are therefore placed in the outer layer of an information system architecture. The kernel layer (the enterprise layer) needs to offer the building blocks required to support the ever changing business process layer. The principal building blocks are the business objects and their dynamic and static features. As the level of granularity of features of business object types is low, the connection between tasks (positioned in the outmost layer) and features of business objects in the (innermost) enterprise layer spans a large difference in granularity level. The rather coarse-grained tasks have to be mapped on fine-grained business object features (see Fig. 5.1). This holds the danger of intertwining the coordination of the updates of business objects with task coordination aspects. The mapping of tasks to business object features can be eased by offering units of behaviour at a more coarse level of granularity than the individual features of a class (cfr. Fig. 5.2). We call these more coarse level units 'business events'. With such approach, the business event is responsible for the coordinated update of business objects, whereas the business process takes care of coordination of tasks.

As the notion of 'event' is used in many different ways and can carry very different semantics depending on the method in the context of which it is used, we briefly review the different potential meanings of an event [34]:

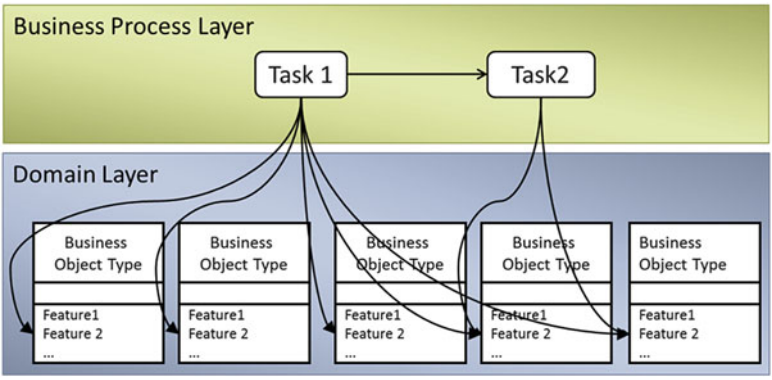


Fig. 5.1 Implementing tasks in the BP layer by directly invoking features of business objects

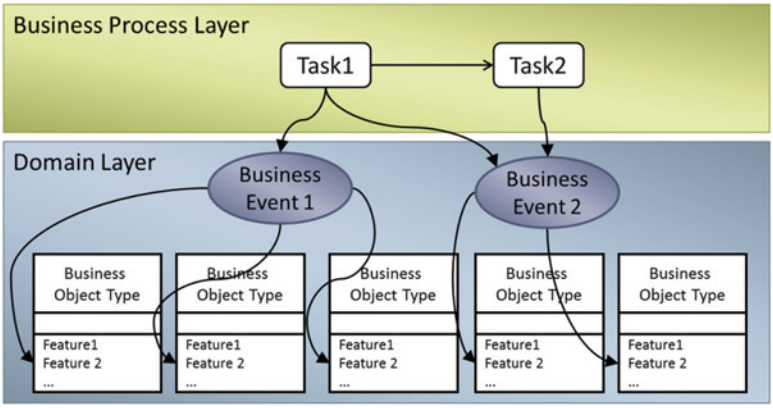


Fig. 5.2 Implementing tasks in the BP layer via intermediate business events

- An event can be used to denote *something that happened*, e.g. Kim Clijsters won from Venus Williams.
- An event can be used to denote *something that happens*: e.g. I'm withdrawing money with an ATM or there's a band performing at Tomorrowland.
- An event can be used to denote *a concept*, e.g. a payment or a festival.
- An event can be used to denote *a request* to register a real-world event in the IS, e.g. scanning a book to request the borrowing of that book. It should be noticed that a request can be refused.

So, when defining the concept of an event, one should discern between a notification of something that happened in the past, an activity that happens at this very moment, a concept and a request. All these concepts are linked to each other. For example, when I pay some acquisition with my VISA card (event as something that happens now; an activity), once the activity has been performed

successfully, the shop's counter system may be notified that an acquisition has been paid (event as a notification). This may be stored as a payment (a concept) in the shop's database. At the same time, the initiation of the payment happens by entering my card in the machine: by entering the card, I request a payment to be performed (and this request can be refused if my pin code is wrong or when the balance of my account doesn't cover my expenses).

In this book, we define a *business event* as a request to perform an activity to register something that happens or that you would like to happen in the real world. So, in the above example, we identify a business event '*pay*' that is a request to register the fact that an acquisition is paid for. The request can be refused (e.g. there's insufficient money) in which case the activity is not executed. If the request is accepted (all conditions are met), the event is subsequently registered. The modeller has the choice to define an event as the creation of an object (e.g. a PAYMENT object), in which case the business event also relates to a business concept.

In the term 'business event', the word 'business' refers to the fact that the action that is considered pertains to the real world and is relevant from a business perspective, and the word 'event' is used to stress the atomicity of the activity executed upon acceptance of the request. Either the request is accepted and then the activity is processed in its entirety or the request is refused and then nothing is executed.

This way of domain modelling with business events is in line with the proposal of Jackson and Zave that requirements should describe what can be observed at the interface between the environment (the real world of the enterprise) and the machine (the enterprise's information systems) and that requirements should be formulated in an action-based way rather than in a state-based way [39, 102]. In particular, the identification of business events allows an action-based way of formulating requirements.

According to the same authors, actions can be environment controlled or machine controlled, and they can be shared between environment and machine or belong to the environment only. Shared actions are observed by both the environment and the machine, whereas unshared actions are not observed by the machine and exist in the environment only.

As an example, consider a library. The real-world library as a physical place with tangible books and people walking in and out with or without books in their hand is what should be considered as the 'environment'. The library system that allows to browse the catalogue, which is used to register loans, renewals of loans and loss and returns of books, is to be considered as the 'machine'.

The 'business events' are the actions for which we express the desire that they should be shared between environment and machine. If in the real world a person walks out of the library with a book, we require that this should be registered as a loan. To ensure that certain events initiated in the environment are shared (and don't go unnoticed), we often make use of security measures such as RFID tags in the books and electronic gates at the entrance and exit of the library. Such measures make sure that a person needs to pass along the desk to have the loan registered

before he or she can walk out of the library with a book. In this way, the security measure enforces the sharing of the business event. The registration of the loan is the combination of a request and an activity:

- The request to borrow a book will be checked against the permissions that apply to the book and to the borrower. The request can be refused, e.g. in case of a blacklisted borrower or in case of a protected book.
- If the request is accepted, the registration of the loan amounts to an activity that will update the state of the book (out) and the state of the member (one extra book in loan). At the same time, a 'loan' object may be created in the library's database (event as a concept).

According to Zave and Jackson, business events can be machine controlled or environment controlled. Losing a book is an example of an environment-controlled event as the loss of a book resides in the real world. An automatic renewal for books that are one day before the day of return would be an example of a machine-controlled shared business event: the initiative is taken by the library system as a service to certain members (hence machine controlled), and the renewal can be observed in the real world by any person looking at the status of the book by means of the library system's services (hence shared). When treating business events as requests that can be refused, there is however also a form of shared control when the system imposes conditions under which an event is accepted. Consider the borrowing of a book. In Zave and Jackson's terms, the borrowing of a book is an environment-controlled event as the initiative for borrowing a book resides in the real world. However, there is also a partial control by the machine. As will be explained in Chap. 7, business events can be subject to preconditions; these are the conditions under which the request for an event is accepted. The borrowing of a book could be subject to the condition that no one is allowed to borrow more than five books at a time. If then a member who already has five books tries to borrow the sixth book, then the borrowing of the book will be refused by the machine. In this case, the initiative for the event is taken by the environment, but the machine jointly controls the acceptance of the business event.

A person sneezing in the library would be an example of an unshared environment-controlled event: the sneezing happens in the real world (hence environment controlled), but this event is not registered and therefore not shared with the library system.

In summary, a business event:

- Is an action shared between the real world and the information system.
- Can be both environment controlled and machine controlled.
- Combines the notion of a request and an atomic action: it is the request to perform an action in its entirety, namely, to register the occurrence of an event in the real world in the enterprise information system; the request can be refused.

5.2 The Object-Event Table

5.2.1 Definition

The object-event table matches object types against event types. The object types are those that appear in the existence-dependency graph. The list of business event types has to be defined in order to build the object-event table. Identification of business events boils down to defining which environment- or machine-controlled events should be shared between environment and machine. Once an initial list of business events has been defined, the object-event table is filled by matching business events against the business object types. Each business event is a request to perform an atomic activity to mirror the occurrence of the event in the real world in the enterprise information system. This registration may require the creation, update and termination of business objects. The goal of the object-event table (or OET for short) is to model the participation of object types to business event types.

As UML does not offer a tabular modelling technique, we use an adapted form of the 'CRUD' table from the Information Engineering method from James Martin [53]. A CRUD-matrix maps business processes against information entities and indicates which process creates, reads, updates or deletes records of an information entity. Likewise, the object-event table maps business events against business object types and indicates which business event **c**reates, **m**odifies or **e**nds objects of the business object type. Additionally, the table will also capture the conditions imposed by a business object type to accept the request for this business event. In this way, the OET captures the requirements associated to both the request component and the activity component of a business event:

- A request can be refused. So, when filled, a cell specifies what preconditions[2] are imposed on a business event by a business object type.
- The activity is distributed across participants. So, assuming that the request has been accepted, when filled, the cell specifies what the effect is of that business event on the participating business object type (modified attributes, operations to call).

The fact that several business objects can jointly be involved in a business event is exactly what embodies the 'interaction' between the business objects. When a business event happens, all participating objects will jointly have to decide whether or not to accept the request. If accepted, they will all execute their corresponding part of the (distributed) activity. In this respect, we can say that objects synchronise their behaviour on the occurrence of business events in which they jointly participate.

[2] See Chap. 7 for more details on how to formulate preconditions.

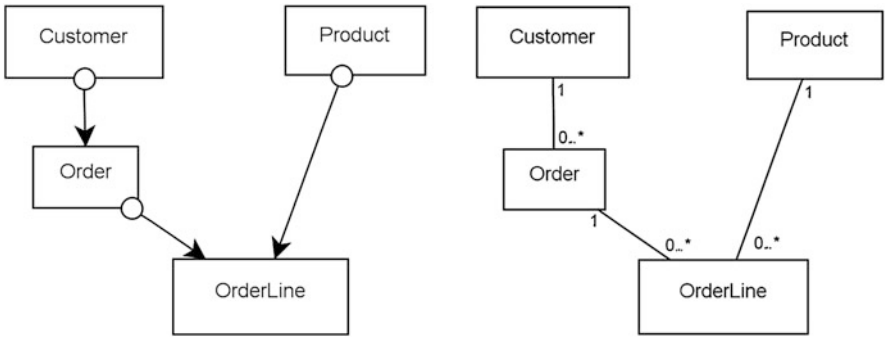


Fig. 5.3 A simple ordering model

Example 5.1 Ordering: Handling a Modify Quantity Event

Let us assume the EDG of Fig. 5.3. Assume, for example, that a customer modifies an order by changing the amount of ordered products in an order line. This will give rise to the definition of a business event *modify quantity*. This business event involves at least the order line (modification of the ordered quantity), the product (condition to accept request is that if the quantity is raised, the product should be available, and the effect will be that the stock is updated) and the order (condition to accept the request is that the order is still modifiable). Notice how 'modify quantity' differs from a method in the class ORDERLINE: handling the business event involves the coordination across different business objects; the status of the order and the product need to be checked in order to decide whether or not to accept the request, and then—upon acceptance of the business event *modify quantity*—the order line and the involved product need to be updated. This coordination embodies the interaction part of system behaviour.

Certain events end the life cycle of an object; however, such ending of an object is not necessarily a synonym for destroying or deleting an object. An ending event brings an object to a final state. Objects that are in a final state are 'dead' and can no longer be subject to changes and can, depending on the archiving, deletion and backup policies, be removed from the database. In other words, ending an object is not the same as physically removing the object, but it only means that a physical removal is allowed.

5.2.2 Graphical Representation

The object-event table (OET) contains one row for each event type and one column for each object type. Each cell in the table indicates whether an object type is involved in an event type or not. A 'C', 'M' or 'E' on a row-column point of intersection indicates that this particular event type is respectively creating,

	MEMBER	COPY	LOAN
register_member	C		
stop_membership	E		
acquire		C	
classify		M	
borrow	M	M	C
renew		M	M
return		M	E
sell		E	
lose		E	E

Fig. 5.4 Simplified (and incomplete) OET for the library example

modifying or ending objects of type P, where P is the object type corresponding to the column. Figure 5.4 is a graphical representation of Example 5.2.

Example 5.2 The Library: Simplified and Incomplete Object-Event Table

Let us assume that for a library, the list of business event types is register_member, stop_membership, acquire, classify, borrow, renew, return, sell and lose. Then a (simplified, see further) object-event table could be represented as in Fig. 5.4.

5.3 Existence-Dependency Graph Versus Object-Event Table

In case multiple objects participate to one business event, we will witness that all these objects are related to each other through associations in the existence-dependency graph. This leads to the definition of a number of rules to fill the object-event table. These rules depart from the principle that the existence-dependency graph and the dynamic model (object-event table and sequence constraint specifications) are dual perspectives of the same reality: they respectively model the static and the dynamic perspective of the universe of discourse. Therefore, they should be consistent with each other. Consequently, the semantics of the existence-dependency graph must also be in accordance with those of the object-event table. This means (among other things) that for each object type in the existence-dependency graph, there is one column in the object-event table (and vice versa).

5.3.1 Propagation Rule

One of the key rules in filling the object-event table is that a master object type by definition participates in all event types in which one of its dependent object type participates. We call this the propagation rule:

Propagation Rule

If P is existence dependent on Q, then Q participates in each event in which P participates.

In other words, each event marked for the dependent P must also be marked for the master Q.

This can be explained as follows. When an existence-dependent object is involved in an event, its master objects are automatically involved in this event as well. For example, if a copy is created, the corresponding title is (implicitly) involved as well, as we now count one more copy for this title. Similarly, a state change of a loan, e.g. because of the return of the copy, automatically implies a state change of the related copy and member: the copy is back on shelf and the member has one copy less in loan. By marking each event type the dependent object type participates in also for the master object type, all implicit participations are made explicit, and as a result, all possible places for information gathering and constraint definition are identified. For example, the *borrow* method of the class MEMBER is the right place to update the number of copies a member has in loan and to check a rule such as 'a member can have at most five copies in loan at the same time'. The *borrow* method of the class COPY is the right place to count the number of times a copy has been borrowed. At implementation time, methods that are empty because no relevant business rule or effect was identified can be removed to increase efficiency.

In addition, by explicitly including the master as participant in the event types of the dependent object types, sequence constraints that concern business event types of different dependent object types can be specified as part of the behaviour of the master.

Example 5.3 The Library: Constraints Spanning Several Object Types

Let us assume a library where books can be reserved. This can be modelled by means of an additional object type RESERVATION that is existence dependent on COPY and MEMBER. Sequence constraints such as 'a reservation can only be made for copies that are on loan' relate to more than one object type, namely, to LOAN and RESERVATION. They can therefore not be specified as a constraint within the context of only a LOAN or of a RESERVATION. Such constraint that needs to combine the context of two (or more) object types can be specified as part of the behaviour of a common master of these object types, COPY in the given example. Similarly, assume that when a copy has been reserved, the renewal of a loan for that copy is not allowed anymore: the copy should be returned to the library. A LOAN object cannot check whether a reservation exists for the copy that is on loan. Such conditions need to be checked at the level of the master, COPY in this case. As a final example, consider the case where members can borrow a maximum of five books at once. This is again an example of a constraint that should be checked by MEMBER rather than by LOAN: the member possesses the information of how many books he or she already has in loan; the loan object to be created does not have that information.

Because of the propagation rule, all of the object types participating in a business event type are connected through the existence-dependency relation. The class (or classes) that is(are) at the lowest position in the EDG is called the **owner class**(es) of an event type. An object type P **owns** an event type *e* if P is involved in *e*, and there is no object type which is existence dependent on P that also participates in *e*.

The identification of owner classes facilitates the construction of the OET. The owner class is most of the time also the class to which one would intuitively allocate an event type. For example, in the library, event types such as *borrow*, *renew* and *return* are intuitively allocated to the LOAN object type. This object type is also the owner class for the above event types. The easiest way to build an OET is to first indicate the involvement of the owner classes and then to propagate these involvements according to the propagation rule.

Example 5.4 The Library: Fully Specified OET

Figure 5.5 gives the indications of the owner classes for the library example of Fig. 4.6: these participations are preceded by 'O/'. Subsequently, the owner participations are propagated along the existence-dependency relations: from LOAN to COPY, from COPY to TITLE and from LOAN to MEMBER. These participations are marked as 'acquired' participations by having the C, M or E preceded by 'A/'. Notice that the propagation is transitive: when propagating from copy to title, also the acquired participations from copy are propagated one step up to title.

Example 5.5 Project Administration: EDG and OET

Imagine a project administration system where people have to register the time they spend on projects. When a project is finished, the project and all registrations for that project are 'closed' (close_project and close_registration event types). This means that all data related to that project cannot be modified anymore, but that it is retained for reporting and as documentation for future cost estimations. Once the information related to a project is considered not relevant anymore (e.g. for being outdated), the project can be ended. Two object types in this domain are project and

	MEMBER	TITLE	COPY	LOAN
register_member	O/C			
stop_membership	O/E			
cr_title		O/C		
end_title		O/E		
acquire		A/M	O/C	
classify		A/M	O/M	
borrow	A/M	A/M	A/M	O/C
renew	A/M	A/M	A/M	O/M
return	A/M	A/M	A/M	O/E
sell		A/M	O/E	
lose	A/M	A/M	A/E	O/E

Fig. 5.5 Owner and acquired participations for the library example

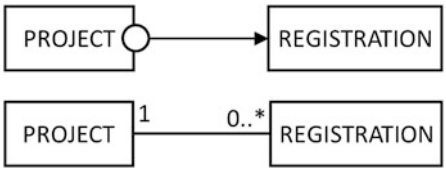


Fig. 5.6 EDG for the project administration example

	PROJECT	REGISTRATION
create_project	O/C	
register	A/M	O/C
close_project	O/M	
close_registration	A/M	O/M
end_project	O/E	
end_registration	A/M	O/E

Fig. 5.7 Owner and acquired participations for the project administration example

registration, the latter being existence dependent on the first. The EDG and OET for this example are given in Figs. 5.6 and 5.7.

5.3.2 Type of Involvement Rule

As explained in Chap. 4, the existence-dependency relationship has life cycle implications (cfr. Sect. 4.2.5). As a result, additional restrictions can be put on the type of involvement: a dependent object type cannot be created before its master exists nor can it exist after its master has been ended. Creating an existence-dependent object means that either the master is created at the same time (e.g. creating the first-order line creates the order) or that the master object type already exists (e.g. opening an account for an existing customer). In the latter case, the creation of a dependent object type modifies the state of the master. Since each event in which a dependent object participates also involves its master objects, this means that the set of creating event types of the dependent object type is a subset of the creating and modifying event types of the master object types. Modifying a dependent object always modifies the state of its master. Finally, ending a dependent object also modifies the state of its master. If the last dependent object is ended, then the master can be ended at the same time or later. We call these constraints the type of involvement rule. This is shown graphically in Fig. 5.8.

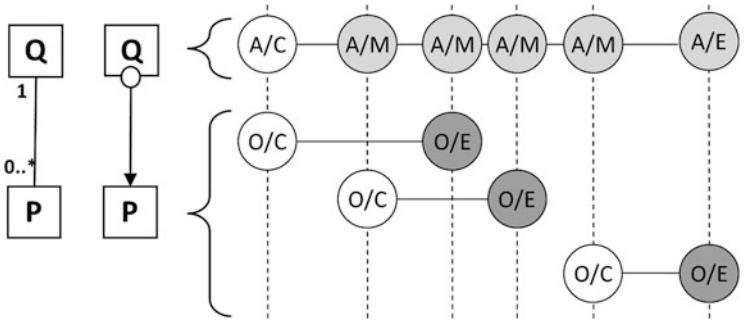


Fig. 5.8 Relationship between the life cycle of a master and its dependent

Type of Involvement Rule

- If in the column of an existence-dependent object type a row contains a 'C', then on the same row a 'C' or 'M' must appear in the column of each master object type.
- If in the column of an existence-dependent object type a row contains an 'M', then on the same row an 'M' must appear in the column of each master object type.
- If in the column of an existence-dependent object type a row contains an 'E', then on the same row an 'E' or 'M' must appear in the column of each master object type.

Example 5.6 The Library: Type of Involvement Rule

LOAN is existence dependent on COPY and MEMBER. The OET in Fig. 5.5 satisfies both the propagation and the type of involvement rule. Each event type marked for LOAN is also marked for both MEMBER and COPY. The event type *borrow* creates a loan, modifies a member and modifies a copy. So for the 'C' in the LOAN column, we have an 'M' in the MEMBER and COPY column. The event type *renew* modifies the state of a loan, a member and a copy. And thus for the 'M' in the LOAN column, we have 'M' in the MEMBER and COPY column. Finally, *return* and *lose* are both ending the life of a loan and are both modifying the state of a member. For the 'E's in the LOAN column, we have 'M's in the MEMBER column. *Return* is modifying the state of a copy and *lose* is ending the life of a copy. So for the 'E's in the LOAN column, we respectively have an 'M' and 'E' in the COPY column. As a result, the object-event table satisfies the type of involvement rule and the propagation rule.

The propagation rule and the type of involvement rule together ensure that the life span of the dependent object type is embedded in the life span of the master.

Notice that we only indicate participations when **exactly one** occurrence of the object type participates in events of the given type. In the given example, in a 'borrow' event, there is exactly **one** copy, **one** loan and **one** member that are involved. Participation of multiple occurrences usually is incorrect. It certainly is incorrect when the exact number of participating occurrences is unknown and/or when the business event has an identical effect on all involved occurrences.

Example 5.7 Project Administration: Wrong OET for Closing Projects

Consider again example 5.5. The correct OET was given in Fig. 5.7. Some people would however argue that when a project is closed, all registrations for that project should be closed as well. Hence in the REGISTRATION column, an 'M' should be put on the *close_project* row. A similar reasoning applies to *end_project*: when a project is ended, all registrations for that project should be ended as well. This would then be indicated by putting an 'E' in the REGISTRATION column on the *end_project* row. The result is the object-event table given in Fig. 5.9.

However, this way of modelling is not correct according to the MERODE rules. In MERODE, we require that per object type, only one occurrence is involved in an event per marking in the OET. In fact, it is impossible to know how many registrations have to be closed when a project is closed. The 'O/M' in the REGISTRATION column on the *close_project* row involves more than one and even an *unknown* number of registrations. In addition, when modelled in this way, the event type *close project* can hardly be considered atomic. Analogous arguments hold for the *end_project* event type. The 'O/E' in the REGISTRATION column is in fact the expression of a cascading end. The elegant solution to model cascade effects such as the closing and ending of projects is to define the appropriate *groups* of events (see Chap. 9).

One might object to this that after an event has occurred, the set of objects might be in an inconsistent state. In the given example, this means that, e.g. when a project is ended, there would exist registrations for that project which are not ended yet. Then some requirements such as referential integrity could be violated. This is not much of a problem. **Atomic** event types do not have to keep the object base in a consistent state. In this sense *close_project* should be understood as the archiving of a project whereby it is assumed that all existence-dependent registrations already have been closed. This assumption can be specified as a precondition to *close_project* (cfr. Chap. 7). Maintenance of consistency will be taken care of in a later stage where rules such as referential integrity and mandatory relationships will give rise to the grouping of atomic events (see Chap. 9).

An exception to this is when the exact number of participating occurrences can be determined (mostly two), and in addition, the event type has a different meaning for each of the participating occurrences. For example, in a sales transaction, two owners are involved in the same event: one is buying the property and one is selling

	PROJECT	REGISTRATION
create_project	O/C	
register	A/M	O/C
close_project	A/M	O/M
close_registration	A/M	O/M
end_project	A/E	O/E
end_registration	A/M	O/E

Fig. 5.9 Wrong OET for project administration

the property. Typically, the participating occurrences have different roles to play in the event. This special case will be dealt with in Chap. 7, Sect. 7.4.

5.3.3 *Contract Rule*

As a consequence of the propagation rule, the set of event types in which an object type that is in essence an AssociationClass (cfr. Sect. 4.1.2) participates is always a subset of the set of event types the object types it relates participate in, because it is existence dependent on these object types. Moreover, when two (or more) object types share a number of common event types, it makes sense to model this relationship between object types by a common dependent object type that has the role of a 'contract'.[3] We call this the **contract rule**. Sometimes, the shared event types can be spread across more than one existence-dependent object type, such as in the case of RESERVATION and LOAN of Example 5.8 below.

Contract Rule

When two object types share two or more event types, these common event types should be owned or shared by one or more common existence-dependent object types.

Example 5.8. The Library: Contract Rule for Reservation Event Types

Suppose that besides borrowing books, it is also possible to reserve books that are not on shelf. If a member changes his/her mind and decides not to fetch the copy, he or she can cancel the reservation. The events *reserve*, *cancel* and *fetch* are added to the object-event table (see Fig. 5.10). The shaded area shows the common event types of COPY and MEMBER. The dependent object type LOAN also participates in some of these events, but *reserve* and *cancel* do not appear in the column of a common dependent object type. According to the contract rule, either LOAN should also participate in these two event types or there should be a new object type, dependent of both MEMBER and COPY that participates in these events. The latter solution is to be preferred, because a loan can occur without a reservation and a reservation can occur without being followed by a loan. The correct object-event table is as in Fig. 5.11.

Contracts between objects are thus the counterpart of associations between objects in a class diagram: common event types between object types always indicate the presence of an association between these object types. The association can either express existence dependency, in which case the event types in which the dependent class are involved are all shared with the master object type. Either the association does not express existence dependency; in that case, MERODE requires the association to be transformed into a class (association as a class), and the shared

[3] The notion of contract will be further elaborated in Chap. 6.

	MEMBER	COPY	LOAN
register_member	O/C		
stop_membership	O/E		
acquire		O/C	
classify		O/M	
borrow	A/M	A/M	O/C
renew	A/M	A/M	O/M
return	A/M	A/M	O/E
sell		O/E	
reserve	?/M	?/M	
cancel	?/M	?/M	
fetch	A/M	A/M	O/C
lose	A/M	A/E	O/E

Fig. 5.10 New and incorrect OET for the library example

	MEMBER	COPY	LOAN	RESERVATION
register_member	O/C			
stop_membership	O/E			
acquire		O/C		
classify		O/M		
borrow	A/M	A/M	O/C	
renew	A/M	A/M	O/M	
return	A/M	A/M	O/E	
sell		O/E		
reserve	A/M	A/M		O/C
cancel	A/M	A/M		O/E
fetch	A/M, A/M	A/M, A/M	O/C	 O/E
lose	A/M	A/E	O/E	

Fig. 5.11 Correct OET for the library example

business event types will be owned by this new (association as a) class and from there propagated to the masters.

Reversely, when in the class diagram, an association is modelled between two object types, and then either this relationship expresses existence dependency, or if not so, the association should be transformed to a new class that is existence dependent on the classes related by the original association. In the first case, the set of event types marked in the column of the existence-dependent object type is a subset of the event types marked in the column of its master, and in the second case, the set of event types marked in the column of the new relationship object type is the set of event types common to all its master object types.

Notice that the contract rule is only applicable in case of **two or more** common event types and that at least one of these must create and another one must end the

existence-dependent object types. If there is only one common event type (e.g. 'fetch' is common to RESERVATION and LOAN in Fig. 5.11), it would be wrong to model an additional object type: a single-event type is a contract with no duration. A contract should however always have a duration that is delimited by at least one creating event type and one ending event type. An additional object type cannot be the result of only one common event type.

Remark also that the result of this is that the event type *fetch* has two owner classes, namely LOAN and RESERVATION, and that it propagates twice to MEMBER and COPY: once from LOAN to MEMBER and COPY and once from RESERVATION to MEMBER and COPY. This is consistent with the fact that there might be two members and two copies involved in a *fetch* business event: the member that made the reservation and the member that fetches the copy and the copy that was reserved and the copy that is fetched.

5.4 Multiple Propagation

In some cases, object types are existence dependent on the same master object type in two different ways, either directly or indirectly. For example, in Fig. 5.12, the object type MARRIAGE is *directly* existence dependent on PERSON in two ways. This means that in any event type of marriage, for example, in 'marry' or 'divorce', two people are involved. As a result, the method PERSON.marry needs two interpretations. Similarly, in a sales transaction, two persons are involved as well: the buyer and the seller.

Sometimes an object type is *indirectly* dependent on the same master object type in two different ways. In Fig. 5.13, both research programs and research contracts are existence dependent on the financing fund. Some but not all research contracts

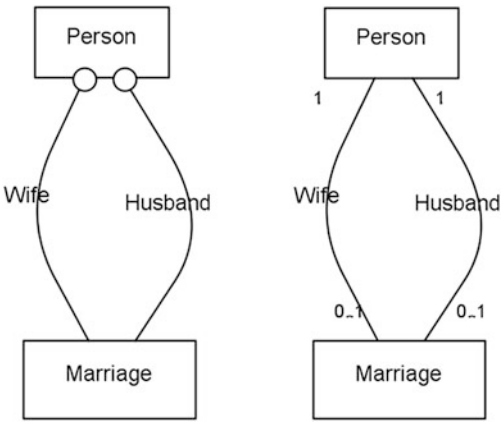


Fig. 5.12 A marriage depends on two people

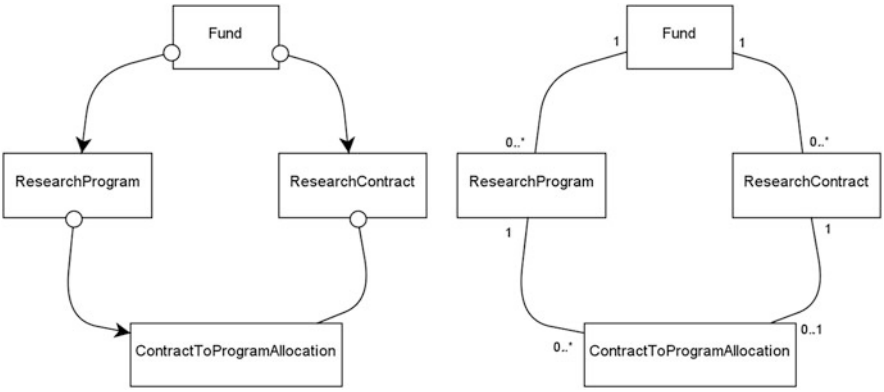


Fig. 5.13 EDG for research funding administration

are set up in the context of a research program, but this relationship does not express existence dependency. As a result, the allocation of a contract to a program is a fourth object type, existence dependent on both RESEARCH PROGRAM and RESEARCH CONTRACT. As one can see from Fig. 5.13, the object type ALLOCATION is existence dependent on the object type FUND in two different ways. Each allocation object will thus refer to two funds, one for the research program and one for the research contract. This type of structure also leads to the *multiple propagation* of event types to one object type: each event type of ALLOCATION will be propagated two times to the FUND object type, once through RESEARCH PROGRAM and once through RESEARCH CONTRACT. This reflects the fact that in each of these event types, two occurrences of FUND are involved.

However, in this example, the business rules state that the fund of the research program has to be the fund of each contract allocated to that program. This should be explicitly specified by adding a constraint to the ALLOCATION object type, namely, research_program.fund = research_contract.fund. In this way, it is ensured that each allocation object refers to only one fund and that only one occurrence of fund is involved in each event type of ALLOCATION.

In situations where more than one occurrence of a single object type are involved in one event type, the event has one alias for each object involved: each alias allows to define a different interpretation of the event. Constraints and aliasing event types will be further explained in Chap. 7.

5.5 Formal Definition of the Object-Event Table

The object-event table assigns a subset of event types to each object type. This subset is called the *alphabet* of an object type P and is denoted by $S_A P$. It contains all the event types that are relevant for that particular object type. In addition, the alphabet of P is partitioned in three mutually disjoint sets: c(P), m(P) and e(P) with

$c(P) = \{\text{The set of event types that create an object of class P}\} \subset S_A P$
$m(P) = \{\text{The set of event types that modify an object of class P}\} \subset S_A P$
$e(P) = \{\text{The set of event types that end an object of class P}\} \subset S_A P$
$c(P) \neq \emptyset$
$e(P) \neq \emptyset$

c(P) and e(P) are never empty: for each object type, there is at least one event type that creates occurrences of this object type, and there is at least one event type that ends occurrences of this object type. As sometimes there are no modifying event types, m(P) possibly is empty.

Example 5.9 The Library: Formal Definition of the OET

Let us assume that for the library example, the list of business event types is register_member, stop_membership, acquire, classify, borrow, renew, return, sell, and lose.

The partitions of the alphabets of the object types COPY, MEMBER and LOAN are as follows:

$S_A\text{COPY} = \{\text{acquire, classify, borrow, renew, return, sell, lose}\}$ with
$c(\text{COPY}) = \{\text{acquire}\}$
$m(\text{COPY}) = \{\text{classify, borrow, renew, return}\}$
$e(\text{COPY}) = \{\text{sell, lose}\}$

$S_A\text{MEMBER} = \{\text{register_member, borrow, renew, return, lose, stop_membership}\}$ with
$c(\text{MEMBER}) = \{\text{register_member}\}$
$m(\text{MEMBER}) = \{\text{borrow, renew, return, lose}\}$
$e(\text{MEMBER}) = \{\text{stop_membership}\}$

$S_A\text{LOAN} = \{\text{borrow, renew, return, lose}\}$ with
$c(\text{LOAN}) = \{\text{borrow}\}$
$m(\text{LOAN}) = \{\text{renew}\}$
$e(\text{LOAN}) = \{\text{return, lose}\}$

Propagation Rule

$$P \leftarrow Q \Rightarrow S_A P \subseteq S_A Q$$
$$\forall P \in \mathcal{M}, e \in A : P \text{ owns } e \Leftrightarrow e \in S_A P \wedge \neg(\exists Q \in \mathcal{M}, Q \leftarrow P \wedge e \in S_A Q)$$

Type of Involvement Rule

$$P \leftarrow Q \Rightarrow c(P) \subseteq c(Q) \cup m(Q) \wedge m(P) \subseteq m(Q) \wedge e(P) \subseteq e(Q) \cup m(Q)$$

Contract Rule

$$\forall P,Q \in \mathcal{M} : \#(S_A P \cap S_A Q) \geq 2 \quad \text{and} \quad \neg(S_A P \subseteq S_A Q \text{or} S_A Q \subseteq S_A P)$$
$$\Rightarrow \exists R_1, R_2, \ldots R_n \in \mathcal{M} : \forall i \in \{1, \ldots, n\} :$$
$$R_i \leftarrow P,Q \quad \text{and} \quad S_A R_1 \cup \ldots \cup S_A R_n = S_A P \cap S_A Q$$

5.6 Meta-Model

EVENT TYPE is added to the meta-model as a new meta-object type representing the rows of the object-event table. Each cell in the object-event table represents a METHOD. Methods can be of two types: they can represent owner participations or acquired participations. An acquired participation is the result of propagating another method along a dependency. Therefore, an acquired method is existence dependent of both another method and a dependency. Figure 5.14 shows the EDG representation of the meta-model.

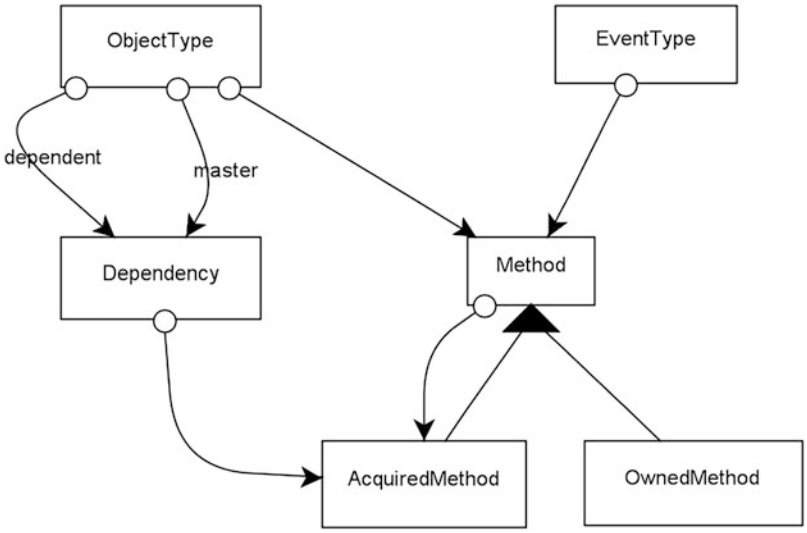


Fig. 5.14 EDG of the meta-model including the definition of the OET

Chapter 6
Object and System Behaviour

6.1 Object Behaviour Modelling

6.1.1 Finite-State Machines

Business events are not allowed to occur in a random order during the life cycle of an object. For example, a book cannot be returned if it has not previously been borrowed. In order to specify this kind of constraints, each object type can specify sequence constraints on the business events it participates in by means of a finite-state machine (or FSM for short).

A finite-state machine allows specifying sequence constraints by defining how business events bring an object from one state to another. In UML, the initial state of a finite-state machine is depicted as a black dot (this is the state in which the object does not exist yet), intermediate states are depicted as circles or rounded rectangles and final states are drawn as double circles. Transitions from one state to another are drawn as arrows and are labelled with

eventname (parameters) [guard]/action

To ensure the correspondence with the OET, the event name should be the name of one of the business event types the object type participates in. The action corresponds to the object type's method that is executed as a response to the business event. In newly developed software, it can be given the same name as the business event that causes its execution. In the absence of parameters and guards, the label of a transition becomes eventname/methodname. In the remainder of this text, we will follow the convention of methodname being equal to eventname, and so eventname/methodname will be abbreviated as 'methodname'.[1]

[1] To distinguish between event names and method names, in JMermaid, event names are by default prefixed with EV_ and method names with ME_.

© Springer International Publishing Switzerland 2014

M. Snoeck, *Enterprise Information Systems Engineering*, The Enterprise Engineering Series, DOI 10.1007/978-3-319-10145-3_6

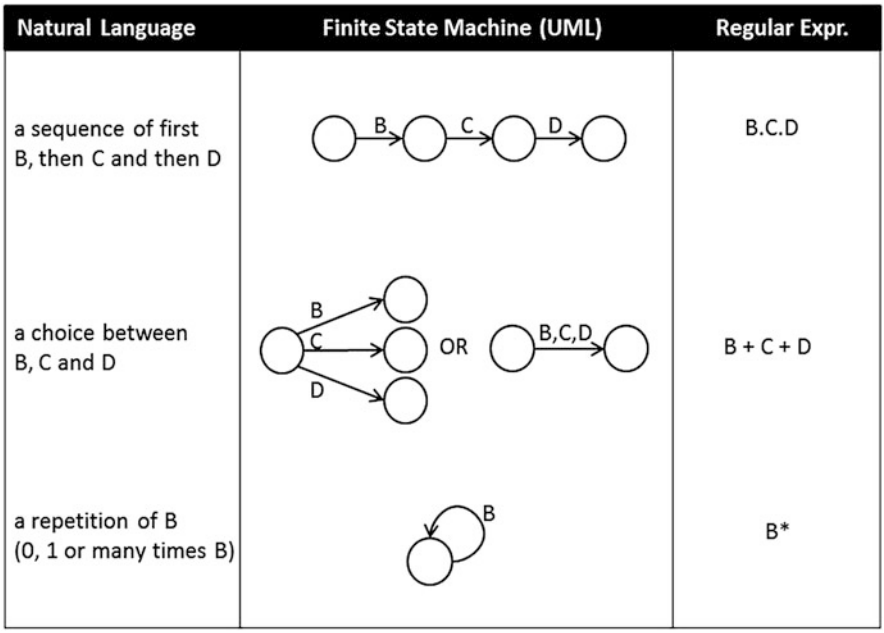


Fig. 6.1 Natural language, finite-state machine and equivalent regular expression for sequence, selection and iteration

Finite-state machines are a well-known kind of automaton that generates a 'regular language'. As will become clear in Sect. 6.3, the theory of automata and their languages (see, e.g. [2, 36]) will prove useful in checking the behavioural model for consistency.

Figure 6.1 depicts in the second column how sequence, selection (exclusive and exhaustive selection) and iteration are modelled with finite-state machines. The third column shows the equivalent regular expression: the '.' sign is used to denote sequence, the '+' sign is used to denote choice and the '*' is used to denote iteration.

The object-event table defines which event types create, modify or end objects of a certain type. It therefore already specifies some sequence constraints. The default assumption is that objects first must be created before they can be modified and that an ending method is always the last method in an object's life. But sometimes, it is necessary to put additional constraints on sequences in which methods can be called as a result of events that happen. As the transitions are labelled with 'eventname/methodname', the sequences defined in the finite-state machine apply both to the methods and to the business event types. As a result, a finite-state machine defines the order in which business events are allowed to happen from the perspective of that business object type.

Example 6.1 The Library: Behaviour Definition

Figure 6.2 shows the finite-state machines for the library example of Figs. 4.6 and 5.5. The equivalent regular expressions are as follows:

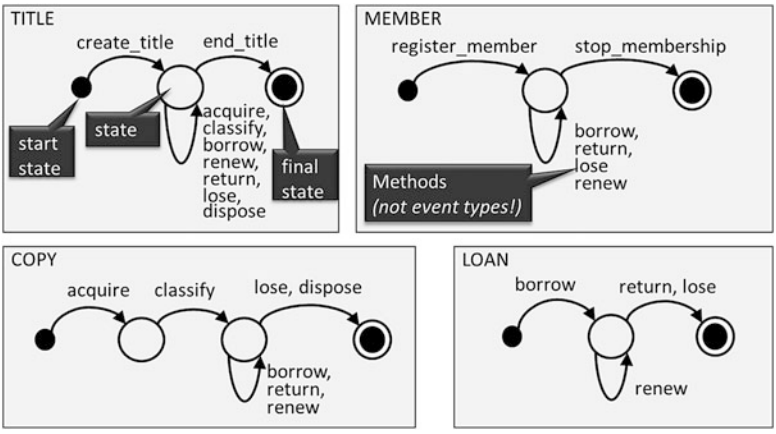


Fig. 6.2 Finite-state machines for the library example

TITLE = create_title. (acquire + classify + borrow + renew + return + sell + lose)*. end_title
COPY = acquire . classify . (borrow + renew + return)* . (sell + lose)
MEMBER = register_member . (borrow + renew + return + lose)* . stop_membership
LOAN = (borrow + fetch) . (renew)* . (return + lose)

The business object type LOAN imposes the constraint that the return of a copy can only happen after a borrow or a fetch, while there may be intermediate renew events. The loan object type imposes these constraints on its own methods but also on the corresponding business event types that initiate the execution of the methods.

Notice that TITLE, LOAN and MEMBER have a default life cycle. Therefore, drawing finite-state machines for these object types is not really necessary. The object type COPY needs an explicit definition of its sequence constraints because the event type *classify* must occur exactly once after the acquisition.

The above example also shows the notation for the different kinds of states: a black dot for the start state (there should be exactly one), a white circle for the intermediate state and a filled circle with a double line for the final states. There can be multiple intermediate and final states in a state machine. States can also be named. When naming a final state, the black fill is omitted and the state is just drawn with a double line. When there are multiple transitions between two states, the arrows representing the transitions are merged into a single arrow, and all the labels are put next to this arrow separated by commas. So, the loop in the second intermediate state of COPY actually represents three transitions: for the *borrow*, *renew* and *return* methods respectively.

In principle, for every object type, a finite-state machine should be defined to specify the sequence constraints that this object type imposes on its methods and the business event types it participates in. However, if the object type does not define additional sequence constraints on top of the default constraints (creation first, then

modifications in an arbitrary order and terminating by one of the ending events), drawing an FSM is not required as the OET captures these sequence constraints.

6.1.2 Basic Correctness Checks

When defining a finite-state machine, care should be taken to make sure that all states can be reached from the initial state (forward accessibility) and that from each state a final state can be reached (backward accessibility). Also, the finite-state machine should be deterministic: from every state, only one transition should be labelled with a given event type, such that business event type determines only one possible next state.

Example 6.2 The Library: Correctness Check of a Finite-State Machine for ACCOUNT

Figure 6.3 shows an example with such basic errors:

- The state 'blocked' is forward accessible via the scenario open^block, but when going backward from the final state, it cannot be reached. So it is a backward inaccessible state.
- From the state 'ForwardInaccessible', we can reach the final state with *close,* but this state cannot be reached from the initial state. It is therefore a forward inaccessible state.
- When an account is in the state 'open', the event *withdraw* can trigger a transition to the state 'open' as well as to the state 'Overdrawn'. This means that the FSM is non-deterministic.

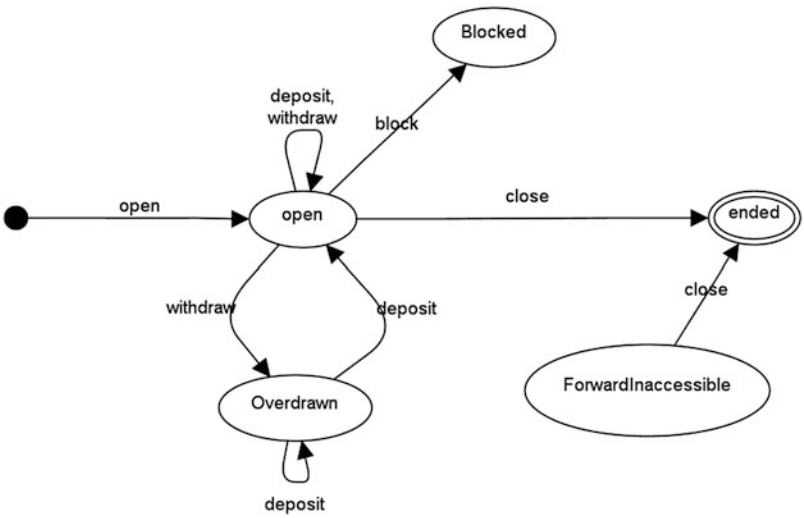


Fig. 6.3 FSM with basic errors

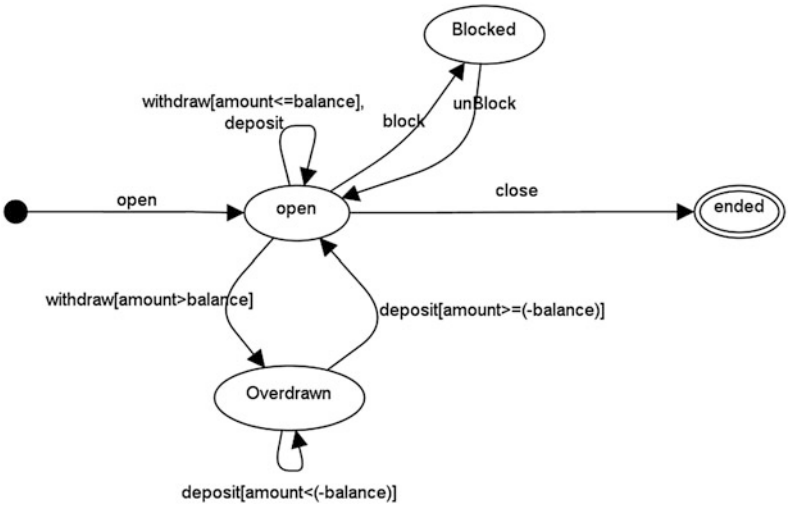


Fig. 6.4 Corrected FSM

– Similarly, in the state 'Overdrawn', there are two transitions labelled *deposit* and that go to two different states.

Backward and forward inaccessible states should be removed or transitions need to be added to ensure both forward and backward accessibility. Non-determinism can be resolved by adding guards or removing transitions. Also minimising the state machine can resolve problems of non-determinism. This requires however an algorithmic calculation of the equivalent minimised FSM. Such algorithm would merge the states 'open' and 'Overdrawn' since each scenario corresponding to a path from the state 'open', through the state 'Overdrawn' back to the state 'open' (i.e. withdraw.(deposit)*.deposit), can also be performed in the state 'open' itself. The corrected FSM is given in Fig. 6.4.

6.1.3 Stratification of Finite-State Machines

Sometimes, modelling all the sequence constraints in a single finite-state machine yields an overly complex finite-state machine. Two techniques exist to keep finite-state machines legible and understandable. The first technique, stratification, is purely a matter of layout. The second technique is explained in the next section: it exploits parallelism and splits a single FSM into multiple parallel FSMs.

Stratification is purely a matter of visual representation. Just as the proper page layout can make a text more readable, the disposition of states and transitions can make a difference in the readability of a FSM. Explaining the stratification algorithm is beyond the scope of this book. However, the general idea of stratification is easy to capture: a stratified FSM contains two axes. Progress towards the final state

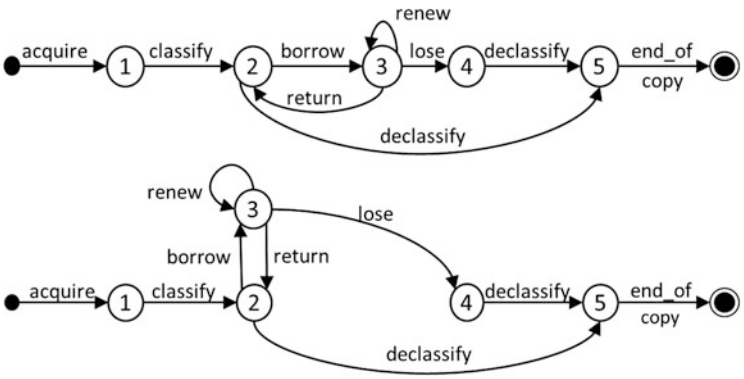


Fig. 6.5 Unstratified and equivalent stratified FSM

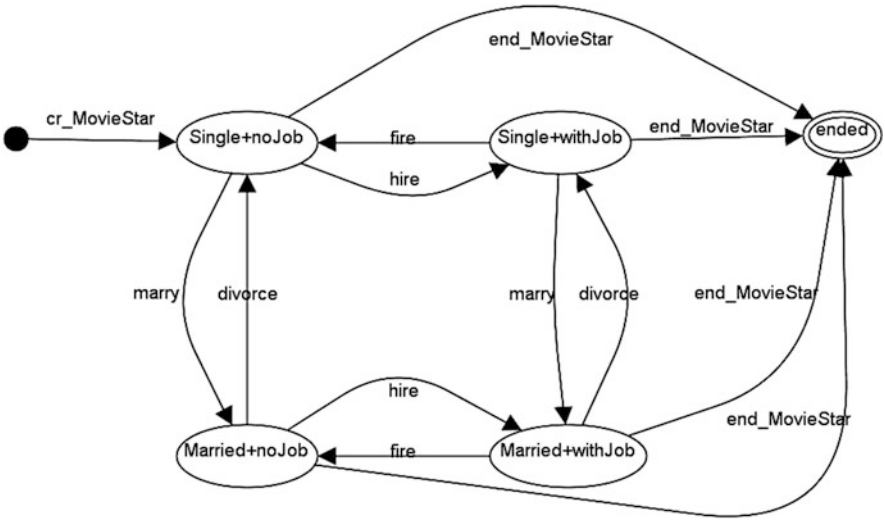


Fig. 6.6 Finite-state machine for movie star

is depicted along one axis, while cycles in the behaviour are drawn along a perpendicular axis. Figure 6.5 shows an unstratified and equivalent stratified FSM.

6.1.4 Exploiting Parallelism

A well-known example (taken from [8]) of the usefulness of roles is modelling the life of a movie star: he or she has a private life and a professional career which gives rise to two independent sets of event types, namely, *marry* and *divorce* for the private life and *hire* and *fire* for the professional career. The life of such an object type can be modelled in a single finite-state machine as shown in Fig. 6.6. We

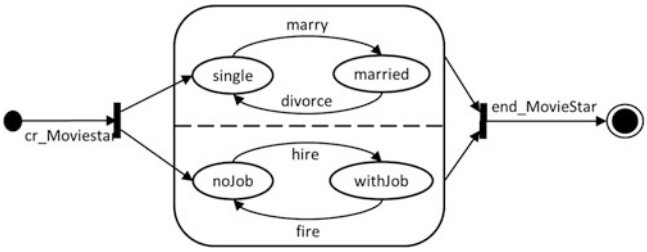


Fig. 6.7 Harel Statechart for movie star

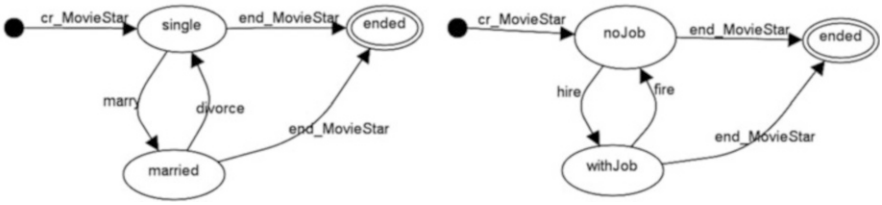


Fig. 6.8 Roles for movie star

assumed that the movie star is initially single and without contract. The end of his/her relevant life can occur at any time. However, by exploiting the concept of parallelism, we can substantially simplify the diagrams. Harel Statecharts [32] are an extended form of finite-state machines allowing using parallel regions and complex transitions with splits and joins. A Harel Statechart for MOVIESTAR is given in Fig. 6.7 using the UML notations for denoting parallel regions and complex transitions. Another solution is to define several roles for the MOVIESTAR object type: one for the private life and one for the professional career, as shown in Fig. 6.8. These roles exist and are executed in parallel with the base class definition and can be stopped at any moment. The base class has the default life cycle unless otherwise specified. This is the way of working we will rely on in this book. Notice that roles are **not** independent object classes: they only exist in the context of the base class and are therefore not visible in the EDG. They only exist as multiple FSMs for one class. The global behaviour of the object type will be the parallel composition of the default life cycles with all the roles.[2] This combination with the default life cycle implies that business events/methods that do not appear in any of the roles are allowed to happen in any state. In order to understand this behaviour, a calculated FSM can be requested that shows this global behaviour. As an example, adding a *dress_up* modifying method in the default life cycle of the movie star results in the global behaviour depicted in Fig. 6.9.

[2] Notice that this requires defining the semantics of the parallel operator (denoted as ||). This will be done in section 6.4.

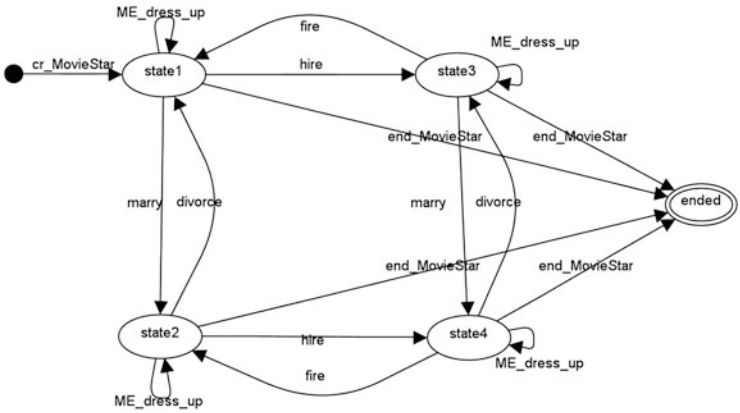


Fig. 6.9 Calculated FSM for movie star

Roles are also further discussed in Sect. 8.3.4, as they show some similarity to the concept of subclasses.

6.2 Global System Behaviour

Besides being subject to sequence constraints, business event types are also the basis for interaction between objects. Object interaction means that the global behaviour of a system is not the simple superposition of the individual behaviour of the objects that compose the system. Rather, objects will influence each other's behaviour because they interact with each other. As a result, the global behaviour of the system is the result of combining the individual behaviour of the objects with the way they interact.

The conceptual model proposed in this book has no separate object interaction model besides the identification of common events in the object-event table. For the purpose of conceptual modelling, we use *multiparty interaction* rather than interaction by means of message passing. This means that when a business event is triggered, all objects that are involved in this event according to the OET will execute their method for this event. For example, if a copy is returned to the library, the user interface will trigger a business event *return (loan_instance),* and the involved loan and its related member, copy and title object will respond and each execute their *return* method, provided that all of them are in a state where a *return* event is allowed to happen according to the specified sequence constraints. This means that if one of the involved objects is not in such a state, the business event will not be allowed to happen. We call this way of interacting 'interaction by joint participation to (common) events'.

This way of interaction can also be seen as the fact that objects 'synchronise' their states when they jointly participate to events. The 'synchronisation' refers to the combination of 'agreement' on accepting the request and the subsequent



Fig. 6.10 Cog-wheel and stopper

performance of the activity. When two or more objects are involved in the same business event, in case of acceptance of the request by each involved object, performing the activity means that the class's method associated with the event has to be executed in each of the participating objects, which puts all participating objects in a next state.

Informally, the mechanism of parallel composition can be explained as follows. The finite-state machine of an object type defines the individual behaviour of an object. This definition determines the set of valid scenarios from the object's perspective. A scenario is a sequence of business event types, e.g. borrow^renew^return. If iteration appears in the life cycle (meaning that a loop occurs in the FSM or that the *-operator is used in the equivalent regular expression), the number of valid scenarios for an object type is infinite.

A system is composed of many objects that all run in parallel and synchronise on the events they jointly participate in. The parallel composition of object must express the fact that when two or more objects run concurrently, only those scenarios are valid for which all objects agree on the sequence of common events. Let's illustrate this with a cog-wheel and a stopper as shown in Fig. 6.10. Assume that the cog-wheel supports two events appearing in a random order, *turnLeft* and *turnRight*, but that the stopper only accepts scenarios that are a sequence of *turnRight* events. Their life cycles are thus respectively determined by the following regular expressions: COG-WHEEL = (*turnLeft* + *turnRight*)* and STOPPER = (*turnRight*)*. If we abbreviate these two event types als L and R, then the set of scenarios accepted by the stopper is {R, RR, RRR, RRRR, ...}, and the set of scenarios accepted by the cogwheel is {L, R, LL, LR, RL, RR, LLL, LLR, LRL, RLL, LRR, RLR, RRL, RRR, . . .}.

The system that constitutes of an interacting stopper and cog-wheel will only accept scenarios that are OK both for the cog-wheel and the stopper. The parallel composition therefore amounts to an operator that calculates the intersection of sets of scenarios: it returns the scenarios that are accepted by all interacting parties. In this case, the parallel composition of the cog-wheel and the stopper is a system that only accepts a sequence of *turnRight* events, i.e. the set of scenarios {R, RR, RRR, RRRR, ...}. So, COG-WHEEL || STOPPER = (*turnRight*)*.

Let's further illustrate parallel composition and the principle of interaction by means of common events with the extract of the library example shown in Figs. 6.11 and 6.12.

	COPY	LOAN
acquire	C	
classify	M	
borrow	M	C
renew	M	M
return	M	E
lose	E	E

Fig. 6.11 OET for an extract of the library example

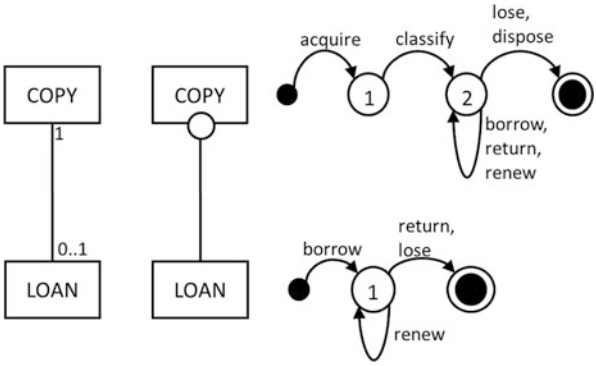


Fig. 6.12 EDG and FSMs for the extract of the library example

Example 6.3 The Library: Global System Behaviour

Table 6.1 gives a sequence of business events offered to the system, starting with the acquisition of a copy of the book entitled *Harry Potter*. For each event, the object ID is given between brackets. The second column indicates how the objects involved in this business event evaluate whether or not to accept the request for the business event (and why). The third column summarises the final outcome (accept or reject) and summarises the main effect.

From this example, it becomes clear that the global behaviour of a system is the combination of the individual behaviour of objects as defined in the FSMs, the way they interact by participating jointly to events as defined in the OET, the pre-conditions they impose on events (see Chap. 7) and the associations defined in the class diagram.

6.3 Consistency Checking

From a quality perspective, it is important to ensure the correct behaviour of the system. This can be achieved by performing several quality checks. First, we need to check the correctness of the behaviour of the individual object types. From a syntactical perspective, we need to ensure that each finite-state machine has only one start state; it is free of inaccessible states and non-determinism (see Sect. 6.1.2). From a semantic perspective, each FSM should be a complete and

Table 6.1 Execution of a sample scenario for the library

Event	Evaluation	Final outcome and effect
Acquire (*Harry Potter*)	• Validated only by copy (*Harry Potter*): accepted	Accept; copy (*Harry Potter*) is in state 1
Borrow (L1, *Harry Potter*)	• Validated by copy (*Harry Potter*): rejected because copy (*Harry Potter*) is in state 1, and in that state, a borrow event is not allowed • Validated by loan (accepted)	Reject
Classify (*Harry Potter*)	• Validated by copy (*Harry Potter*) only and accepted	Accepted; copy (*Harry Potter*) is put in state 2
Borrow (L1, *Harry Potter*)	• Validated by copy (*Harry Potter*) and accepted • Validated by loan and accepted	Accepted; loan(L1) is created
Dispose (*Harry Potter*)	• Validated by copy (*Harry Potter*) only. Request is refused because of referential integrity constraint: master object cannot be ended as long as it has living dependents	Reject
Borrow (L1, *Harry Potter*)	• Validated by copy (*Harry Potter*) and rejected because of cardinality constraint: a copy can have at most one loan • Validated by loan (accept)	Reject
Return (L1, *Harry Potter*)	• Validated by copy (*Harry Potter*) and accepted • Validated by loan (L1) and accepted	Accepted; loan(L1) is put in state 2
Dispose (*Harry Potter*)	• Validated by copy (*Harry Potter*) only and accepted	Accepted; copy (*Harry Potter*) is put in the final state

valid representation of the life cycle of the business objects it represents. Next, we need to guarantee consistency between static and dynamic characteristics of object types. This requires verifying the FSMs against the OET and the EDG. Finally, a check of the global system behaviour should be performed.

6.3.1 Consistency Checking with the OET

The OET defines the set of event types an object type participates in and partitions this set in creating, modifying and ending event types. These elements have an influence on what can be considered a valid behaviour definition of an object type. In the first place, there is the alphabet rule.

Alphabet Rule

The structure diagram or expression that defines the behaviour of an object type must contain all and only the event types for which there is a ‘C’, ‘M’ or ‘E’ in the column of that object type in the OET.

As said before, the partitioning of the participations in creating, modifying and ending event types for an object type imposes a default life cycle. This default life cycle must be respected by the sequence constraints.

Default Life Cycle Rule

The events marked with C, M or E in the object types' column in the OET must appear as creating event types, modifying event types or ending event types respectively in the sequence restrictions of the object type's finite-state machines. This means that the sequence constraints imposed by the roles must not violate the default life cycle of create, modify and end.

6.3.2 Consistency Checking with the EDG

As explained in Chap. 4, the semantics of existence dependency induce life cycle constraints. In Chap. 5, this has given rise to the propagation rule, which reflects the fact that a master object type by definition participates in all event types in which one of its dependent object types participates (cfr. the propagation rule explained in Sect. 5.3.1). Together with the principle of interaction by means of joint participation to common event types, this therefore implies that a master object and a existence-dependent object will always have to synchronise on all business events in which the existence-dependent object participates. As a result, to ensure the consistency of the global behaviour, it is necessary to investigate the life cycle of the master and of the existence-dependent object type for possible contradictions.

In this paragraph, we outline the main principles along which such consistency and correctness can be done and give an informal definition of the consistency rules that should be verified to assess the correctness of a domain model. Then, in Sect. 6.4, we formalise the parallel operator and outline how regular expressions (the mathematical equivalent of FSMs) can be used for formally checking consistency between the FSMs and the EDG.

The essence of consistency checking between the FSMs and the EDG is based on two principles: propagation of events and existence-dependent objects as contracts between master object types.

Propagation of Events. Event types of existence-dependent object types are propagated to the master object type (see Chap. 5).

Existence-Dependent Objects as Contracts. According to the OET, two or more object types jointly participate to two or more common business event types, at least one existence-dependent object type must be modelled that participates in all the common business event types. This existence-dependent object type acts as a contract between the master object types: the contract ensures that the participating object types agree on the allowed behaviour (sequences of events).

In order to understand the consequences of this notion of contract for sequence constraints, let us imagine an enterprise with only two object types, a master object type ACCOUNTHOLDER and a dependent object type ACCOUNT. According to the

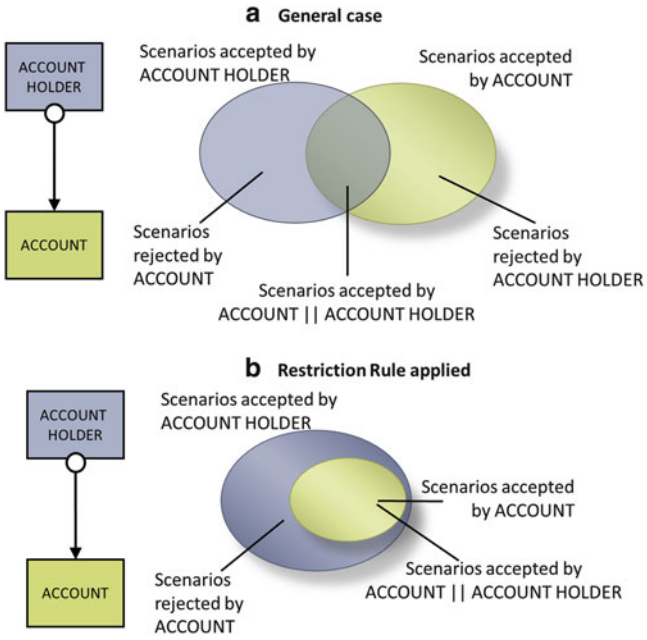


Fig. 6.13 Restriction rule for existence-dependent object types

principle of parallel composition, both object types will run concurrently, and such a system will only accept sequences of events that satisfy the sequence constraints of both ACCOUNT and ACCOUNTHOLDER. So, if we look at the sets of scenarios as depicted in Fig. 6.13a, we see that the possible scenarios for ACCOUNT || ACCOUNTHOLDER are those that are in the intersection of the set of scenarios accepted by ACCOUNT and the set of scenarios accepted by ACCOUNTHOLDER. If ACCOUNT is dependent on ACCOUNTHOLDER, this means that only those scenarios of ACCOUNT that are at the same time in the set of scenarios of ACCOUNTHOLDER will be possible in practice. Any scenario of ACCOUNT that is not acceptable from the point of view of ACCOUNTHOLDER will *always* be rejected. It can therefore be removed from the life cycle definition of ACCOUNT because it is completely superfluous: it will never be accepted. Consequently, it seems sensible to require that a master object type ACCOUNTHOLDER can accept all scenarios of its dependent object type ACCOUNT. This means that the existence-dependent object type ACCOUNT must have more stringent sequence constraints or must be *more deterministic than* the master object type ACCOUNTHOLDER. This situation is depicted in Fig. 6.13b.

As a result, when an object type has more than one master object type, the set of scenarios accepted by the dependent object type is in the intersection of the sets of scenarios accepted by all its master object types. Notice that not all scenarios in the intersection are necessarily relevant. The dependent object type can put additional constraint such that we obtain a *subset* of the intersection of the sets of scenarios of

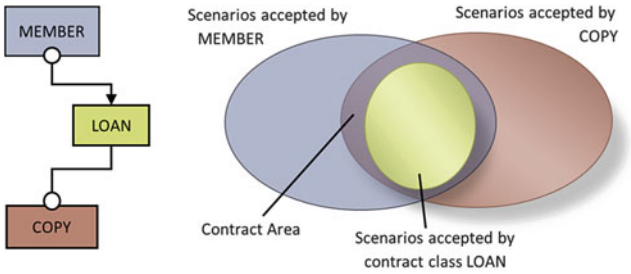


Fig. 6.14 LOAN as a contract between MEMBER and COPY

the master object types. In this sense, the intersection acts as a contract area and the dependent object type acts as the agreed contract between the master object types: for the common event types, it defines the set of scenarios that will be accepted by all participants. For the library example, LOAN is a contract between MEMBER and COPY (see Fig. 6.14). The fact that the set of scenarios of the contract object type is a *subset* of the set of scenarios of the contracting object types conforms to the intuitive meaning of a contract: a contract implies a number of commitments (restrictions) and not extra degrees of freedom.

How can this relationship *more deterministic than* be checked? The basic idea is to project the sequence constraints of the master object type on the set of common event types (i.e. the set of event types the dependent object type participates in) and see if these sequence constraints are less stringent than those of the dependent object type. The projection consists of replacing all the non-relevant event types with a 'do-nothing' event type (denoted by '1').

Definition An object type P is **more deterministic than** an object type Q (which is written as $P \leq Q$) if (and only if) the set of event types P participates in is a subset of the set of event types Q participates in and the scenarios of P are all acceptable for Q.

Example 6.4 The Library: Verification of the Restriction Rule

With the sequence constraints as below and LOAN existence dependent on COPY and MEMBER, we have:

COPY = acquire.classify.(borrow + renew + return)*.(sell + lose)
MEMBER = register_member.(borrow + renew + return + lose)*.stop_membership
LOAN = borrow.(renew)*.(return + lose)

The set of event types in the life cycle of LOAN is a subset of the set of event types in the life cycle of COPY. In order to compare the scenarios defined by COPY and LOAN, the expression of COPY is projected on the set of event types in the life cycle of LOAN:

COPY$\backslash S_A$LOAN
= acquire.classify.(borrow + renew + return)*.(sell + lose)$\backslash S_A$LOAN
= (borrow + renew + return)*.(1 + lose)

Apparently, the scenarios defined by LOAN are all acceptable for COPY.
As a result, LOAN $\leq$ COPY: LOAN is more deterministic than COPY.
The same reasoning applies to MEMBER:

MEMBER$\backslash S_A$LOAN
= register_member.(borrow + renew + return + lose)*.stop_membership$\backslash S_A$LOAN
= (borrow + renew + return + lose)*

Which is also less deterministic than LOAN and thus LOAN $\leq$ MEMBER.

From the point of view of object behaviour, the partial order **more deterministic than** ($\leq$) can be defined as the dual counterpart of the existence-dependency relation. We therefore formulate the following consistency rule :

Restriction Rule
If P is existence dependent on Q, then P must be more deterministic than Q.

Notice that the restriction rule contains a one-direction implication. It might indeed happen that P is more deterministic than Q (P $\leq$ Q) without P being existence dependent on Q, as shown in the next (simplified) example.

Example 6.5 House Owners: 'More Deterministic than' Not Resulting from Existence Dependency

HOUSE = build.(buy + sell)*.destroy
PERSON = (. . .).(build + buy + sell + destroy)*.(. . .)

Then HOUSE $\leq$ PERSON, but a house is never existence dependent on a person. As HOUSE and PERSON have more than two event types in common, we need at least one existence-dependent object type in which all these event types are involved. In this example, this is the object type:

PROPERTY_OF = (build + buy).(sell + destroy)

6.3.3 Checking Global Behaviour

Some readers might have wondered why in the library example (Example 6.1), the sequence constraints for COPY do not include the constraints that apply to a loan, for example, that a *return* event should always be preceded by a *borrow* or *fetch* event. The reason for this lies in the fact that COPY and LOAN objects will synchronise on common events.

The domain model specifies object *types*, but the actual system will consist of object *occurrences*. These objects run concurrently and have to synchronise on events in which they jointly participate. The fact that these objects exist

concurrently can be expressed by means of the parallel operator ||. For example, a system composed of one object p of type P and one object q of type Q can be described as p || q. Now since in a domain model we define object *types*, the system can be defined as the *class* P running in parallel with the *class* Q, by which we mean that many instances of P will run in parallel with many instances of Q and whereby some instances of P may interact with some of the instances of Q. The fact that many instances of a class P exist and run simultaneously can be denoted as P~. So the global behaviour of a system composed of two object types P and Q can be written as P~||Q~. In the general case, calculating such global behaviour cannot be done with FSMs. Nevertheless, in one specific case, namely, in the case P and Q are related by an existence-dependency relationship with maximum cardinality one, this global behaviour can be partially calculated by means of FSMs. We'll illustrate this with the example of COPY and LOAN.

Example 6.6 The Library: Calculating Global Behaviour

Suppose that we have the following (slightly modified) definition for the object types COPY and LOAN:

COPY = acquire.classify.(borrow + renew + return)*.(sell + lose)
LOAN = borrow.(renew)*.(return + lose)

The basic definition of the global behaviour of a system composed of many copies and loans can be written as

COPY~ || LOAN~

The definition of the global behaviour of a system composed of only one copy and many loans can then be written as

COPY || LOAN~

We can further simplify this expression by taking the cardinality of the existence-dependency relationship between COPY and LOAN into account. The existence-dependency graph specifies that a copy can have at most one loan at one point in time. Hence, during its life, a copy will participate zero, one or more times in a loan, which is an *iteration* of loan. So, whereas LOAN~ denotes many loans in arbitrary order and with overlapping life spans, LOAN* denotes a series of consecutive loans. The definition of the global behaviour of a system composed of only one copy and a sequence of zero, one, or many loans can then be written as

COPY || LOAN*

So, to find out what the set of accepted scenarios will be, we must calculate the parallel composition of COPY and the iteration of LOAN. The behaviour of a copy that can be on loan zero, one or more times consecutively is

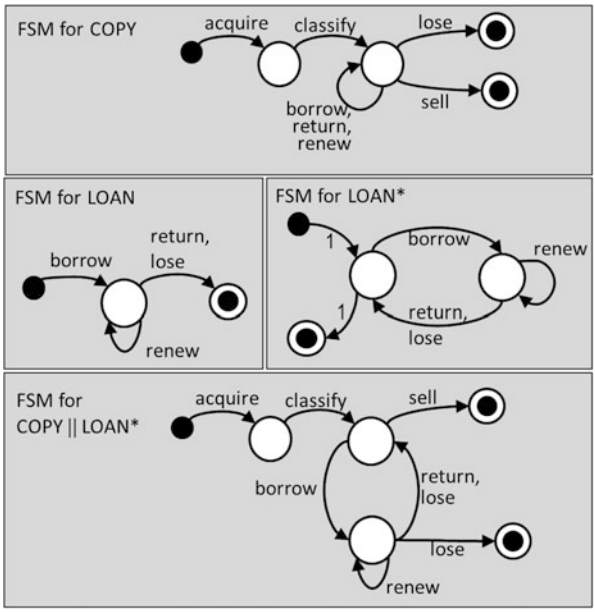


Fig. 6.15 Calculating the behaviour of a copy on loan

COPY || (LOAN)*
= COPY || (borrow.(renew)*.(return + lose))*
= acquire.classify.[borrow.(renew)*.return]*.[sell + borrow.(renew)*.lose]

How exactly such a result is calculated is beyond the scope of this book. It can be done in an axiomatic way or by using algorithms on finite-state machines. Figure 6.15 illustrates the latter method by calculating the same expression by means of finite-state machines.

Notice how the finite-state machine of COPY does not say that a *return* can only occur after a *borrow* event. We can however find this constraint both in the finite-state machines of LOAN and LOAN*. Similarly, the latter finite-state machine does not contain the constraint that *lose* is an ending event type, while this is included in the finite-state machine of COPY. Putting the finite-state machines of COPY and LOAN* in parallel gives the expected result: both constraints are now included in the final finite-state machine. As a result, the sequence constraints of the calculated expression are exactly expressing what we understand by a copy that can be borrowed many times consecutively. For this reason, it is not necessary to include the sequence constraints of LOAN in the definition of COPY: they will automatically be enforced when COPY and LOAN objects run concurrently.

Checking global system behaviour for correctness would require calculating the global behaviour using a process algebra that is able to handle the || and the ~-operator. How this can be done is explained in [82] and [17]. In these publications, we also formally demonstrate how the restriction rule contributes to the guarantee that the global behaviour of a system is free of deadlocks.

6.4 Formal Definition of Interaction by Joint Participation to Business Events

In order to formally calculate and check the behaviour of a system composed of many concurrent objects, the semantics of the parallel operator || must be defined. This operator must express the fact that when two or more objects run concurrently, only those scenarios are valid for which all objects agree on the sequence of common events.

The semantics of the parallel operator could be defined by outlining an algorithm for calculating the parallel composition of two finite-state machines. It is however easier to understand the semantics by considering this as a mathematical operator on regular expressions. So, to explain the semantics of the parallel operator (denoted as ||), we'll first consider the equivalent regular expressions for each finite-state machine.

In order to take into account that for the business events that are not shared by two object types, synchronisation is *not* required, a projection operator '\B' is defined (B will be the set of common event types) that replaces each event type that is not in B by a 'do-nothing' event type. The projection can be applied to individual scenarios as well as to the regular expressions that define the sequence constraints. When it is applied to scenarios, the 'do-nothing' event types can just be dropped from the scenario. When applied to regular expressions, the resulting regular expression can be simplified by using the following axioms:

- For any event type a, choosing between a and a is just a: a + a = a.
- The 'do-nothing' event type can be dropped in a sequence: 1 . a = a = a . 1.
- 'Do nothing' is always included in an iteration (namely, do it zero times), so (a + 1)* = a*.

Example 6.7 The Library: Applying the Projection to Scenarios

Given the life cycle definitions of Example 6.1, the following is a valid scenario for copy:

acquire^classify^borrow^renew^return^borrow^renew^renew^lose

Now let's consider only the events that involve LOAN. Then B = {borrow, renew, return}. If we apply the projection \B to the above scenario, we obtain the scenario:

1^1^borrow^renew^return^borrow^renew^renew^1
= borrow^renew^return^borrow^renew^renew

Example 6.8 The Library: Applying the Projection to Regular Expressions

If the same projection onto the events of LOAN is applied to the regular expression for COPY, the following expression is obtained:

COPY \ {borrow, renew, return}
= [acquire . classify . (borrow + renew + return)* . (sell + lose)] \ {borrow, renew, return}
= 1 . 1 . (borrow + renew + return)* . (1 + 1)
= (borrow + renew + return)*

This regular expression can now be compared to the regular expression of LOAN to assess the absence of contradictions (see Sect. 6.3.2).

The full formal definition of scenarios, projection and parallel composition is given below:

Formal Definition of the || Operator

Regular Expression
Let A be the universe of event types.
R*(A) is the set of all regular expressions over A where e is a regular expression over A if and only if

(a) e = 0 or (b) e = 1 or (c) ∃ a ∈ A: e = a or (d) ∃ e', e" ∈ R(A) such that e = e' + e" or e = e'.e" or e = (e')*

Regular Language
Each regular expression defines a set of scenarios, called its *language*

(1) A* is the set of scenario's over A. A* is defined by

(a) 1 ∈ A*
(b) ∀ a ∈ A: a ∈ A*
(c) Let s, t ∈ A*, then s^t ∈ A*
(d) ∀ s ∈ A*: 1^s = s = s^1

(2) The regular language of a regular expression is a subset of A* defined by

L(1) = {1}
∀ a ∈ A : L(a) = {a}
∀ e, e' ∈ R*(A):
L(e + e') = L(e) ∪ L(e'), L(e.e') = L(e).L(e'), L(e*) = L(e)*
where L(e).L(e') = {s^t | s ∈ L(e) and t ∈ L(e')}
and L(e)* = {1} ∪ L(e) ∪ L(e).L(e) ∪ L(e).L(e).L(e) ∪ . . .

Projection Operator
Let B $\subseteq$ A. Then
$1\backslash B = 1$
$\forall$ a $\in$ A : $(a\backslash B = 1 \Leftrightarrow a \notin B)$ and $(a\backslash B = a \Leftrightarrow a \in B)$
$\forall$ s, t $\in$ A* : $(s \wedge t)\backslash B = s\backslash B \wedge t\backslash B$,
$\forall$ e, e' $\in$ R*(A) :
$(e + e')\backslash B = e\backslash B + e'\backslash B$, $(e.e')\backslash B = e\backslash B \,.\, e'\backslash B$, $(e^*)\backslash B = (e\backslash B)^*$

Object Type and Domain Model
An object type P is a tuple $<\alpha,e> \in <\mathcal{P}(A), R^*(A)>$.
α is the alphabet, which we denote by $S_A P$ and e is the behaviour definition denoted by $S_R P$.
A domain model $\mathcal{M}$ is a subset of $<\mathcal{P}(A), R^*(A)>$.

Parallel Operator
Let P, Q be object types
The alphabet of P || Q is the union of the alphabets:

$$S_A (P \parallel Q) = S_A P \cup S_A Q$$

and the behaviour of P || Q is defined by an expression

e'' $\in$ R*(A) such that
$L(e'') = \{s \in (S_A P \cup S_A Q)^* \mid s\backslash S_A P \in L(P) \text{ and } s\backslash S_A Q \in L(Q)\}$

The formal definition of the consistency checking rules is given below:

Alphabet Rule
$\varphi(S^R P) = S_A P$.
with $\varphi : R^*(A) \rightarrow \mathcal{P}(A)$: $e \rightarrow \varphi(e)$ such that
$\varphi(a) = \{a\}$
$\varphi(e + e') = \varphi(e) \cup \varphi(e')$
$\varphi(e \,.\, e') = \varphi(e) \cup \varphi(e')$

Default Life Cycle Rule
$\forall P \in \mathcal{M} : S^R P \leq (\Sigma\, c(P)).(\Sigma\, m(P))^*.(\Sigma\, e(P))$

Formal definition of *more deterministic than*
$P \leq Q$ if and only if $S_A P \subseteq S_A Q$ and $L(P) \subseteq L(Q\backslash S_A P)$

Restriction Rule
$P \leftarrow Q \Rightarrow P \leq Q$

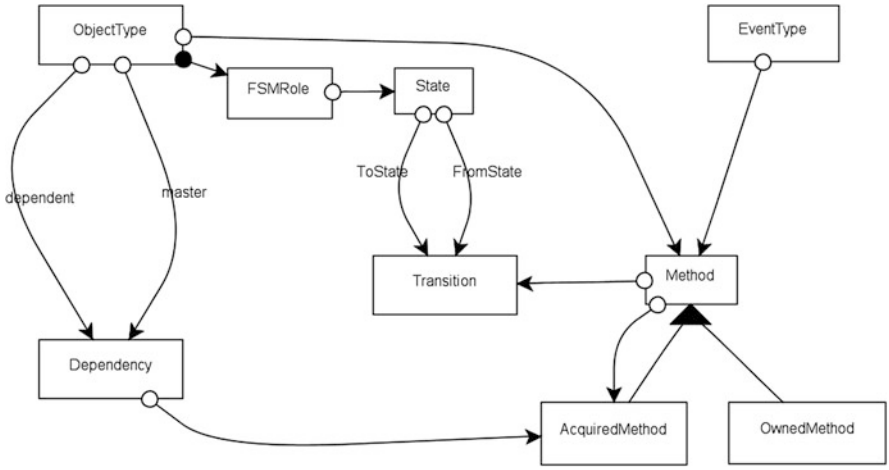


Fig. 6.16 Meta-model including finite-state machines

Notice that if the sequence constraints imposed by each object type on event types are written as a regular expression or as an equivalent graphical representation (such as a finite-state machine), checking the contract can be done automatically by a CASE-tool.

6.5 Meta-model

As parallel behaviour can be specified by defining multiple roles per OBJECTTYPE, each OBJECTTYPE can have one to many FSMs. There is always at least one FSM that represents the default life cycle. Next to this default FSM, zero to many user-defined FSMs can be specified. States only exist in the context of a particular FSMs. TRANSITIONS specify how a METHOD causes a transition from one STATE to another.

The EDG of the meta-model is represented in Fig. 6.16.

Chapter 7
Attributes and Constraints

7.1 Defining Attributes

7.1.1 What Are Attributes?

In essence, without attribute values, objects and business events cannot be identified by human users.[1] In order to make objects and events identifiable for the outside world (including the user), we have to adorn them with values by which we can recognise them. For example, a customer becomes identifiable through his/her name, address and telephone number. An account becomes identifiable through its account number, and a deposit event becomes identifiable through the amount deposited, the account number of the account to which the money is to be deposited and the date and time at which the deposit took place. To this end, in a domain model, the definition of a business object type includes a number of attributes (called static[2] features in UML). In terms of a relational database, the attributes will correspond to the columns of a table definition. Each object will have a value for each of the attributes in the definition of its object type. Together, all these values form the 'state vector' of an object. The range of values an attribute can take can be constrained to a particular type such as a numerical value, a date, a time, text and so on. This is called the data type of the attribute. For example, a customer may have a street name that is of data type 'text', and a ZIP code of that is of data type 'numeric'.

Attribute values provide us with information about objects and business events. Information is the combination of data, meaning (or interpretation) and context. For example, '25061994' is a data item. We don't know a lot when we see such a sequence of digits. But if we are told that the meaning of this sequence of digits is a date in the format DDMMYYYY, then we know that it represents the 25th of June 1994.

[1] They may have a system-generated ID, but this is not practical for users.

[2] Not to be confused with the keyword 'static' in Java.

© Springer International Publishing Switzerland 2014

M. Snoeck, *Enterprise Information Systems Engineering*, The Enterprise Engineering Series, DOI 10.1007/978-3-319-10145-3_7

We can also be told that it represents an amount of money in Euros. We will then interpret the string as the value '25,061,994 €'. But apart from the fact that we know how to interpret the data item, we do not have valuable information yet. For that purpose, we need a context. In the case of the date, the context could be that June 25, 1994, is the date of birth of John Johnson. In the case of the amount, the context could be that this is the 1997 budget of our department. Once we know the context, we have valuable information.

In an object-oriented domain model, object types and their designations provide us with the context for data items. The name of an attribute and its data type provide us with the necessary interpretation. Consequently, attributes describe object types and define information that is of value for the enterprise. It is important to notice that in a domain model, *the information should be of value to the enterprise*. For example, although the shoe size of a customer might be an interesting attribute for a shoe store, it is probably of no value at all for a bank. As a result, it makes no sense to model an attribute 'shoe size' for the object type CUSTOMER in a banking domain model. But it can be sensible to do so in a shoe store domain model.

Besides being of value to the enterprise, attributes should only be included *if the information can be traced and maintained*. For example, it would probably be very interesting for a bank to have detailed information on the total wealth of its customers, but due to the private nature of this information, it is doubtful whether customers would be willing to provide their banker with this information. Even when information can be traced, it may well be too costly to update this information. For example, for an association of alumni, it may well be interesting to know the professional address of their members. Although it is probably not difficult to get this information when a new member is registered, it might turn out to be too costly to update this information as people change jobs. In this case, including the professional address as an attribute for the object type MEMBER would result in 'polluted information': since the attribute is available, users might use this information. But if the information is not updated properly due to the high cost or impossibility to trace changes, users may well be unaware of the fact that they are using outdated and wrong information.

In summary, attributes are essential in satisfying information needs. In this respect, the specification of attributes should be as complete as possible, but at the same time other aspects such as cost and data quality should be taken into account.

7.1.2 The Difference Between Object Types and Attributes

One of the difficult questions in conceptual modelling is whether something should be modelled as an additional object type or as an attribute. Unfortunately, it seems to be impossible to define rules that enable analysts to make clear-cut choices in all situations. The rules listed below are intended as a help for analysts: they outline a number of directives to consider when one is doubting between modelling

something as an attribute or as an object type. However, relying on common sense and intuition will remain necessary.

Before deciding to model something as an object type, one should always check whether the concept has the characteristics defined in Sect. 3.2.3.1. Some of these characteristics are definitely not applicable to attributes:

- Objects have an identity that is independent of the value of their attributes. Attribute values don't. For example, let us assume that student Jef and child Pete are related to the same person Mary who is their teacher and mother, respectively. Assume that PERSON has an attribute 'name' and that Pete tells us that his mother's name is not 'Mary' but 'Mary-Ann'. If you change the value of the name of a person, in this case change the value 'Mary' into 'Mary-Ann', the student and the child will still remain connected to the same person (see Fig. 7.1). The same reasoning does not apply to, for example, the concept of a date. Imagine that the date of birth of Mimi Smith is the same as the date of promotion of John Doe. If date is defined as a business object type with attributes 'day', 'month' and 'year', then Mimi Smith and John Doe will be connected to the same date. However, if you change the value of the 'year' attribute because the year of birth for Mimi is wrong, then it is not so obvious that the connection between John Doe and that date remains valid. The reason for this is that date should be defined as an attribute rather than as an object type.
- Objects exist for a certain time and have a meaningful life cycle with at least one creating and one ending business event. Attributes have no meaningful life cycle. A date exists once and forever; it does not exist for a certain period of time only. Nor has it a life cycle with a creating and ending event. In particular, *end_of_date* can hardly be seen as a meaningful business event type. This does not mean that there are no operations on attributes. But the difference is that operations on business object types have to be the result of business events. In general, operations on attributes are simply the operations associated with their data type. For example, 'to_upper' is an operation associated with the data type 'string' that turns all characters in the string to uppercase characters. It is not a business event.
- Objects can be described by a number of properties. Attributes are the describing elements themselves. For example, a product can be described by its product number, its colour, its price, etc. But the product number cannot be described by

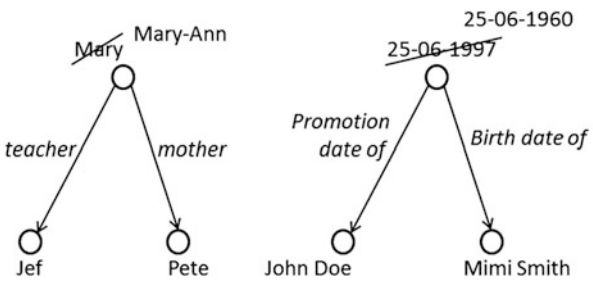


Fig. 7.1 Object types versus attributes

means of other properties. Therefore, product number is an attribute and not an object type.

Additional considerations are:

- Each object is the context in which attribute values become information. Values of attributes are not information on their own; they need the context of an object to become relevant information.
- Adding an object type to a domain model costs more in terms of development effort and required maintenance than adding an attribute. On the other hand, modelling something as an object type allows for more functionality than modelling something as an attribute. For example, in the library model introduced in Chap. 3, the privileges of members were said to be an attribute of MEMBER. A more elaborated model where PRIVILEGE would be modelled as an object type allows for more functionality. For example, it would then be possible to define categories of members and attach privileges to certain categories. It is then also easily possible to change the privileges of all members of a same category. This is not possible, or at least more difficult to handle, if privileges are modelled as attributes of MEMBER.
- Users must be willing to manage the information as an object type. This means, for example, that each time a value is entered, users must check whether a new occurrence must be created or whether the object already exists. In a library, 'author' can be an attribute of a book or an object type on its own. In the latter case, when a new book is classified, the users have to search the existing authors to see if the author of the new book already exists in the system. Only if the author cannot be found, a new author object must be created before classifying the book. If 'author' is an attribute, it suffices to enter the appropriate value for the 'author' attribute. The value of the author will then be duplicated as many times as there are classified books of this author.

Although the considerations listed above can help to identify attributes, choices are domain dependent, and even within one domain, in some cases it is not always clear whether something should be modelled as an attribute or as an object type. This is further illustrated by the following two examples.

Example 7.1 Stock Management

In a stock management system, the object type PEN has an attribute 'colour': we can have blue pens, red pens, green pens and black pens. In this context the value 'ultramarine blue' has no meaning on its own. It is just a describing element for pens. We should therefore model 'colour' as an attribute of STOCK ITEM. On the other hand, in a paint factory, the value 'ultramarine blue' probably evokes a lot of information: it represents a particular mixing of pigments and has a number of characteristics like resistance to sunlight, for example. In this kind of domain, it is valid to model COLOUR as a business object type.

Example 7.2 Product Pricing

In a (simple) sales company domain model, one of the attributes of the object type PRODUCT is 'price'. Since it must be possible to modify the price of a product, for example, to correct a mistake, an event type *change_price* or *change_product_details* is required. On the other hand, changing the price of a product might also be the result of the fact that prices evolve over time (rather than a mistake). Let us now assume that it is required that there is an information need about the evolution over time of product pricing. In this case, it is a valid option to model PRICING_SCHEME as an object type and to include attributes for describing the period of validity of a pricing scheme in the definition of PRICING_SCHEME. As a result, changing the price scheme of a product because of new sales conditions now results in the definition of the business event types *cr_pricing_scheme, mod_pricing_scheme* and *end_pricing_scheme*.

This last example also illustrates how archiving and versioning considerations can make a domain model grow. It is likely that in a first pass of modelling, price will be modelled as an attribute of PRODUCT but that later on, archiving and versioning consideration may lead to converting price/pricing scheme into an object type.

7.1.3 Data Types

Each attribute is assigned a data type: number, string, date and so on. In object-oriented implementation technology, some of these data types are object types on their own. For example, in Java, STRING is a class with own attributes (such as length) and a whole bunch of methods to manipulate strings. However, although STRING is an object type in an object-oriented implementation, it is not a *business* object type. It rather is an *implementation* class.

As business objects are connected to other object types via associations, object types can refer to one another. *At implementation time*, the associations may be converted to attributes having a business object type as their data type. For example, as a loan refers to a member and a copy, *at implementation time*, the class definition for LOAN can have an attribute 'borrowing_member' of type MEMBER and an attribute 'borrowed_copy' of type COPY.

The phrase 'at implementation time' in the above paragraph is crucial. In a conceptual model, one should never define attributes that have a business object type as data type: this should be modelled as an association between the two object types.

Once the domain model has been augmented with attribute definitions, it is sufficiently complete for (partial) code generation. The actual rules and principles for the transformation of a model to platform-specific models and code along the principles of model-driven engineering will be the subject of Chap. 1.

7.2 Defining Constraints

The domain model has to capture all relevant business rules. Some of these rules are expressed in the EDG, some are expressed in the OET and finally, some are expressed as sequence constraints in FSMs. Some business rules however cannot be expressed by any of these three techniques. They have to be formulated as additional constraints in the class definition of the business object type. The first type of constraints has to be satisfied at all times. They are called *class constraints* or invariants. The second type of constraints is formulated as preconditions to methods and applies only when the method is invoked: objects of that business object type will only accept an event if all the preconditions for the method are satisfied. These constraints are called *method constraints*. The constraints will be formulated in an OCL-like way or, to avoid complex OCL-expression, in a pseudocode-like way. A detailed definition of OCL is beyond the scope of this book. However, for the sake of clarity, we briefly outline the basic principles of OCL:

– All attributes defined for object types and all variables and parameters of methods can be used in constraint expressions. Attributes of master object types and dependent object types can be addressed by using the path expressions self.master.attribute_name and self.dependent.attribute_name, respectively.[3] If the associations have role names, the role names can be used instead of the name of the class: rolename.attributename.
– Attributes of dependent object types with a cardinality of one result in a single value on which operations can be applied according to the data type of the attribute (e.g. mathematical operations on attributes with a numerical data type). Attributes of dependent object types with a cardinality of many are more difficult to handle as the reference is in fact a set of occurrences. Constraints on sets can be expressed in terms of set operations such as size(), count(), sum(), forall(), min(), max(), etc. preceded by an arrow. For example:
  - The number of occurrences in a set is referred to by means of the operation → Size().
  - Iteration over all instances in a set is denoted by the operation → Forall.

Example 7.3 Constraint: No More than 5 Loans

A constraint that specifies that a member should always have less than 5 (not ended) loans can be written as:

```
class Member
invariant
loan → Count (state <> ended) < 5
```

[3] The use of the prefix 'self' is optional.

It is important to notice that the specification of constraints does not mean that the constraints are automatically enforced. The philosophy of preconditions is that they specify under which conditions the methods will perform correctly. Different implementation strategies can be followed to ensure the satisfaction of preconditions. The simplest strategy is that the class that calls the method has the responsibility to ensure, before calling the method, that the preconditions are satisfied (e.g. by including the necessary if-then-else statements) [57]. An alternative strategy is that the method checks the precondition satisfaction itself and throws an error in case of non-satisfaction. Finally, the method can also be implemented in such a way that it covers all exceptional cases. The choice between different implementation strategies will result in different transformation rules (see Chap. 11).

The following paragraphs describe four different types of constraints that can be specified.

7.2.1 Uniqueness Constraints

Uniqueness constraints are class constraints that are the equivalent of defining candidate primary keys in the relational model. Objects have a unique (system-defined) identity, but this identity is not visible for the user. However, it may well be required to have a visible and unique identification for objects. For example, it may well be necessary to have unique personnel numbers, invoice numbers or order numbers. These kinds of attributes are subject to uniqueness constraints. Sometimes, uniqueness constraints are spread across more than one attribute. For example, if order lines are numbered, we would probably want this number to be unique within one order, but not across all orders. This can be specified by defining that the combination of the order number and the order line number has to be unique.

Uniqueness constraints are part of the class definition of the object type to which the uniqueness constraint applies. They are specified by means of the keyword **isUnique** followed by the list of attributes, the combination of which has to have unique values over all occurrences of the class. For example:

```
class Order: invariant isUnique (order_number)
class OrderLine: invariant isUnique (Order, line_number), isUnique
(Order, Product)
```

One has to be careful not to define uniqueness constraints on attributes when the uniqueness is only accidental. We should only define uniqueness constraints when this is a real business rule.

7.2.2 Attribute Constraints

Attribute constraints are another kind of class constraints, which, more in particular, restrict the possible values of attributes. Of course, assigning a data type to an attribute is already some kind of constraint, but defining additional constraints can be necessary. For example, the name of a member can be of data type STRING, and in addition it can be required that the name is fully alphabetical. This particular type of attribute constraint is also called attribute domain constraint or validation constraint.

Sometimes constraints on the possible values of attributes refer to the value of another attribute within the same class. For example, in the library model, a constraint can be put on the attribute 'date_to_return' of the object type LOAN so as to ensure that this date is always later than the date the loan is registered.

Example 7.4 Constraint: Return Date After Loan Date

```
class Loan
  invariant
  date_to_return > date_of_loan
```

[4]

7.2.3 Method Constraints

Some constraints are related to a single event rather than as an invariant to an object type as a whole or to attributes. For example, in the project administration example in Chap. 5, it was said that when a project is closed, all registrations for that project should be closed as well. As *close_project* and *close_registration* are not ending events, this type of constraint is not a referential integrity constraint. The requirement that all registrations should be closed doesn't hold permanently: it only holds at the moment the closing of a project is requested. It must therefore be specified explicitly and it is attached to the *close_project* method. Method constraints can refer to all the attributes of the class and to the parameters of the method.

Example 7.5 Constraint: Closing Projects Require Closed Registrations

```
class Project
close_project ()
  precondition
    registration → Forall (state = "closed" or state = "ended")
```

[4] Strictly speaking, the comparison operator '>' cannot be directly applied to dates: one should use the features defined in an implementation class Date, e.g. after(d:Date). For the sake of simplicity, we omit this and use direct comparison for those data types where the intended semantics of comparison and equality is intuitively clear.

Example 7.6 Constraint: No Refunding After the Deadline

For conference registrations, refunding the registration fee is only possible before a particular deadline:

```
class Conference
refund ()
  precondition
    date(today) ≤ refund_deadline
```

Example 7.7 Constraint: No Overdrawing of Accounts Beyond Credit Limit

A bank requires that when money is withdrawn, the resulting sum of the balances of all the accounts of the customer is not below his/her credit limit:

```
class Customer
withdraw (amount)
  precondition
    (account.balance → sum()) - amount >=credit_limit
```

7.2.4 Referential Integrity and Sequence Constraints

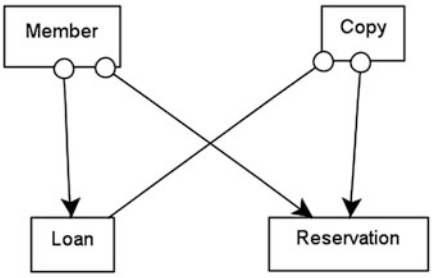
Referential integrity constraints and sequence constraints are a kind of method constraints as well, but their explicit specification is in fact redundant. Indeed, these constraints can be generated from the EDG and finite-state machines, respectively. Hence, their elaboration is a part of the transformation to implementation. However, in order to make a class definition fully self-contained, we can include a description of the generated (or specified) preconditions for the relevant methods. In any way, the preconditions as described here are not readily fit for implementation: they need to be transformed to a form suited for the target implementation platform.

Since they are the consequence of the concept of existence dependency, all referential integrity rules can be generated from the EDG and the finite-state machines. They are generated (or specified) as preconditions for ending and creation events. Each ending method of a master object type will have a precondition for each of the dependent object types of one of two forms:

```
dependent → Forall(state = a_final_state)
```

or

```
dependent.state = a_final_state
```



	Member	Copy	Loan	Reservation
register_member	O/C			
stop_membership	O/E			
acquire		O/C		
sell		O/E		
classify		O/M		
borrow	A/M	A/M	O/C	
renew	A/M	A/M	O/M	
return	A/M	A/M	O/E	
lose	A/M	A/E	O/E	
reserve	A/M	A/M		O/C
cancel	A/M	A/M		O/E
fetch	A/M	A/M	O/C	O/E
	A/M	A/M		

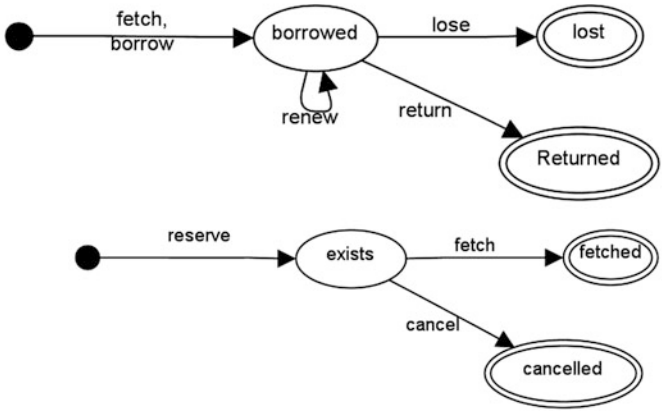


Fig. 7.2 Domain model for a simple library with reservations

Assume a simple model for a library with reservations. EDG, OET and the non-default FSMs are given in Fig. 7.2.

For this model, the preconditions for *end_copy* and *lose* methods in the class COPY are:

```
class Copy
end_copy ()
  precondition
    reservation → Forall(state = fetched or state = cancelled)
    loan.state = lost or loan.state = returned
lose ()
  precondition
    reservation → Forall(state = fetched or state = cancelled)
    loan.state = lost or loan.state = returned
```

Similarly, existence-dependency relations with cardinality one give rise to additional preconditions. In the class of the master object type, we have to ensure that when a new dependent object with cardinality one is created, either it is the first dependent or the previous dependent is in a final state. The general form of this precondition is as follows:

```
class master_object_type
create_dependent - with cardinality 1
  precondition
    dependent = void or dependent.state = a_final_state
```

Assuming that ended objects are not physically removed, then the cardinality 1 prohibits having more than one *living* dependent, but there might be many dependents in a final state.

For the library example, this means that the borrow method in the class COPY has a precondition that is as follows:

```
class Copy
borrow ()
  precondition
    loan = void or loan.state = returned or loan.state = lost
```

Finite-state machines are the basis for defining the state indicators and preconditions for methods. The easiest way to illustrate how FSMs give rise to preconditions is to represent them as state transition tables. From these state transition tables, we can derive the required value of the state indicators before acceptance of a business event and the new value of the state indicators after occurrence of the business event.

A state transition table can be represented in two ways. The most compact representation has three columns. In the first column the methods are listed; the second and third columns represent the transition by giving the state from where the transition comes and the destination state of the transition. The initial state is indicated as a dash and the final states are indicated with an 'F' between parenthesis. For example, the state transition table for the finite-state machine in Fig. 6.5 on page 109 is given in Fig. 7.3.

Fig. 7.3 State transition table for Fig. 6.5

Method	From state	To state
acquire	-	1
classify	1	2
borrow	2	3
renew	3	3
return	3	2
lose	3	4
declassify	4	5
declassify	2	5
end_of_copy	5	6(F)

Fig. 7.4 Second representation for state transition tables

Method	From state						
	-	1	2	3	4	5	6(F)
acquire	1	ε	ε	ε	ε	ε	ε
classify	ε	2	ε	ε	ε	ε	ε
borrow	ε	ε	3	ε	ε	ε	ε
renew	ε	ε	ε	3	ε	ε	ε
return	ε	ε	ε	2	ε	ε	ε
lose	ε	ε	ε	4	ε	ε	ε
declassify	ε	ε	5	ε	5	ε	ε
end_of_copy	ε	ε	ε	ε	ε	6	ε

A more elaborated representation has one row per event type and one column per state. Each cell contains either the destination state of the transition or the error symbol 'ε' (meaning that the transition is not allowed from the state indicated in the column head). Figure 7.4 shows such a transition table for the same finite-state machine.

This second representation has the advantage that all erroneous transitions are identified and that a meaningful error message can be attached to each of the error symbols. It is also easy to see that the finite-state machine is deterministic because there is at most one destination state in each of the cells.

Notice that even for default life cycles, a state transition table can be specified: when an object type has more than one ending event type, it can be interesting to define different final states in order to make a difference between different causes for ending a life cycle. For example, one can specify that a *return* event brings a loan in state 'returned' (normal end) while a *lose* event brings a loan in state 'lost' (abnormal end).

When the technique of roles is used to specify sets of parallel sequence constraints by means of multiple finite-state machines for one object type, there are two possible options. The first one is to have one state transition table and one state indicator associated to each role (including the default life cycle). The second solution is to integrate all role definitions into a single finite-state machine by using the algorithms for parallel composition. We then need only one state indicator and a single state transition table.[5]

[5] For the generation of a working prototype with the JMermaid tool suite, it is required to work along the second option: all roles should be combined into a single FSM that is indicated as the one to be used for code generation.

7.3 Defining Methods

Just as attributes make objects identifiable, the difference between events becomes identifiable by defining the effect of an event on the participating objects as (pseudo) code in the method bodies and by defining the attributes that are required to perform the operations in these method bodies. The definition of the method parameters, variables and bodies is however considered to be part of the design step of a project and is therefore not further addressed here.

7.4 Aliases

As briefly explained in Chap. 5, sometimes an object type is *directly* or *indirectly* dependent on the same master object type along two different existence-dependency paths. Such joining existence-dependency paths lead to the *multiple propagation* of business event types to one object type. Whenever two or more objects of the same class simultaneously participate in a single event, the event might have a different effect on each object, leading to the definition of aliases for a business event type. The following examples further illustrate the use of aliases in combination with multiple propagation, making use of preconditions and method body to illustrate the use of aliasing event types.

Example 7.8 Sales and Billing: Multiple Propagation Constraint

As an example, consider the EDG in Fig. 7.5. This EDG describes the sales activities of a company. Every customer company has several existence-dependent contacts: these are the people that work for that customer company and are the contacts for sales and billing.

As can be seen from the OET in Fig. 7.6, multiple propagation occurs for the event types of invoice_line in the classes CONTACT and CUSTOMER. There are indeed two paths from INVOICE LINE to CONTACT and CUSTOMER: one via INVOICE and one via ORDER LINE and ORDER. According to the business rules, it is allowed to have different contacts for the invoice and the order: the person connected to the order is the sales contact (i.e. the person who signs the order form) and the person connected to the invoice is the billing contact (i.e. the person to send the bills to). It is however required that these people should belong to the same customer company. As a result, a constraint is required in the class INVOICE LINE to ensure that orders are invoiced to people from the same company as the one from those who placed the order:

```
class Invoice_line
invariant
  invoice.billing_contact.customer
  = order_line.order.sales_contact.customer
```

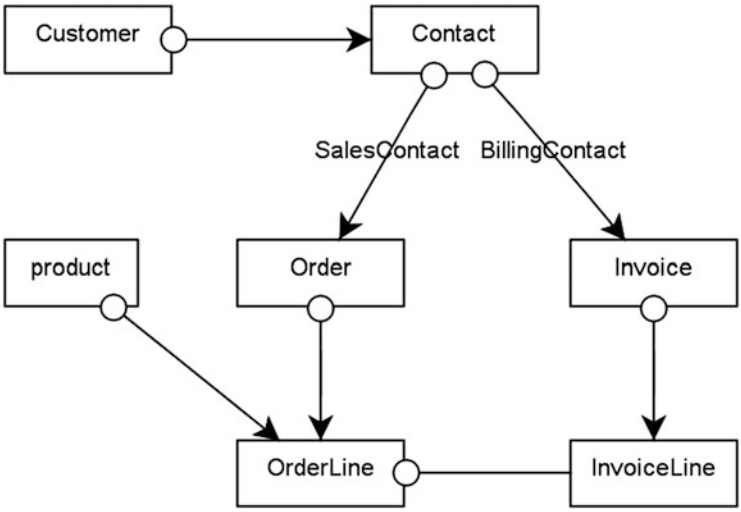


Fig. 7.5 EDG for sales and billing

	CUSTOMER	CONTACT	PRODUCT	ORDER	INVOICE	ORDERLINE	INVOICELINE
EV_cr_Customer	O/C						
EV_end_Customer	O/E						
EV_cr_Contact	A/M	O/C					
EV_end_Contact	A/M	O/E					
EV_cr_product			O/C				
EV_end_product			O/E				
EV_cr_Order	A/M	A/M		O/C			
EV_end_Order	A/M	A/M		O/E			
EV_cr_Invoice	A/M	A/M			O/C		
EV_end_Invoice	A/M	A/M			O/E		
EV_cr_OrderLine	A/M	A/M	A/M	A/M		O/C	
EV_end_OrderLine	A/M	A/M	A/M	A/M		O/E	
EV_cr_InvoiceLine	A/M A/M	A/M A/M	A/M	A/M	A/M	A/M	O/C
EV_end_InvoiceLine	A/M A/M	A/M A/M	A/M	A/M	A/M	A/M	O/E

Fig. 7.6 OET for sales and billing

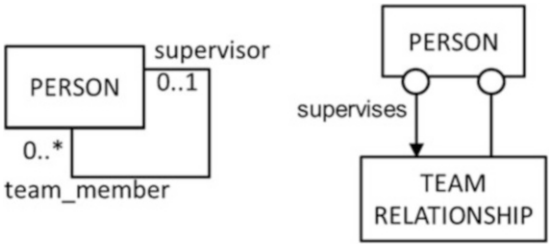


Fig. 7.7 Team relationships between people

	PERSON	TEAM RELATIONSHIP
…		
cr_team_rel	A/M, A/M	O/C
end_team_rel	A/M, A/M	O/E
…		

Fig. 7.8 OET for team relationships between people

Because of this constraint, only one occurrence of CUSTOMER will be involved in the owned event types of INVOICE_LINE. This is indicated by greying out one of the participations. For CONTACT, two occurrences remain involved in the aforementioned event types. However, as long as the method bodies and preconditions remain empty or the same for both occurrences, there is no need to define an alias.

Example 7.9 Team Relationships: Aliases and Constraints

Figure 7.7 shows an example of a team relationship between people. On the left is the UML representation defining that each person supervises zero to many team members and that each person has zero or one supervisor. On the right side, the unary relationship has been transformed to an existence-dependent object TEAM RELATIONSHIP. Every person is involved in zero or one TEAM RELATIONSHIP instance as a team member and each person is involved in zero to many TEAM RELATIONSHIP instances as a supervisor (namely, one for each supervised team member). Notice that it is important to provide the existence-dependency relationships with a name since they play different roles. In the class definition, the names of the dependency relationships will be used to reference the dependent object types along the two different existence-dependency relationships.

The object-event table for this example is given in Fig. 7.8.

As we can see from this example, the creation of a team relationship involves two people: the supervisor and the team member. The method body and precondition required for these persons might be different depending on the role they play. Suppose, for example, that a person cannot be assigned a supervisor if he/she has the rank of director and that teams are limited to 20 people at most. This requires the definition of two aliases for the event type *cr_team_rel*. This means that the business event cr_team_rel should relate to two methods with distinct names. In

	PERSON	TEAM RELATIONSHIP
…		
cr_team_rel	A/M (become_team_member) A/M (become_supervisor)	O/C
end_team_rel	A/M (end_team_membership) A/M (end_supervisorship)	O/E
…		

Fig. 7.9 OET with aliases for cr_team_rel

the OET given in Fig. 7.9, we represent these distinct names in a line below the indication of the object's participation to the business event.[6]

Below is the class definition of the object type person. Notice that if a business event type with multiple participants from the same class does not require a different interpretation for each of the involved instances, it is not necessary to define aliases for this business event type: only one method needs to be defined that will be invoked on all participating instances.

The required business rules for this model are:

- A person with the rank of director cannot supervise a team.
- A person should not be his/her own supervisor.
- The supervisor should not be supervised by his/her team/member, directly or indirectly.
- A person should not be twice in the same team.
  This constraint is in fact already realised by means of the cardinality constraint of the ED-relationship 'in _team_of'.
- A team can never count more than 20 members:

```
class Person
become_team_member (t: Team_relationship)
  precondition
    -- A person with the rank of director cannot supervise a team.
    rank <> director
    -- Generated precondition enforcing the cardinality of one:
    in_team_of = void or in_team_of.state = ended

become_supervisor (t: Team_relationship)
  precondition
```

[6] In the JMermaid tool, the distinct names will not be visible in the OET, but only by opening the method's details.

```
  -- A team can never count more than 20 members
  supervises → count (state<>ended) < 20

class Team_relationship

invariant
  -- A person should not be his/her own supervisor:
     supervisor <> team_member

cr_team_rel (new_boss, new_member: Person) is
  precondition
  -- A person should not be his/her own supervisor
     new_boss <> new_member
  -- Member not already in team of supervisor
     not (new_boss.supervises.team_member → includes(new_member))
  -- Boss is not in team of member.
  -- This should be checked by means of recursively.
  -- Collecting all team_members of the member.
  -- This can be done with the 'Closure' function in OCL:
     not (new_member → Closure(supervises.team_member) → includes
     (new_ boss))
```

In the case of unary relationships, the definition of aliases is always required if one wishes a bidirectional implementation of the existence-dependency relations. The example below is a variation on the unary relationship. This second example is a many-to-many relationship taken from a network installation and configuration problem domain where the installation of one part requires other parts to be already installed (a typical bill-of-material type of problem).

Example 7.10 Part-Type Prerequisites: Aliases and Constraints

The UML diagram and EDG in Fig. 7.10 shows the part of the model that specifies which types of parts are prerequisites for other types of parts.

In the OET (Fig. 7.11) aliases are identified for the event types cr_prerequisite and end_prerequisite. The specifications are similar to the previous example. The major difference is that because a part type can be required by many part types, both dependencies have a cardinality of many.

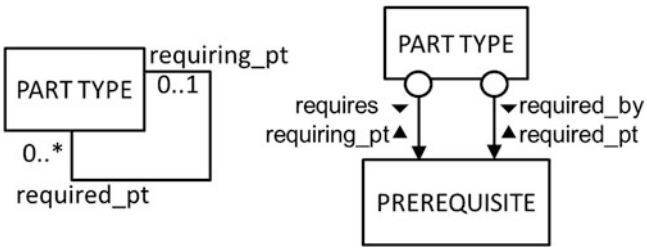


Fig. 7.10 EDG for part-type prerequisites

	PART TYPE	PREREQUISITE TYPE
cr_Part Type	O/C	
end_Part Type	O/E	
cr_Prerequisite	A/M (cr_prerequisite_as_requiring) A/M (cr_prerequisite_as_required)	O/C
end_Prerequisite	A/M (end_prerequisite_as_requiring) A/M (end_prerequisite_as_required)	O/E

Fig. 7.11 OET for part-type prerequisites

```
class Prerequisite
invariant
  required_part <> requiring_part

cr_prerequisite (required, requiring: Part_type)
  precondition
    -- A part type cannot require itself:
    required <> requiring
    -- A part type can be required by another part type only once:
    not (requiring_part.requires.required_part → includes(required)
    -- Requiring part type is not required by required part type
    --(e.g. if a requiring part A requires required part B, then part B
    --as a requiring part cannot require part A as a required part).
    -- This condition should be satisfied across the complete hierarchy.
    -- For example if A requires B, which in turn requires C, then C
    -- should not require A.
    -- This should be checked by means of recursively collecting all
    -- required part types of the object 'required' with the Closure
    -- function
    not (required → Closure(requires.required_part) → includes(requiring))
```

A final example is taken from a domain model for a pharmacy.

Example 7.11 Pharmacy: Multiple Propagation

Suppose that a pharmacist is allowed to replace the medicine prescribed by the doctor to the patient by another medicine that is, e.g. cheaper or better suited for the patient, provided the medicine is of the same category. The pharmacist is however required to keep a record of the exact prescription and of what has been sold to the patient. A sample domain model is proposed in Fig. 7.12 (EDG) and Fig. 7.13 (OET).

As one can see from the OET in Fig. 7.13, the owned event types from SALES_ON_PRESCRIPTION are propagated two times to medicine and category. In every sales there are indeed two medicines involved: the one that is sold and the one that was prescribed by the doctor. They should however belong to the same category. As a result, an invariant constraint is specified in the class

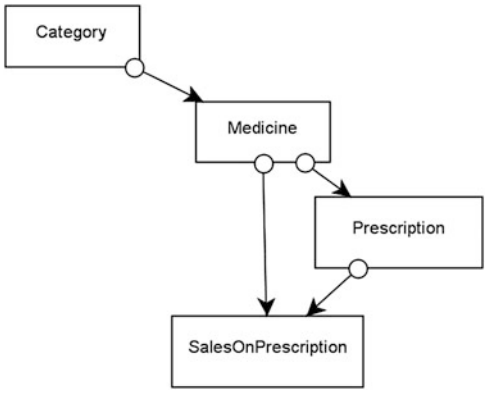


Fig. 7.12 EDG for a pharmacy

	CATEGORY	MEDICINE	PRESCRIPTION	SALES ON PRESCRIPTION
cr_category	O/C			
end_category	O/E			
cr_medicine	A/M	O/C		
end_medicine	A/M	O/E		
cr_prescription	A/M	A/M	O/C	
end_prescription	A/M	A/M	O/E	
cr_sales on prescription	A/M, A/M	A/M (cr_sales_sold) A/M (cr_sales_prescribed)	A/M	O/C
end_sales on prescription	A/M, A/M	A/M, A/M	A/M	O/E

Fig. 7.13 OET for the pharmacy

SALES_ON_PRESCRIPTION, requiring that the category of the sold medicine be the same as the category of the required medicine:

```
class Sales_on_prescription
invariant
  self.prescription.medicine.category = self.medicine.category
```

The class MEDICINE requires the definition of aliases for the event type *cr_sales_on_prescription*, since only the stock of the sold medicine should be lowered with the amount of sold medicine. In addition, the selling of a medicine requires sufficient stock. No method body or preconditions were identified for *end_sales_on_prescription*. As a result, there is no difference in method depending

on the role played by the medicine and hence there is no need to identify aliases for this event type:

```
class Medicine
cr_sales_sold (quantity : Integer)
precondition
    stock >=quantity
```

7.5 Meta-model

The meta-model can be further expanded with the concept of attributes. In order to capture the fact that the model can also contain constraints, INVARIANT is added as a dependent of OBJECT TYPE and PRECONDITION as a dependent of METHOD. The meta-class CONSTRAINT can be used as anchor point to merge the MERODE meta-model with the OCL meta-model (Fig. 7.14).

Additionally, many constraints can be defined on this meta-model to capture various model quality rules.

An example of a syntax rule is that a transition can only connect states that belong to the same FSM. This can be specified as the following multiple propagation constraint:

```
class Transition
invariant
  ToState.FSMRole=FromState.FSMRole
```

As an example of a semantic consistency check, the alphabet rule states that a finite-state machine can only refer to the methods for an event the object type

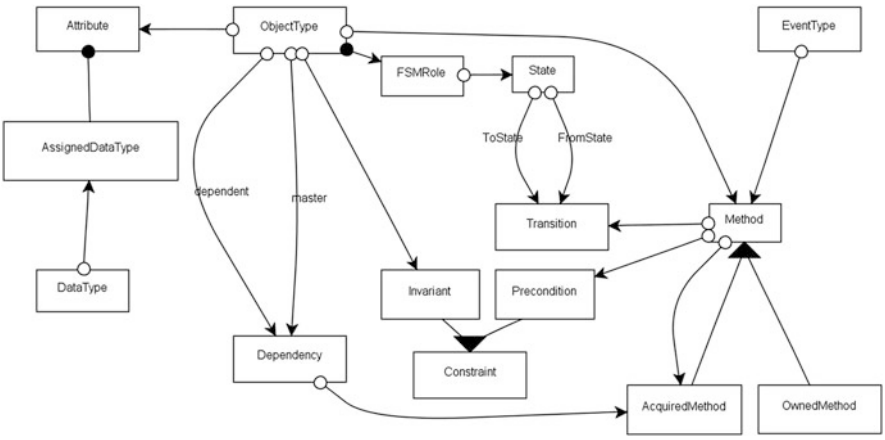


Fig. 7.14 Meta-model with attributes and constraints

participates in. This rule can be formulated as the following multiple propagation constraint:

```
class Transition
invariant
  ToState.FSMRole.ObjectType = Method.ObjectType
```

Many more constraints need to be defined to ensure the quality of models. However, the full list of meta-model constraints is beyond the scope of this book.

Chapter 8
Inheritance

This chapter presents the use of the principle of generalisation/specialisation in conceptual modelling. Generalisation/specialisation is an abstraction principle allowing the definition of a class as a refinement of another class. The more general class is called a supertype, generalisation or parent class and the refined class is then called a subtype, specialisation or child class. With the advent of object-oriented analysis methods, many different ways of how to concretely use the generalisation/specialisation hierarchy in modelling were proposed. Most of the time, however, these proposals were not well integrated with the behavioural aspects in object-oriented modelling. Also today the definition of the generalisation/specialisation relationship is mostly formulated in the context of a static model only. In this chapter we will first address the definition of the concept from the perspective of the class diagram. Subsequently, Sect. 8.2 addresses the behavioural aspects of inheritance. To conclude Sect. 8.3 will address the question on when to use or not to use this construct.

8.1 Inheritance in the Class Diagram

8.1.1 Definition of Generalisation/Specialisation

Generalisation/specialisation has everything to do with classification of objects into types. So far, we have classified business objects uniquely into a single business object type. However, sometimes business objects are much alike, without being really similar enough to be put in the same class. In such cases, we can revert to some sort of classification hierarchy. For example, in a banking context, we say that checking account and savings account both are accounts. We hereby acknowledge the fact that there is a more general classification ‘account’ of which two more specialised versions exist, namely, ‘checking account’ and ‘savings account’. We

© Springer International Publishing Switzerland 2014

M. Snoeck, *Enterprise Information Systems Engineering*, The Enterprise Engineering Series, DOI 10.1007/978-3-319-10145-3_8

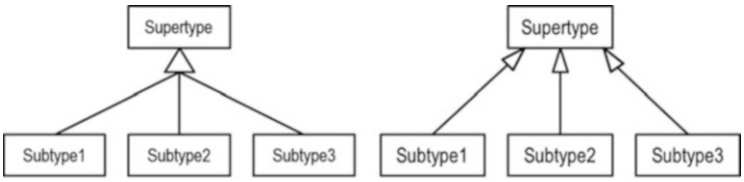


Fig. 8.1 Two UML notations for generalisation/specialisation

model this by capturing the three concepts as business object types and defining ACCOUNT as a supertype or generalisation of both SAVINGS_ACCOUNT and CHECKING_ACCOUNT, which are the subtypes or specialisations. Moreover, the subtypes are said to inherit all the features of the supertype. In this respect, objects that belong to a subtype 'conform' to the definition of the supertype: a checking account conforms to the definition of an account; it has the same properties, but on top of that it has extra properties, which makes it a special case of an account.

Notation-wise, in UML, an inheritance relationship is denoted as a line with a white triangle pointing as an arrowhead to the supertype. In case of multiple inheritance relationships pointing to the supertype, the triangles may be joined. Figure 8.1 shows both notation styles.

Two additional considerations need to be taken into account. First, classification of objects into types can be static or dynamic and, second, classification can be single or multiple [65]. The first distinction refers to the question whether or not an object can change type over time, e.g. like when a student becomes a professor. In the past, some authors (e.g. [25]) have argued that a specialisation has a permanent character as opposed to roles, which model temporal aspects, while other authors have allowed objects to move between classes in a generalisation/specialisation hierarchy (e.g. [31]). As UML targets a very flexible use of the language, it takes the most flexible position on not excluding dynamic classification. In this book we, however, take a stricter stand and favour static classification (also called strong typing), as this leads to a much more simple and clear formalisation.

Single and multiple classification refer to the question of whether or not an object can belong to multiple classes at once. Some authors adhere to multiple classification, not only allowing overlapping subtypes but also viewing generalisation/specialisation as a superset/subset hierarchy [23, 79, 80]. Strong typing-oriented approaches will adhere to single classification and assume the generalisation and specialisation types to have disjoint instance sets.

UML adheres to multiple classification and allows overlapping of subtypes and a subtype to inherit from multiple supertypes, a situation called 'multiple inheritance'. As we aim at the transformation of models to code, and not all programming languages allow multiple inheritance, from a methodological point of view we stick to non-overlapping subtypes and single inheritance only.

The following definition formalises the structural aspects of generalisation/specialisation:

Definition The IS-A relationship is a partial order between classes. When P IS-A Q, we say that P is the specialisation and Q is the generalisation class. In MERODE, the IS-A hierarchy must satisfy the following restrictions:

1. Object types have at most one generalisation type: multiple inheritance is not allowed.
2. Objects belong to a single subtype. Subtypes are by definition disjoint.
3. The IS-A hierarchy must be acyclic.

The classes that are directly or indirectly a generalisation type of a type P are called the *parent* types of P. The *topmost parent type* of a type P is that parent that is on the top of the generalisation hierarchy. Object types that are directly or indirectly a specialisation of type P are called the *descendants* of P. An object type P is both a parent and a descendant of itself. The *cluster* of an object type P is the set of all parents, descendants and siblings of this object type. The siblings of an object type are *all* the objects that are in the same hierarchy. The *extent* of a type P is the set of all object instances of this type. The *deep extent* is the union of extents of all object types that have P as a parent, including the extent of P.

A total specialisation means that the extent of the generalisation class itself is empty: all objects in the hierarchy are in the extent of one of the specialisation types. In such a case, the generalisation type is sometimes also called an 'abstract type'. In UML this is represented by writing the name of the class in italic font and adding the keyword 'abstract' next to the class's name. Since we prefer a purely graphical notation, we'll represent this by a black triangle. When a specialisation is not total, we say it is optional. In this case, the extent of the generalisation object type can contain objects. An optional specialisation is represented by a white triangle (or normal font and no keyword next to the class's name in UML). Figure 8.2 shows the UML and MERODE notation for an abstract class.

Example 8.1 Animals: Generalisation/Specialisation Hierarchy

Imagine an animal world with a limited number of types of animals. Suppose we have cartoon animals which can be either land animals or fish, and within the class of land animals, we discern cats and dogs. We also assume that all animal objects are either cats, dogs or fish. As a result the classes CARTOON ANIMAL and LAND ANIMAL are abstract classes. The corresponding hierarchy is shown in Fig. 8.3.

Single classification means that object types have disjoint extents, even in the presence of generalisation/specialisation. Static classification requires that each object belongs to exactly one and always the same class for the whole time of its existence. As a result, the generalisation and specialisation types have disjoint extents. For the same reason, overlapping subclasses are not allowed, and objects are not allowed to migrate between subclasses or between subclass and superclass. This is different from approaches where the specialisation type is defined as a subset of the generalisation type [23]. However, with a static and single classification approach, the superset/subset hierarchy can still be reconstructed by working with the deep extent of object types (see Fig. 8.4): the deep extent of a specialisation type is always a subset of the deep extent of its generalisation type. Figure 8.4

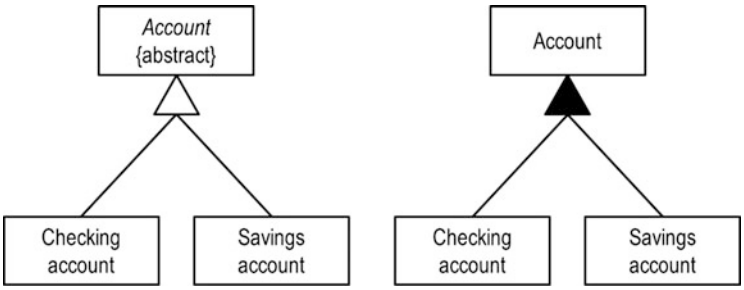


Fig. 8.2 UML notation and MERODE notation for an abstract class

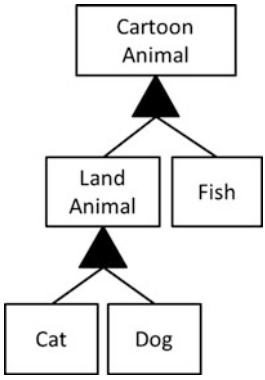


Fig. 8.3 A hierarchy of animal types

pictures the concepts of inheritance hierarchy, extent and deep extent. Figure 8.5 shows the cartoon animal hierarchy with the corresponding set/subset hierarchy of the deep extents of the classes. This demonstrates that apparently contradictory methodological options can be reconciled when using the proper definitions (see Sect. 8.4).

Notice that there is no existence-dependency relationship between supertypes and subtypes. Existence dependency requires the existence of two different objects and that the life of one object is embedded in the life of another object. In a generalisation/specialisation hierarchy, the life of generalisation objects and that of specialisation objects are completely independent of each other. In some methodologies (e.g. [79]), the life of an object in a generalisation/specialisation hierarchy is modelled by decomposing the life of an object across different occurrences in the super-/subclass hierarchy. In that case, each occurrence in a subclass has a matching occurrence in the superclass. Consequently, the conceptual object of the universe of discourse is seen as a composition of all matching objects in the generalisation/specialisation hierarchy. In the definition presented here, each object in the universe of discourse belongs to exactly one and always the same class in the conceptual model. As a result, in MERODE there is not a generalisation object for each specialisation object.

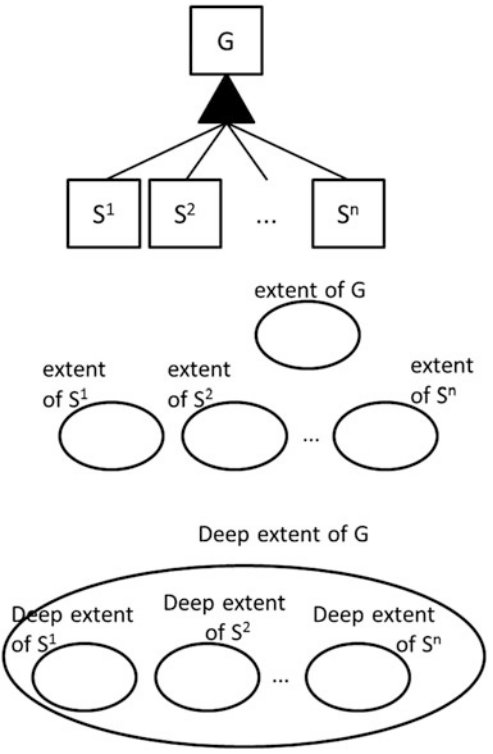


Fig. 8.4 Extent/deep extent and superset/subset hierarchy

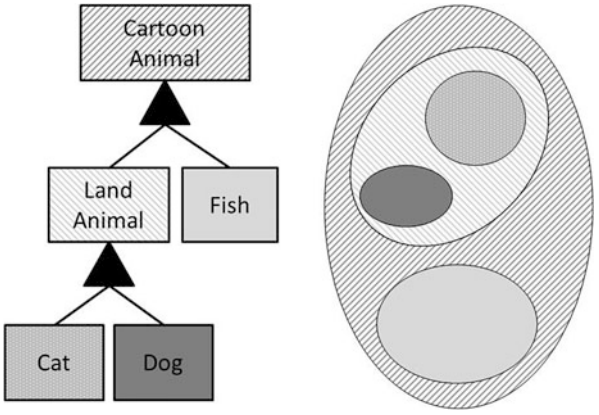


Fig. 8.5 Extent/deep extent and superset/subset hierarchy for cartoon animals

8.1.2 Inheritance

The concept of generalisation/specialisation is strongly connected to the notion of inheritance as it has been defined in object-oriented programming languages. In object-oriented programming an important principle is the Liskov principle of substitutability which states that at any time, when an object of a particular type is expected, any object that *conforms to* that type can be supplied. In other words, when an object of a generalisation type is expected, the object that is actually supplied is allowed to be of one of the subtypes of the generalisation type. So for example, if in a web form one of the input fields is defined as being of type ACCOUNT, then both a checking account object and a savings account object can be provided as input, since CHECKING ACCOUNT and SAVINGS ACCOUNT are subtypes of ACCOUNT and hence *conform to* the definition of ACCOUNT. If on the other hand the type of the input field is defined as being of type CHECKING ACCOUNT, then only a checking account can be provided. Since an object-oriented conceptual model should be platform independent and transformable to (among others) object-oriented implementation technology, the principles for dealing with inheritance at conceptual modelling level should be maximally aligned with the way inheritance is defined in object-oriented programming languages. Fortunately, most concepts can be used at conceptual modelling level in the same way as they are defined at programming level. We say that a specialisation class 'inherits' structure and behaviour from the generalisation class. Specialisation classes are also allowed to 'extend', 'refine' or 'override' the inherited behaviour and attributes. In order to understand the remainder of this paragraph, it is important to understand what is meant exactly by the terms 'inherit', 'extend', 'refine' and 'override'.

Inheritance. A specialisation type inherits all the attribute definitions of the generalisation type. This means that attributes of the specialisation type have the same name and domain as the attributes of the generalisation type. The specialisation type also inherits the methods of the generalisation type. This is the logical consequence of the Liskov principle of substitutability. As objects of the supertype can be substituted by an object of a subtype, subtype objects need to be 'conformant' to the supertype. This implies having the same attributes and the ability to handle the same events as objects from the supertype. Objects of the subtype hence need to define what the effect of each of these event types is on their state. The consequence is that object-event involvements are also inherited from the generalisation type: if the generalisation type has a method to react to a certain event type *e* because it participates in that event type *e*, then the specialisation will inherit that method and will also be able to be involved in that event type *e*.

Next to inheritance of attributes and methods, the associations the generalisation type participates in are also inherited by the subtype. So if an object type P is existence dependent of a master object type Q, then any subtype of P inherits the fact of being existence dependent of Q.

Extension. A specialisation type can extend the class definition it inherits from its generalisation type by adding new attributes, by participating in extra associations

or by being involved in more event types and hence by having additional methods for responding to these additional event types.

Overriding and Refinement. A specialisation type can override the methods it inherits from its generalisation type. Overriding is used when the specialisation type reacts differently to an event type than the generalisation type. Very often, overriding is used to achieve the idea that a specialisation can refine the attributes and behaviour it inherited from its parent. It should be remarked, however, that refinement can lead to situations where the principle of substitutability is violated, namely, in the case where refinement leads to a situation where a subtype refuses actions that are accepted by the parent hereby and breaks the conformance with the generalisation type (see discussions on covariance, novariance and contravariance in [57]). When overriding inherited methods, it is therefore required that the signature of the method remains the same. The signature of the method is its name plus the list of parameters with their domain definition.[1] When the specialisation type needs additional parameters for reacting to an event, we cannot use the technique of overriding. We then need to rely on the technique of extension. In programming, extensions in the class definition of the subtype are not visible to the supertype.

8.2 Behavioural Aspects of Inheritance

8.2.1 Specialisation of Event Types

Example 8.2 Loans at the Bank

In a bank, customers can be granted loans. Excellent customers can acquire different types of bonuses, one of which is a credit bonus. Customers who have a credit bonus can be granted a special discount loan. The domain model for this example is given in Fig. 8.6. The *black* triangle indicates that the specialisation of bonuses is total: there are no 'general' bonuses; each bonus is of a particular type. The specialisation of loans into special discount loans is not total, which is indicated by the *white* triangle.

A first remark should be made about the specialisation of loans. Subtypes inherit all the methods and hence the object-event participations of the supertype. According to the Liskov principle, whenever a loan is expected, a special discount loan can occur as well. As SD_LOAN inherits all the features of LOAN, a special discount loan object can always behave as a loan object, and in this way the Liskov principle is satisfied. However, whenever we wish to create a loan, it should be clear whether we wish to create an object of class LOAN or of the class SD_LOAN. As a result, we

[1] For example in a class ACCOUNT, a method to register the withdrawal of money will need a parameter for the amount (e.g. of type Double), will return no value (return type void) and hence have a signature as follows: void withdraw (Double amount).

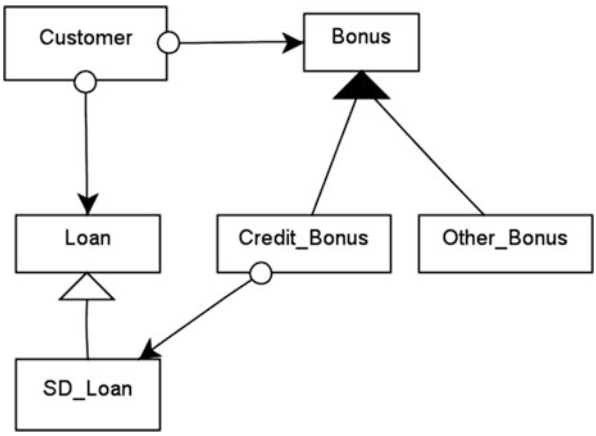


Fig. 8.6 A partial domain model for a bank

cannot use one single business event type to trigger in one case the creation of a loan and in another case the creation of a special discount loan. The way the event type is propagated to master object types will also slightly differ: in case of a special discount loan, we also need to propagate the business event along the proper dependencies of the subtype (in this case this would be propagation to CREDIT BONUS) in addition to the propagation along the inherited dependencies. We thus need two different business event types, such that it becomes clear which kind of loan needs to be created and along which path the event needs to be propagated. As the processing of both events has a shared logic, we will define the event type to create a subtype as a specialisation of the creating event of the supertype.[2] In this case *cr_SD_loan* is a specialisation of the event *cr_loan* and is the creating event type for SD_LOAN.

Secondly in the same line of reasoning, we argue that *any* creating event type of a subtype needs to be a specialisation of a creating event type of its supertype. In other words, a subtype cannot have an own creating event type that is not a specialisation of a creating event type of its supertype. The argument behind this restriction is that a subtype inherits all existence-dependency relationships from its supertype. For that reason, the subtype needs to obey the rules of life cycle start (and ending) imposed on the supertype, in relation to the dependents and masters of that supertype. Each creation of a subtype therefore needs to obey all the rules that apply to the creation of the supertype. This can be made explicit by defining that the creation of a subtype is a specialisation of the creation of its supertype.

Finally, an analogous reasoning can be made for ending methods: as a subtype inherits all existence-dependency relationships from its supertype, it needs to obey the rules of life cycle ending imposed on the supertype in relation to the dependents and masters of the supertype. Each ending of a subtype therefore needs to obey all the rules that apply to the ending of the supertype. This can be made explicit by

[2] Notice that in UML, events are not considered as classifiers and can therefore not be specialised.

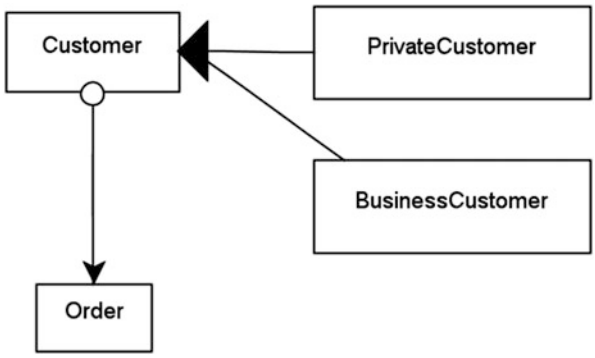


Fig. 8.7 Customer subtypes and orders

defining that the ending of a subtype is either the unmodified inherited version of the ending method of the supertype or a specialisation of the ending of a supertype. Figure 8.7 illustrates this with an example. Customers can be business customers or private people. Both inherit from the object type CUSTOMER that they can place orders. Rules of referential integrity stipulate that a customer cannot be ended as long as it has outstanding orders. This rule is inherited by and needs to be obeyed by the ending events for BUSINESS CUSTOMER and PRIVATE PERSON. Similarly, according to the model in Fig. 8.6, a CUSTOMER object cannot be ended as long as it has outstanding LOAN objects. As these LOAN objects can be SD_LOAN object (according to the rule of substitutability), we need to inform the master CUSTOMER object when an SD_LOAN object is ended.

When a specialisation is total, the generalisation class will have no instances. In this particular example, this means that the class of BONUS object types has no instances. Each bonus is always a credit bonus (instance of BONUS) or another type of bonus (instance of OTHER_BONUS). As a result, the event type *e_bonus* will always apply to a credit bonus or an 'other bonus'. But different with a creation event, the ending event will always be called upon a particular object instance, the type of which can be casted. This means that it can always be determined which 'variant' of the code needs to be executed: the one in the class of BONUS or the code in the class of OTHER_BONUS. For the ending of a subtype, we can therefore opt for the inherited method (either overridden or not) or choose to specialise the method further.

The same reasoning applies to modifying methods: they can be inherited or specialised methods from the supertype. Finally, a subtype can *extend* the features inherited of its supertype, not each ending or modify-event of a subtype is a specialisation of an ending or modifying event of the supertype. Subtypes can therefore have their own ending and modifying event types. In the given example, we can define *change_rate* and *change_duration* event types that are applicable to both types of loans even though the effect may be different depending on the object type. Although an object type can extend the features it inherited with its own ending methods, the advantage of using only inherited or specialised methods is that the ending methods of the subtypes inherit all the rules of the ending methods of the supertype.

The decision whether or not to specialise is determined by the question whether the event method of the subtype will have the same signature as the event method of the supertype. If the signature remains the same, specialisation is not required, but the method body can still be overridden in the subtype. If the signature is different, extension is required. In the case of extension, we allow the new event type to be defined as a specialisation of an inherited event type. This is an extension to standard UML practice where inheritance of events is not defined. If the method interface of *change_rate* and *change_duration* remains the same, there is no need to define specialisation event types in the given example.

In case of event specialisation, the subtype *replaces* the event type of the supertype by its own specialised version. For example, *cr_loan* is an O/C method for loan. The event *cr_loan* cannot be used to create an SD_LOAN object. Therefore, a specialised event *cr_SD_loan* is created leading to an S/C method in SD_LOAN. Notice that, in an object-oriented implementation, the event method of the supertype is always inherited and can be reused to implement the specialised event method. However, we do not want to allow the IS-layer using the inherited *cr_loan* method to create an SD_LOAN. To indicate that, in case of specialisation, the inherited creating and ending methods should not be invoked by other components but that the specialised event method should be invoked instead, the inherited method should be marked as 'hidden'. In the given example, the SD_LOAN class will inherit the *cr_loan* and *end_loan* methods from the LOAN class as hidden methods and can use these hidden methods to implement their (public[3]) *cr_SD_loan* and *end_SD_loan* methods. How the 'hidden' character of a method is transformed to code (public, private, etc.) has to be defined in the transformation from PIM to PSM.

In summary, we need to determine for each inherited method whether or not it is applicable without change to objects of the subtype. Inherited methods are indicated with an 'I/' or 'S/' in the OET. If the inherited method is applicable to the specialised object type (possibly after overriding the corresponding method), the I/ must be followed by a C, M or E to indicate the nature of the involvement. If it is not the case, the method is specialised for each subtype, which is indicated by an S/ followed by a C, M or E.

In summary, the rules for filling the OET are as follows:

- Creating event types must always be specialised: a 'C'-involvement therefore always gives rise to an 'S/C'-involvement for the specialisation type.
- Modifying and ending methods can be inherited or specialised.
- There cannot be an owned creating method for a subtype.
- Owned modifying and ending methods are allowed for subtypes.

In the OET, specialisations are indicated by means of subdivided rows and columns. Empty generalisation object types (resulting from a total specialisation) and the rows corresponding to their own event types are shaded. The OET for the

[3] Public in the sense of visible to the IS-layer.

	CUSTOMER	LOAN		BONUS		
			SD_LOAN		Credit_BONUS	Other_ Bonus
cr_customer	O/C					
end_customer	O/E					
cr_bonus cr_credit_bonus cr_other_bonus				*O/C*	*I/C* S/C	*I/C* S/C
end_bonus end_credit_bonus end_other_bonus				*O/E*	*I/E* S/E	*I/E* S/E
cr_loan cr_sd_loan		O/C	*I/C* S/C			
end_loan edn_sd_loan		O/E	*I/E* S/E			
change_rate		O/M	I/M			
change_duration		O/M	I/M			

Fig. 8.8 OET for the banking example, before propagation. Methods in *italics* are hidden methods

banking example with the basic object type/event type involvement and inheritance indications is given in Fig. 8.8.

In the class definition, empty classes (shaded columns) give rise to the definition of an *abstract* class with *abstract* or *deferred* methods for the own event types.

8.2.2 Propagating Along Inherited Dependencies

The principle of inheritance states that the associations in which the generalisation type participates are also inherited by the subtype. So if an object type P is existence dependent of a master object type Q, then any subtype of P inherits the fact of being existence dependent of Q. This means that next to a number of explicitly modelled existence-dependency relationships, a model with inheritance also contains a number of implicit existence-dependency relationships. Figure 8.9 shows these inherited dependency relations for the banking example.

Since LOAN is existence dependent on CUSTOMER, all event types in which LOAN is involved involve a CUSTOMER as well. Therefore, event type involvements are propagated from LOAN to CUSTOMER. It can be argued that this principle of propagation should be extended to the inherited existence-dependency relationships as well.

Suppose that a special discount loan is subject to the possibility that its conditions are changed throughout time, depending on how interest rates fluctuate on the market. This change of conditions is modelled by means of a *change_condition* business event, for which SD_LOAN has an owned method. SD_LOAN has an implicit existence-dependency relationship with CUSTOMER, stating that it is existence dependent of CUSTOMER. Now it seems logical that if the conditions of the special discount loan are changed, this affects the customer as well and may need a verification (precondition check) or some registration on the involved customer as well. For this reason, the participation of SD_LOAN to the *change_condition* event (O/M method in

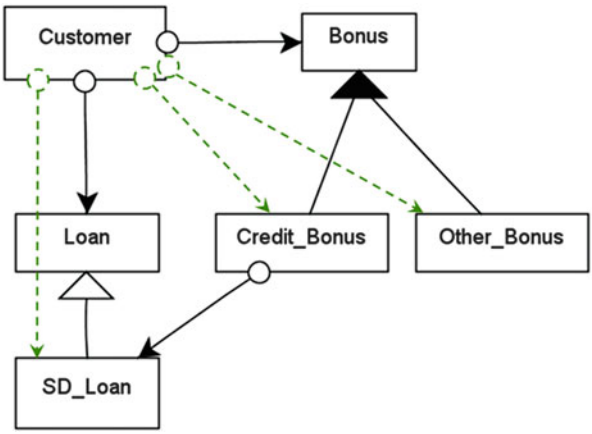


Fig. 8.9 Inherited existence-dependency relationships for the banking example shown as *dashed arrows*

the column of SD_LOAN) is propagated to CUSTOMER along the inherited dependency and yields an A/M method for CUSTOMER.

It is important to remark that it is the participations to a business event that are propagated and not the methods. For the ease of speaking, in the remainder of this chapter, we will formulate propagation as 'the O/C *method* is propagated from SD_LOAN to CUSTOMER', whereas in fact we mean that *the participation* to the business event change_conditions is propagated from SD_LOAN to CUSTOMER. This nuance is important to understand that the propagation does not mean that the *change_conditions method* in class SD_LOAN becomes visible to the class customer. When handling the business event *change_conditions*, a method will be invoked in the class CUSTOMER to deal with the effect of the business event on the CUSTOMER object and a method will be invoked in the class SD_LOAN to deal with the effect of the business event on the SD_LOAN object, but the method in the class CUSTOMER will not be able to invoke the method in the class SD_LOAN. The latter is the consequence of rules of visibility of methods in object-oriented programming languages.

The general application of the principle of propagation along inherited existence-dependency relationships leads to the fact that an object type acquires the business event participations of its existence-dependent object types through propagation (for which it will have to define a method) *plus* all the specialised and public[4] inherited methods of the object types that are directly or indirectly a specialisation of any of the existence-dependent object types. Let us call this the *accumulation principle*. The propagation is to the master object type only and not to a possible generalisation type of this master object type. This propagation rule is represented graphically in Fig. 8.10.

When we apply this rule of propagation along inherited dependencies to the OET of the banking example, we obtain the OET of Fig. 8.11. After propagation,

[4] An inherited method is public unless it has been specialised. In that case the inherited method is hidden and replaced by its specialised version.

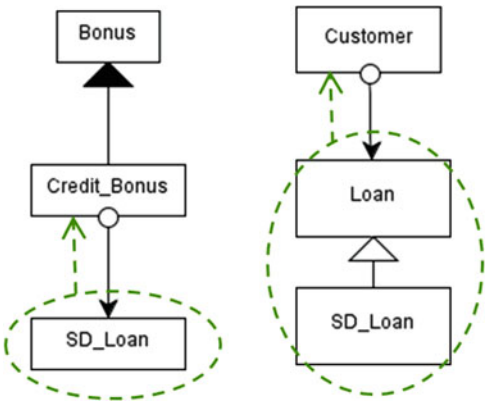


Fig. 8.10 Inheritance and propagation along inherited dependencies

inheritance and propagation must be rechecked until no more modifications are necessary. In this example, the OET of Fig. 8.11 is the final one.

In addition to the own event types of LOAN, *cr_sd_loan* and *end_sd_loan* are also propagated to CUSTOMER as a result of the propagation along inherited dependencies. Similarly, *cr_credit_bonus, cr_other_bonus*, *end_credit_bonus* and *end_other_bonus* are propagated to customer. The private inherited methods are not propagated according to the accumulation principle. The *cr_sd_loan* method propagates a second time to customer: it propagates to CREDIT BONUS via the existence-dependency relation and then propagates from there to CUSTOMER following the accumulation principle. This is consistent with the fact that two customers are potentially involved in a special discount loan: the customer the loan is granted to and the customer who owns the credit bonus used to obtain the loan.[5]

The *change_rate* and *change_duration* methods also propagate twice to CUSTOMER: one time the A/M in the column of LOAN propagates directly from LOAN to CUSTOMER (via the existence dependency). The second propagation is the I/M method in the column of SD_LOAN that propagates to CREDIT BONUS via the existence-dependency relation and then propagates from there to CUSTOMER following the accumulation principle.

The I/M method for *change_rate* and *change_duration* in SD_LOAN do, however, not propagate to customer according to the accumulation principle (this would cause a third participation in the column of customer). The reason is that from the perspective of the customer, the change_rate (or change duration) applies to either a loan or a special discount loan, but not to both at once. The I/M signifies that SD_LOAN *replaces* the original O/M method not that it makes an additional method.

Finally, also the O/M method for *change_conditions* in the column of SD_LOAN propagates twice to customer: one time the O/M propagates via LOAN to CUSTOMER

[5] If the customer owning the bonus should be the same as the customer the loan is granted to, a multiple propagation constraint should be added. If not (e.g. children can make use of the bonus of their parents) then no constraint needs to be added.

Fig. 8.11 OET for the banking example after propagation

	Customer	Loan	SD_Loan	Bonus	Credit_Bonus	Other_Bonus
cr_customer	O/C					
end_customer	O/E					
cr_bonus	A/M			O/C	I/C	I/C
→ *cr_credit_bonus*	A/M				S/C	
→ *cr_other_bonus*	A/M					S/C
end_bonus	A/M			O/E	I/E	I/E
→ *end_credit_bonus*	A/M				S/E	
→ *end_other_bonus*	A/M					S/E
cr_loan	A/M	O/C	I/C			
→ *cr_sd_loan*	A/M		S/C		A/M	
	A/M					
end_loan	A/M	O/E	I/E			
	A/M					
→ *end_sd_Loan*	A/M		S/E		A/M	
	A/M					
change_rate	A/M	O/M	I/M		A/M	
	A/M					
change_duration	A/M	O/M	I/M		A/M	
	A/M					
change_conditions	A/M		O/M		A/M	
	A/M					

(according to the accumulation principle). The second propagation is to CREDIT BONUS via the existence-dependency relation and then from there to CUSTOMER following the accumulation principle.

Notice that although LOAN and CREDIT BONUS share the *change_rate* and *change_duration* event types, there is no need to define a common existence-dependent object type. The contract rule is satisfied by the fact that SD_DISCOUNT LOAN is existence dependent on CREDIT BONUS.

Inherited object-event participations are propagated along explicitly modelled and inherited existence-dependency relationships in the same way as owned and acquired participations are propagated except in one case. Inherited participations are not propagated along an inherited dependency when the method of the generalisation that is the source of the inherited method has already been propagated

along the explicitly modelled existence-dependency relationship source of the inherited existence dependency. As a result inherited methods that have been specialised will not be propagated. Instead, the specialised methods are propagated.

8.2.3 More Examples

Example 8.3 Material Products and Services

The model in Fig. 8.12 represents a partial model of a company that sells both tangible products and services. The delivery of tangible products happens through orders. Services are delivered by means of business processes, and their delivery is agreed through a service contract. According to the inheritance relationship, SERVICE will inherit the methods from PRODUCT. SERVICE will also have a number of own methods (e.g. to manage SLA[6] templates) and methods acquired from REQUESTED SERVICE (e.g. to manage the SLA agreed in the SERVICE CONTRACT). All these methods will be propagated to BUSINESS PROCESS. The resulting OET is shown in Fig. 8.13.

Example 8.4 The Library: Specialised Items

A second example comes from a library environment. In a library, different types of items are available: CDs, single issues of journals, bound volumes of journals and copies of books. CDs and single issues of journals must not leave the library. The other types of items can be borrowed (and returned). Only loans of copies can be renewed. A first (and wrong) EDG for this example is given in Fig. 8.14. The corresponding OET with basic participations and inheritance is given in Fig. 8.15.

Figure 8.16 explains the propagation of the business event *renew*. The propagation rule causes the participation of RENEWABLE LOAN to *renew* to propagate to COPY (cfr.

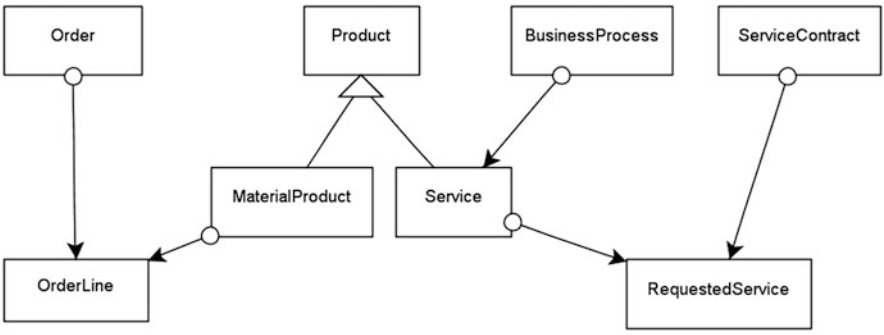


Fig. 8.12 Material products and services

[6] Service-level agreement.

	Order	OrderLine	ServiceContract	RequestedService	BusinessProcess	Product	MaterialProduct	Service
EV_cr_Order	O/C							
EV_end_Order	O/E							
EV_cr_OrderLine	A/M	O/C					A/M	
EV_end_OrderLine	A/M	O/E					A/M	
EV_cr_ServiceContract			O/C					
EV_end_ServiceContract			O/E					
EV_cr_RequestedService			A/M	O/C	A/M			A/M
EV_Check_SLA			A/M	O/M	A/M			A/M
EV_end_RequestedService			A/M	O/E	A/M			A/M
EV_cr_BusinessProcess					O/C			
EV_end_BusinessProcess					O/E			
EV_cr_Product						O/C	I/C	I/C
→ *EV_cr_MaterialProduct*							S/C	
→ *EV_cr_Service*					A/M			S/C
EV_modify_details					A/M	O/M	I/M	I/M
EV_AddSLATemplate					A/M			O/M
EV_end_Product					A/M	O/E	I/E	I/E
→ *EV_end_Service*					A/M			S/E
EV_end_MaterialProduct							O/E	

Fig. 8.13 OET for material products and services

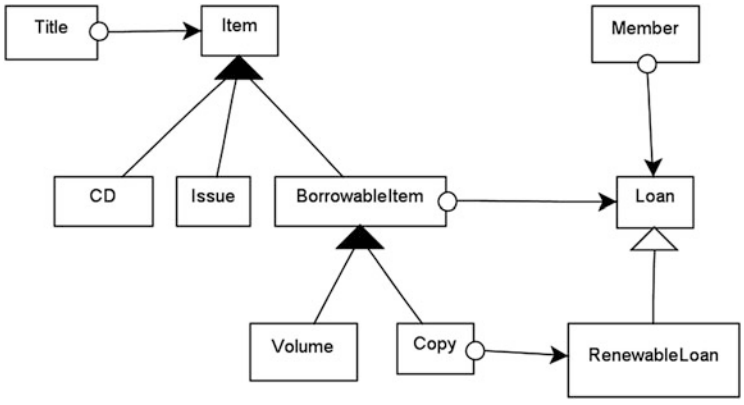


Fig. 8.14 (Wrong) EDG for the library

	Title	Item	CD	Issue	BorrowableItem	Volume	Copy	Member	Loan	RenewableLoan
EV_cr_Title	O/C									
EV_end_Title	O/E									
EV_cr_Item	A/M	O/C	I/C	I/C	I/C					
→ *EV_cr_CD*	A/M		S/C							
→ *EV_cr_Issue*	A/M			S/C						
→ *EV_cr_BorrowableItem*	A/M				S/C	I/C	I/C			
→ → *EV_cr_Volume*	A/M					S/C				
→ → *EV_cr_Copy*	A/M						S/C			
EV_end_Item	A/M	O/E	I/E	I/E	I/E					
→ *EV_end_CD*	A/M		S/E							
→ *EV_end_Issue*	A/M			S/E						
→ *EV_end_BorrowableItem*	A/M				S/E	I/E	I/E			
→ → *EV_end_Volume*	A/M					S/E				
→ → *EV_end_Copy*	A/M						S/E			
EV_cr_Member								O/C		
EV_end_Member								O/E		
EV_borrow	A/M				A/M	I/M	I/M	A/M	O/C	I/C
→ *EV_cr_RenewableLoan*	A/M				A/M	I/M	A/M	A/M		S/C
	A/M						I/M			
EV_end_Loan	A/M				A/M	I/M	A/M	A/M	O/E	I/E
	A/M						I/M			
renew	A/M				A/M	I/M	A/M	A/M		O/M
	A/M						I/M			
return	A/M				A/M	I/M	A/M	A/M	O/M	I/M
	A/M						I/M			
pay_fine	A/M				A/M	I/M	A/M	A/M	O/M	I/M
	A/M						I/M			

Fig. 8.15 OET for the (wrong) library EDG

green/dotted arrow). Because of the accumulation principle, the *renew* event also propagates from RENEWABLE LOAN to BORROWABLE ITEM and MEMBER (cfr. blue/dashed arrows). Because of inheritance, VOLUME and COPY will inherit the *renew* event from BORROWABLE ITEM (red/full line). But this is not what we specified: loans of volumes cannot be renewed. The error stems from the erroneous existence-dependency relation between BORROWABLE ITEM and LOAN. Indeed, a generalisation/specialisation hierarchy must always be interpreted as 'a generalisation object or a specialisation1 object or a specialisation2 object or . . .'. The erroneous existence-dependency

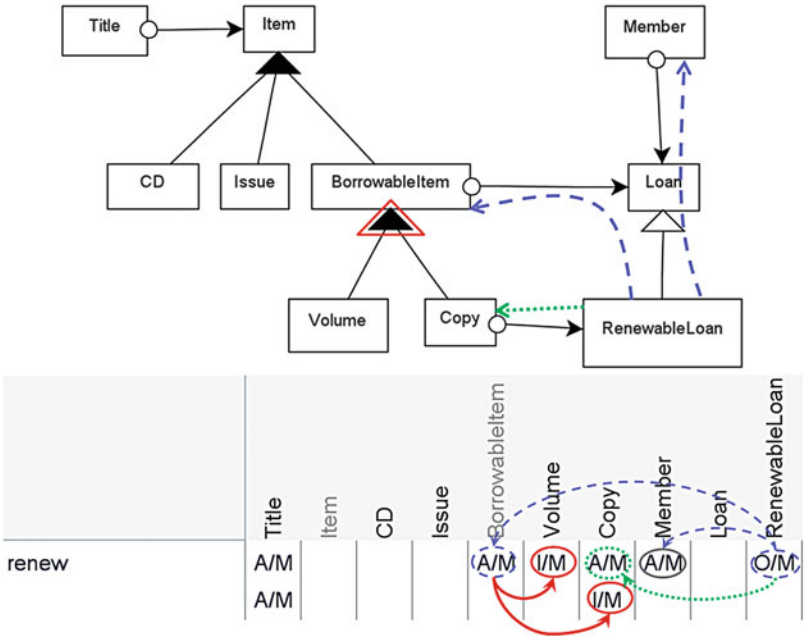


Fig. 8.16 Explanation of the propagation paths for *renew*

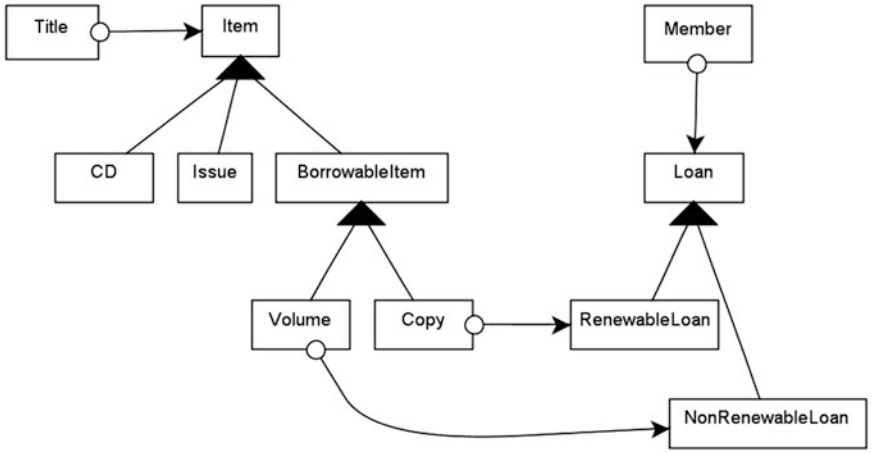


Fig. 8.17 Correct EDG for the library

relation therefore means 'a LOAN *and by inheritance a RENEWABLE LOAN* are existence dependent on a VOLUME or on a COPY'. But a RENEWABLE LOAN cannot be existence dependent on a VOLUME.

The library diagram must be corrected as in Fig. 8.17. Notice that in this diagram, the existence-dependency relation is not between BORROWABLE ITEM and

	Title	Item	CD	Issue	BorrowableItem	Volume	Copy	Member	Loan	RenewableLoan	NonRenewableLoan
EV_cr_Title	O/C										
EV_end_Title	O/E										
EV_cr_Item	A/M	O/C	I/C	I/C	I/C						
→ *EV_cr_CD*	A/M		S/C								
→ *EV_cr_Issue*	A/M			S/C							
→ *EV_cr_BorrowableItem*	A/M				S/C	I/C	I/C				
→ → *EV_cr_Volume*	A/M					S/C					
→ → *EV_cr_Copy*	A/M						S/C				
EV_end_Item	A/M	O/E	I/E	I/E	I/E						
→ *EV_end_CD*	A/M		S/E								
→ *EV_end_Issue*	A/M			S/E							
→ *EV_end_BorrowableItem*	A/M				S/E	I/E	I/E				
→ → *EV_end_Volume*	A/M					S/E					
→ → *EV_end_Copy*	A/M						S/E				
EV_cr_Member								O/C			
EV_end_Member								O/E			
EV_borrow								A/M	O/C	I/C	I/C
→ *EV_cr_RenewableLoan*	A/M						A/M	A/M		S/C	
→ *EV_cr_NonRenewableLoan*	A/M					A/M		A/M			S/C
EV_end_Loan	A/M					A/M	A/M	A/M	O/E	I/E	I/E
	A/M										
renew	A/M						A/M	A/M		O/M	
return	A/M					A/M	A/M	A/M	O/M	I/M	I/M
	A/M										
pay_fine	A/M					A/M	A/M	A/M	O/M	I/M	I/M
	A/M										

Fig. 8.18 Correct OET for the library after propagation

NOT RENEWABLE LOAN, because then, through inheritance, COPY would be a master object type of both RENEWABLE LOAN and NOT RENEWABLE LOAN. The OET for this new EDG is given in Fig. 8.18.

The following example is another illustration of the fact that one must be very careful when using generalisation/specialisation.

Example 8.5 The Pharmacy: Free and Restricted Products

In a pharmacy, some products can be sold freely, others only on doctor's order. This is represented in the EDG of Fig. 8.19.

One would be tempted to draw an existence-dependency relation between PRODUCT and SALE because the general case is that a sale is always existence dependent on

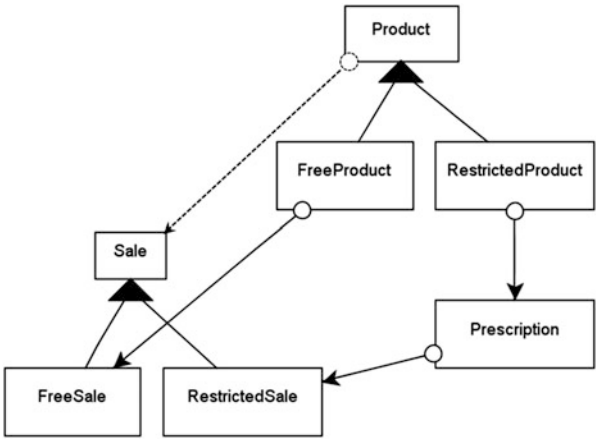


Fig. 8.19 EDG for the pharmacy

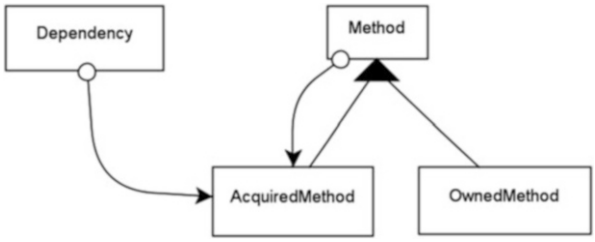


Fig. 8.20 Excerpt from the MERODE meta-model

a product. But again, this existence-dependency relation must be interpreted as 'a sale, and by inheritance a free sale and a restricted sale are existence dependent on a product, which is either a free product or a restricted product'. Since a restricted sale is existence dependent on a restricted product only (by the intermediary of a prescription), the dashed existence-dependency relation in the above model is wrong and should be removed.

In some particular situations, the propagation rules could lead to an endless propagation loop. Consider the extract of the MERODE meta-model in Fig. 8.20. It defines that an ACQUIRED METHOD is existence dependent on a DEPENDENCY relationship and another METHOD (that can either be an owned or another acquired method). In the OET (Fig. 8.21), *cr_method* is inherited by ACQUIRED METHOD (①) and then specialised to *cr_Acquired_method* (②). This specialised method is propagated to METHOD (③) and then inherited again by OWNED_METHOD and ACQUIRED_METHOD (④). Now ACQUIRED_METHOD has two participations in *cr_acquired_method*. This reflects the fact that when an acquired method is propagated, two acquired methods are involved: the one that is propagated (corresponding to the I/M participation) and the one that is created (corresponding to the S/M participation). However, the propagation is a chain reaction as the newly created acquired method needs to be propagated to the masters, their masters, etc. In the OET this is reflected by the fact that the inherited method in ACQUIRED METHOD should be propagated again to

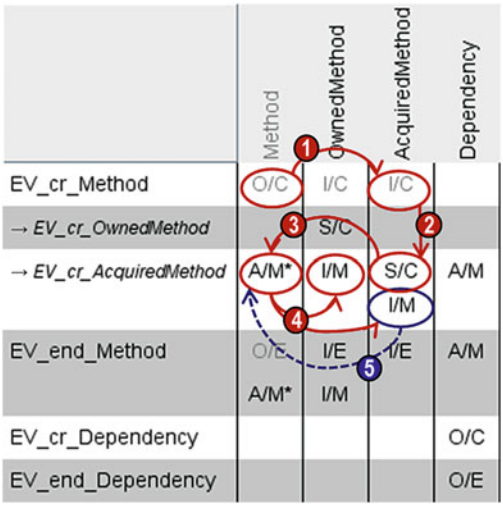


Fig. 8.21 OET for the MERODE meta-model excerpt

METHOD (blue/dashed arrow ⑤), inherited again, etc., and this would lead to an endless propagation loop. We therefore state the rule that when an acquired method is acquired from an inherited or specialised method, it should not be propagated to the same object type again. So the propagation should stop when the I/M is propagated again: the blue/dashed arrow should therefore not lead to another A/M in the column of METHOD. Similarly, the A/M participation for *end_method* in the column of METHOD is the result of the propagation of the I/M participation for *end_method* in the column of ACQUIRED METHOD. This A/M is inherited by ACQUIRED METHOD but should then not be propagated again to method. The fact that the A/M method is not further propagated is shown as a '*' next to the already existing A/M method.

8.3 On the Use of Generalisation/Specialisation Hierarchies and Roles[7]

The concepts of role presented in Chap. 6 and of generalisation/specialisation are somewhat similar in the sense that they both allow specifying special cases of one object type. Using specialisation types gives rise to the definition of a new class, whereas role types will incorporate special behaviour in the base class. The role

[7] Sections 8.3 and 8.4 have been previously published as the following paper: Snoeck, M., Dedene, G. (1996). Generalization specialization and role in object oriented conceptual modeling, Data &

concept as described in ORM [74] constitutes a way to model temporal aspects of objects. Using roles to describe temporal properties of objects is compatible with the role concept as used in JSD ([38], p. 104). Also OMT [77] had a role concept, but this one is completely different from the ones included in JSD and ORM. As a matter of fact, it is identical to the concept of existence-defined subclass (see Sect. 8.3.2).

The similarity between both constructs can make it difficult to determine when which construct should be used. In addition it is not always clear whether the constructs should be used at all: a number of situations can be modelled with as well as without specialisations or roles. As a result, and according to the principle one should always keep a model as simple as possible, in the area of conceptual modelling, one should be very careful about using the notion of generalisation/specialisation. The next sections investigate the different ways in which a base class can be partitioned in subsets and whether it is appropriate to identify these subsets as new object types, in particular as subclasses of the original class. Three types of subclasses are discussed: attribute-defined, existence-defined and state-defined subclasses [31]. By means of a number of examples, we will illustrate how in some cases the generalisation/specialisation and role constructs can be replaced by more simple constructs while retaining the same semantic content. Section 8.3.4 will then propose a number of criteria for determining the situations in which the generalisation/specialisation and role constructs are to be used.

8.3.1 Attribute-Defined Subclass

Example 8.6 Books: Attribute-Defined Subclass, Based on Colour

Let BOOK be an object type defined within the context of a library. This object type has an attribute named 'colour', the data type of which is an enumeration type (red, blue, grey, etc.). We can define subclasses of BOOK based on the value for this colour attribute:

RED-BOOK contains the objects book of class BOOK where colour = 'red'
BLUE-BOOK contains the objects book of class BOOK where colour = 'blue'
...

Although it is hard to imagine that it is useful to define the set of red books as a specialisation type of BOOK, there are examples where attribute-defined subclasses might seem useful. Let's look at the following example.

knowledge engineering, 19(2), 171–195. © Elsevier 1996. The material from this paper has been integrated in this book with the kind permission of Elsevier.

Example 8.7 Cars: Attribute-Defined Subclass, Based on Price

Suppose that the VAT percentage for cars depends on the price of the car. Cars of 100,000 Euro or more are considered to be of a deluxe model and are subject to the deluxe VAT of 33 %. Standard cars of less than 100,000 Euro are subject to a VAT of 20.5 %

Therefore, the attribute 'price' defines a total and disjoint specialisation, and the event *invoice* will have a different content for each specialisation, namely:

```
class Standard_model
  Invoice () {
    ...
    Invoice_amount = price * 1.205;
    ...
  }
class Deluxe_model
  Invoice () {
    ...
    Invoice_amount = price * 1.33;
    ...
  }
```

The same effect can be obtained if a single class CAR is defined with the following method for 'invoice':

```
class Car
  Invoice () {
    ...
    if price < 75000
      then invoice_amount = price * 1.205
      else invoice_amount = price * 1.33;
    ...
  }
```

This example also shows that the concept of generalisation/specialisation is the dual of choice.

8.3.2 Existence-Defined Subclass

A subclass S of a base class G can be defined as consisting of all members of G that are involved in a relation with another object. This can be done for any optional relationship type. Two examples illustrate this concept. Notice that the class diagrams in these examples are not EDGs. The relationships are drawn according the UML notation and hence do not always express existence dependency.

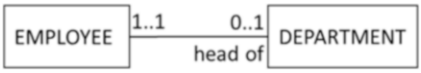


Fig. 8.22 Some employees manage a department

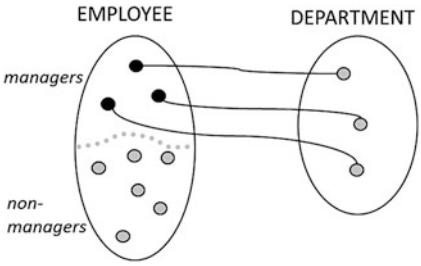


Fig. 8.23 Occurrences of EMPLOYEE, DEPARTMENT and HEAD OF

Example 8.8 Managers of Departments: Existence-Defined Subclass

Every department has exactly one manager. Some employees are the manager of one department. This is illustrated in Fig. 8.22.

Since the association HEAD OF is optional for employees, some employees will not participate in that association (see Fig. 8.23). Consequently, the set of employees can be split in two disjoint subsets: employees that are a manager and employees that are not, as shown in Fig. 8.24. As a result, the relationship type HEAD OF is now mandatory for MANAGER.

It can be argued that the mere fact of participating in an association is not a sufficient reason for defining a generalisation/specialisation hierarchy. First, if one cares to apply the principle that a conceptual model should be kept as simple as possible, the solution with the optional relationship type is to be preferred. Second, the constraint that only managers can be heads of departments can be modelled by means of a precondition. It then is required that the method *assign_to_department* in class EMPLOYEE imposes the precondition that the grade of the employee that is assigned head of a department is 'manager'. Third, being a manager may in fact be a temporal property of employees: usually an employee is promoted to manager only after a few years, and possibly a manager can also be demoted to the rank of employee. If this is the case, the specialisation becomes temporary, and, as explained in Sect. 8.1.1, we cannot use inheritance if we choose to stick to static classification.

Example 8.9 Conference Organisation

The CRIS-case [66] describes the organisation of an IFIP Working Conference. This example shows part of the UML class diagram for this case study. Figure 8.25 shows a UML class diagram without generalisation/specialisation, and Fig. 8.26 shows a class diagram with generalisation/specialisation as proposed by [25]. Notice how the use of generalisation/specialisation makes cardinality restrictions change from optional to mandatory.

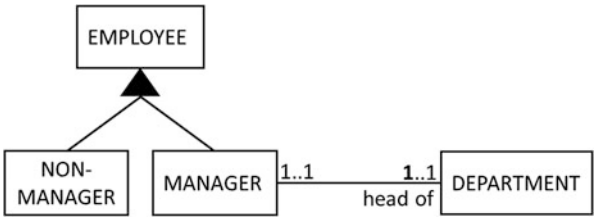


Fig. 8.24 Managers and non-managers

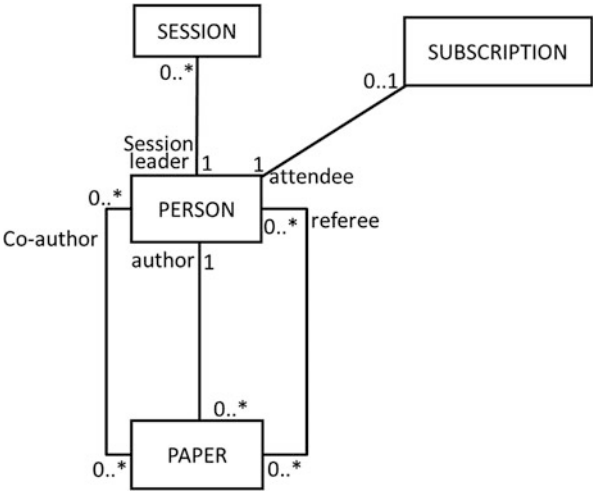


Fig. 8.25 UML class diagram for the CRIS-case

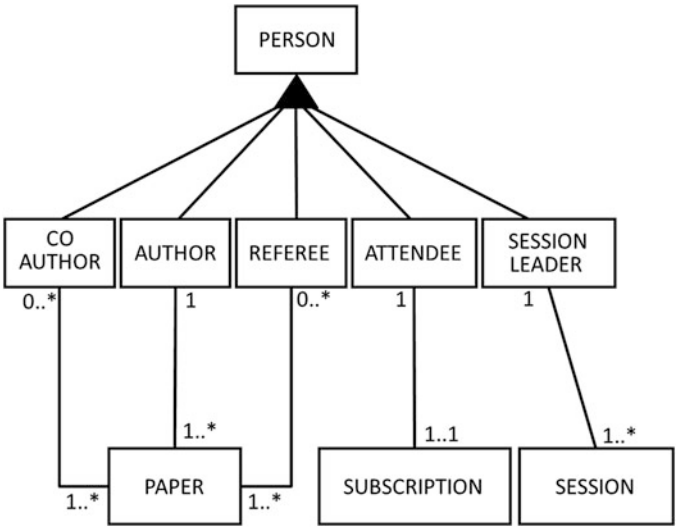


Fig. 8.26 Specialisations for person in the CRIS-case

In [25] ATTENDEE, REFEREE, AUTHOR, CO-AUTHOR, etc., are defined as *roles* of PERSON, and this is motivated by the fact that being an attendee, a referee, an author and so on are temporal rather than permanent aspects of a person. The role concept can be used when behaviour restrictions are too complicated to be modelled in a single FSM for person (see Chap. 6). In case of simple behaviour, the default FSM or a single FSM may be sufficient.

8.3.3 State-Defined Subclass

A state-defined subclass is equivalent to the 'user controllable subclass' mentioned in [31]. In this case a subclass S is defined as consisting of all members of the base class G that are in a particular (set of) state(s). The first example is taken from [31].

Example 8.10 Ships: State-Defined Subclass

The class of SHIPS can be divided into two disjoint subclasses BANNED_SHIPS and SAFE_SHIPS. Originally every ship is a member of SAFE_SHIPS. If it is banned from territorial waters, a ship becomes a member of the class BANNED_SHIP. If authorities decide to lift the ban, the ship is moved to the set of SAFE_SHIPS again. The fact that ships are *moving* between subclasses indicates that being banned is a temporal property of a ship and can therefore be modelled by means of behaviour descriptions. Figure 8.27 shows a finite state machine that describes the life cycle of a ship. The set of banned ships can then be defined as the set of instances of the class SHIP that are in state 'banned', while the safe ships are the set of instances that are in state 'safe'. This same situation can also be modelled as an attribute-defined subclass if the attribute 'banned' with domain ('yes', 'no') is added to the class definition of SHIPS.

These examples show that defining subsets of classes can be done in various ways and that the same universe of discourse can be represented by different models. The following section presents a number of guidelines that help the analyst to decide upon the concepts to use to model subsets.

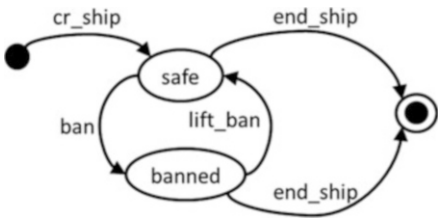


Fig. 8.27 Banning a ship and lifting the ban

8.3.4 Guidelines for Using Roles and Generalisation/ Specialisation

In order to obtain a complete set of criteria for choosing between the generalisation/ specialisation and the role concept, we consider the components of an object type definition. The static aspects of object types contain attribute definitions. A subclass can have additional attributes, and it can have different constraints on inherited attributes. The dynamic aspects of an object type are described by means of methods and sequence constraints. A subclass can respond to more events than its parent; it can override inherited methods and it can override the sequence constraints on inherited methods. Because a domain model should be kept as simple as possible, we propose as general guidelines that attribute, existence or state subclasses are not modelled as specialisation types or roles of a base type *unless* at least one of the following conditions is satisfied:

Condition 1. Does subclass A have *additional attributes* compared to base class B?
Condition 2. Do objects from subclass A respond differently to events than objects from base class B? That is, does subclass A have *other methods* than class B *for the same events*?
Condition 3. Does subclass A have *different attribute constraints* than class B?
Condition 4. Does subclass A respond to *more events* than class B?
Condition 5. Does subclass A have additional *sequence constraints* compared to class B?

In order to choose between the use of the role concept or the generalisation/ specialisation concept, the following question must be answered:

Condition 6. Do objects belong permanently or temporarily to subclass A?

Depending on the answers to these six questions, one of the following alternatives must be chosen:

Action 1: model A as a specialisation type of B
Action 2: model A as a role of B (i.e. an additional FSM)
Action 3: A is the same type as B

These alternatives are exhaustive and exclusive in the sense that we cannot, at the same time, model a type A as being a specialisation type of B and as a role of B. The decision rules should therefore lead to exactly one of the three conclusions.

Let us assume that we have two object types A and B, A being a candidate specialisation type or role of B. The following decision rules are proposed:

Rule 1: According to a static and single classification approach, objects belong to only one class for the whole time of their existence. As a result, an object cannot first belong to a generalisation class and then some time later to a specialisation class nor can an object switch between specialisation classes during its lifetime. Hence, roles are used to model temporal

aspects while the generalisation/specialisation hierarchy is used to model permanent aspects.

Rule 2: If A has other methods than B for the same event types, then A is a specialisation type of B. In this way, A can override the methods it inherited from B for these event types.

Rule 3: If A has other attribute constraints than B, then A is a specialisation type of B. If A is a specialisation class, then A can override the attribute definitions it inherited from B.

Rule 4: If A only has additional event types and additional sequence constraints, A can be modelled as a role of B, the additional event types being added to the set of events B participates in.

The fact that we adhere to a static and single classification approach has led to rule 1, namely, that a specialisation is always permanent. In addition, this static and single classification approach also implies that different specialisation classes have to have disjoint extents.

The rules have been investigated by means of a decision table workbench, which reveals unnecessary conditions, incomplete specifications and contradictions. The final decision table with guidelines for the use of the role concept and the generalisation/specialisation concept is shown in Fig. 8.28. The conditions are listed in the upper left quadrant; the decision alternatives are listed in the lower left quadrant. The upper right quadrant shows the possible alternatives for the conditions of the corresponding row. For the five first conditions, a 'Y' stands for 'Yes', an 'N'

C1: Does A have *additional attributes* compared to base class B?	-						
C2: Does A have *other methods* than class B *for the same events* ?	Y		N				
C3. Does A have *different attribute constraints* than class B?	-		Y		N		
C4: Does subclass A respond to *more events* than class B?	-		-		Y	N	
C5: Does subclass A have more stringent *sequence constraints* than class B?	-		-		-	Y	N
C6: Do objects belong permanently or temporarily to A?	P	T	P	T	-	-	-
1. A is a specialisation type of B	x	-	x	-	-	-	-
2. A is a role type of B	-	-	-	-	x	x	-
3. A = B	-	-	-	-	-	-	x
4. Contradiction	-	x	-	x	-	-	-

Fig. 8.28 Guidelines for using the role concept and the generalisation/specialisation concept

stands for 'No' and a '-' stands for 'not relevant'. For the sixth condition, a 'P' stands for 'permanent' and a 'T' for 'temporal'. Finally, in the lower right quadrant an 'x' means that the action is applicable and a '-' means that it is not.

It turns out that in case at least one of the four conditions C2–C5 is satisfied, the question of additional attributes is irrelevant. In case none of the four conditions is satisfied (column 7), the mere fact that A has additional attributes compared to B is not a sufficient condition to model B as a specialisation or role. Indeed, if A has additional attributes, we can expect that the methods of A are different from those of B or that A responds to additional events such that these additional attributes can be handled. If no such methods are present, we assume that the additional attributes belong to the base class and can possibly be assigned a null value.

Columns 2 and 4 denote real contradictions: according to the first rule, the temporal aspect of A leads to the use of the role concept while the presence of other methods and/or different attribute constraints leads to the use of the generalisation/specialisation concept (rules 2 and 3). In these situations it is recommended that the contradiction be resolved by modifying the conceptual model. This can for example be done by explicitly modelling the temporal aspect as an extra class that is further specialised, as shown in Example 8.7.

Example 8.11 Real Estate Management

Each building consists of a number of rooms. Each room serves a specific purpose: a didactic, research or logistic purpose. When a building is rearranged, rooms can receive another destination.

Suppose we have other attributes, other events, other methods and other attribute constraints for each kind of room, then we would like to model CLASS_ROOM, RESEARCH_ROOM and OFFICE as specialisations of ROOM. But since the destination is a temporal aspect of rooms, we have to model these as role of ROOM. A possible solution is to model the destination of a room as a separate object type as shown in Fig. 8.29.

Notice that a DESTINATION is existence dependent on ROOM with cardinality 1: a room can have only one destination at one point in time. But with this solution we have also the flexibility to allow rooms to have more than one destination at one point in time by changing the cardinality to many. In combination with time attributes, this will also allow keeping track of past and future destinations.

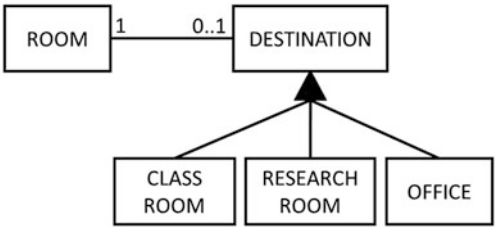


Fig. 8.29 Kinds of rooms

8.4 Formal Definition of Inheritance

Inheritance, denoted as $<^+$, is a hierarchical partial order on $\mathcal{M}$ and is defined as follows:

(1) $\forall\, G \in \mathcal{M} : G <^+ G$

(2) $\forall\, G, S \in \mathcal{M} : G <^+ S$ and $S <^+ G \Rightarrow G = S$

(3) $\forall\, G, S, T \in \mathcal{M}$: $G <^+ S$ and $S <^+ T \Rightarrow G <^+ T$

$<$ is the cover of $<^+$. $<^+$ can be recovered by taking the reflexive transitive closure of $<$. $<$ must be a hierarchy:

(4) $\forall\, G, G', S \in \mathcal{M}$: $G < S$ and $G' < S \Rightarrow G = G'$

Let $G, S \in \mathcal{M}$. If $G < S$ then we say that G is a generalisation of S and S is a specialisation of G.

$\gamma_+(P) = \{G \in \mathcal{M} \mid G <^+ P\}$ (parents)
$\gamma_-(P) = \{S \in \mathcal{M} \mid P <^+ S\}$ (descendants)
$\gamma_{max}(P) = \{G \in \mathcal{M} \mid G <^+ P$ and $\neg\, \exists\, H \in \mathcal{M} : H < G\}$
$\text{cluster}(P) = \gamma_-(\gamma_{max}(P))$

oids^+ is a function that takes the deep extent of an object type and is defined as:

$$\text{oids}^+(P) \;=\; \cup\, \{\text{oids}(S) \mid P \in \gamma_+(S)\}$$

where oids(P) denotes the extent of P.

If a generalisation is total, this implies that its extent will always be empty:

$$G <_0 (S_1, \ldots, S_n) \;\Rightarrow\; \text{oids}(G) \;=\; \varnothing$$

The principle of single classification requires that

$$\forall P, Q \in \mathcal{M} : \text{oids}(P) \cap \text{oids}(Q) \;=\; \varnothing$$

8.5 Meta-model

The meta-model can be further expanded with the concept of inheritance. The inheritance association connects two object types. According to the principle of single classification, an object type can play only once the role of subtype, but it can play the role of supertype many times. A specialised event type is created as dependent of an event type but only because of an existing inheritance association. Moreover, it can only be created for events that have an owned create method for the supertype of the object that is the supertype in the inheritance relation that is used to motivate the creation of a specialised event (Fig. 8.30).

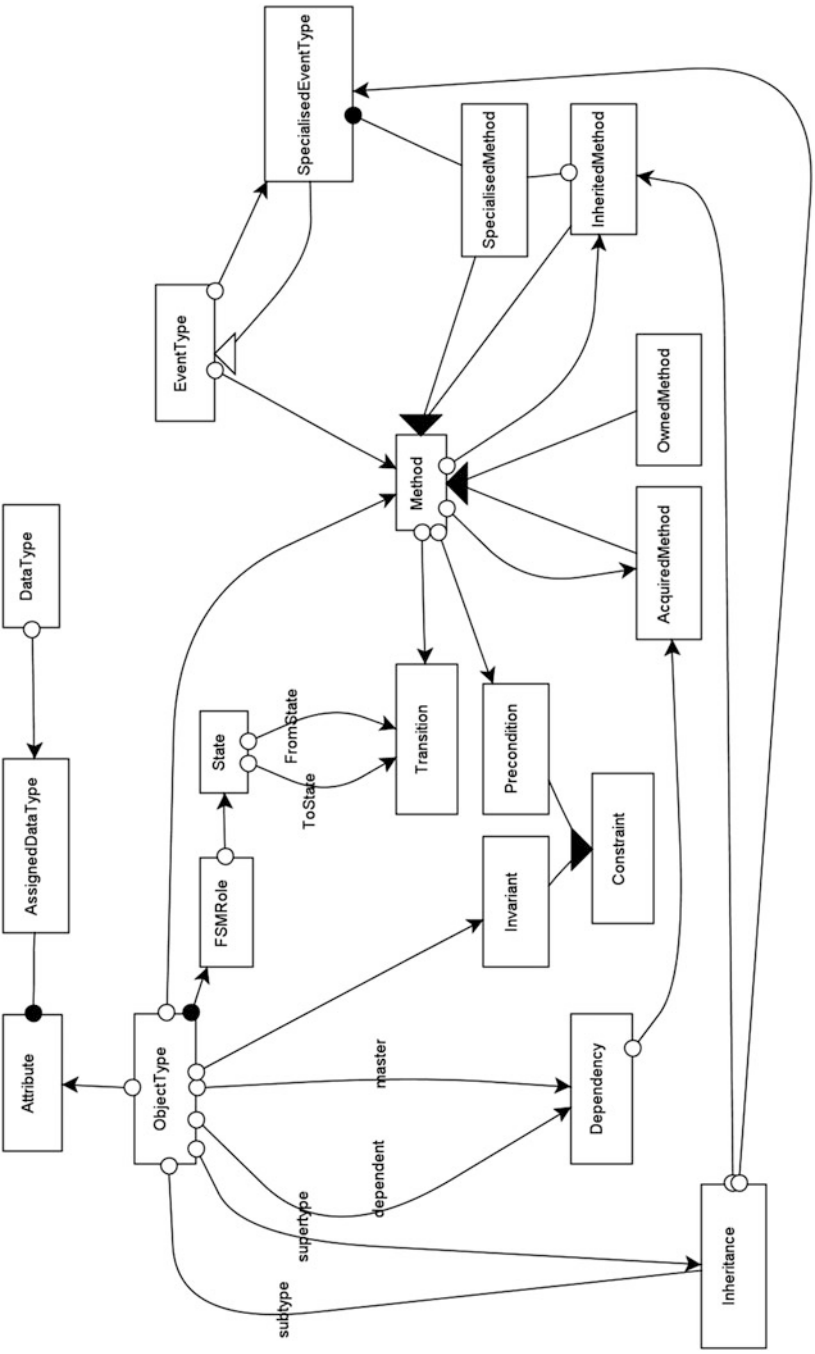


Fig. 8.30 Meta-model with inheritance

```
class SpecialisedEventType
invariant
    Inheritance.supertype.Method → Select(isOCLTypeOf
    (OwnedMethod)).EventType → Includes(EventType)
```

Similarly, inherited methods are the inherited versions of an existing method of any kind but only exist because of an inheritance relationship. In the class INHERITEDMETHOD an invariant constraint needs to be formulated that enforces that the object type of the inherited method should be equal to the subtype in that inheritance relation. Also, the object type of the method that is inherited should play the role of the supertype in the inheritance association that is used to motivate the creation of an inherited method.

```
class InheritedMethod
invariant
  ObjectType = Inheritance.subtype
  ViaMethod.ObjecType = Inheritance.supertype
```

Also the SPECIALISEDMETHOD is subject to different constraints. It is a specialised version of an inherited method for the same object and that refers to a specialised event of the corresponding event. Moreover, an additional invariant needs to be formulated that specifies that the inheritance relation that motivated the creation of the specialised event should be the same as the inheritance relation that motivated the creation of the inherited method.

```
class SpecialisedMethod
Invariant
  ObjectType = InheritedMethod.ObjectType
  SpecialisedEventType.EventType = InheritedMethod.EventType
  InheritedMethod.Inheritance = SpecialisedEventType.Inheritance
```

Part III
The Information System Layer and the Business Process Layer

Chapter 9
The Information System Service Layer

One of the main goals of building enterprise information system is to satisfy the information needs of the different stakeholders of the organisation. But before information needs can be satisfied, information obviously needs to be registered first. As a consequence, an enterprise information system needs to offer two types of services: input services to register information and output services to satisfy information needs. In combination with the enterprise layer, this leads to three loosely coupled subsystems, each addressing a specific concern:

- The business logic as encapsulated in the domain model
- The registration of data as supported by input services
- The information needs as satisfied by information object types and output services

These concerns give rise to different components in the final enterprise information system. Work organisation issues are addressed by the business process layer, which is the subject of Chap. 10. In this chapter we first describe a set of new concepts that are required for delivering information services that address the registration of data and the satisfaction of information needs: *information object types*, *output information services* and *input information services*. Subsequently, we explain how the connection with the enterprise layer is realised by means of an *event handling layer,* leading to yet another set of concepts: *consistent events* and *transactions*. The result is a refinement of the layered architecture as was proposed in Chap. 2. We illustrate the overall resulting architecture by means of the architecture of JMermaid.

© Springer International Publishing Switzerland 2014

M. Snoeck, *Enterprise Information Systems Engineering*, The Enterprise Engineering Series, DOI 10.1007/978-3-319-10145-3_9

9.1 Information Objects

Business objects are persistent, meaning that they are permanently stored in a database so as to make them accessible to any application that can access the database. But very often the information contained in enterprise objects is not sufficient to realise the desired functionality. Typical examples of such persistent information object types are classes used for the logging of executed business events and the logging of executed services. Another example is the logging of customer activity on a web interface which is logged for web analytics purposes to improve the customer experience. In such cases we need to define additional information object types as persistent data, next to the already identified business object types. We define information object types as follows:

An *information object type* is a class that represents persistent information required to satisfy information needs, but which is not a business object type.

The general idea behind information object types is that all persistent information is contained in the enterprise object types and information object types. All other objects that are part of or called by the services are volatile (non-persistent) objects that are created to run the requested service and that are no longer needed when the service is terminated.

Information object types do not obey the rules of business object types. Some information object types are strongly linked to and existence dependent on enterprise object types. Assume for example an online banking service that allows a customer to view the history of financial movements (i.e. deposits and withdrawals) on his/her bank account. The realisation of such a service requires the introduction of information objects that record the movements on an account. The events *deposit* and *withdraw* each create occurrences of this information object type, one for each event. In this particular case, this information object type is existence dependent of ACCOUNT as the logs of movements can only exist in the context of the account for which the movements were logged. Another example of an information object type is a 'user preferences' class that is required to manage (and remember) user preferences, such as choices about various types of options in the user interface. In this second example, the information object type is not existence dependent of any business object type. For that reason, the use of the existence-dependency relations is advised only for the domain model. Information object types can be added to the UML class diagram and can be related to other information object types and to business object types with conventional UML associations. A stereotype or a different graphic symbol can be used to differentiate information object types from business object types.

Another difference is that information object types may not have a life cycle. Information object types that are introduced to keep the trace of events have an 'instantaneous' life: they are just created by the event of which they keep trace and are further no more involved in any other events. For such object types there is no ending event type. In the above example, MOVEMENT is an example of such an

instantaneous object type: occurrences are created by the *deposit* and *withdraw* events and no other events affect these objects.

A final difference is that information object types do not only respond to business events: they can also respond to information system events, also called application events. For example, the ERP system of my organisation allows me to select candidates that applied for a vacancy and to generate a single PDF containing all the relevant data for that candidate combined with uploaded CVs, motivation letter, etc. This PDF report is stored in the system and stays available for a while (e.g. 1 week). It is therefore an information object type, and its creation is not by means of a business event but happens as response to a service request by the user. In this example the request is the application event resulting from pushing a button in the user interface of the application, but such request could also be a call by a web service. Its ending is also not a business event but a timer event. Another example are user preferences that are entirely managed by information system services and are never affected by business event types.

In terms of layers, care should be taken to position information object types in an outer layer compared to business object types. In practice, this means that information objects can have a reference to a business object, but as business objects should never be aware of the presence of other layers, business objects cannot refer to information objects.

9.2 Information System Services

9.2.1 What Is an Information System Service?

The easy way to define information system services is to say they correspond to the 'use cases' of an information system. Yet use cases can be defined at various granularity levels, and moreover there is the danger of confusing use cases with business processes. Complex use cases, and in particular when they involve multiple users and/or correspond to multiple tasks, may actually be a business process description rather than a single information system service. Information system services are services rendered by the information system to support the execution of activities. They therefore only refer to automated services and activities. Business processes describe work organisation: they may be constituted of manual tasks and activities outside the information system as well. Business process enactment will make use of the information system services to realise the part of the business process that needs enterprise information system support.

Business processes should be modelled as a separate layer on top of the information system services (see Chap. 10), and the information system services are put in between the business process layer and the enterprise layer. They will offer the required automated support e.g. for user and service tasks and message events. The lowest level of granularity of a business process decomposition is the atomic

activity (task) or event. Therefore, the (maximal) granularity of an information system service should be defined such that it fits into the scope of a single task or message event. In particular, this means that an information system service always refers to a single (executing) business process actor.

The more coarse-grained information system services can be further decomposed and refined using the «include» and «extend» dependencies between use cases (see UML use case definition in [67], Chap. 18). One can easily start the description of information system service by defining a series of fine-grained basic and trivial services that are subsequently composed into more complex services. For the lowest level services, we distinguish between input and output services and make the link to the enterprise layer explicit by specifying which business objects are queried and which business events are triggered.

9.2.2 Output Services

Output information services are services that query enterprise and information objects. They are activated by messages coming from the user interface or by messages coming from other services. Their output is consumed by the user interface or by other services.

To represent the connection between information system services and the enterprise layer, we use the concept of a dependency association from the UML using the following conventions:

- The enterprise layer is represented as a package.
- The query is represented as a dependency using the «access» stereotype having the use cases representing the output service as a source and the enterprise layer package as a target. The «access» stereotype represents the fact that the source element accesses the target element.
- The relevant part of the domain model may be shown in the enterprise package. One might consider having a specific business object type as target of the «access» association, but in case multiple business object types are accessed, representing the relevant part of the domain model will yield a more readable diagram.

A template for an output service specification diagram using this notation is given in Fig. 9.1.

The further (textual) specification of the output service can be given according to a use case template or by addressing the dimensions of the Zachman framework:

- When: specification of preconditions to be satisfied and how to invoke the service (e.g. timer or user invocation).
- What: specification of input and output data (query parameters, screen layout, message format) and of postconditions that will be satisfied (minimally always satisfied postconditions, postconditions satisfied when successfully executed).

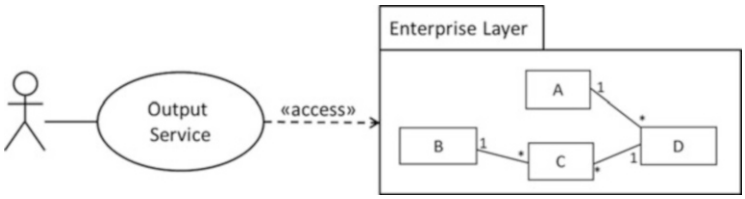


Fig. 9.1 Template use case diagram for an output service

- How: specification of the execution flow, e.g. by means of an activity diagram or interaction diagram. This may also include the specification of an SQL query.
- Why: motivation of the necessity and goal of the services.
- Where: identification of network aspects (e.g. client side versus server side).
- Who: identification of the owner, authorised invoker and/or destination of the output (e.g. the user, another service, an external service, etc.).

Trivial output services are:

- One output service per business object type to view the list of *all* objects in that class
- One output service per business object type to search for a specific objects in that class, using search criteria
- One output service per business object type to select and view the details of one object in that class

9.2.3 Input Services

Input information services collect information about events or groups of events occurring in the real world (environment controlled) or triggered by some condition (machine controlled), and they trigger the execution of a business event in the enterprise information system in order to share the event between environment and enterprise information system. They may also produce some output, for example, to share a machine-controlled event with the real world (see Chap. 5). As will be explained in Chap. 10, the transition from a real-world (business) event to the triggering of an event in the domain model may happen via the intermediate of business process events, tasks and application events. For example, upon arrival of a new patient in the hospital (an environment-controlled event in the real world), a business process may be started, involving several tasks. One of these tasks may be the registration of the patient into the hospital's information system. This means that an input service is needed, which allows collecting all required information about this patient and which will trigger a *cr_patient* event (business event in the domain model) to mirror this environment-controlled event (the arrival of the new patient) into the enterprise information system. Similarly, when a new customer arrives at a bank to request the opening of an account (environment-controlled

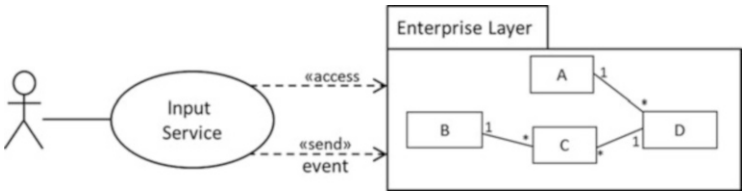


Fig. 9.2 Template use case diagram for an input information service

event in the real world), one of the tasks of the business process may require the use of an input service which allows triggering a *cr_customer* (business event in the domain model) followed by an *open_account* event (business event in the domain model).

Any service that triggers one or more business events is called an *input* serviceinput service. Figure 9.2 shows a template for an input information service specification diagram by means of the use case notation. The triggering of a business event is represented as a dependency labelled with the stereotype «send» having the use case representing the input information system as a source and the enterprise layer as a target. This «send» dependency is further annotated with the business events that are triggered by the input service.

The further (textual) specification of the input service can be refined by following a use case template, by adding an activity diagram or by addressing the dimensions of the Zachman framework:

- When: specification of preconditions to be satisfied and how to invoke the service (e.g. timer or user invocation).
- What: specification of input and output data (parameters, screen layout, message format) and of postconditions that will be satisfied (minimally always satisfied postconditions, postconditions satisfied when successfully executed).
- How: specification of how input data will be collected (e.g. screen shot of user interface) and specification of the execution flow, e.g. by means of an activity diagram or interaction diagram. This must include the business events that will be triggered, and this may also include the specification of SQL queries that are used to retrieve data to check preconditions or data needed to trigger the events.
- Why: motivation of the necessity and goal of the services.
- Where: identification of network aspects (e.g. client side versus server side).
- Who: identification of the owner, authorised invoker and/or destination of the output (e.g. the user, another service, an external service, etc.).

In all cases, it is important that the description identifies which business events will be triggered in what order.

Trivial input services are:

- One input serviceinput service per business event in the domain model

9.2.4 Complex Services

Complex services will use the trivial basic services to compose more intelligent support for user or service tasks. The basic services can be incorporated into the composed service by means of the «include» and «extend» relationships between use cases.

For example, the above-mentioned trivial input services may be linked to the trivial output services in the following way:

- From the list of all objects in a class (trivial output service) or from the service that allows to view the details of one object, invoke the input service for the modifying and ending events for that object.
- The input service for triggering a creating event may invoke the trivial output services to search and select the master objects for the to-be-created dependent object, as these will be required as input parameters for the creating event.

Example 9.1 A Car Rental Company

Let's consider a simplified case of a car rental company in which customers can rent cars. Each customer can rent one or more cars. After a car was rented, it has to be returned to the car rental company and the customer has to pay the agreed price. If he/she doesn't pay, reminders are sent. Finally, the rental is ended when either the rental has been paid or it is ended unpaid.

As the company wants to trace how often it has to send reminders, an information object has been added to log the 'remind' event. The resulting UML class diagram is shown in Fig. 9.3, using stereotypes to distinguish between business object types («BOT») and information object types («IOT»).

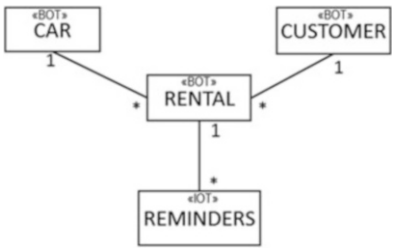


Fig. 9.3 Class diagram for the car rental company with stereotypes distinguishing between business object types («BOT») and information object types («IOT»)

	CAR	CUSTOMER	RENTAL
cr_car	O/C		
end_car	O/E		
cr_customer		O/C	
end_customer		O/E	
rent	A/M	A/M	O/C
return	A/M	A/M	O/M
remind	M	A/M	O/M
pay	A/M	A/M	O/E
end_rental	A/M	A/M	O/E

Fig. 9.4 OET for a car rental company

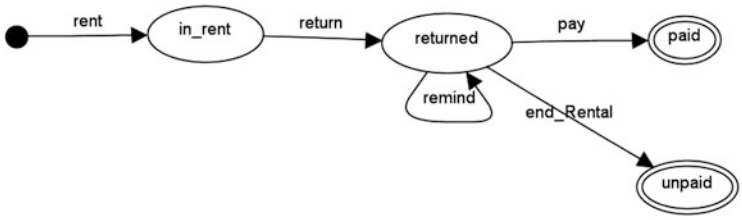


Fig. 9.5 Life cycle of RENTAL

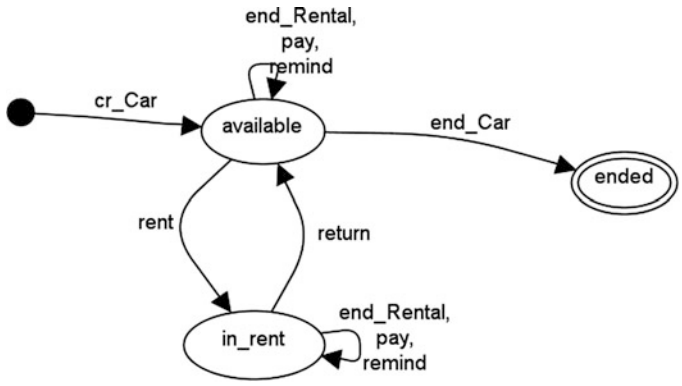


Fig. 9.6 Life cycle of CAR

The OET is given in Fig. 9.4, and the sequence constraints for RENTAL and CAR are given in Figs. 9.5 and 9.6. CUSTOMER has a default life cycle.

Trivial services for this example are given in Table 9.1.

Figure 9.7 shows some sample non-trivial services for the car rental company.

'Rentals Overview': On request, produce a list of all persons, indicating per person the number of cars in rent at this moment This service needs only to query the state of PERSON and RENTAL objects. Hence, it is an output service. This type of service can be implemented in many different ways, for example, by means of an SQL query on a database. Such a query would need to inspect the state of both customer and rental objects. Another implementation could keep track of the number of cars in rent for each person by defining a counter. In order not to clutter the domain model with implementation-related attributes, such a counter could be defined as an attribute of a persistent information object type, existence dependent on CUSTOMER. At implementation time, this object type can be merged with the customer object type for efficiency reasons. Using the Zachman keywords, this service can be further described as follows:

– When: user invocation.
– What: no input parameters.

Table 9.1 List of trivial services for the car rental example

Business object type	Output services			Input services
	List	Details	Search	
CAR	View list of cars	View car details	Search cars	Create car
				End car
CUSTOMER	View list of customers	View customer details	Search customers	Create customer
				End customer
RENTAL	View list of rentals	View rental details	Search rentals	Register rental (customer_ID and car_ID known)
				Register return
				Remind
				Register payment
				End unpaid rental

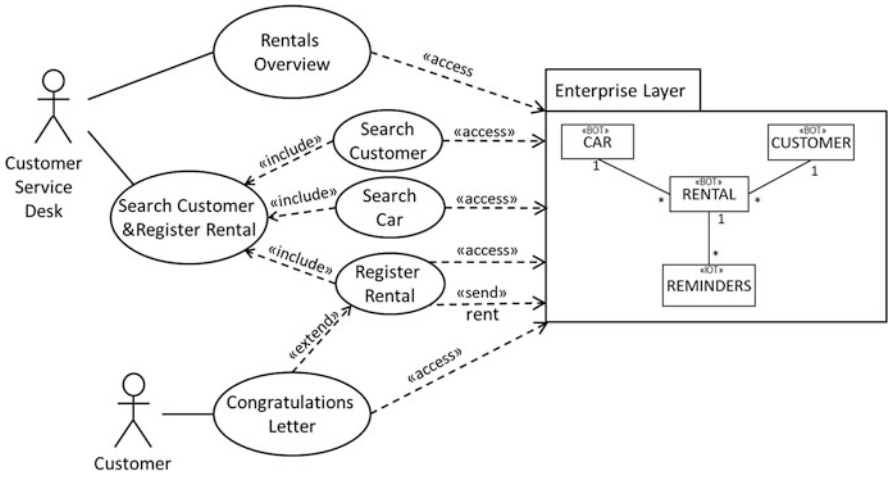


Fig. 9.7 Sample services for the car rental example

– How: no input is required. Data is retrieved by means of an SQL query:

```
SELECT Customer.Name, Count(Rental)
FROM Customer INNER JOIN Rental ON Customer.ID = Rental.Cust_ID
GROUP BY Customer.Name;
```

Report is shown in printable format.

– Why: to be able to monitor the average number of rentals per customer.
– Who: employees of the customer service desk and employees of the sales department are authorised to invoke this service.

'Search Customer and Register Rental': Register the rental of a car by a customer that has already been registered before This service is triggered by the user at the customer service desk through the user interface. In the user interface, the user searches the customer using the customer name as search criterion and the car using the car type and the rental dates as search criterion. Once customer and car have been selected, the user interface will trigger the registration of *rent* event for the selected customer and car. Because of the latter, it is an input service. This service requires («include»s) the use of two output services allowing searching for a particular customer and car in order to obtain the correct customer_ID and car_ID and the trivial input service to register the rental.

Using the Zachman keywords, this service can be further described as follows:

- When: user invocation.
- What: input parameters (customer name, car type, start date of rental, return data of car). Output is confirmation of success or (in case of failure) error message and reason of failure.
- How: user provides customer name via user interface. All customers with this name are shown (show full name, address, birth date). User provides car details via user interface. Only available cars are shown (show full car details). Once customer and car have been selected, the rental registration will be triggered by clicking OK button. Upon success, confirmation is given and rental details are shown in printable format. Upon failure, reason of failure is given.
- Why: to be able to register a rental.
- Who: employees of the customer service desk and employees of the customer help desk are authorised to invoke this service.

'Congratulation Letter': Automatically produce a letter to congratulate a person who rents his/her fifth car This time, the service is triggered by the invocation of an input service to register a rental. This service is therefore an extension of the 'Register Rental' service as indicated by the «extend» relationship between both use cases. Again, there are several possible implementation choices to obtain the information to decide when to call the extension. For example, a counter can be defined that counts the number of cars ever rented by each customer. Or we can query the number of rentals for a given person.

Generally speaking, different implementation choices can be made for information system services. As an example one can differentiate dynamic database approaches making use of stored procedures (e.g. to automatically calculate and update the total amount of expenditure in a view table to be accessed by an output service) versus static databases combined with applications (using programmatic techniques for data access and manipulation, e.g. data access objects and/or SQL queries). The choice between different implementation options can be left to preferences at implementation side (see also Chap. 11).

9.3 Refining the Architecture of the Information System Layer

9.3.1 The Event Handling Layer

The input service is responsible for triggering the event, but the actual broadcasting of the event to the participating business objects and information objects is the responsibility of an event handling layer positioned between the information system service layer and the enterprise layer. For example, a *withdraw* event must be handled by the account and by the customer and it must create a new movement object. The input service is not responsible of making sure that all participating objects handle the request to execute the event. This is the responsibility of the event handling layer. If needed, input services can inspect the business objects and information objects to obtain all information required for issuing the event request. Taking into account information objects as well as event handling, the layered architecture is therefore further refined as shown in Fig. 9.8. In Chap. 2, we defined the information system service layer as a layer on top of the enterprise layer. The enterprise layer is refined to a (lowest) business and information objects layer and a business event handling layer residing on top of the business and information objects layer. The information system service layer consists of two types of services: the input and output services. Both types of services can directly 'read' objects in the business object layer, but the triggering of business events by input services happens by invoking the services of the business event handling layer (see Fig. 9.8).

9.3.2 Consistent Event Types and Transactions

Up to now, the event types that have been defined are 'atomic' event types. These fine-grained event types provide the building blocks for complex input services. The grouping of atomic event types into complex input services needs to happen for two reasons: consistency and user friendliness.

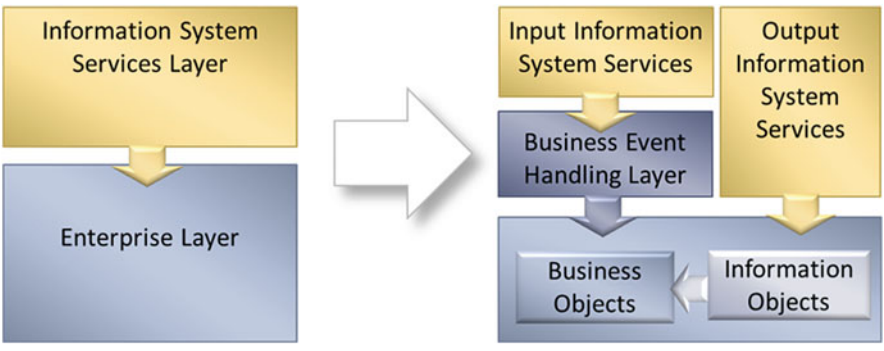


Fig. 9.8 Refinement of the layered architecture with an event handling layer

Consistency: In Chap. 5 it was said that atomic event types do not necessarily keep the object base in a consistent state. For example, when we consider the cardinalities in the existence-dependency graph, whenever there is a mandatory relationship, a master cannot be created without creating a dependent at the same time. As a result, creating the master would lead to an inconsistent state as the master is (temporarily) missing a dependent. A similar situation occurs when ending the last dependent in a mandatory dependency. For this reason, some atomic business events are consistent and some are not. To solve this issue, the atomic events identified so far need to be grouped to 'consistent composed events'.

User friendliness: Although business events are already more coarse than individual class operations, they still are too fine-grained to allow for efficient user input. In many cases, the grouping of business events will be required to allow for efficient user input. As an example, the implementation of a cascading end (ending an order and all its order lines) will require the grouping of several *end_orderLine* and an *end_order* event. Such grouping for efficiency purposes are 'transactions' that invoke multiple events.

Example 9.2 Assigning Employees to Departments

Consider an EDG that requests that each employee is assigned to exactly one department at any point in time (Fig. 9.9). Atomic event types are (a.o.) *cr_assignment, cr_employee, end_assignment* and *end_employee*. The fact that the relation from EMPLOYEE to ASSIGNMENT is mandatory (each employee *must* be assigned to a department at any time) implies that when an employee is created, an assignment must be created as well. Similarly, when the last assignment is ended, either the employee must be assigned to a new department or the employee must be ended as well.

Ensuring this type of consistency gives rise to the definition of *consistent* event types that are the grouping of atomic event types. Before and after the occurrence of a consistent event, the set of business objects must be in a consistent state, i.e. a state where **all** constraints are satisfied. In this example, maintaining consistency implies the definition of the consistent event types (the name of a consistent event type starts with 'C_').

- *C_cr_Employee*, which is a sequence of the atomic event types *cr_employee* and *cr_assignment*
- *C_end_employee*, which is a sequence of the atomic event types *end_assignment* and *end_employee*

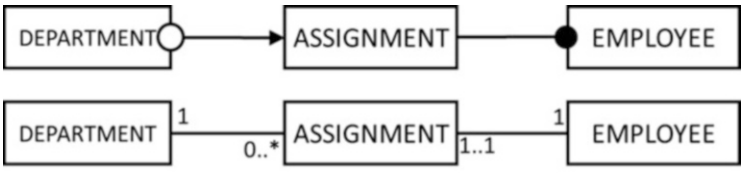


Fig. 9.9 Employees always belonging to a department

Table 9.2 Atomic events, consistent events and transactions for employees and departments

Enterprise layer		
Atomic events	*Consistent?*	*Comment*
cr_employee	No	Requires assignment to be created at the same time
end_employee	No	Requires deletion of assignment at he same time
cr_department	Yes	
end_department	Yes	
cr_assignment	No	Always happens together with cr_employee
end_assignment	No	Always happens together with either end_employee or cr_assignment
Composed events	*Consistent?*	*Composed of*
C_cr_employee	Yes	cr_employee.cr_assignment
C_end_employee	Yes	end_assignment.end_employee
C_move_employee	Yes	end_assignment.cr_assignment
Information system layer		
Single-event transactions	*Invoked event*	*Comment*
Create department	cr_department	Creates an empty department
End department	end_department	Terminates an empty department
Multiple-event transactions	*Invoked event*	*Comment*
Merge(dept1, dept 2)	(C_move_employee)*. end_department	Moves all employees from dept1 to dept2 and deletes dept1

– *C_move_employee*, which is a sequence of the atomic event types *end_assignment* and *cr_assignment*

In addition to these consistent event types, events may be grouped to offer efficient input services to users. An input service allowing merging one department into another is an example of a 'multi-event transaction', invoking a series of *C_move_employee* events, followed by the ending of the merged department. Simple input services that invoke just one event are 'single-event transactions'. Another example of grouping is the closing of projects as was described in Example 5.7: closing a project requires all registrations to be closed first. Rather than requiring the user to close all registrations one by one, a multi-event transaction can be defined that invokes a series of *close_registration* events followed by a *close_project* event.

Table 9.2 gives an overview of the atomic events, required consistent events and some sample transactions for the employee and department example.

As a consequence of defining consistent events and transactions, the architecture of the information system layer and the event handling layer can be conceived as a set of successive layers with increasing granularity of event handling. At the lowest and finest granularity level are the atomic event types that have been defined in the enterprise model. The second granularity level groups these atomic event types to consistent event types where necessary. The next granularity level is the one of input services, where we can distinguish between single-event and multiple-event transactions. These are only allowed to invoke the consistent atomic events and the consistent composed events. Only consistent events (either atomic or composed) are available to information system services. Inconsistent atomic events stay hidden for the information system layer. Finally, the business process layer will add still higher levels of granularity with tasks that will invoke (groups of) information system services and with business processes as conglomerates of tasks and subprocesses.

9.3.3 Cross-Cutting Concerns

Although most functional requirements can be organised in one of the three layers (enterprise, information system services, business process layer), some aspects are cross-cutting the layers. A typical example is authorisation. Authorisation can be defined at the three levels. At the business process level, it refers to task responsibilities and authorisations; at the level of information system, it defines who has access to what information system service; and at the enterprise layer level, it defines what domain objects one is authorised to access.

Example 9.3 Student Grading: Authorisation as Cross-Cutting Concern

At the level of domain objects, we define objects such as STUDENT, COURSE, COURSE ENROLMENT and TEACHER. A teacher will be allowed to determine the value of a grade (an attribute of COURSE ENROLMENT) only for students that are enrolled for a course he/she teaches. At information system service level, all teachers have access to the input service that allows them to register grades. Students don't have access to that service. But the permissions of the teachers are further constrained by the domain objects as they are only allowed to enter or update the grades of their own students. Furthermore, at business process level, it has been determined that they need to do this before the end of the examination period. Once the examination period has been closed, teachers lose access to the grade input service. On the other hand, student administration will have the permission to update the grades of all students enrolled at the faculty, even though these people do not teach courses. Furthermore, they are also allowed to do so outside the examination period.

As a result, the actual authorisation is a combination of business process aspects (role played and timing constraints), access to information system services and access to business object instances.

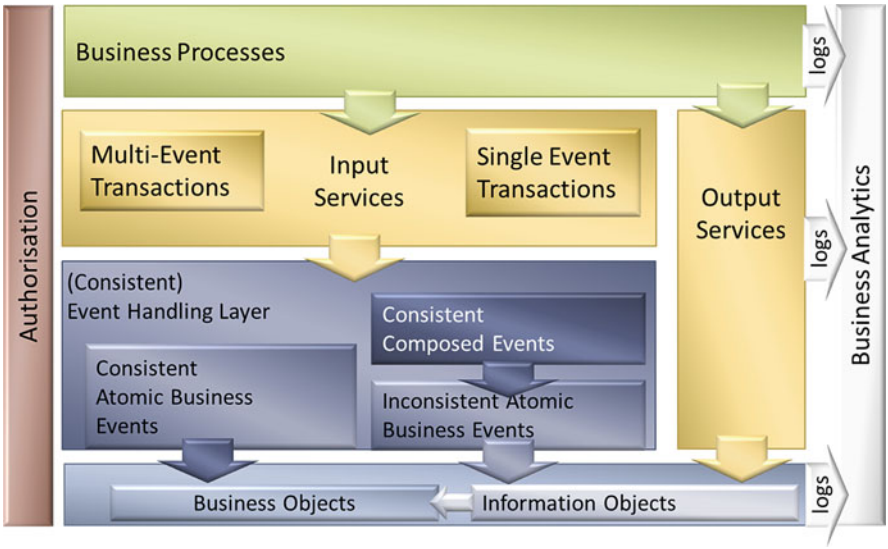


Fig. 9.10 Extended layered architecture with cross-cutting concerns

Data collection for business intelligence purposes is another example of a transversal aspect. Business objects that are in a final state may be kept stored rather than physically deleted such as to make them available for further (big) data analysis purposes. Also event logs may be kept that reflect the triggering of business events (enterprise layer), the use of information system services (information system service layer) or the execution of tasks (business process layer). All these logs constitute valuable sources for data analytics.

Figure 9.10 presents the refined layered architecture, including the aforementioned cross-cutting concerns.

9.4 Case Study: The JMermaid Architecture

9.4.1 Information Object Types

JMermaid is the computer-aided software engineering (CASE) tool that supports the domain modelling part of MERODE and it has (obviously) been architected following the MERODE principles. The meta-model described at the end of Chaps. 4–8 defines the business object types for the MERODE CASE-tool. As the CASE-tool needs to represent the objects in a graphical way, each meta-model object type has zero to many corresponding 'graphical model' object types. These graphical model objects keep track of layout information such as the x and y

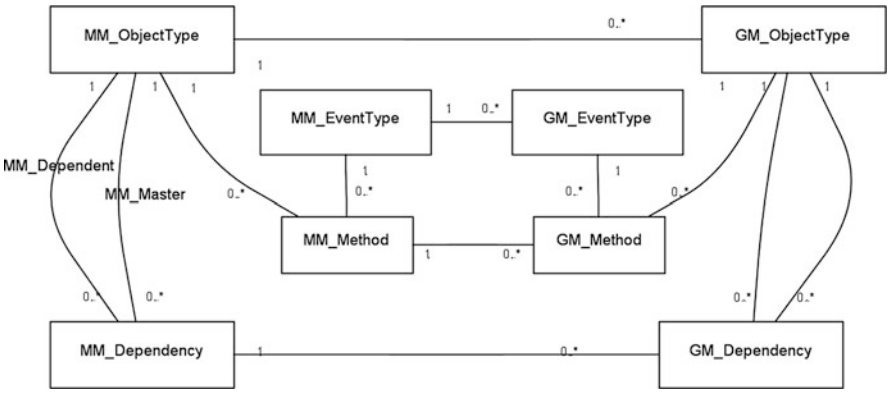


Fig. 9.11 Extract of the JMermaid business and information object types diagram showing the business object types on the left (names prefixed by MM_) and the information object types on the right (names prefixed by GM_)

coordinates, line colour, fill colour and visibility of labels. They are information object types. A meta-model object (names prefixed by MM_) can have many corresponding graphical model objects (names prefixed by GM_) because it may appear on different diagrams. For example, a business object (domain model class) is represented both in the EDG and in the OET, but an attribute has no graphical representation. Figure 9.11 shows an excerpt of the combined business (meta-model) and information (graphical model) object type class diagram showing the business object types on the left and the information object types on the right.

9.4.2 Event Handling

Meta-model business events are events that manipulate meta-model objects. For example, adding a class to the EDG corresponds to a *cr_mm_objectType* event type. When a domain class (instance of MM_OBJECTTYPE) is represented graphically, one or many corresponding graphical model objects (instances of GM_OBJECTTYPE) need to be created as well. For the given example, a box needs to be drawn on the EDG tab and a column needs to be added to the OET. On the other hand, certain actions affect only the graphical object. For example, moving a class on the EDG or changing its line colour or fill colour only affects the graphical model object. To this end, in JMermaid, a separate series of 'graphical model events', has been identified. These events only affect the 'GM_' classes. In JMermaid, events have been implemented as commands (using the command pattern), and to distinguish between the two types of events, the event handling layer contains two types of commands: meta-model commands and GUI commands. The former manipulate meta-model objects, while the latter manipulate graphical model objects.

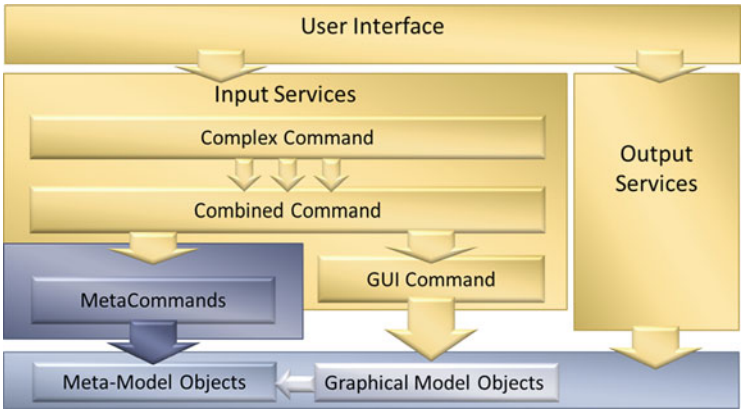


Fig. 9.12 General architecture of the JMermaid tool

When the user performs some action at user interface level, JMermaid will determine whether the action implicates only the graphical model (moving, resizing, zooming, changing colours, etc.) or whether it affects meta-model objects (creating, updating or deleting an object type, a dependency, an event type, etc.). In the first case, GUI commands will handle the event. To handle requests of the second type, 'combined commands' have been defined. These are multi-event transactions that first validate and invoke a meta-model command and then invoke the required GUI commands. Assume, for example, that a user wants to create an object type 'product' in JMermaid. The combined command 'Add_ObjectType' first invokes the 'cr_ObjectType' meta-command. This command validates the request, e.g. by checking whether the proposed object type name is unique within the model under construction. If accepted, the object type 'product' is added to the model. After the meta-model command has successfully been executed, the GUI command 'GUI_create_ObjectType' command is invoked, which will create two instances of GM_OBJECTTYPE for the 'product' instance of MM_OBJECTTYPE: one for its representation in the EDG and one for its representation in the OET.

Complex commands are an active way to ensure the consistency of a model: by grouping a set of commands into one complex command, the completion of specifications entered by the user can be enforced. For example when an object type is added to a model, besides creating the object type itself, also default events, default methods and a default finite-state machine are created for that object type. To achieve this, the complex command to add an object type consists of a list of 11 commands: one to create the object type, two to create the creating and ending event types for that object type, two to create the corresponding creating and ending methods and six commands to create a finite-state machine, its initial, intermediate and final state and the two transitions for the creating and ending method. In that way, the alphabet rule and the default life cycle rule are obeyed, and the tool itself can at any point in time rely on the existence of the default finite-state machine.

The resulting overall architecture is shown in Fig. 9.12.

9.5 Meta-model

The meta-model can be further expanded with the concept of INFORMATION OBJECT TYPE and INFORMATION SYSTEM SERVICE. Both types of information services can inspect persistent object types (OBJECTINSPECTION), but only input services will trigger events (EVENTTRIGGER). Notice how input services trigger the events, but not the individual methods of object types. The meta-model in Fig. 9.13 still makes abstraction of the distinction between consistent composed events and atomic events, the full detailed meta-model being out of scope of this book.

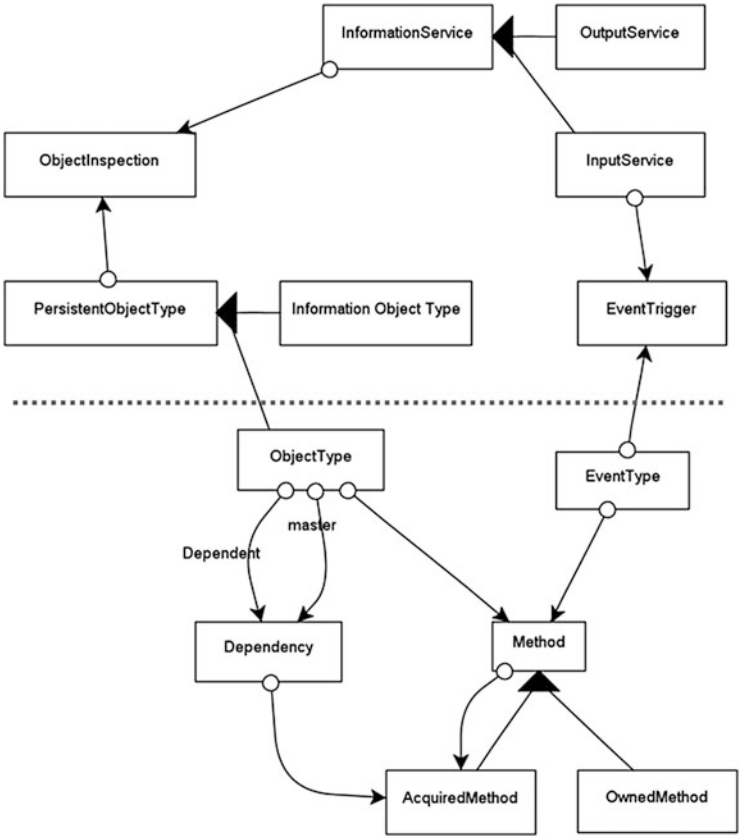


Fig. 9.13 Meta-model for the information system layer

Chapter 10
Bridging Business Process Modelling and Domain Modelling

10.1 Business Process Modelling

As explained in Sect. 1.3.2 most Enterprise Architecture frameworks seem to agree that at least the following viewpoints should be included in a Business Architecture description [5]:

- The 'why' describing the goals and the motivations of why things are done a certain way.
- The 'what' describing the main concepts or business objects of the organisation.
- The 'how' describing the business processes that are performed at the operational level.
- The 'who' describing the actors and resources used to perform tasks in the business process models.

The previous chapters of this book have concentrated on the 'what' by defining the business object types and part of the 'how' by defining business events and the behaviour of business object types. Business process modelling complements domain modelling by capturing the organisational dimension in terms of actors, activities and workflows. While business process modelling is commonly stated to address the 'how' of the enterprise, it actually has a much broader scope in practice. Business process modelling addresses the control flow ('what needs to be done and when') by defining sequence relationship between activities and business process events. It addresses the relation between the process and the data perspective ('what do we need to work on') by defining the input/output data to activities as a complement to the control flow. The data definition itself is not addressed: it is assumed that the modelling and definition of the persistent data required for the process has been defined in a domain model. Finally, business process models also address the resource perspective ('who's doing the work') by defining the actors (human participants and systems) that perform activities and generate business process events. If business processes are moreover clearly linked to goals (the

© Springer International Publishing Switzerland 2014

M. Snoeck, *Enterprise Information Systems Engineering*, The Enterprise Engineering Series, DOI 10.1007/978-3-319-10145-3_10

'why' dimension), one can state that business process modelling involves the six columns of the second row of the Zachman framework.

Business process modelling and object-oriented technology have undoubtedly been some of the most important domains of interest of information systems and software engineering research over the past three decades. For a long time, both domains have largely evolved independently, and still not much research can be found in which business process modelling principles are fully linked to domain modelling or vice versa [63, 85]. In fact, both views are complementary: what is (partly) abstracted from in one view is (more) explicitly present in the other. In domain modelling, the primary emphasis is on the specification of a structural model of business object types and associations between these object types, whereas the business process part is largely neglected or supposed to be given. Especially when object-oriented analysis and design methods are used for domain modelling [e.g. 12, 13, 21, 27, 67], business process modelling is still treated in a fairly limited way [85]. Similarly, in business process modelling, the domain model is often neglected or supposed to be given [22, 81]. When the complementary aspects are ignored, the danger exists that these aspects are not properly separated in the specifications. In particular, in object-oriented requirements engineering the danger exists that business process aspects are modelled as part of the behaviour of domain classes, resulting in hard-coded business processes and hence inflexible systems that are hard(er) to adapt. Separation of concerns is, however, one of the main principles to ensure adaptability and flexibility of information systems. This principle dictates to keep process aspects separated from the domain model, but nevertheless the link between domain concepts and the organisational elements should be clearly established. For example, sequence constraints on business events that result from the business logic should be part of the domain model (e.g. in a library, the return of a copy to the library must be preceded by a borrowing event). These sequence constraints are less likely to change over time than sequence constraints that are the result of work organisation aspects (e.g. if a member of the library does not show up after five reminders, set all the books he/she borrowed to the state 'lost'). A clear vision of what should be put in what layer and how these aspects are linked to each other is important in designing more adaptable systems.

In order to achieve this, it should be clearly defined how the business process model links to the domain model in order to be able to compose the full picture of the business. Exactly this link is missing in many domain modelling approaches: domain models and business process models are created side by side, without clear guidelines on what to model where and without clearly established links. In business process models, the definition of input and output data of activities and of data required for decisions in data-based gateways refer to the 'what' dimension, but business process modelling does not genuinely address domain modelling. Reversely, domain models provide the necessary ingredients for defining work organisation, and although certain business object types reflect units of work (e.g. ORDER), a domain model can't adequately reflect work organisation. Integrating domain modelling and business process modelling allows obtaining an integrated picture of the enterprise.

Apart from the need for a 'global' picture to understand the business, another reason why domain modelling and business process modelling need to be integrated is that both address behavioural aspects of the universe of discourse and might therefore be the source of conflicting requirements. The dynamic aspects of the domain model define the behaviour of business objects by specifying internal object behaviour (state machine diagrams) and interactions between simultaneously living objects (the object-event table in this book, or sequence diagrams in a general UML-based approach). The business process model uses process diagrams to describe sets of activities that need to be executed in order to achieve certain goals. Both the business process model and the domain model are concerned with defining behavioural aspects of the enterprise, but whereas the diagrams in the domain model describe the behaviour on a per object basis, the business process diagrams will describe the same reality from a per work process basis. For instance, an order handling business process will describe the work organisation as a set of consecutive steps such as order intake, order acceptance, order shipment and order payment. The domain model will describe concepts such as CUSTOMER, PRODUCT, ORDER and ORDERLINE and describe a life cycle for each of these business concepts. An example of constraints emanating from work organisation could be that a product cannot be ordered when it is out of stock or that an order can be modified by the customer as long as it is in the state 'in registration' but cannot be modified by the customer anymore once it has reached the state 'confirmed'. The company's employees on the other hand still can modify it to account for special situations. Obviously, the business process model will assume that the business objects involved in the business process support the business process steps adequately. If the domain model defines that no order deliveries are to proceed before the payment is registered, the proposed business process is not consistent with the domain model, as the business process requires order shipment to be performed before order payment. Obviously we face a situation where common elements of the dynamic aspects are included in different views of a same model, holding the risk of inconsistencies. The (in)consistency issues stem from the fact that both the business process model and domain model document the business functionality from a different but overlapping point of view. It is precisely this overlapping aspect that causes the inconsistency problems to arise. Therefore, an integrated approach which supports a seamless and consistent design of all views is needed to ensure a successful implementation of the system.

While the need for an integrated approach in which models can be cross-checked for consistency requires a little more motivation, the way to achieve this integrated model is, however, less straightforward. In research literature, essentially, three approaches are discussed to achieve this: transforming the business process to object-oriented concepts, adding object information to the business process model and linking the models while preserving their differences. The latter approach is the most adequate as it conforms to the principle of multiple views addressing separate concerns. Specifically for UML, authors have identified links between the business process model and different UML diagrams, such as the class diagram, state charts,

use case diagram and sequence diagram. However, the interpretation of the link and the focus of discussion may differ. Some focus on the relationship between business process models and the UML use case diagrams [64] while others advocate that the main relationship is provided between the business process model and the class diagram [49]. Multiple papers on this topic have been published [7, 48, 49, 64, 75, 78, 92, 95]. A common observation for many of these papers is that the authors present an informal definition of the link between both models and refrain from discussing the ways to verify the consistency between the models.

10.2 Linking Business Processes to the Enterprise Layer and the IS Service Layer

As a full-fledged enterprise, information system development method should take all aspects into account: business processes, the domain model and information system services should be linked together. In the layered architecture proposed in this book, business events and information system services are used as hinge concepts between business process modelling and domain modelling.

Business process modelling takes an action- and process-oriented view on the domain. Business processes are therefore formulated in terms of subprocesses, tasks and events. Hence, next to business events and information system events, there are also business process events.

- In the domain modelling perspective, only business event types are of particular interest. Business events are events that are shared between environment and information system. Although they are (also) part of the information system, *business* events are only those events that are relevant from a business perspective, even if there is no information system around (see Sect. 5.1).
- The information system service layer handles information system events such as keyboard actions and mouse clicks (also called application events). Information system events are directly related to the presence of a computerised system. They are designed to allow the external user to register the occurrence of or invoke a real-world event. For example, the use of an ATM machine to withdraw money from one's account will invoke the business event '*withdraw*' by means of several information system events such as '*insert card*', '*enter PIN code*', '*enter amount*' and so on.
- A business process event is 'something that "happens" during the course of a process' [68]. A further distinction is made between events that start, interrupt, resume or end workflows. They can be of many different origins such as clocks, messages, errors, cancellations, signals, etc. A timer that reminds me that I should start preparing my lectures, a message that warns me that certain exam grades are missing and a monthly timer that initiates a transfer of money to my savings account are examples of workflow events.

Obviously, these different kind of events relate to each other. Once business event types have been identified in the domain model, the whole domain model can be considered as one component, the interface of which is the set of all event types that allow the creation, modification and updating of the information contained in the domain model. Input information system services are then nothing more than a way to invoke these business events. The input information system service will translate information system events (e.g. originating in the user interface) into the invocation of one or more business events. An example would be a web-based form accepting input such as customer name, address and email that triggers a *register customer* business event upon hitting the 'send' button. Business process events describe how events that originate in the real world or in information systems affect the work organisation. Some business process events represent business events. For example, a 'none' start event can represent an environment-controlled event, such as me deciding to bake my own bread on a Sunday morning (see Fig. 1.4). Likewise, the researcher deciding to buy a book is represented by the 'none' start event (the green circle that starts the business process) in the lane of the researcher in Fig. 10.2. Business process events can also be information system-related, like message events. As an example, the intermediate events 'receive notification' in the lane of the researcher symbolise the arrival of an email.

In the domain model a business event represents 'a request to perform an activity to register something that happens or that you would like to happen in the real world' (see definition on p. 93). In a business process model, the aspects pertaining to the (environment) control, request, activity and registration are modelled in more details.

Example 10.1 Borrowing a Book: Relationship Between Different Kinds of Events

Figure 10.1 represents the business process for borrowing a book. The fact that the business event *borrow* is an environment-controlled event is represented by the 'none' start event in the lane of the member. The task 'register loan' is the task that will trigger the execution of the business event *borrow* through some user interface and information system events. The request aspect of this business event is represented by the 'error' interrupting intermediate event, which symbolises that the loan may be refused. In that case the loan is not registered and the business process defines that it is the librarian's task to put the book back on shelf.

When modelling business processes a two-step approach can be taken to integrate business process models with the enterprise layer and the information system service layer. First, each task is identified as an input or information task. In the first case, the task involves input, modification or ending of business objects, and the to-be-triggered business events need to be identified. In the second case, the task will only consult business objects, which can be indicated by linking the task or message event to the consulted business object types.

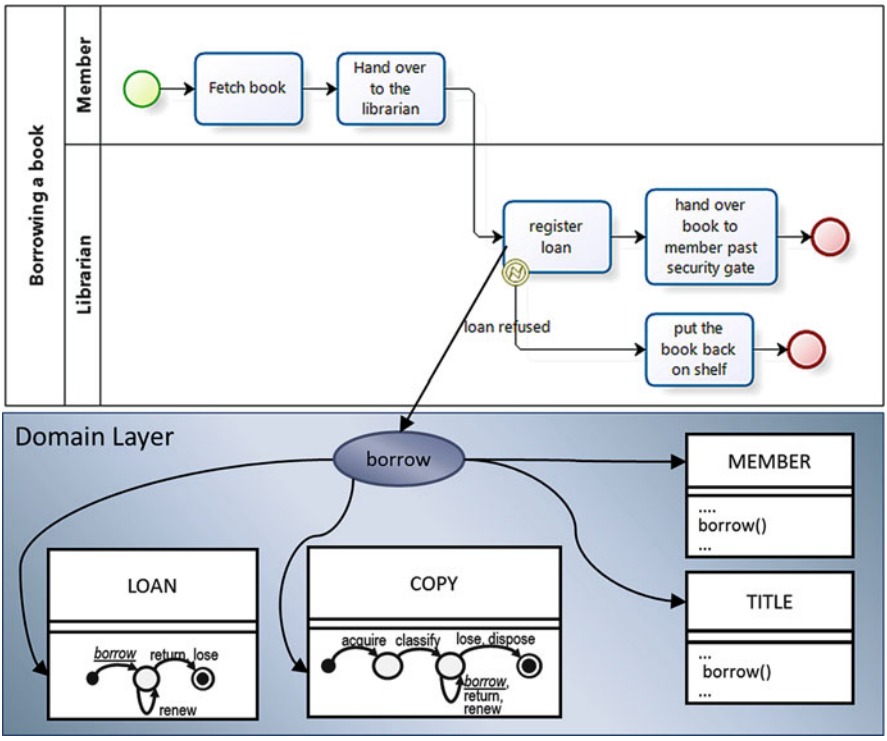


Fig. 10.1 Business process for borrowing a book

In a second step the business processes are analysed in search for required information support through information system services. The tasks that have to be performed by actors can be further classified as manual, interactive or fully automated. Interactive and automated activities are realised by means of an information system service. These information system services are put in between the business process layer and the enterprise layer and will offer the concrete support to invoke business events via the event handing layer.

Example 10.2 Book Acquisition: Linking the Business Process Model to the Domain Model

Let us consider the business process for acquiring a book as presented in Chap. 3; see Fig. 10.2.

As only the acquisition officer will work with the enterprise information system, the business process of the researcher will not link to the domain model. Table 10.1 relates the tasks of the business process of the acquisition officer to the library domain model.

Tasks 6 'Order book from editor' does not link to the domain model. In this example it is assumed that task 6 will be performed using the editor's sales channel and that this will not be replicated in the own organisation's systems. At KU

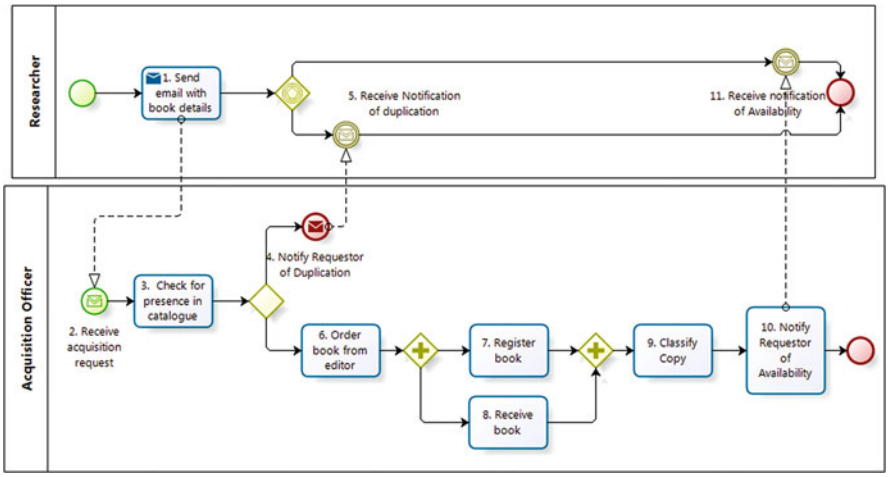


Fig. 10.2 Acquisition business process in the library

Table 10.1 Linking the acquisition process to the domain model

Task	Input or output?	Triggered business events	Consulted business object types
1. Send email	–		
3. Check for presence in catalogue	Output		TITLE, COPY
4. Notify requestor of duplication	–		
6. Order book from editor	–		
7. Register book	Input	*cr_title*	
8. Receive book	Input	*acquire*	
9. Classify copy	Input	*classify*	
10. Notify requestor of availability	Output		COPY

Leuven, it works differently as a purchase always has to be registered as a purchase order (PO) in KULoket. In such case, the domain model needs to be extended with the concept of PURCHASEORDER and task 6 will trigger a *cr_PO* event, and task 8 'Receive book' will update the PO to register the arrival of the ordered goods. Task 4 does not link to the domain model as it is performed using the email platform.

If information system services are considered as well, the model can be further refined as in Table 10.2.

Table 10.2 Linking the acquisition process to information system services

Task	Input or output?	Information system service	Triggered business event/consulted object types
1. Send email with book details	–	*Outside the EIS (manual email)*	
2. Receive acquisition request	–	*Outside the EIS (manual email)*	
3. Check for presence in catalogue	Input	**Output service** that allows to consult the list of registered titles and copies	TITLE, COPY
4. Notify requestor of duplication		*Outside the EIS (manual email)*	
5. Receive notification of duplication		*Outside the EIS (manual email)*	
6. Order book		*Outside the EIS (seller's sales channel)*	
7. Register book	Input	**Input service** that allows to register the title	*cr_title*
8. Receive book	Input	**Input service** that allows registering an additional copy for an existing title	*acquire*
9. Classify copy	Input	**Input service** that allows to assign a placement number to the copy	*classify*
10. Notify requestor of availability	Output	**Output service** that enables to look up recently acquired copies and their details in order to compose the email (manual)	TITLE, COPY
11. Receive notification of availability		*Outside the EIS (manual email)*	

10.3 Distributing Constraints Across Layers

10.3.1 General Principles

Both the business process model and the domain model allow to define constraints. A natural question to ask then is where to put what kind of constraints. Unfortunately, the answer to this question is not clear-cut. However, the following general advice can be formulated:

1. In the enterprise layer, preconditions and invariants *prevent* something bad to happen *but cannot enforce* something good to happen. We assume that the constraints formulated in the domain model are enforced by the event handling layer (see p. 127).

2. The enterprise layer model should only contain constraints that enforce rules that should *never* be broken. 'Flexible' rules, i.e. rules that can be overridden in certain circumstances, should not be enforced by the enterprise layer but rather be captured in the higher-level layers, namely, the information system service layer and the business process layer.
3. The enterprise layer should only contain constraints that enforce rules that are unlikely to change often. 'Variable' rules, i.e. rules that are frequently adapted, should not be enforced by the enterprise layer, as their adaption would require adapting the enterprise layer, which might entail changes in all layers above the enterprise layer. They should rather be captured in the higher-level layers, namely, the information system service layer and the business process layer.

The following example illustrates the first rule.

Example 10.3 Returning Parcels

Assume an online shop that delivers the goods to the customers through some parcel collection points. Once the goods have arrived at the parcel point, the customer is notified of their arrival. The customer has two weeks to collect the goods and decide whether or not to return some goods.

Every day, the employee of the parcel collection point needs to check for the presence of overdue parcels that were not collected by the customer. These are returned to the shop, together with the returned goods. The employee needs to do this first thing in the morning, before the shipper arrives to bring the new parcels and take the returned parcels. The employee will register the to-be-returned parcels by scanning their bar code and confirming their return in the system.

The above specification has led to the definition of the object type PARCEL in the domain mode, and the business events *deliver, collect* and *return* that change the state of a PARCEL object and which are used to register the fact that a parcel has arrived at the parcel point and is ready to be collected (*deliver*), a customer has collected a parcel (*collect*) and that an uncollected parcel is returned to the shop (*return*). Additionally, in the class PARCEL, there is a precondition for the method *return* that specifies that a parcel should not be returned earlier than two weeks after its arrival at the parcel collection point.

```
class Parcel
return ()
  precondition
    today > (arrival_date + 14)
```

This precondition will *prevent* the employee from erroneously registering the return for a parcel that is not yet overdue. The precondition can, however, *not ensure* that the employee performs the daily check and returns all overdue parcels.

In order to ensure the daily check to happen, a daily timer can start (trigger) the business process 'check and return overdue parcels' each day at 8 o'clock. Figure 10.3 shows the relationship between the trigger in the business process model and the precondition in the domain model.

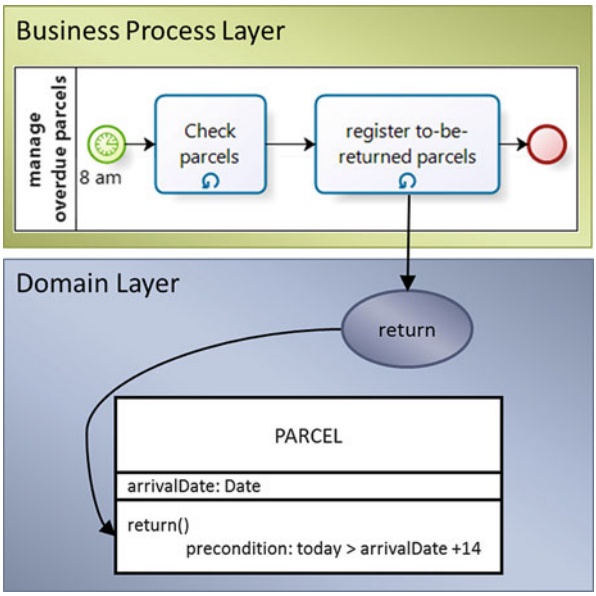


Fig. 10.3 Triggers versus preconditions

The second rule is motivated as follows. As the domain model shapes the kernel of the enterprise system, it constrains all above layers. Indeed, every time a business process task invokes a business event (via an information system service), this event is validated by the domain model. If the event violates a constraint defined by one of the involved domain objects, the event is rejected. Hence, only constraints that should never be violated can be put in the domain model.

The third rule is motivated by the desire of lowering the maintenance cost of software.

Constraints have a strong link with business rules as they are one of the mechanisms than can be used to enforce business rules. The SBVR standard from the OMG [69] distinguishes between business rules and business policies. It defines both concepts as follows (cfr. [69], Sects. 12.2.1 and 12.2.2):

Business policy:

– *Definition: element of governance that is not directly enforceable whose purpose is to guide an enterprise.*
– *Note: Compared to a business rule, a business policy tends to be:*
  - *Less structured*
  - *Less discrete or not atomic*
  - *Less carefully expressed in terms of a standard vocabulary*
  - *Not directly enforceable*

Business rule:

– *Definition: rule that is under business jurisdiction. General concept: rule, element of guidance.*

– *Note: A rule's being under business jurisdiction means that it is under the jurisdiction of an authority that can opt to change or discard the rule at its own discretion.*

A business rule may be used to implement a business policy. A special case of business rules are operational business rules, which are enforceable. The Business Motivation Model standard from the OMG [70] defines different levels of enforceability, one of which is 'strict'. The other enforcement levels allow deferred enforcement or overriding of the rule (cfr. [69], Sect. 12.2.3 and [70], Sect. 8.3.7).

As a consequence, the enterprise layer should only contain constraints that enforce business rules that can never be broken and hence require a 'strict' enforcement. Business rules with lower enforcement levels or business policies may be captured (and enforced) in the business process layer. This implies, for example, that sequences that arise from guidelines about a particular way of working should be put in the business process layer, whereas sequences that are essential and inherent to the business should be put in the enterprise layer. The underlying idea is that essential business domain rules are supposed to be quite stable and hence can be put in the inner layer of the system, i.e. the enterprise layer. Organisational aspects on the other hand tend to change more frequently and may require flexibility to allow for exceptions. Their enforcement should therefore be handled by the upper layers. When deciding whether or not to enforce a strict business rule through constraints in the domain model, one must keep in mind that business processes must be flexible in order to adapt easily to a changing business environment. In order to allow for flexibility at the business process level, the domain model should leave sufficient degrees of freedom.

The next paragraph illustrates this issue of 'sufficient degrees of freedom' by means of an example of event sequencing in the domain model versus allowed event sequences required by the business processes. The formal elaboration of this example has been published in [88].

10.3.2 An Order Handling Example

The following example is inspired by a real-life case in the domain of telecom services. A company sells products that must be installed at the customer's site. The domain model defines the concepts of product, sales order, order line and customer. The EDG is given in Fig. 10.4 and the OET is given in Fig. 10.5.

Analysis of business processes allows discovering more relevant business event types and sequence constraints between event types. As an example, let us consider the business process for creating a sales order (represented in Fig. 10.6, for the sake of simplicity we assume that orders never are cancelled and that installations eventually always succeed):

A sales person has received a request for proposal, visits the customer, consults technical people for the optimal choice of products, reaches an agreement on what the customer wants to buy, registers the sales order, prints it and has it signed by the customer. The sales

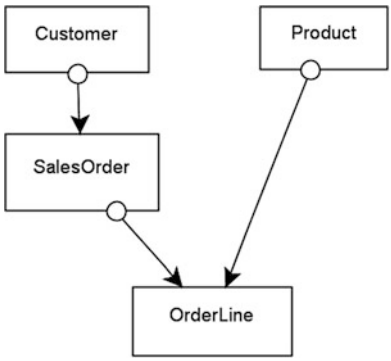


Fig. 10.4 EDG for sales orders (adapted from Fig. 2 in [88])

	Customer	Product	SalesOrder	OrderLine
cr_Customer	O/C			
end_Customer	O/E			
cr_Product		O/C		
end_Product		O/E		
cr_SalesOrder	A/M		O/C	
mod_SalesOrder	A/M		O/M	
end_SalesOrder	A/M		O/E	
cr_OrderLine	A/M	A/M	A/M	O/C
mod_OrderLine	A/M	A/M	A/M	O/M
end_OrderLine	A/M	A/M	A/M	O/E

Fig. 10.5 Object-event table for sales orders (adapted from Fig. 3 in [88])

order is then passed on to Customer Operations. They do a final technical check, accept the order, plan its installation and install the products at the customer's site. At the end of the installation process, the customer signs the acceptation of the products and from then on the billing of the installed services is started.

By analysing each task, additional business events are discovered. In terms of domain modelling, the following additional business events are required for a sales order:

– To register the signature of the customer: *customer_sign*
– To signal the hand over to Customer_Operations: *operations_accept*
– To register the successful installation: *customer_accept*

The sequence constraints that emerge from this business process description are the following:

– A sales order is first signed and then accepted by Customer Operations.
– After the *operations_accept* event, installation will proceed and eventually the customer will accept the installation.

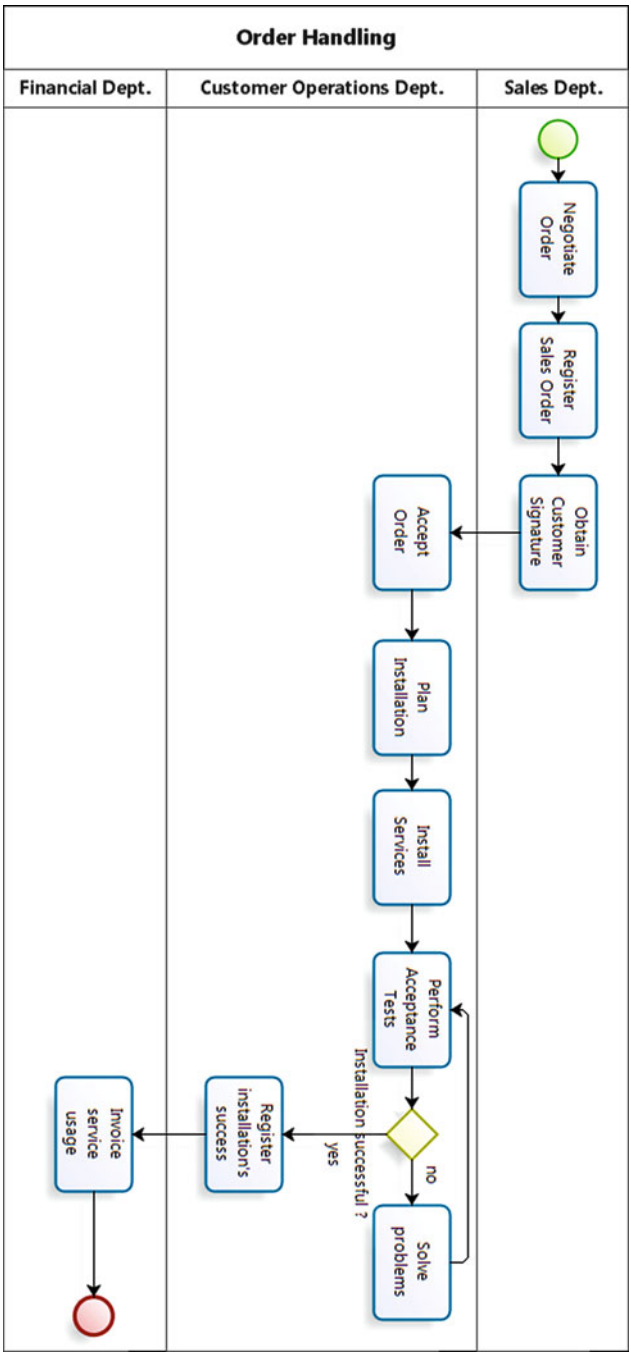


Fig. 10.6 Standard business process for sales (adapted from Fig. 5 in [88])

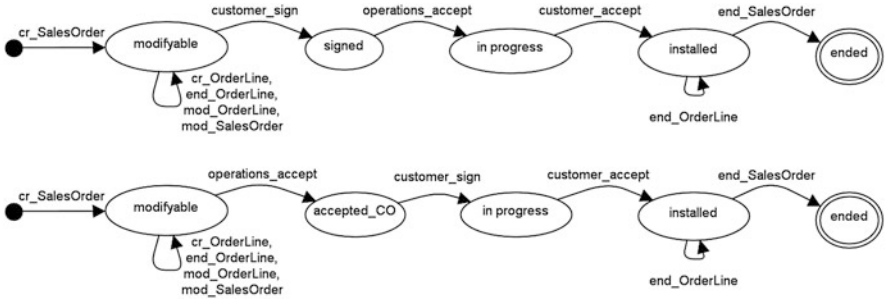


Fig. 10.7 FSMs for sales order corresponding to the standard business process (*top*) and corresponding to the process for complex sales orders (*bottom*) (adapted from Fig. 6 in [88])

– It is assumed that once a sales order has been signed, it cannot be modified anymore: it is a contract with a customer and one cannot change the terms of a contract after it has been signed.

Because the signature of a sales order by the customer and the (technical) acceptance of the sales order by Customer Operations are considered to be essential for the business, it is initially decided to model these as business events and to enforce their sequence through the domain model. The corresponding FSM for the sales order is given in Fig. 10.7 (top).

After interviewing more domain experts, it appears that sometimes a different business process is followed. In the case of complex configurations, the technical background of sales people is not always sufficient to ensure that the products on the sales order meet all technical requirements. The possibility that a change of the sales order will be required during the installation process is real, although it occurs only in a minority of cases. The finite-state machine given above, however, enforces that a signed sales order cannot be modified anymore. For this reason, in such cases, the sales order must be reviewed by Customer Operations prior to the registration and signature of the order by the customer. A successful review by Customer Operations is also considered as hand over of the sales order. Once Customer Operations has successfully reviewed the sales order, they also are bound to install the order without further modifications. So, besides the standard business process that is followed in most cases, there exists an alternative business process for complex sales orders, represented in Fig. 10.8.

This alternative business process puts the *operations_accept* business event before the *customer_sign* event as represented in the new state chart for sales order depicted in Fig. 10.7 (bottom).

As a result, the analysis of the two scenarios results in two conflicting state machines for sales order, namely, in one case the customer's signature precedes the operation acceptance and in the other case it is vice versa. Hence, some of the sequence constraints should not be put in the domain model but rather in the business process model. In this case study, the sequence constraints between modifying events and signature of the sales order are an example of a business

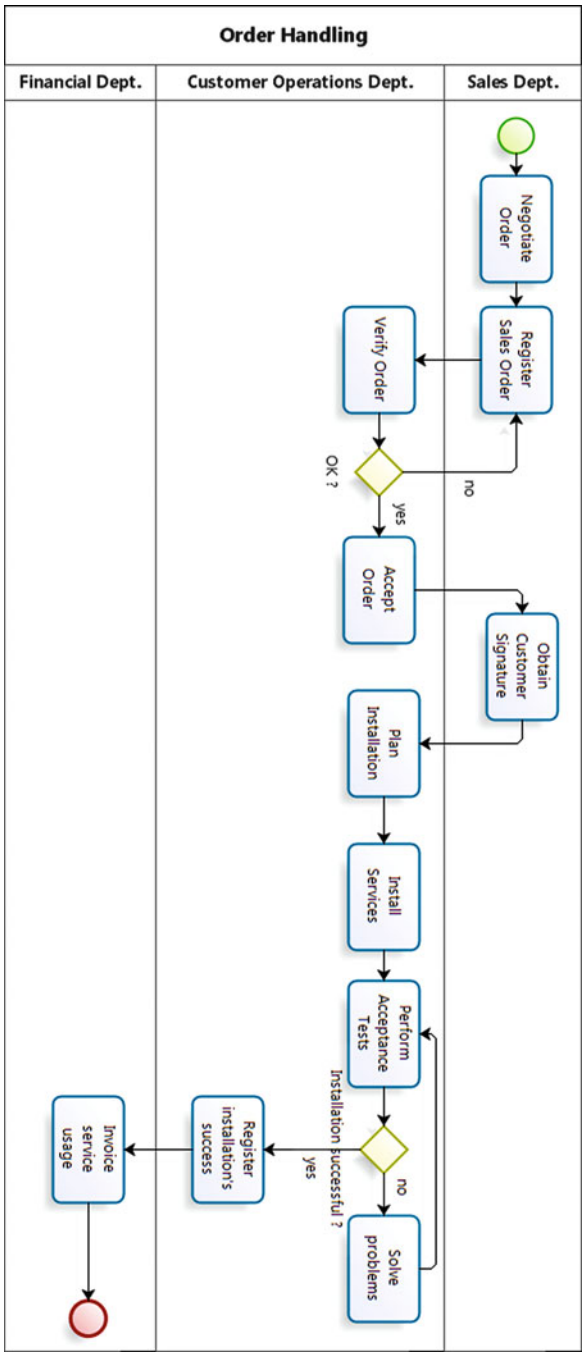


Fig. 10.8 Business process for complex sales orders

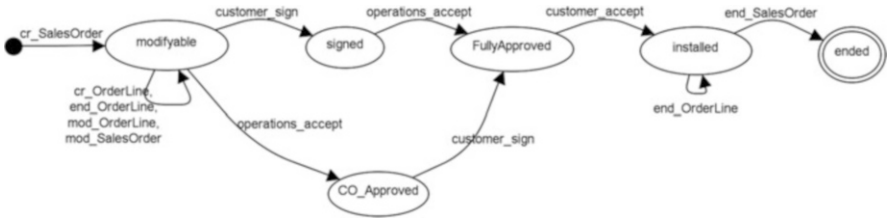


Fig. 10.9 State chart for sales order that allows for the two business processes

rule that has a 'strict' enforcement: these sequence constraints should never be violated. Once a sales order has been signed, it cannot be modified anymore without having it re-signed by the customer (which amounts to creating a new sales order and have this new sales order signed). The sequence constraints between *customer_sign* and *operations_accept* can, however, be overridden in certain circumstances: they represent a business rule with a lower enforcement level. For efficiency purposes, it is the policy of the business to have a sales order signed before passing it to Customer Operations. However, in complex situations (the notion of 'complex' can in its turn be defined by a decision rule), the sales person must check the sales order with Customer Operations prior to the signature by the customer. In order to allow different scenarios in the business process layer, the domain model must leave sufficient degrees of freedom. In this particular case, the behaviour definition of sales order must not enforce sequence constraints between *customer_sign* and *operations_accept*. The resulting state chart is given in Fig. 10.9.

10.3.3 Cross-Layer Consistency

The example of the previous section illustrates the fact that an incomplete analysis of business processes may result in a too strict set of constraints in the domain model. But also the specification of new business processes at a later point in time may result in the fact that the new business process is in conflict with the domain model constraints. Given the fact that the business process models and the domain model might contradict each other, the question raised is how to check consistency across these models. As there is a many-to-many relation between processes and business objects (one process may affect many objects and one object may be affected by many processes), an automated cross-model consistency check of sequence constraints is far from evident. Yet it is possible to achieve this to a certain level by using the power of formal languages. The remainder of this section briefly outlines the main idea behind such check.

Today, many practitioners model business processes with BPMN. In the academic community, Petri Nets are the preferred modelling paradigm to formalise business process models as their expressive power is larger than the expressive

power of regular expressions and finite-state machines. Its ability to model interleaving and the ability to model individual cases as tokens makes Petri Nets a more attractive theory for the formalisation of business processes. By using Petri Nets as modelling language, a business process can be defined as a Petri Net language over the set of tasks [19]. In its turn, the execution of a task implies the triggering of business events, and therefore a task can be defined as a formal language (regular or Petri Net) over the set of business event types. By nesting the formal language of the tasks into the Petri Net definition of a process, a business process can be considered as a formal Petri Net language in terms of business event types. Each business process thus defines a set of scenarios of business event types in the same way as a business object type defines sets of scenarios (its language).

The business events triggered by the business process layer will only be accepted provided they satisfy all constraints defined by the domain model. As a result, in order to be consistent with the domain model, each of the scenarios defined by a business process must be accepted by the domain model (see also Fig. 10.10). Comparing scenarios of events generated by a business process with the behaviour definitions of object types is, however, not straightforward since the life cycle of one object can be spread across different tasks and even across different business processes. If we consider, for example, the task of modifying an order from the point of view of a single order line, this task will generate either a single *cr_order_line* event, a single *mod_order_line* event or a single *end_order_line* event. Usually, a single task will move a domain object one step further in the life cycle. In addition, a single task can treat several domain objects of a *same* type. For example, the 'modify order' task can manipulate several order lines at once. Hence, if we analyse the *types* of events that are generated by this task, we see that this task generates scenarios with *cr_order_line, mod_order_line* and *end_order_line* events in a random order. As a result, the analysis of business event type scenarios generated by tasks and business processes must be done by looking at the events that affect a single object occurrence and not by looking at the

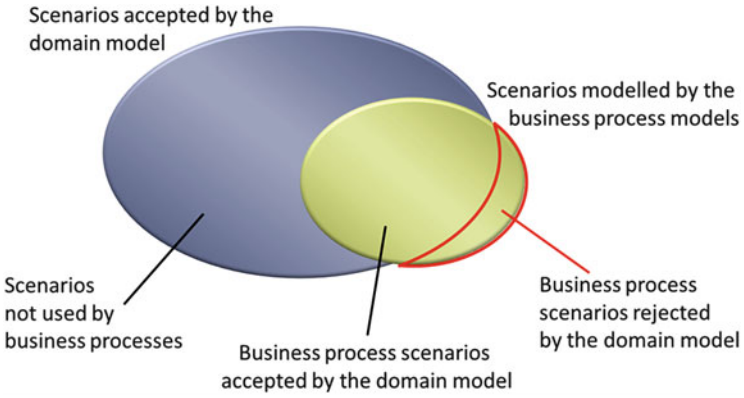


Fig. 10.10 Relationship between the set of scenarios accepted by the domain model and the set of scenarios accepted by the business process models

types of the events only. Checking the consistency between a business process model and a domain model is therefore not so obvious. BPMN models can be translated to Petri Nets and then verified against the life cycles in the domain model. Previous research has already verified the feasibility of such approach [18]. However, the issue of how to translate the errors back to the original BPMN models is not yet solved, which is an absolute requirement for the practical use of such verification algorithm.

10.4 Meta-model

Figure 10.11 represents the meta-model further expanded with the most important concepts of the business process layer that have a link with information system service layer concepts. Notice how the three layers can easily be distinguished in the meta-model.

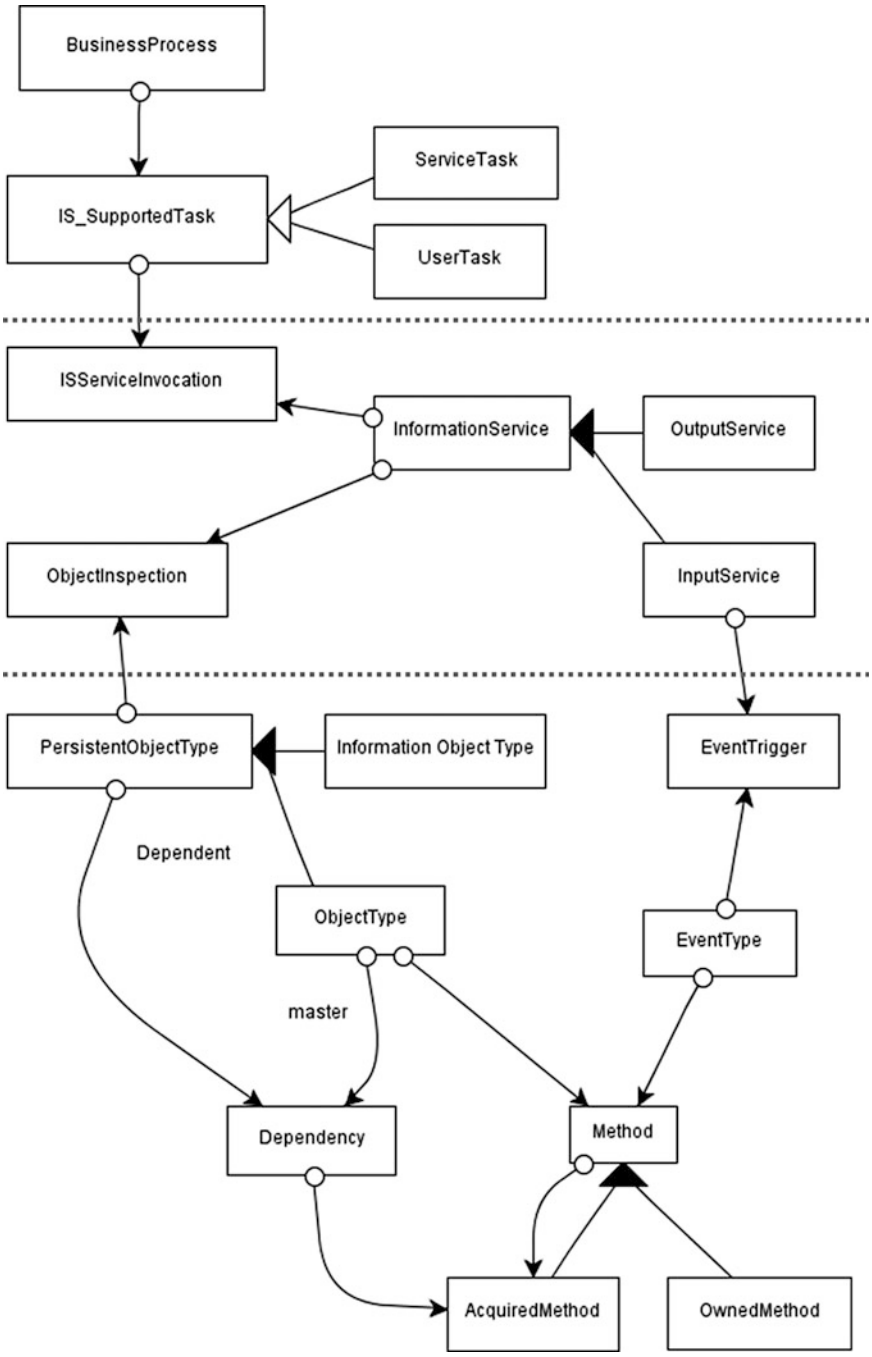


Fig. 10.11 Meta-model for the domain model, information system services and business process models

Part IV
Model Transformation

Chapter 11
Model Transformation

11.1 Model-Driven Development

In a classic approach to software development, the requirements and analysis stages are a largely informal and iterative process. At a certain point in time, specifications are considered to be (more or less) finalised and they are handed over to the implementation team, which proceeds with the technical design, coding and testing. Also the sequence of technical design, coding and testing is actually an iterative process. If problems are discovered during coding and/or testing, one will return to the previous stage to take corrective actions. It is, however, rare that the analysis documents are updated accordingly. Surely, discovered errors may lead to some extra requirements elicitation and analysis activities during the implementation phase, but the results of these will be immediately incorporated into the code rather than documenting them first via updated models and requirements documentation. Figure 11.1 represents this way of working.

In model-driven development, models are the prime artefact based on which software is developed. Requirements analysis should result in formal 'platform independent models' (PIMs), possibly creating first 'computational independent models' (CIMs) as preliminary artefacts. These PIMs are then transformed into 'platform specific models' (PSMs) by means of model-to-model (M2M) transformations. These PSMs can at their turn be transformed into code by means of model-to-code (M2C) transformations. The basic idea behind this transformational approach (represented in Fig. 11.2) is that when testing fails to produce the desired results, rather than going back to the coding phase, it is the PIM that is updated accordingly, and the PSMs and code are regenerated.

Hence, model-driven engineering focuses on (1) designing platform-independent ***models*** as the main representation of a system-to-be and having a sufficient level of completeness to generate other models or code from them and (2) on ***transformation(s)*** (mappings) from platform-independent to platform-specific models or code, a process that may pass through a number of mappings

© Springer International Publishing Switzerland 2014

M. Snoeck, *Enterprise Information Systems Engineering*, The Enterprise Engineering Series, DOI 10.1007/978-3-319-10145-3_11

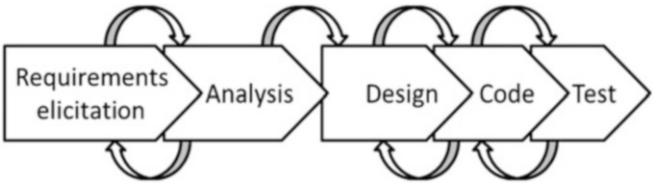


Fig. 11.1 Classical software development cycle

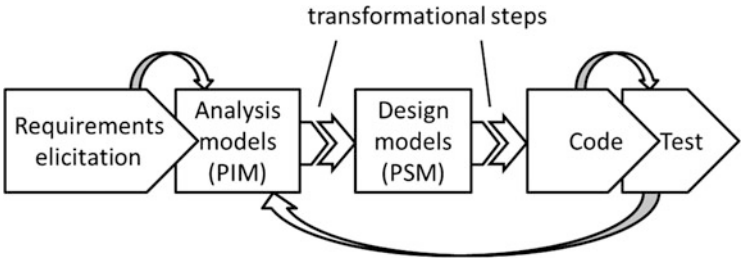


Fig. 11.2 Model-driven software development cycle

before a software artefact can be generated. Although model-driven engineering can be achieved in a manual way as well, today, the practical realisation of a model-driven approach to software development benefits from a variety of tools and technologies. The OMG offers the MDA as a set of standards to realise this MDE approach. The key standards include (a.o.) the UML, Meta-Object Facility (MOF), XML Metadata Interchange (XMI) and Object Constraint Language (OCL). While UML is the language used to write down models describing a system's design, MOF and XMI are used to store, transport and exchange models between tools. The main purpose of XMI is to enable the interchange of metadata between tools in heterogeneous environments.

A full explanation of MDE and MDA technology is beyond the scope of this book (see, e.g. [41] for an introduction). We therefore focus on the main components: models and transformations. First, models need to be stored as machine-readable artefacts. Meta-models and meta-meta-models are the prime artefacts underlying this aspect of MDE. Second, the transformation process needs to be supported by transformation engines and formalisms to express transformations. The transformation engine will read the machine-readable models, extract the relevant information and execute the transformation. This requires a 'transformation definition' that specifies how the information contained in a model needs to be transformed (see Fig. 11.3). Such transformation definitions usually consist of a set of transformation rules grouped together into a transformation definition. The transformation rules will typically be determined by platform choices and non-functional requirements. For example, the desire for open software will lead to a different platform choice and subsequent transformation rules than a choice dictated by the desire to stick to a specific software vendor. Non-functional

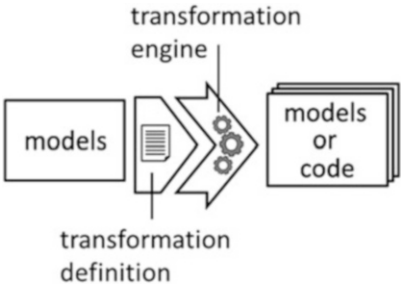


Fig. 11.3 Model transformation

requirements about security, speed, confidentiality, etc. will typically be different per domain. For example, for a stock exchange site the speed in terms of millisecond can matter while for another domain such high performance solutions can be unnecessarily costly. All these issues are taken care of when designing the transformations that are to be applied to the models, and different non-functional requirements may dictate different transformation rules. In this way, the model-driven approach fosters a separation of functional requirements (captured through models) and non-functional requirements (captured in the transformation rules).

The remainder of this chapter explains by means of general principles and a number of simple examples how MERODE domain models can be transformed to working applications. Since MDE is not the core subject of this book, intermediate model-to-model mappings are skipped, limiting the explanation to the demonstration of how models can be directly transformed into code. The next section explains the basic transformation rules used to transform a MERODE model into a prototype application and illustrates them by means of the resulting code. Section 11.3 briefly explains the technology chosen for the code generation and its underlying mechanism, such as the transformation rule formalism.

11.2 Transformation Rule Examples

11.2.1 General Implementation Architecture

The general transformation architecture used for prototype generation follows the layered principle as outlined in Sect. 2.1. As shown in Fig. 11.4, the existence-dependency graph and the finite-state machines are used to generate the enterprise layer responsible of persistence and the business logic, while the object-event table is used to generate the event handling layer. As information system services are documented by informal techniques such as use case diagrams and textual explanations, these models are not fit for transformation purposes. Except for ‘standard’ input and output functionalities, the code for these layers will therefore need to be manually produced. Process models on the other hand can, if specified in a formal

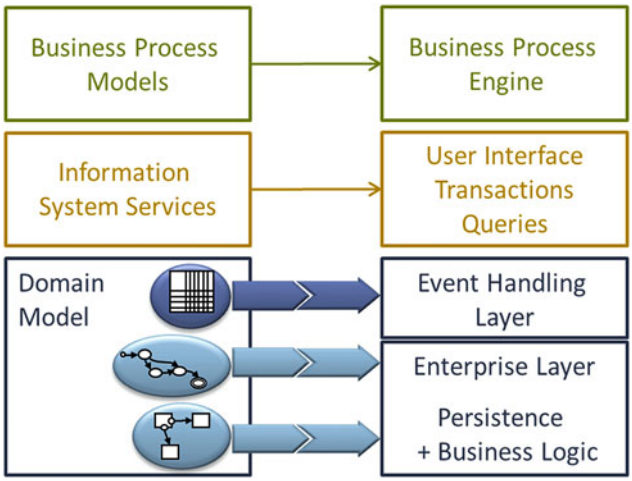


Fig. 11.4 General implementation architecture

language such as BPMN or Petri Nets, also be used for code generation purposes. The creation of executable models and their use for business process enactment falls beyond the scope of this book. We refer the reader to, e.g. [22], for a more in-depth discussion of business process modelling fundamentals and the use of process models for process enactment.

11.2.2 The Enterprise Layer

This layer forms the basis of the target architecture. All other layers in the prototype's architecture will make use of the services provided by the enterprise layer. The main sources for the MDE generation of this layer are the PIM elements of the class diagram and the finite-state machines. However, also the columns of the object-event table are taken into account as these define the list of methods to be included in each class, and each cell also defines the preconditions that apply to a method.

11.2.2.1 The Database Schema

To cater for persistence, the EDG may be transformed to a database schema. When doing so, in principle each enterprise object type and each information object type will be transformed to one database relation. In order to cater for a proper implementation, associations will typically be converted to additional attributes referring to the ID of the corresponding master object type. For example, as a loan refers to a member and a copy, the class definition for LOAN can have an attribute 'borrowing_member' of type MEMBER-ID and an attribute 'borrowed_copy' of type

COPY-ID. This amounts to the use of foreign keys to implement the reference from the dependent to the master.

When formulating constraints in Chap. 7, it was assumed that associations were navigable in both ways using the role names. Such a two-way implementation of associations is, however, not directly feasible in relational databases for associations with a cardinality of many. A cardinality of many means that from the master one navigates to a *collection* of dependents. This amounts to a set-valued reference attribute, which does not comply with the rules of the first normal form. In such a case, the associations can only be implemented in one direction, as a reference from the existence-dependent object to the master object. Finding the dependents of a master object will then require querying the database. In addition, the implementation of constraints formulated by using the set operators *Forall(), Count(), Sum()*, etc. will need to take account for the fact that navigation from a master object to its dependents must be done by querying the database.

Another point of attention when converting the EDG to a database schema is managing the cardinalities of one at the side of the dependent. In Chap. 4 it was explained that the cardinalities in the EDG reflect reality at one point in time. As a result, in the given example, a COPY is associated with at most one LOAN at one point in time. The EDG itself is, however, not the database schema. Indeed, if the EDG is transformed into a logical database schema using the same cardinalities, the database management system will enforce a one-to-one association between LOAN and COPY. This means that each loan object must be *physically* removed when it reaches its final state to allow a new loan for the same copy. A much more flexible database design is to transform such a one-to-one association to a one-to-many association. In this way, physical removal can be postponed and LOAN objects can be used for archival and data analysis purposes. Since putting an object in a final state does not necessarily demand the physical deletion of an object, allowing several LOAN objects per COPY object in the database naturally allows keeping the history of loans. It is then up to the clean-up and back-up policies to determine how long this history must be kept. In addition, if later on the cardinalities of the EDG are modified, it is more likely that the database schema can remain the same. For example, if in the enterprise model for the library we introduce the notion of fines, copies can have many loans associated with them: as long as the fine is not paid, the loan does not reach a final state. However, once a copy is returned, it can be borrowed again. Hence, a copy can have several ongoing loans: one for each fine that is not yet paid and possibly one more for the copy which is on loan. If the database schema already allows a one-to-many association between copies and loans, there is no need to adapt the database schema because of the introduction of fines.

On the other hand, if no history is required, dependents with a cardinality of one might be considered to be merged with the master. Consider, for example, the assignment of employees to departments. Each EMPLOYEE has exactly one (existence-dependent) ASSIGNMENT. If no history of assignments is needed, the ASSIGNMENT object type can be embedded in the EMPLOYEE class as a foreign key to DEPARTMENT. Attributes of ASSIGNMENT can also be merged into the EMPLOYEE class.

11.2.2.2 The Business Logic

Object Types, Attributes and Associations

Next to generating a database schema, the domain object types are also used to generate code that contains the business logic. As an example, for the generation of the MERODE prototype applications, the following transformation rules have been defined (see [60] for a more extended explanation). Each object type in the EDG is transformed into:

Rule 1: An *abstract class* and *implementation class*. By making a distinction between an abstract and implementation class, it is easier to see the difference between generated PSM elements and PSM elements added manually after the PIM-to-PSM transformation took place. This leads to a better traceability [41]. Furthermore we use the object-relational mapping (ORM) technique of Hibernate to make this type of objects persistent.

Rule 2: A *factory* which makes it easier to create and collect objects of the same type [29].

Rule 3: Each attribute of an object type is transformed into an attribute of the mapped class, and for each attribute a getter and setter is declared.

Rule 4: One *method* (in the mapped abstract class) for each event which the object type participates in and one *method* for checking the corresponding preconditions. The (columns of the) object-event table contains the information needed for this transformation rule.

Rule 5: In order to cater for a proper implementation, associations are converted to additional attributes having a business object type as their data type. In this case associations are implemented in both directions and therefore each 'many' association end in our class diagram is transformed into an association attribute in the mapped classes with a getter and setter. For example, as a loan refers to a member, the class definition for LOAN will have an attribute 'member' of type MEMBER. Reversely, the class definition of MEMBER will have an attribute 'loan' of type COLLECTION (of LOANS).

In the remainder of this section, the transformation rules are illustrated by means of the resulting code snippets. Readers that would like to see the full code can simply generate an example. After generating a prototype, the source code can be found in the map ..\merode_application\src\.

Example 11.1: Code Examples for Class, Attribute and Associations for the Library Example

For the library example, the code generation for the object type copy results in the following list of Java classes:

- Copy.java (as a result of rule 1)
- CopyImpl.java (as a result of rule 1)
- CopyFactory.java (as a result of rule 2)
- CopyState.java (as a result of rule 6, see further)

– CopyStateAllocated (as a result of rule 6, see further)
– CopyStateAvailable (as a result of rule 6, see further)
– CopyStateExists (as a result of rule 6, see further)
– CopyStateEnded (as a result of rule 6, see further)

Generating the code for an attribute 'location number' of type String in the class COPY results in the following Java code:

```
//---------------- attributes ----------------
private java.lang.String locationnumber;
public java.lang.String getLocationnumber(){
   return this.locationnumber;
}
public void setLocationnumber(java.lang.String locationnumber){
   this.locationnumber = locationnumber;
}

The association to the master title results in the following code in
Copy.java
//---------------- relations ----------------
public dao.Title getTitle() {
   return this.title;
}
public void setTitle(dao.Title title){
   this.title = title;
}
   private dao.Title title;
```

One may have noticed that the generative approach may compromise the readability of the code, which is one of the shortcomings of this approach. Of course a judicious definition of patterns may address this issue to a certain extent, but it will not always be possible to define generic patterns that yield perfectly understandable code.

11.2.2.3 Sequence Constraints

Sequence constraints can be implemented in different ways. One way is to express them as preconditions. This requires transforming the finite-state machines to state transition tables as explained in 7.2.4. From these state transition tables, one can derive a precondition that uses a 'state' attribute for the object and specifies the required value of this state indicator before acceptance of an event. The state transition table also specifies the new value of the state indicators after occurrence of the event.

A second way to implement sequence constraints is to make use of the state pattern [29]. This is the way sequence constraints are implemented for the prototype

code generation in JMermaid. In this case all roles need to be combined into a single finite-state machine that is used for code generation. For this kind of transformation, the finite-state machine is transformed into (see [60]):

Rule 1: *An abstract state class* (associated with the mapped class) with *state subclasses* for each state in the FSM.

Rule 2: *Methods in the state subclasses* (1) for checking state conditions for each event which the object type participates in and (2) to execute the method. For each method that is not enabled in a particular state, an exception will be thrown.

Example 11.2: Code Example for Transforming Finite-State Machines for the Library Example

The FSM of COPY defines that from the state 'exists' the event *sell* causes a transition to the state 'ended' and that the event 'classify' causes a transition to the state 'classified'. This results in a Java class 'CopyStateExists' with the following code for these two events:

```
public void me_sell (org.hibernate.Session sess, Copy object)
throws org.hibernate.HibernateException {
   CopyStateEnded state = CopyStateEnded.getObject(sess);
   object.setState(state);
}
public void me_classify (org.hibernate.Session sess, Copy object)
throws org.hibernate.HibernateException {
   CopyStateAvailable state = CopyStateAvailable.getObject(sess);
      object.setState(state);
}
```

11.2.2.4 Invariants and Preconditions

In Sect. 7.2 an OCL-like language was used to formulate constraints. In case no formal language is used, constraints will need to be manually incorporated in the code. The corresponding code can be injected in the check_event method of each class or directly in the event handler (see further). In case OCL would be used, then transformation rules for transforming OCL expressions to code can be defined.

Example 11.3: Code Snippets for Preconditions

Examples of (incomplete) code snippets that could be injected in the code are as follows.

The precondition of registering the return date for a loan to be after the day of registration can be programmed as follows in Java:

```
//checking a precondition for a returnDate to be later than today
if (input_returnDate.compareTo(today) > 0){
   //precondition -> return date is after today: satisfied...
   this.returnDate = input_returnDate;
} else {
   //precondition is violated, consequence is programmed here...
```

```
    throw new RuntimeException("return date is not valid.");
}
```

The next code snippet is an example of implementing the check of an attribute precondition constraint. The piece of code will check a name attribute for being not empty, with a minimum and maximum length of characters to be 2 and 20, respectively.

```
/**
* Precondition
* @param name is expected to be not null,
* min length: 2, max length: 20
*/
if (name == null || "".equals(name)) {
   //name is either not provided or an input is an empty string
      throw new RuntimeException("Precondition: 'NAME NOT EMPTY'
violated.");
} else if (name.length() < 2){
   throw new RuntimeException("Precondition: 'NAME MIN SIZE'
violated.");
} else if (name.length() > 20){
   throw new RuntimeException("Precondition: 'NAME MAX SIZE'
violated.");
} else {
   //preconditions are satisfied
}
```

The precondition check of maximum of 5 allowed loans per member could be formulated in Java as follows:

```
/**
* Precondition.
* @param loans should contain maximum 4 loan objects
*/
if (loans.size() <= 5) {
   //precondition is satisfied, member can have another loan
} else {
   throw new RuntimeException("Member cannot borrow anymore...");
}
```

The deadline precondition check for refund method presented in Example 7.6 could be formulated in Java as follows:

```
/**
* Precondition
* @param refundDeadline should be greater than today
```

```
*/
//declaring an attribute of type Date: by default it is the current day
   Date today = new Date();
if(refundDeadline.compareTo(today) > 0){
   //deadline is not over: proceed further to refund
   ...
} else {
   //the precondition is violated, consequence is programmed here...
   throw new RuntimeException("Refund deadline is over.");
}
```

11.2.2.5 Cardinality and Referential Integrity Constraints

To ensure referential integrity a restricted approach to object ending is used. This requires that master objects cannot be ended as long as there are other dependent objects referring to the master (the life cycle of which is not ended either). This leads to the generation of a check for living dependent objects in each ending method of a class.

A cardinality of maximum one also requires the generation of extra constraints. In particular, the creating event of a dependent of which there should be maximum one should be equipped with a precondition to verify the non-existence of a dependent.

Example 11.4: Cardinality and Referential Integrity Verification for COPY

As COPY is a master of LOAN, a copy cannot be ended if there are still ongoing loans for that copy. This leads to the following code for the check_sell method in CopyImpl.java:

```
public void check_me_sell() throws MerodeException {
   if(hasLivingDependents()){
      throw new MerodeException ("Precondition violation in ending
      method \"me_sell()\" in class Copy:\nObject has living depen-
      dents" + livingDependents);
   }
}
```

As a copy can have at most one loan, this results in a precondition on the *borrow* event. The generated code for checking the preconditions of *borrow* results in the following code in CopyImpl.java

```
public void check_me_borrow() throws MerodeException {
   if ((getLoan() != null) && (!getLoan().getState().isFinalState()))
      throw new MerodeException ("Precondition violation in creating
      method \"ME_borrow\" in class Copy:\nObject already has a living
      dependent Loan");
}
```

11.2.3 The Event Handling Layer

The event handling layer sits in between the enterprise layer and the information system services. It will accept requests to trigger an event from the input information system services and coordinate the handling of these event requests. The main source to design the event handling layer is the object-event table. Each row in the OET defines an event for which a handler needs to be defined. Furthermore, the row defines the participating classes, and for each class, the preconditions that need to be checked and the methods that are to be invoked.

The main tasks of an event handler are to ensure that the 'request' is handled by having the preconditions imposed by all participants checked and by having the effect operationalised on each participating object. The precondition check can be performed by the event handler itself; it can be delegated to some rule engine or to the participating classes. Also a combination of these three ways of checking preconditions can be adopted. If all preconditions are satisfied, then the event needs to be handled by all participants, requiring some mechanism to notify participants of the necessity of handling the event. Finally the atomic character of the event requires that the whole procedure is well coordinated such as to ensure that either all participants handle the event or it is not handled by any of the participants. This coordination can be achieved by either checking the preconditions before notifying the participants or by directly notifying the participants without preliminary check, but providing for some rollback mechanism if the event is not successfully handled by one of the participants. Finally event handling can be either centralised in one component handling all events or distributed across several event handlers.

The easiest way to transform the object-event table to an event handler PSM and to code is to use a standard collaboration pattern that implements the broadcasting and handling of an event. Obviously, the different choices mentioned in the previous paragraph will lead to different design patterns for event handling PSMs. As an example, the generation of the MERODE prototype applications follows a centralised approach: one event handler class is generated which bundles all needed event handler methods. In the JMermaid implementation a decentralised approach was followed: the use of the command pattern for meta-model events amounts to the generation of a PSM with one class per event type.

Below we summarise the pattern used for the prototype generation. The event handler in a MERODE prototype application contains one 'HandleEvent' method per business event. This method consists (roughly speaking) of four steps that need to be performed for each participating object. Figure 11.5 visualises the interaction for an event that is broadcasted to a single participating object (see [60] for more details).

Step 1: The event handler 'asks' a participating object (an object which is involved in the processing of a business event) whether all preconditions set by the object are met. These preconditions can be defined by the analyst but also include conditions that can be derived from other parts of the model. For example,

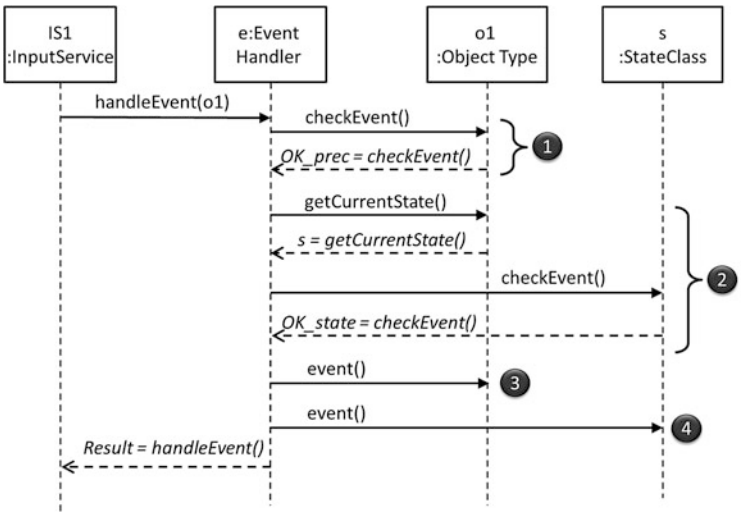


Fig. 11.5 Sample interaction pattern for handling an event

associations between classes will lead to preconditions to maintain referential integrity.

Step 2: Similarly to the previous step, the event handler retrieves from the participating object its current state (or reference to the state object) and checks whether that state allows further processing of the event.

Step 3: If the results of step 1 and 2 are positive (this means 'no exceptions are thrown'), the event handler invokes the method in the participating object which correspond with the triggered event.

Step 4: Next, the event handler executes in the state object retrieved in step 2 the method for modifying the state (according to the triggered event).

11.3 Transformation Technology Example

11.3.1 The mxp File

The starting point for a transformation is the machine-readable form of the model. In the case of JMermaid, the model information is stored in XML format in the mxp file.[1] The structure of this XML file is in line with the meta-model outlined in the previous chapters. As explained in Sect. 9.4.1, next to the meta-model objects, information objects are used to store graphical information. As a result, the mxp file

[1] The mxp is actually a ZIP archive. In order to retrieve the XML file, one needs to unzip the mxp file first.

contains a node ‘meta-model’ with all information relating to the meta-model objects and a ‘guimodel’ node with all the graphical information. The highest level of the mxp file is therefore as follows:

```
<?xml version="1.0" encoding="UTF-8"?>
<mxp:mermaidmodel xmlns:mxp="http://merode.econ.kuleuven.ac.be/
mermaid/mxp/1.5" mxp.version="1.5" jmermaid.version="1.0"
timestamp="1400253727535">
  <mxp:metamodel lastid="13">
  .....
  </mxp:metamodel>
  <mxp:guimodel currentview="1" topview.lastid="1">
  ....
  </mxp:guimodel>
</mxp:mermaidmodel>
```

If we create a model containing only a business object type named ‘a’, with two attributes ‘name’ and ‘nr’ of type String, then the meta-model node contains the following information:

```
<mxp:metamodel lastid="13">
  <mxp:datatypes/>
  <mxp:metaobjects>
    <mxp:metaobject id="1" name="a" abstract="false">
      <mxp:metadescription/>
      <mxp:metaconstraints/>
      <mxp:metamultiplepropagationconstraints/>
      <mxp:metaattributes>
        <mxp:metaattribute id="12" name="name" type="String"
        type-id="1">
          <mxp:description/>
          <mxp:validation/>
          <mxp:implementation/>
        </mxp:metaattribute>
        <mxp:metaattribute id="13" name="nr" type="String" type-
        id="1">
          <mxp:description/>
          <mxp:validation/>
          <mxp:implementation/>
        </mxp:metaattribute>
      </mxp:metaattributes>
      <mxp:metafsms>
        ... (lines defining the default fsm deleted here)
      </mxp:metafsms>
    </mxp:metaobject>
  </mxp:metaobjects>
```

```
  <mxp:metaevents>
    <mxp:metaevent id="2" name="cr_a">
      <mxp:metadescription/>
      <mxp:metaattributes/>
    </mxp:metaevent>
    <mxp:metaevent id="3" name="end_a">
      <mxp:metadescription/>
      <mxp:metaattributes/>
    </mxp:metaevent>
  </mxp:metaevents>
  <mxp:metaspecialisedevents/>
  <mxp:metadependencies/>
  <mxp:metainheritances/>
  <mxp:metamethods>
    <mxp:metamethod id="4" name="cr_a" provenance="OWNED"
        type="CREATE" ownerobjectid="1" ownereventid="2">
      <mxp:metaprecondition/>
      <mxp:metaimplementation/>
      <mxp:metapostcondition/>
      <mxp:metaattributes/>
      </mxp:metamethod>
    <mxp:metamethod id="5" name="end_a" provenance="OWNED"
        type="END" ownerobjectid="1" ownereventid="3">
      <mxp:metaprecondition/>
      <mxp:metaimplementation/>
      <mxp:metapostcondition/>
      <mxp:metaattributes/>
    </mxp:metamethod>
  </mxp:metamethods>
</mxp:metamodel>
```

11.3.2 Template-Based Approach to Transformation

The JMermaid code generator uses the Velocity Template Engine[2] as transformation technology to generate the code. A template writer will write generic code using markup statements called *references* in those places where model elements should be filled. These references are pulled from a *Context* object (collection of objects that hold the information on chosen properties from the models to be used for mapping, e.g. object name, attributes, dependencies, states, transitions, etc.),

[2] http://velocity.apache.org/

and the corresponding values are inserted in a generated file as a result of merging a context with a template.

The MERODE code generator first uses an XML parser to extract data from the mxp file and populates a set of Java meta-object classes. It then distributes the properties collected from XML into the Velocity Template contexts. It is the template engine's responsibility then to merge each context with a specified template to generate a set of files, e.g. a database script, software application code (event handlers, data access objects to communicate with a database, user interface code or configuration files). Finally, the compiler module transforms the bunch of files into a compiled executable application.

The Velocity Template contexts and templates act as mapping between the model and the prototype code. Templates are formulated in a specific template language, which use references (starting with a $-sign) to refer to objects in the generated application code.

Below is a simplified example of such template code for mapping a business object type defined in the MERODE model into a Java class, including the object type's properties such as attributes and dependencies (with control statements in green and starting with a '#' and references in red and starting with a '$'):

```
public abstract class $object.objName {
   //---------------- attributes ----------------
#foreach ($attribute in $object.attributes.entrySet())
        private $attribute.attributeType $attribute.attributeName;
#end
   //---------------- dependencies --------------
#foreach ($master in $object.masters.entrySet())
       private $master.masterName $master.masterName.toLowerCase();
#end
   ...
}
```

Transformation of the template using the data of the business object type LOAN of the library model (assuming attributes loanDate and returnDate) will result in the following code:

```
public abstract class Loan {
    //---------------- attributes ----------------
    private int loanID;
    private Date loanDate;
    private Date returnDate;
    //---------------- dependencies --------------
    private Member member;
    private Copy copy;
   ...
}
```

A short extract from one of the templates that generates the piece of code to check if referential integrity constraints are satisfied (see Example 11.4) is as follows:

```
#foreach ($method in $object.ownedMethods)
    ....
    #if ($method.isEndingMethod)
        if(hasLivingDependents())
        throw new MerodeException ("Precondition violation in ending
        method \"${method.methodName}\" in class ${object.
        objectName}:\nObject has living dependents");
    ....
#end
```

When applied to the above example of class 'A' and its method '*end_a*', this results in the green (bold) lines of code below in the class AImpl:

```
public class AImpl
   extends A
{
   ...
   public void check_end_a() throws MerodeException {
      if(hasLivingDependents())
      throw new MerodeException ("Precondition violation in ending
      method \"end_a\" in class A:\nObject has living dependents");
   }
   ....
}
```

Further adjustments of the templates might be needed based on preferences about architectural choices, technologies or programming languages' syntax or naming conventions. For example the naming conventions in the Java language require that a class name always starts with a capital letter which is different from how variables are named. Another example requiring adjustment is wording used in a template which conflicts with certain key words of a programming language resulting in compilation errors.

Chapter 12
Application and Component Integration

12.1 Business Event-Based Coordination

Although the MERODE approach was initially developed for the development of new intranet-based applications, its concepts also suit the problem of application or component integration very well [45, 46]. Essentially, the event layer leads to an event-driven target architecture. The principles of such architecture can easily be applied to achieve component[1] integration. Components are typically coarser grained than individual business objects and are usually conceived as a black box to the outside world: they may contain multiple business objects but do not publish individual interfaces to each of these individual business objects. Therefore, in a component integration context, event-based interaction will occur at the level of components rather than at the level of individual business objects.

Two major types of architectures for application or component integration can be conceived. In the first case, off-the-shelf components are positioned as information system services on top of an (newly developed) enterprise layer. The second type of architecture consists of distributed components, without unifying enterprise layer.

In the case of *intra-enterprise* application integration, an enterprise layer can be used as an integration layer for off-the-shelf software. In many cases, off-the-shelf software will manage information about business objects, and different off-the-shelf applications may manage information about different but also about the same business objects. Figure 12.1 illustrates this with an example taken from the

This chapter is a reworked and shortened version of the following journal paper 'Snoeck M, Lemahieu W, Goethals F, Dedene G, Vandenbulcke J, Events as atomic contracts for application integration, *Data & Knowledge Engineering*, 51(1), 81–107. © 2003 Elsevier'. The material from this paper has been integrated in this book with the kind permission of Elsevier.

[1] We will use the generic term 'component'. Depending on the actual environment, this component may also be a genuine application or a web service.

© Springer International Publishing Switzerland 2014

M. Snoeck, *Enterprise Information Systems Engineering*, The Enterprise Engineering Series, DOI 10.1007/978-3-319-10145-3_12

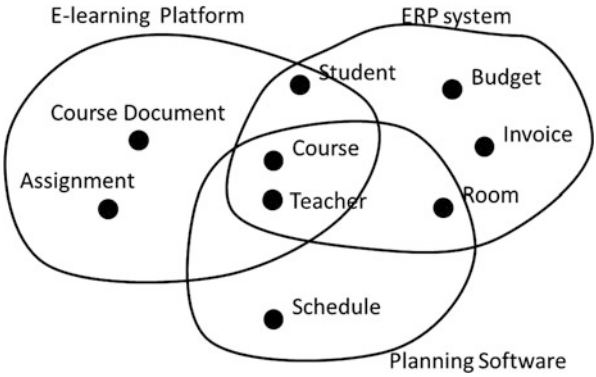


Fig. 12.1 Information sharing between off-the-shelf software

university context. Three software packages are being used: an ERP system for the support of administrative processes, a planning software for creating course schedules and an e-learning platform to support the teaching processes. The e-learning platform and the ERP system need to manage and share information about students, courses and teachers. Course documents (e.g. slides) and course assignments are proprietary information of the e-learning platform. Budgets, invoices and payments (e.g. of tuition fees) are proprietary information of the ERP system. The ERP system will have to share information on rooms, teachers and courses with the planning software to enable the scheduling of courses. The schedule is, however, only managed by the planning software. Such situation is very common and holds the danger of information replication, inconsistency and inefficiency. The integration of these applications can benefit from the creation of an enterprise model that defines business object types in a uniform company-wide and application-independent way and will also benefit from an implemented independent enterprise layer that holds the 'unique' and correct information about business objects. Having an independent enterprise layer that contains the most important business objects will also increase the organisation's independence of software vendors.

In order to solve the problem of overlapping information, the enterprise layer should contain the business objects that are shared across applications. And in order to increase the independence from the software vendor, the enterprise layer should in addition contain all information that is vital to the organisation and that the organisation wishes to retain if the off-the-shelf software is dismissed.

To coordinate between the off-the-shelf software and the enterprise layer, the acquired software is positioned on top of the event handling and enterprise layer as a component that consists of both input and output services. In this way, the information system layer consists of independent off-the-shelf software and possibly self-developed information services (see Fig. 12.2). In the ideal situation, the off-the-shelf software doesn't manage its own copy of business objects but is able to retrieve this information from the enterprise layer, as all applications can inspect the state of business objects in the same way as regular output services (see arrows labelled 1). Applications that need to keep a local copy of business objects will have to synchronise with the enterprise layer making use of event handlers. The event

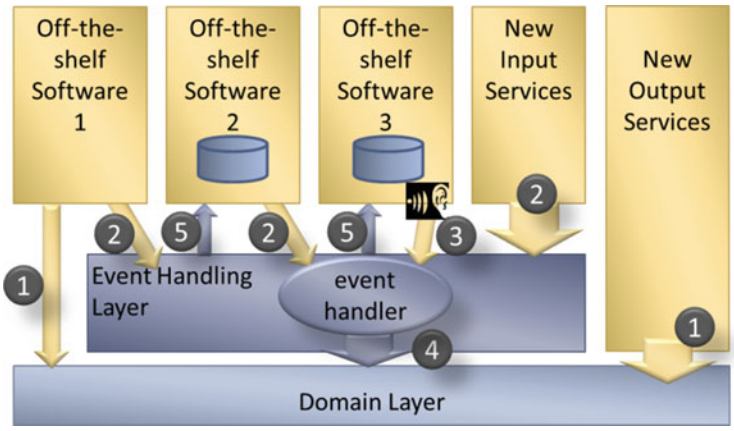


Fig. 12.2 Using event handlers as coordinating agents

handlers take the role of coordination agents for the update of shared information. Each off-the-shelf software uses the services of the event handlers to trigger business events whenever it wishes to update business objects in the enterprise layer, both in the case the application relies on the external enterprise layer and in the case the application maintains its own local copy. It does this in the same way as a regular input service (see arrows labelled 2). In case the interface of the off-the-shelf software doesn't allow to trigger business events, the event handler could be equipped with a listening function to detect the occurrence of a business event in the application (represented as arrow labelled 3). The event handler notifies business events to the business objects residing in the enterprise layer as described previously in Sect. 11.2.3 (arrow labelled 4). When an off-the-shelf software wants to be notified of updates in the enterprise layer, it can subscribe to the corresponding event handler in order to be notified upon occurrence of the event (arrows labelled 5). In this way, the off-the-shelf software can keep its own information synchronised with the information contained in the enterprise layer.

In a distributed, more loosely coupled environment, the enterprise layer may itself be distributed among multiple components, applications or web services, possibly developed and controlled by different authorities. This may, for example, be the case when different organisations decide to collaborate. Such inter-organisational collaboration may require the integration of existing, independently developed components. In other situations, the respective partners in an extended enterprise develop their components from a 'unified' enterprise model. This may hold the danger of conflicting business rules, but as will be explained in Sect. 12.4, business event-based coordination facilitates the discovery and management of such event sequence conflicts.

In both situations of intra-enterprise integration and distributed, loosely coupled environments, there might be no unique enterprise model nor a unique location for the enterprise layer, but the business objects are distributed across components.

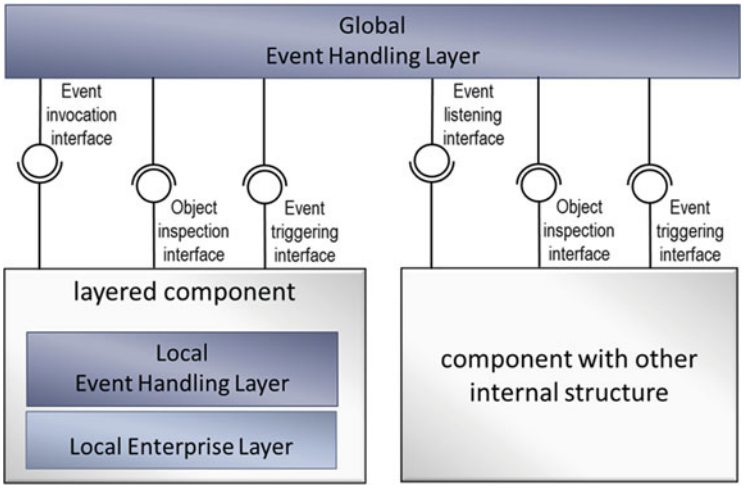


Fig. 12.3 Inter-component and intra-component event dispatching in a loosely coupled distributed environment

Existing components can be integrated as explained above. The design of to-be-developed components may be conceived according to the event paradigm. They then have a similar layered structure consisting of a 'local' enterprise layer comprising several business objects, which incorporate their business logic. The component offers a unified public interface to its (hidden) business objects, which supports two types of interaction: attribute inspection and event notification. This internal layered architecture is not a requirement: any kind of component can be integrated in the event-based architecture as long as the two types of interactions are supported. At the highest level, business events are units of action that span one, two or more components and that provoke some processing in each of them. This general architecture is represented in Fig. 12.3.

12.2 The Component-Event Table

In both cases (intra-enterprise integration and distributed, loosely coupled environments), the modelling of object interaction can be done in a similar way as with the object-event table, except that the columns represent applications which keep a local replicated copy of business objects. The table is used to define which application/web service/component is involved in which type of business event, needs to be notified of these and therefore needs to subscribe to business event notifications. As an example, the table in Fig. 12.4. represents such a partial table for the information sharing example of Fig. 12.1.

In the same way as in the object-event table, the coordination between components is achieved by having components specify preconditions for business events.

	E-learning Platform	ERP system	Planning Software
cr_student	X	X	
end_student	X	X	
cr_course	X	X	X
end_course	X	X	X
cr_teacher	X	X	X
end_teacher	X	X	X

Fig. 12.4 Example of an event-application subscription table

As a result, a business event becomes a small-scale contract between involved components: each component can insert its own clauses into the contract by specifying preconditions. Additionally, postconditions can be formulated to express what results are guaranteed. Obviously, this interaction pattern easily accounts for changes in the number of components that need to be notified of an event: it suffices to equip the event handler with a subscription mechanism. Whenever a component needs to be notified of a particular type of business event, it subscribes with the appropriate event handler to the corresponding event type. When a component subscribes to an event type, it means that the corresponding cell is marked in the component-event table. Similarly as in the OET, three kinds of information are associated with an entry in the component-event table:

- The preconditions specify the conditions that need to be satisfied (from the component's point of view[2]) in order to accept the execution of the event.
- The actions specify the services or methods that must be called in order to notify the component of the occurrence of an event.
- The postconditions specify the guaranteed results.

12.3 Architectures for Event Handlers

Section 11.2.3 outlined the architecture of the event handling layer for simple applications. In the case of component integration, the same architecture can be followed, but the implementation architecture for the component-event table can also be based on the widely used event-based architectures for event generation and notification [9, 16, 56]. The component or service that triggers the event is an event producer; the components with marked participations are event consumers. Existing event-based systems however don't offer coordination capabilities; their architecture is mainly guided by subscription and notification mechanisms. This implies that the event handler should add a coordination capability on top of the publish-subscribe architecture. Just as in newly developed applications, the event

[2] For applications that also adopt the event-based approach for the interaction between their internal components, the application level preconditions to an event can be derived as the combination of the preconditions imposed by the application's internal components that participate in the event.

handler will be responsible for having the event notification broadcasted to all participating components, coordinating the responses (success or failure) and providing the triggering component with feedback about the execution status.

The use of a publish-subscribe architecture allows for a dynamic runtime management of the component-event table. Rather than hard-coding the interaction (as was the case with the architecture proposed in Sect. 11.2.3), components that wish to share information will subscribe as publisher or for notification to the corresponding events by marking the corresponding entry in the component-event table. When a component no longer needs to be notified of a particular type of event, it suffices to remove the subscription by removing the marked cell from the component-event table.

As explained in Sect. 11.2.3, different patterns of precondition verification and coordination can be conceived for event handling. Although the possible implementation architectures for the coordination mechanism are a topic of ongoing research, some significant pros and contras can be formulated for the different options.

The different options for precondition verification and coordination have different strengths and weaknesses with regard to performance and the distribution of components. Locating precondition checking in the event handler leads to the most efficient architecture: precondition checking is centralised and the respective components are only notified of events that do not violate any preconditions. In this way, the risk of a costly global rollback is avoided in case some precondition is not met in one of the participating components. Distributed precondition checking prior to event notification still avoids the necessity of rollbacks,[3] whereas a distributed architecture without preliminary precondition checking will require a rollback if a precondition is not met.

Centralised precondition checking by the event handler will, however, be difficult to realise in a strongly distributed environment. Especially in a situation where components are developed independently, e.g. in a business-to-business web service context, the enforcement of preconditions may to a certain extent be encapsulated and hidden from the outside world. For example, a supplier web service may not want to reveal the implementation of the precondition that checks a customer's credit worthiness before shipping an order. As a consequence, the precondition should be checked locally in the supplier web service, not at the level of the central event handler. The latter should only be notified of whether or not the precondition was violated.

Therefore, in a strongly distributed environment, distributed precondition checking is much easier to realise. In that case, there is still the choice between precondition checking prior to method invocation and event dispatching without prior precondition checking. The former will require a *locking* mechanism to ensure

[3] At least it avoids the necessity of rollbacks caused by failed events. An underlying transaction management mechanism might still use rollbacks to enforce transaction atomicity at the database level.

data consistency in the time interval between the precondition check and the actual processing of the event, which may put a burden on performance. The latter architecture simply tries to resolve precondition violations by inducing a rollback on all participating components *after* they had already processed the event. Whereas a rollback may be a costly operation in terms of performance, the latter may still be the most efficient approach in the case where the number of events for which some precondition is violated is much smaller than the number of 'succeeded' events. Furthermore, in more advanced implementations, the components or the event dispatcher may be equipped with the intelligence to induce the so-called compensating actions if an event partially fails instead of inducing a costly global rollback. For instance, if the precondition *order quantity* $\leq$ *inventory* is violated for a certain order line, the PRODUCT component where the precondition was violated may try to induce a preliminary *refill_stock* event instead of inducing a rollback of the entire transaction.

Regardless of the actual architecture, it is of utter importance that it is not limited to *event notification* but that the *atomicity* of an event is enforced among all participating components. For that purpose a transaction management mechanism is to be used underneath. The transaction management mechanism enforces the all-or-nothing aspect of events. The latter is key to the coordination mechanism: the participating components are not only notified as in traditional event-based systems [9, 16, 26, 42, 56], but each event is indeed an atomic unit of work that is either accepted and processed in all components or rejected and no action is taken in any component at all. All participating components may enforce constraints that together determine the composite system's behaviour.

12.4 Contract Management Capabilities

The event paradigm is in the first place a composition mechanism, which allows for the coordination of the respective components that together form a more complex system. However, as discussed in this section the role of events as *contracts* between the participating components can be an important element in service *discovery* as it allows searching for components or services that are 'compatible' process-wise.

Indeed, in contrast to existing event-based architectures for event observation and notification, the business event-based coordination model allows components specifying constraints on the invocation of events. By considering each row in the component-event table, we have an overview of the components involved in one event and an overall view of all constraints imposed by the participants on this event. By comparing two or more columns, we have an overview of the set of common event types for two (or more) components. As each component can define its own sequence constraints on the common (and own) event types, one has to check whether components agree on the allowed sequences for the common event types. The analysis of these constraints is an essential step in, for example, web

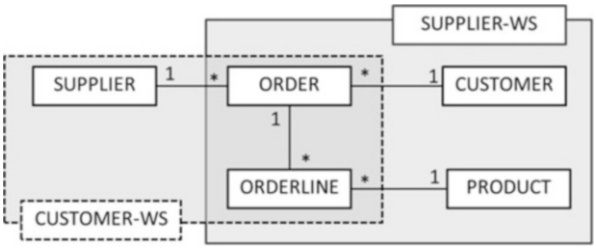


Fig. 12.5 Distributed components for an order handling system

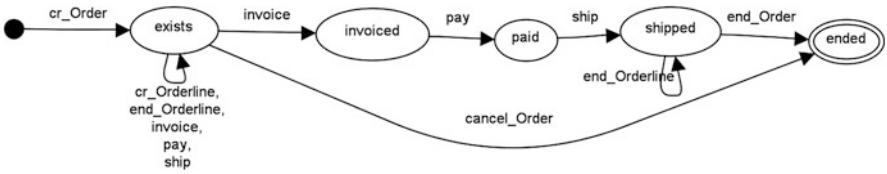


Fig. 12.6 State machine for ORDER from the supplier's point of view

service matching: it is of no use to find a web service that performs the required task if the imposed constraints are in contradiction with constraints imposed by other components involved in the same event.

In a stand-alone or distributed but tightly integrated environment, the sequence constraints of all object types of the domain model are developed in an integrated way, hence reducing the risk of conflicts. In a distributed environment, however, sequence constraints are developed in an independent manner by multiple parties. The danger of conflicting sequence constraints is real. Let us reconsider an ordering example in a web service context (the formal elaboration of this example can be found in [89]).

The **supplier** web service offers the possibility to order products. As shown in Fig. 12.5, the supplier's system keeps track of the products and all orders placed for products. The customer's system keeps track of its own orders. For each order, the customer's system can keep only a stub with a link to the supplier's web service, keep a cache of important attributes of the order or mirror it completely.

Of more importance is the fact that each web service will specify sequence constraints on the events it participates in according to its local business rules. For the sake of simplicity, we assume that both web services use the same names for the events they share[4] and we omit the non-shared events. Let us assume, for example, that the supplier's business rules demand that payment is received before the goods are shipped. The sequence constraints on the order events as specified by the supplier are given in Fig. 12.6. The customer sticks to the sequence constraints as specified in Fig. 12.7.

[4] If this is not the case, a mediation broker could provide a mapping based on ontological descriptions of the events.

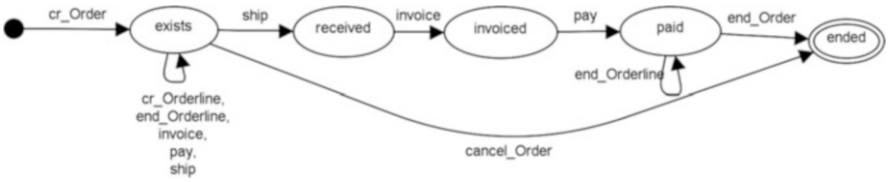


Fig. 12.7 State machine for ORDER from the customer's point of view

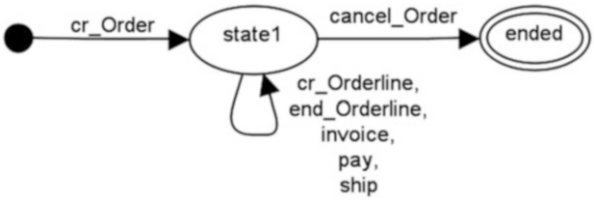


Fig. 12.8 Calculated state machine for the parallel composition of ORDER from the customer's point of view and from the supplier's point for view

Obviously, this customer will not be able to collaborate successfully with this supplier. The parallel composition (global behaviour) of the two finite-state machines reveals that the only scenarios that will be jointly accepted by the supplier's business process and by the customer's process are those that end with *cancel_order*. The parallel composition of both finite-state machines results in Fig. 12.8.

Although the resulting finite-state machine is not empty (which would have indicated a full deadlock), the automaton does not contain all the original business events: the events *ship, invoice* and *pay* are missing in the result. This indicates that shipping, invoicing and payment events will always be rejected by one of the two components because of conflicting sequence constraints. Indeed, the customer will reject the payment event as long as there has been no shipping and invoicing, whereas the supplier will reject the shipping as long as there has been no payment. Obviously, no meaningful collaboration is possible.

The problem of incompatible constraints will be handled differently depending on the system's development context. In a context of fixed partnerships between enterprises, contract analysis can be done at design time. In this kind of situation, conflicting sequence constraints indicate incompatible business rules of the partner companies. The most simple and realistic solution is to resolve the conflicts at the business level by adapting the business rules and the constraints imposed by the components accordingly.

In the context of dynamic web service discovery, incompatible sequence constraints cannot be resolved by modifying the web service, and one will continue to search for another web service or set of web services with compatible sequence constraints.

Finally, in a context of dynamic multiparty interaction, where components can (un)subscribe to an event at any time, an automatic calculation of parallel composition could be used by an event handler to manage the subscription policy. The same way of working can be used to manage system evolution in one of the aforementioned contexts.

12.5 Benefits of Event-Based Coordination

Although the MERODE approach was originally developed to create software from scratch, this chapter has demonstrated that it also allows putting MDE into practice in the context of component-based development. The main benefit of the proposed event-based coordination approach is that it alleviates the maintenance problem caused by one-to-one interaction by proposing a universal communication paradigm that is able to handle notification and coordination when multiple business objects or software components need to interact upon occurrence of an event.

In terms of maintainability, the event dispatching pattern is much easier to adapt than arbitrary message passing patterns. The main advantage is that the dispatching pattern is fully independent from the number of components or objects to which the event must be dispatched. As a result, adding or dropping an object type or a component does not cause any redesign of interaction as opposed to the classical approach [86]. In addition, the dispatching pattern can easily account for distributed components, either tightly or loosely coupled.

Event dispatching can be implemented by means of all known message passing mechanisms, and it can be used both for interaction between peer services and for a complex service to coordinate the behaviour of its components. The clear distinction between attribute inspections and business events allows for focusing on only the latter with respect to *transaction management*. Also, transaction specification is simplified: a single business event results in multiple simultaneous updates in multiple business objects. The latter is much easier to describe than the myriad of one-to-one message exchanges that could make out a single business transaction.

References

1. Agilemanifesto, Manifesto for Agile Software Development, http://agilemanifesto.org
2. Aho AV, Ullman JD (1972) The theory of parsing, translation and compiling. Volume I: Parsing. Series in automatic computation. Prentice Hall, Englewood Cliffs, NJ
3. Ambler S (2005) The agile unified process (AUP). http://www.ambysoft.com/unifiedprocess/agileUP.html
4. Baeten JCM (1986) Procesalgebra. Kluwer programmatuurkunde, Deventer
5. Bernaert M, Poels G, Snoeck M, De Backer M (2013) CHOOSE: towards a metamodel for enterprise architecture in small and medium-sized enterprises. Working papers of Faculty of Economics and Business Administration, 13/856, Ghent University, Belgium
6. Booch G (1994) Object oriented analysis and design with applications, 2nd edn. Benjamin/Cummings, Redwood City, CA
7. Burkhardt R (1995) Modellierung dynamischer aspekte mit dem objekt-prozeß-modell. Dissertation, Universität Ilmenau
8. Cameron J (1985) Course notes, IBM chair. Katholieke Universiteit Leuven, Leuven
9. Carzaniga A, Di Nitto E, Rosenblum DS, Wolf AL (1998) Issues in supporting event-based architectural styles. In: Proceedings of the third international software architecture workshop, ACM Press, New York, November 1998, pp 17–20
10. Chen PP (1977) The entity relationship approach to logical database design. QED information sciences, Wellesley, MA
11. Coad P, Yourdon E (1991) Object-oriented analysis. Prentice Hall, Englewood Cliffs, NJ
12. Coleman D, Arnold P, Bodoff S, Dollin C, Gilchrist H, Hayes F, Jeremaes P (1994) Object-oriented development, The FUSION method. Prentice Hall, Englewood Cliffs, NJ
13. Cook S, Daniels J (1994) Designing object systems: object-oriented modeling with syntropy. Prentice Hall, Englewood Cliffs, NJ
14. Cooke J (1992) Formal methods: mathematics, theory, recipes or what ? Comput J 35(5):419–423
15. de Champeaux D, Lea D, Faure P (1993) Object-oriented system development. Addison Wesley, Boston
16. Cugola G, Di Nitto E, Fuggetta A (2001) The JEDI event-based infrastructure and its application to the development of the OPSS WFMS. IEEE Trans Softw Eng 27(9):827–850
17. Dedene G, Snoeck M (1995) Formal deadlock elimination in an object oriented conceptual schema. Data Knowl Eng 15(1):1–30
18. De Backer M (2007) The use of Petri net theory for business process verification. PhD Dissertation, KU Leuven

© Springer International Publishing Switzerland 2014

M. Snoeck, *Enterprise Information Systems Engineering*, The Enterprise Engineering Series, DOI 10.1007/978-3-319-10145-3

19. De Backer M, Snoeck M, Monsieur G, Lemahieu W, Dedene G (2009) A scenario-based verification technique to assess the compatibility of collaborative business processes. Data Knowl Eng 68(6):531–551
20. Dietz JLG, Hoogervorst JAP et al (2013) The discipline of enterprise engineering. Int J Organ Des Eng 3(1):86–114
21. D'Souza DF, Wills AC (1999) Objects, components and frameworks with UML, the catalysis approach. Addison-Wesley, Boston, p 785
22. Dumas M, La Rosa M, Mendling J, Reijers HA (2013) Fundamentals of business process management. Springer, Berlin, p 399
23. Embley DW, Kurtz BD, Woodfield SN (1992) Object-oriented systems analysis: a model-driven approach. Yourdon Press, Englewood Cliffs, NJ
24. Erl T (2008) SOA design patterns. Prentice Hall/Pearson PTR, London
25. Essink LJB, Erhart WJ (1991) Object modelling and system dynamics in the conceptualization stages of information systems development. In: Van Assche F, Moulin B, Rolland C (eds) Object oriented approach in information systems, Proceedings of the IFIP TC8/WG8.1 working conference on the object oriented approach in information systems, Quebec City, Canada, 28–31 October, 1991, North Holland, pp 89–116
26. Fiege L, Mezini M, Mühl G, Buchmann A (2002) Engineering event-based systems with scopes, 16th european conference on object-oriented programming (ECOOP'02), Lecture Notes in Computer Science, Springer, vol 2374, pp 309–334
27. Fowler M (1997) Analysis patterns, reusable object models. Addison Wesley Longman, Menlo Park, CA, 357 p
28. Fowler M (2002) Patterns of enterprise application architecture. Addison-Wesley
29. Gamma E, Helm R, Johnson R, Vlissides J (1994) Design patterns: elements of reusable object-oriented software. Pearson Education, Upper Saddle River, NJ
30. Goethals F, Snoeck M, Lemahieu W, Vandenbulcke J (2006) Management and architecture click: The FAD(E)E framework. Inf Syst Front 8(2):67–79
31. Hammer M, McLeod D (1981) Database description with SDM: a semantic database model. ACM Trans Database Syst 6(3):351–386
32. Harel D (1998) On visual formalisms. Commun ACM 31(5):514–530
33. Henderson JC, Venkatraman N (1999) Strategic alignment: leveraging information technology for transforming organizations. IBM Syst J 38(2.3):472–484
34. Hens P, Snoeck M, Poels G, De Backer M (2009) The use of the concept of event in enterprise ontologies and requirements engineering literature, FBE research report KBI_0909, 21 p. https://lirias.kuleuven.be/bitstream/123456789/311272/1/KBI_1114.pdf
35. Hoare CAR (1985) Communicating sequential processes, Prentice-Hall International, Series in Computer Science
36. Hopcroft JE, Ullman JD (1969) Formal languages and their relation to automata. Addison Wesley, Reading, MA
37. IBM, The IBM rational unified process, http://www-01.ibm.com/software/rational/rup/
38. Jackson MA (1983) System development. Prentice Hall, Englewood Cliffs, NJ
39. Jackson MA (2000) Problem frames: analysing and structuring software development problems. Addison-Wesley Professional, Boston, 416 p
40. Jacobson I, Christerson M, Jonsson P (1997) Object-oriented software engineering, a use case driven approach. Addison Wesley, Reading, MA, Revised Edition
41. Kleppe AG, Warmer J, Bast W (2003) MDA explained: the model driven architecture: practice and promise. Addison-Wesley Longman, Boston, MA, 170 p
42. Krishnamurthy B, Rosenblum DS (1995) Yeast: a general purpose event-action system. IEEE Trans Softw Eng 21(10):845–857
43. Krogstie J, Jørgensen H (2003) Quality of interactive models. In: Olivé A, Yoshikawa M, Yu E (eds) Advanced conceptual modeling techniques. Springer, Berlin/Heidelberg, pp 351–363
44. Krogstie J, Sindre G, Jørgensen H (2006) Process models representing knowledge for action: a revised quality framework. Eur J Inf Syst 15(1):91–102

45. Lemahieu W, Snoeck M, Michiels C (2003) Integration of third-party applications and web clients by means of an enterprise layer. J Cases Inf Technol 5(1):213–233. doi:10.4018/978-1-59140-061-5.ch014
46. Lemahieu W, Snoeck M, Michiels C (2003) An enterprise layer based approach to application service integration. Emerald 9:21
47. Lindland OI, Sindre G, Solvberg A (1994) Understanding quality in conceptual modeling. IEEE Softw 11(2):42–49
48. Loos P, Allweyer T (1998) Process orientation and object orientation—an approach for integrating UML and event-driven process chains (EPC). Institut für Wirtschaftsinformatik, University of Saarbrücken, Technical report 144
49. Loos P, Fettke P (2001) Towards an integration of business process modeling and object-oriented software development, Chemnitz University of Technology, Chemnitz, Germany, Technical Report
50. Losavio F, Chirinos L, Lévy N, Ramdane-Cherif A (2003) Quality characteristics for software architecture. J Object Technol 2(3):133–150
51. Maes R, Dedene G (1996) Reframing the Zachman information system architecture framework, Tinbergen Institute, discussion paper TI 96-32/2
52. Malone TW, Crowston K (1994) The interdisciplinary study of co-ordination. ACM Comput Surv 26(1):87–119
53. Martin J (1982) Strategic data planning methods. Prentice-Hall, Englewood Cliffs, NJ
54. Martin R, Riehle D, Buschmann F (1997) Pattern languages of program design, vol 3. Addison-Wesley, Reading, MA, pp 395–408. ISBN ISBN: 0-201-31011-2
55. Medina-Mora R, Winograd T, Flores F (1992) The action workflow approach to workflow management technology. Proceedings of the conference on computer supported co-operative work '92, New York, Nov. '92, pp 281–288
56. Meier R, Cahill V (2002) taxonomy of distributed event-based programming systems, Technical report, TCD-CS-2002, Department of Computer Science, Trinity College Dublin, Ireland, (March 2002)
57. Meyer B (1997) Object oriented software construction, 2nd edn. Prentice Hall, Englewood Cliffs, NJ
58. Milner R (1982) A calculus of communicating systems. Springer, New York
59. Mintzberg H (1993) Structure in fives: designing effective organisations. Prentice Hall, Englewood Cliffs, NJ
60. Monsieur G, Snoeck M, Haesen R, Lemahieu W (2006) PIM to PSM transformations for an event driven architecture in an educational tool. In: Proceedings of the European workshop on milestones, models and mappings for model-driven architecture (3M4MDA), European conference, ECMDA-FA, Bilbao, Spain, pp 49–64. http://eprints.eemcs.utwente.nl/3693/
61. Nellborn C (1999) Business and systems development: opportunities for an integrated way of working. In: Nilsson AG, Tolis C, Nellborn C (eds) Perspectives on business modelling, understanding and changing organisations. Springer, Berlin, pp 197–213
62. Nelson HJ, Poels G, Genero M, Piattini M (2012) A conceptual modeling quality framework. Softw Qual J 20:201–228
63. Nilsson AG (1999) The business developer's toolbox: chains and alliances between established methods. In: Nilsson AG, Tolis C, Nellborn C (eds) Perspectives on business modelling, understanding and changing organisations. Springer, Berlin, pp 217–241
64. Nüttgens M, Feld T, Zimmermann V (1998) Business process modeling with EPC and UML: transformation or integration? In: Schader M, Korthaus A (eds) The unified modeling language—technical aspects and applications. Physica-Verlag, Heidelberg, pp 250–261
65. Olivé A (2007) Conceptual modelling of information systems. Springer, Berlin, 455 p
66. Olle TW (1983) Comparative review of information systems design methodologies—stage 1: taking stock. In: Olle TW, Sol HG, Verrijn-Stuart AA (eds) Information systems design methodologies: a comparative review, Proceedings of the IFIP WG8.1 working conference on comparative review of information systems design methodologies, Noorderwijkhout, The Netherlands 10–14 May 1982, North Holland, pp 1–14

67. OMG, The Unified Modelling Language (UML), version 2.5 FTF—beta 1, Retrieved from Object Management Group: http://www.omg.org/uml
68. OMG, Business Process Model and Notation (BPMN), version 2.0.2, Retrieved from Object Management Group: http://www.omg.org/spec/BPMN
69. OMG, Semantics of Business Vocabulary and Business Rules (SBVR), version 1.2, Retrieved from Object Management Group: http://www.omg.org/spec/SBVR
70. OMG, Business Motivation Model (BMM), version 1.2, Retrieved from Object Management Group: http://www.omg.org/spec/BMM
71. OMG, MDA Guide Version 1.0.1. Retrieved from Object Management Group: http://www.omg.org/mda
72. Op't Land M, Proper E, Waage M, Cloo J, Steghuis C (2009) Enterprise architecture, creating value by informed governance. Springer, Berlin
73. Paulk MC, Curtis B, Chrissis MB (1993) Capability maturity model, version 1.1. IEEE Softw 10(4):18–27
74. Pernici B (1990) Objects with roles, proceedings of the conference on office information systems, *SIGOIS Bulletin 11*, 2(3), pp 205–215
75. Philippow I, Burkhardt R, Wolf M (1996) Objektorientierte modellierung mit dem objekt-prozeß-modell. EMISA Forum 6(1):18–22
76. Pohl K (1993) The three dimensions of requirements engineering, in Advanced Information Systems Engineering. Lect Notes Comput Sci 685:275–292
77. Rumbaugh J, Blaha M, Premerlani W, Eddy F, Lorensen W (1991) Object oriented modelling and design. Prentice Hall International, Englewood Cliffs, NJ
78. Schwegmann A, Schlagheck B (1997) Integration der prozeßorientierung in das objektorientierte paradigma: Klassenzuordnungsansatz vs. prozeßklassenansatz, Westfälischen Wilhelms-Universität Münster, Technical report 60
79. Shlaer S, Mellor SJ (1988) Object-oriented systems analysis: modelling the world in data. Prentice Hall, Englewood Cliffs, NJ
80. Shlaer S, Mellor SJ (1992) Object lifecycles: modeling the world in states. Prentice Hall, Englewood Cliffs, NJ
81. Silver B (2011) BPMN method & style, with BPMN implementer's guide, 2nd edn. Cody-Cassidy, Aptos, CA, p 269
82. Snoeck M (1995) On a process algebra approach to the construction and analysis of M.E.R.O.DE.-based conceptual models, PhD Dissertation, Katholieke Universiteit Leuven, Faculty of Science and Department of Computer Science
83. Snoeck M, Dedene G (1996) Generalization specialization and role in object oriented conceptual modeling. Data Knowl Eng 19(2):171–195
84. Snoeck M, Dedene G (1998) Existence Dependency: the key to semantic integrity between structural and behavioral aspects of object types. IEEE Trans Softw Eng 24(24):233–251
85. Snoeck M, Poelmans S, Dedene G (2000) A layered software specification architecture, *Lecture Notes in Computer Science*, vol 1920, Springer, pp 454–469
86. Snoeck M, Poels G (2000) Improving the reuse possibilities of the behavioral aspects of object-oriented domain models. *Lecture Notes in Computer Science*, vol 1920, pp 423–439
87. Snoeck M, Michiels C, Dedene G (2003) Consistency by construction: the case of MERODE. *Lecture Notes in Computer Science*, vol 2814, pp 105–117
88. Snoeck M (2003) Sequence constraints in business modelling and business process modelling. In: Piattini M, Filipe J, Braz J (eds) Enterprise information systems IV. Kluwer Academic, Dordrecht, pp 194–201
89. Snoeck M, Lemahieu W, Goethals F, Dedene G, Vandenbulcke J (2004) Events as atomic contracts for application integration. Data Knowl Eng 51(1):81–107
90. Sowa JF, Zachman JA (1992) Extending and formalizing the framework for information systems architecture. IBM Syst J 31(3):590–616

91. Schwaber K, Sutherland J (2003) The Scrum Guide™, The definitive guide to scrum: the rules of the game. https://www.scrum.org/Portals/0/Documents/Scrum%20Guides/2013/Scrum-Guide.pdf
92. Tesmer O (2005) Integrierte modellierung von geschäftsprozessen und ihrer unterstützung durch software ein methodenvergleich, Master's thesis, Universität Hamburg, Vogt-Kölln-Straße 30, Hamburg
93. The Open Group, TOGAF® Version 9.1, www.opengroup.org/togaf/, Accessed on 15 May 2014
94. The Open Group, The ArchiMate® 2.1 Specification, http://www.opengroup.org/subjectareas/enterprise/archimate, Accessed on 29 May 2014
95. van Hee K, Sidorova N, Somers L, Voorhoeve M (2006) Consistency in model integration. Data Knowl Eng 56(1):4–22
96. van Lamsweerde A (2009) Requirements engineering: from system goals to UML models to software specifications. Wiley
97. Voas J (1997) Can clean pipes produce dirty water? IEEE Softw 15(4):93–95
98. Wand Y, Weber R (1990) An ontological model of an information system. IEEE Trans Softw Eng 16(11):1282–1292
99. Wirfs-Brock R (1990) Designing object-oriented software. Prentice-Hall, Englewood Cliffs, NJ
100. Yu ESK (1993) Modeling organizations for information systems requirements engineering, Proceedings of IEEE International Symposium on Requirements Engineering, pp 34–41
101. Zachman JA (1987) A framework for information systems architecture. IBM Syst J 26 (3):276–292
102. Zave P, Jackson M (1997) Four dark corners of requirements engineering. ACM Trans Softw Eng Methodol 6(1):1–30

Index

A
Accumulation principle, 182
Acquired participations, 116, 125
Adaptability, 6, 18, 32, 34, 49, 224
Aggregation, 79, 99
  composite, 99
  shared, 99
Alias, 123, 161
Alignment, 3, 11, 36, 39, 49
Alphabet rule, 137–138, 146
Analysis phase, 21, 22, 24
Application event, 207, 226
ArchiMate, 6–8, 11, 16
Association
  many-to-many association, 96
  one-to-one association, 96
  unary association, 97, 122
AssociationClass, 81
Authorisation, 218

B
Backward accessibility, 130, 131
Behavioural aspects, 15
Binary association, 95–97
Bunge–Wand–Weber (BWW), 42
Business architecture, 9–11, 15, 16, 33, 223
Business concept, 110
Business event, definition, 57, 110
Business intelligence, 219
Business logic, 33, 34, 205, 224, 247, 250–251, 264
Business motivation model, 233
Business object, definition, 53
Business object layer, 33
Business policy, 232
Business process enactment, 16
Business process event, 209, 226
Business process layer, 33, 34, 107, 108, 205, 207, 218, 219, 228, 238–240
Business process model, 11, 15, 16, 20, 21, 224, 225, 227, 230, 231, 236, 240
Business Process Modelling Notation (BPMN), 14, 23, 29, 238, 240, 248
Business rule, 154, 232
Business rules, 65

C
Capability maturity model (CMM), 41
Cardinality, 62
Cascading end, 119, 216
CASE-tool, 47, 48, 147
Class, 95
Class diagram, 26, 52, 61, 80, 83, 94, 136, 248
Classification
  dynamic, 172
  multiple, 172
  single, 172, 173
  static, 172, 173
Code generator, 30, 258
Common language, 35, 49
Computation independent model (CIM), 25, 26
Computer aided software engineering (CASE), 46
Conceptual Modeling Quality Framework (CMQF), 42
Conforms, 176
Consistency, 21, 23, 26, 29, 30, 45, 83, 89, 103, 136, 221, 225, 238–240

© Springer International Publishing Switzerland 2014

M. Snoeck, *Enterprise Information Systems Engineering*, The Enterprise Engineering Series, DOI 10.1007/978-3-319-10145-3

Consistency (*cont.*)
autocomplete, 23, 47–49
behavioural, 66, 128, 138
checking, 43, 46, 80, 82, 240
consistency by analysis, 47, 48
consistency by construction, 47–49
consistency by monitoring, 47, 48
inconsistency management, 49
inter-diagram consistency, 43
intra-diagram consistency, 43
Consistent event, 205
Constraints, 154, 249
attribute constraint, 156
class constraint, 154
domain constraint, 156
implementation, 155, 230, 251, 252
invariant, 154
method constraints, 154
precondition, 154
sequence constraints, 127, 251
validation constraint, 156
Contract, 138, 140
Contract rule, 120
Correctness, 46
Cost, 21, 34, 49, 58, 116, 119, 150, 232
Cross-cutting concerns, 219
CRUD-matrix, 26, 47, 57, 112

D
Data analytics, 219
Database schema, 86, 248, 249
Data perspective, 15
Data quality, 150
Deadlock, 45, 269
Default lifecycle rule, 138, 146
Delete, 113
Demand side, 10, 11, 29, 31, 49
Designation, 54, 150
Design patterns, 9, 24, 255
Design phase, 21
Domain model, 15–20, 24, 25, 29, 31–33, 36, 107, 205, 208, 236, 239
definition, 11

E
Enforcement level, 233
Enterprise architecture, 9, 11, 12, 15, 24, 29
Enterprise engineering, 4, 29
Enterprise information system, 19
Enterprise layer, 31–34, 205, 207, 208, 210, 215, 218, 228, 231, 247–255, 261–263
Enterprise model, 11, 16, 19
Entity relationship model, 79
Environment, 110
Event
activity, 109
atomic event, 58, 215, 217
concept, 109
consistent event, 58, 216, 217
environment controlled, 111, 209, 227
event log, 219
handling layer, 205, 215, 230
machine controlled, 111
notification, 110
relationships, 227
request, 109
shared, 111
unshared, 111
Existence dependency, 26, 62
definition, 83
motivation, 80
Existence dependency graph, 94
definition, 84
External validity, 27, 43, 48, 49

F
Feasibility study, 21
Finite state machines (FSMs), 26, 66, 127, 143, 269
Foreign key, 249
Formal language, 45, 46
Forward accessibility, 130

G
Generalisation/specialisation, 79
Genericity, 37, 38
Global behaviour, 134, 142, 269
Goal modelling, 15

I
Implementation bias, 36, 49, 54
Implementation phase, 21, 245
Independent model, 25
Individual behaviour, 134
Information engineering, 46, 112
Information needs, 32, 34, 102, 150, 205, 206
Information object type, 205

Information system architecture, 9
application architecture, 9–11
data architecture, 9–11
information architecture, 9–11
Information system event, 57, 207, 226
Information system service, 16, 33, 70
complex service, 211
input service, 32, 34, 70, 74, 205, 209, 210, 214, 215, 262
output service, 32, 34, 70, 205, 208, 209, 212, 214
Information system service layer, 32, 33, 71, 205, 217, 262
Inheritance, 176
abstract class, 173
deep extent, 173, 174
definition, 172
descendant, 173
extension, 176–177
extent, 173, 174
multiple inheritance, 172
overriding, 177
parent, 173
refinement, 177
single inheritance, 172
substitutability, 176
total specialisation, 173
Inherited dependency, 181
Intentional quality, 43, 44, 46
Interaction, 54
Inter-diagram consistency, 69
Internal validity, 43, 47, 50

J
JMermaid, 49, 161, 164, 205, 219, 252, 255, 256, 258

K
A Kind Of, 79

L
Layered architecture, 34, 49, 215, 226
Layers, 32, 36, 218
Lifecycle, 84, 88–94

M
Machine, 110
Maintenance, 21
Minimized FSM, 131
Model-driven architecture (MDA), 25
Model-driven engineering (MDE), 24, 26, 29, 153, 245
Modelling language, 7, 14, 20, 42, 44, 46, 239
Modelling method, 20, 22, 23, 26, 29, 41, 54
Model quality, 20, 22, 23, 26, 29, 31, 42, 44, 48, 168
framework, 42
rules, 168
More deterministic than, 139
definition, 140
Multi-party interaction, 57, 134
Multiple propagation, 161
Multiplicity, 61, 85

N
Navigation, 249
Non-determinism, 131
Notation, 88, 129, 208
Bachman notation, 87
entity relationship, 87
MERODE notation, 86, 173
UML notation, 85, 86, 127, 133, 172, 173, 208

O
Owner, 64, 116
Owner participations, 125

P
A Part Of, 79, 98, 99
Persistence, 206, 248
Petri Net, 238
Physical removal, 113, 159, 219, 249
Platform independent model (PIM), 25, 26, 41, 74, 245
Platform specific models (PSMs), 25, 74, 245
Pragmatic quality, 44–46, 48
Precondition, 111, 112, 251
Problem orientation, 37, 49
Product line, 35
Programme management, 3
Propagation rule, 115, 138
Prototype, 13, 19, 20, 26, 27, 30, 50, 68, 161, 247, 250, 251, 255, 259

Q
Quality assurance, 31, 41, 49, 50
Quality control, 48, 49, 75, 82, 136

R
Real world, 18, 19, 39, 53, 59, 101, 102, 110, 209
Referential integrity, 119, 157, 254
Requirements engineering, 31, 35, 36, 38–40, 47, 48, 50, 52, 86, 108, 224
Requirements gathering, 24, 32, 35–37, 39, 40, 80, 81, 86, 91
Restriction rule, 140, 141, 146–147
Role(s), 147, 160, 172, 191, 197

S
SBVR, 232
Scope, 17, 29, 58, 75
Semantic quality, 43, 45, 48, 168
  completeness, 43, 45, 47–49
  validity, 43
SEQUAL, 42
Software development process, 21, 22, 41
  iterative approach, 22
  plan-driven approach, 22
  software development lifecycle model (SDLM), 22
Software process improvement (SPI), 41
Software quality, 24, 41
Solution orientation, 49
Solution-oriented thinking, 38
Stability, 32
State pattern, 251
State transition table, 159, 251
Strategy development, 3
Stratification, 131
Strong typing, 172
Supply side, 10, 31, 34, 49, 50
Syntactic quality, 44, 48
Syntax checking, 42

T
Technology architecture, 9–11, 33, 34
Testing, 21, 22, 25, 44, 245
Three-tier architecture, 33
TOGAF, 6, 7, 9, 11, 16
Transaction, 205, 216, 217
  multi-event transaction, 217
  single-event transaction, 217
Transformation(s), 41
  definition, 246
  model-to-code transformation, 24, 26, 74, 247
  model-to-model transformation, 24, 26, 74, 245, 247
  rule, 246, 250
Transformational approach, 24, 245
Transformation engine, 24, 31, 41, 246
Type of involvement rule, 117

U
Understandability, 44, 46, 48, 103, 131
Unified Modelling Language (UML), 14, 20, 23, 25, 26, 29, 37, 46, 80, 81, 180, 246
Universe of discourse, 17, 45, 57, 59, 79, 80, 82, 114, 174, 196, 225
Use case, 35, 37–39, 71, 207, 208
  diagrams, 226, 247
  modelling, 38

V
Variability, 35
Virtual enterprise, 19

W
Weak entity, 87
Weak relationship, 87
Work organisation, 15–16, 33, 34, 38, 92, 108, 207, 224, 225, 227, 233

Z
Zachman, 6–8, 16, 224